Le logiciel de simulation MATLAB Introduction Partie I

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Introduction

Bienvenue à l'enseignement supérieur! Si vous voulez réussir, vous devez savoir comment cet endroit fonctionne. Une des principales règles que vous devez savoir est la différence entre les instructeurs que vous aviez avant votre entrée en université et ceux que vous aurez maintenant. Permettez-moi de prendre quelques minutes pour vous expliquer cela.



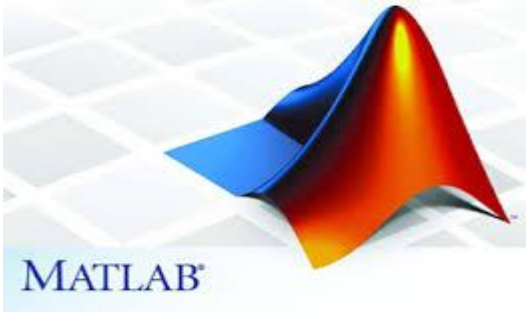
Fountain of Knowledge at Texas Tech University

Jusqu'à maintenant votre instruction a été dans les mains des enseignants, et le travail d'un enseignant est de vous assurer que vous apprenez. Les enseignants sont évalués sur la base des résultats d'apprentissage, généralement mesurée par des tests standardisés. Si vous n'apprenez pas, alors votre professeur est blâmé. Cependant, les choses sont très différentes pour un professeur d'université. Son travail ne consiste pas à vous faire apprendre. À l'université, l'apprentissage est votre travail - et vous seul. Le travail de votre professeur est de vous conduire à la fontaine de la connaissance. Que vous buvez profondément ou seulement se gargariser est entièrement à vous.





MATLAB (MATrix LABoratory) est un logiciel de programmation est de simulation.
Le logiciel MATLAB implémente le langage MATLAB. Il fournit une très vaste bibliothèque de fonctions pré-définies pour rendre la programmation technique une tâche plus faciles et plus efficaces. Avec cette très grande variété de fonctions, il est beaucoup plus facile de résoudre les problèmes techniques dans MATLAB que dans d'autres langages tels que Java, Fortran ou C ??.





Le plus grand avantage de Matlab est qu'il est facile à utiliser.
Ce cours est une introduction aux facilités que procure Matlab aux étudiants et aux programmateurs.



Aujourd'hui Matlab est utilisé par beaucoup d'entreprises pour le calcul et le prototypage rapide lors du developement de nouveaux systèmes.

Exemples d'entreprises qui utilisent Matlab pour le prototypage rapide

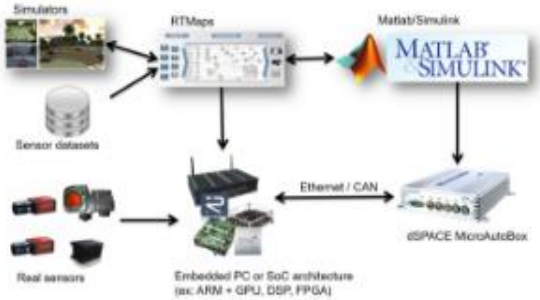
INTEMPORA EDITOR OF RTMAPS

DOWNLOAD NEWSLETTER SIGN UP SIGN IN

HOME PRODUCTS USE CASES NEWS & EVENTS SUPPORT ABOUT US CONTACT



This tutorial shows how to interact with Simulink and dSPACE prototyping systems from the RTMaps studio. The goal is to exchange data between RTMaps and Simulink, while staying compatible with dSPACE prototyping systems like dSPACE MicroAutoBox with complete control of data synchronisation.



dSPACE: Simulation central

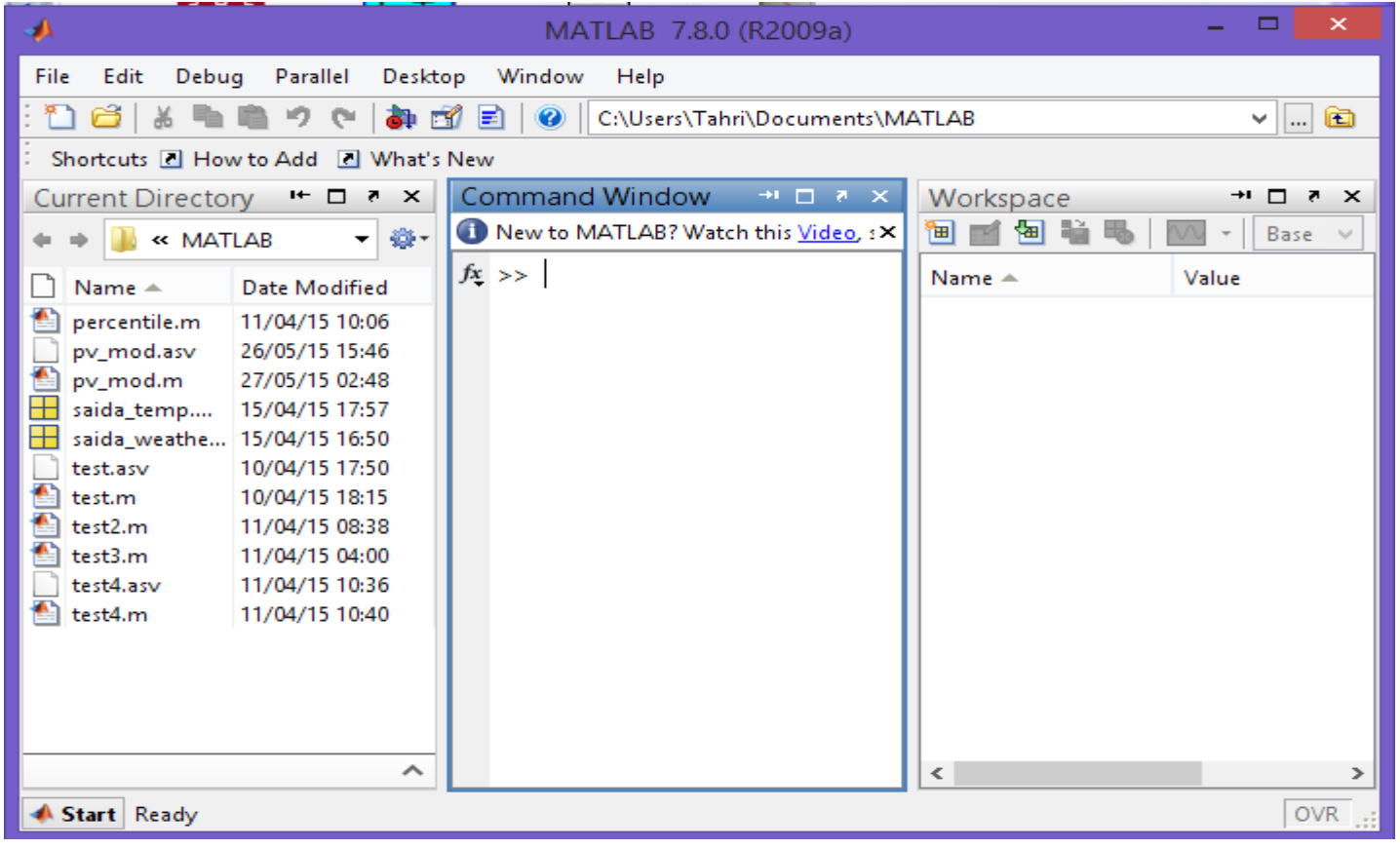
dSPACE's ASMs and ModelDesk bring together all the simulation models the user requires, so they can be run from one user interface

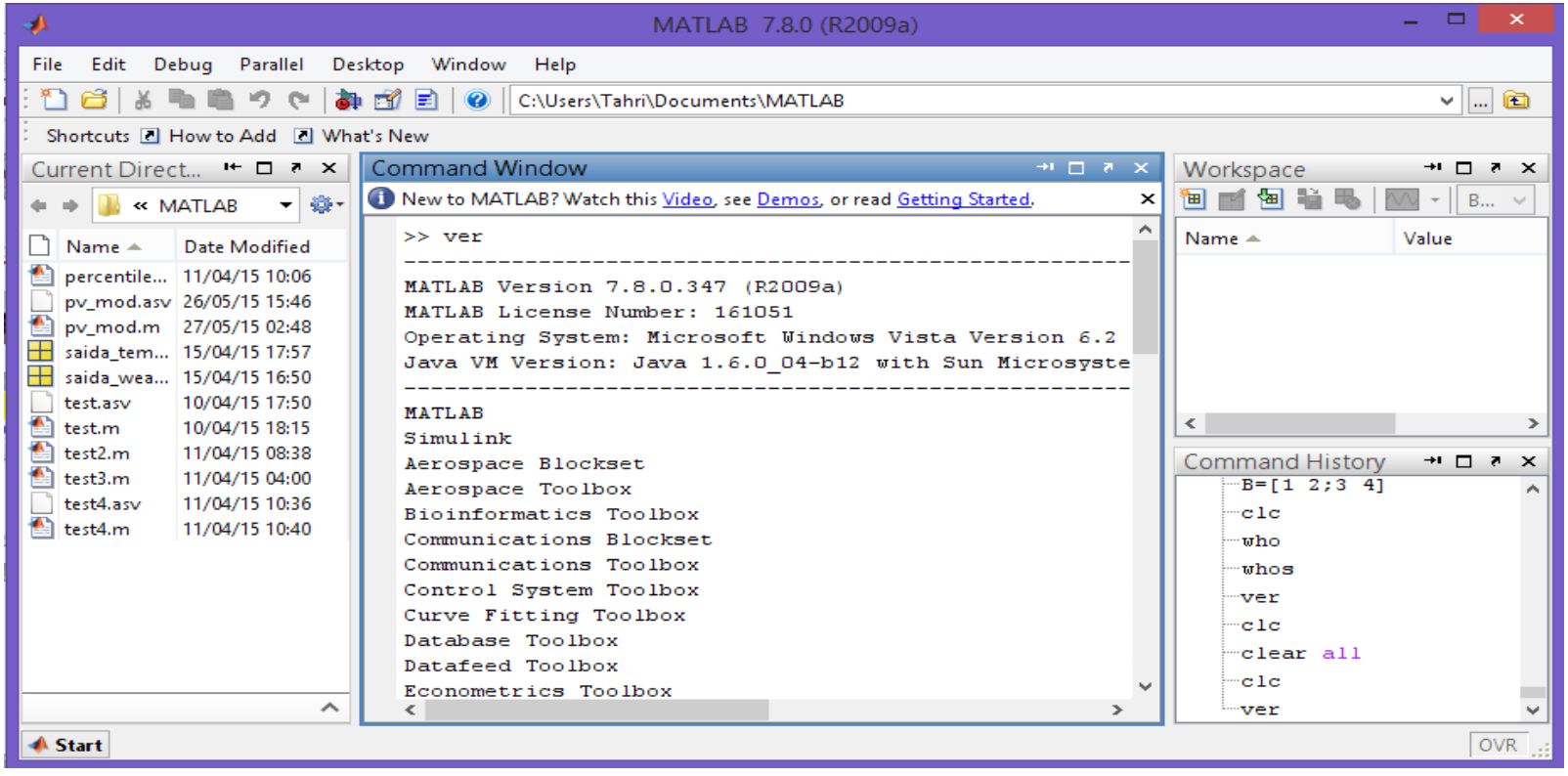


Image: Custom models can be easily included in the open Automotive Simulation Models from dSPACE

Today's development process for automotive control algorithms is highly complex. Modern control algorithms require intensive interaction between distributed functions on different electronic control units (ECUs). A typical development process starts with the PC-based simulation of single control functions and ends with a full-blown ECU network test on a hardware-in-the-loop simulator (HIL). During each development step, functions have to be verified by simulating them together with a model of the device under control (e.g., combustion engine, brake hydraulics, electric motor). dSPACE Automotive Simulation Models (ASM) are the ideal toolkit for this. They are MATLAB/Simulink-based models for simulating essential automotive components and properties, such as combustion engines, electric motors, vehicle dynamics, electrical systems, and traffic for passenger vehicles as well as commercial vehicles.

2. MATLAB


MATLAB 7.8.0 (R2009a)
File Edit Debug Parallel Desktop Window Help
C:\Users\Tahri\Documents\MATLAB
Shortcuts How to Add What's New
Current Directory
« MATLAB
Name Date Modified
percentile.m 11/04/15 10:06
pv_mod.asv 26/05/15 15:46
pv_mod.m 27/05/15 02:48
saida_temp.... 15/04/15 17:57
saida_weathe... 15/04/15 16:50
test.asv 10/04/15 17:50
test.m 10/04/15 18:15
test2.m 11/04/15 08:38
test3.m 11/04/15 04:00
test4.asv 11/04/15 10:36
test4.m 11/04/15 10:40
Command Window
New to MATLAB? Watch this Video,
>>
Workspace
Base
Name Value
Start Ready
OVR



La commande ver permet de vous donner des informations sur la version de la copie du logiciel Matlab ainsi que toutes ses composantes.

2.1 Scalaire

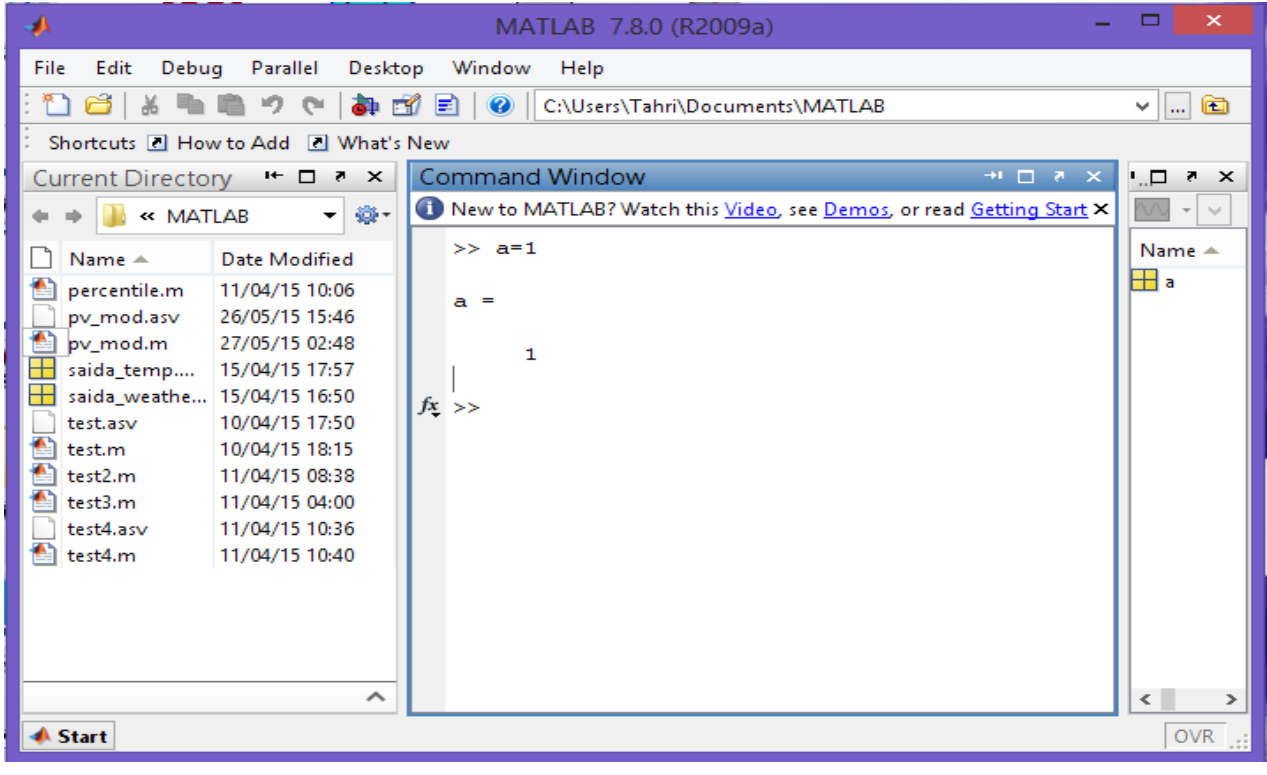


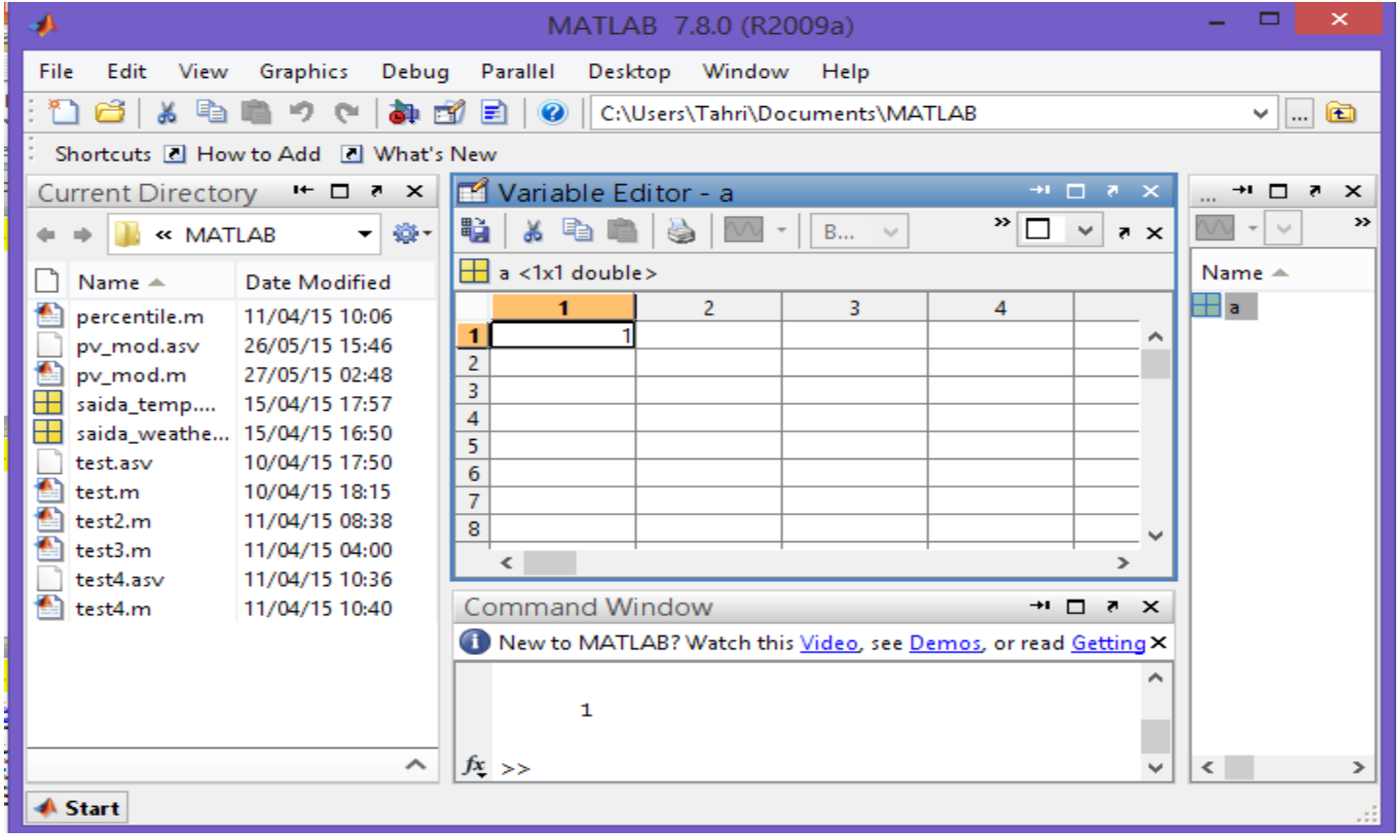
a est un scalaire.

```
>> a=1

a =

     1
```





Après double click sur la variable a, il est évident que la variable a est considérée comme une matrice de dimension (1,1).

2.2 Vecteur



A vecteur de dimension (1,2), une ligne, deux colonnes.

```
>> A=[1,1]

A =

    1    1
```

La commande clc permet d'éffacer l'écran du workspace.

```
>> clc
```

La commande length permet de visualiser la dimension d'un vecteur.

```
>> length(A)

ans =

    2
```

```
>> m=length(A)

m =

    2
```

2.3 Matrice



B matrice de dimension (2,2), deux lignes, deux colonnes.

```
>> B=[1 2;3 4]

B =

    1    2
    3    4
```

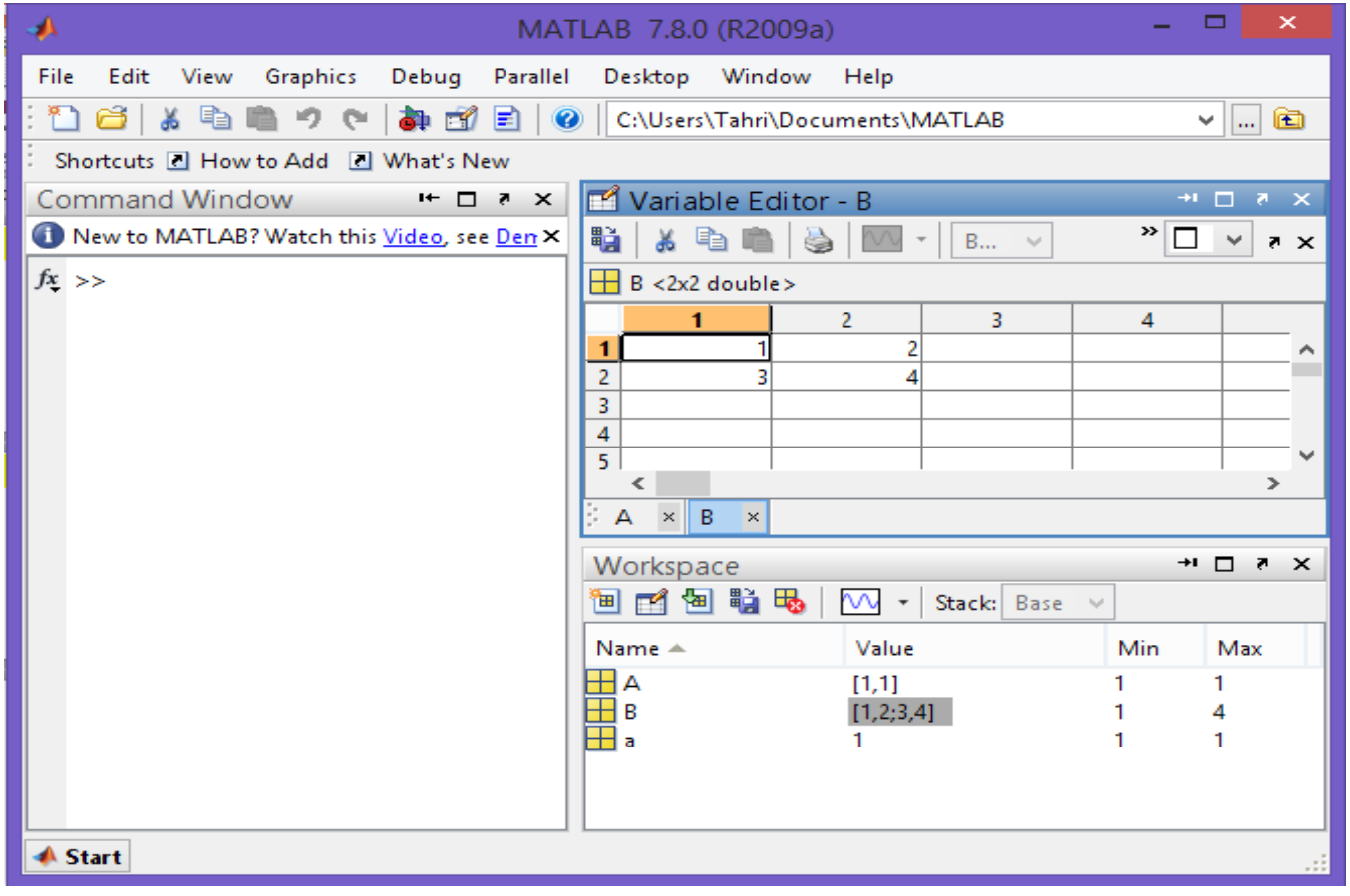
La commande size permet de visualiser la dimension d'une matrice.

```
>> n=size(B)

n =

    2    2
```

```
[l,c]=size(B)
l =
     2
c =
     2
```



Remarquer chaque variable est définit dans le workspace

2.4 Fonctions prédéfinies



Toutes les fonctions courantes et beaucoup parmi les moins courantes existent. La plupart d'entre elles fonctionnent en complexe. On retiendra que pour appliquer une fonction à une valeur, il faut mettre cette dernière entre parenthèses.

Exemple :
>> sin(pi/12)
ans =
0.16589613269342

Voici une liste non exhaustive :
- fonctions trigonométriques et inverses : sin, cos, tan, asin, acos, atan
- fonctions hyperboliques (on rajoute "h") : sinh, cosh, tanh, asinh, acosh, atanh
- racine, logarithmes et exponentielles : sqrt, log, log10, exp
- fonctions erreur : erf, erfc
- fonctions de Bessel et Hankel : besselj, bessely, besseli, besselk, besselh.

Matlab dispose de plusieurs fonctions prédéfinies qui facilitent le travail.

La commande who permet de voir quelles sont les variables définies dans le workspace.

```
>> who
Your variables are:

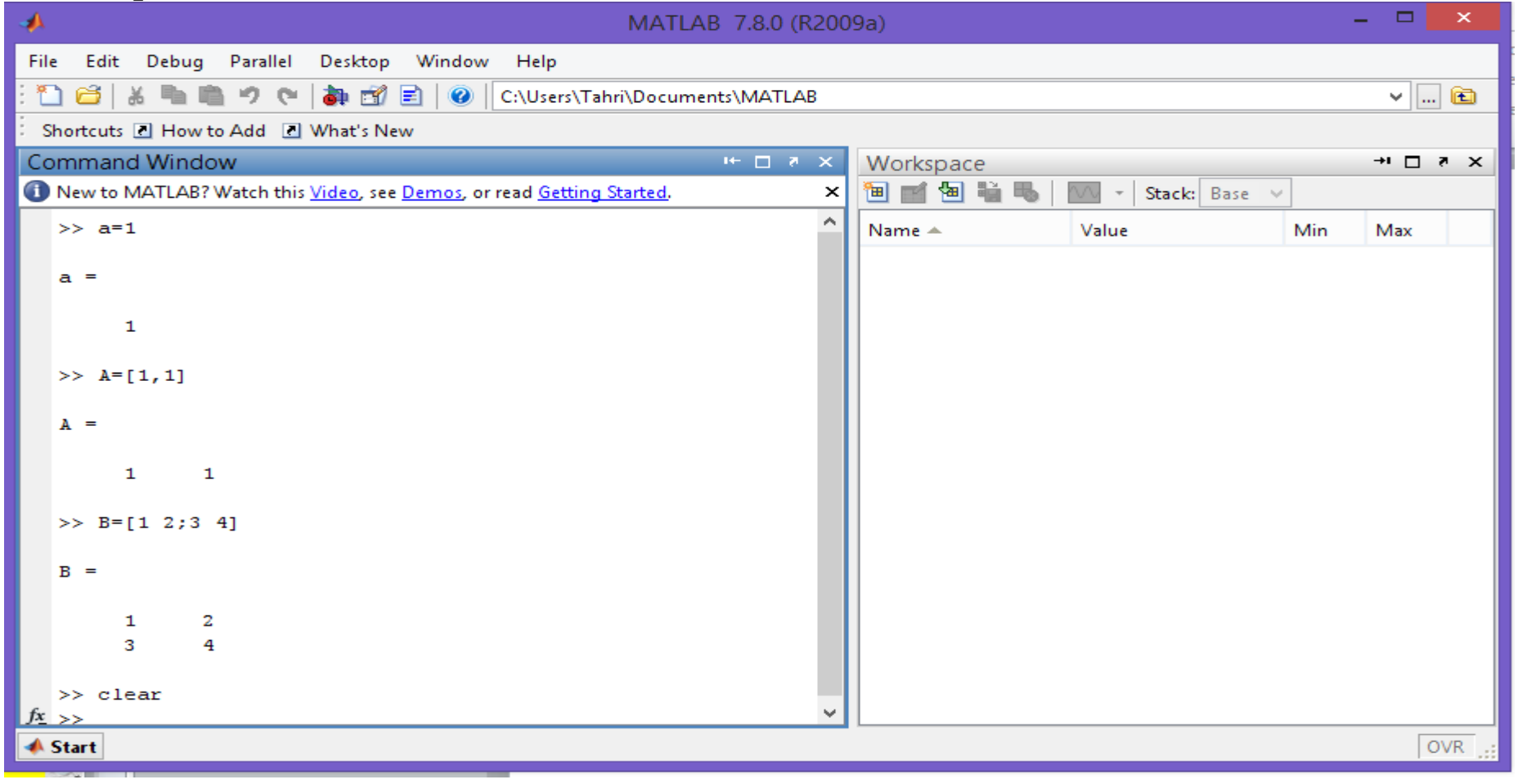
A  B  a
```

La commande whos permet de voir quelles sont les variables définies dans le workspace, leurs dimensions et leurs natures.

```
>> whos
  Name      Size            Bytes  Class     Attributes

  A         1x2                16  double
  B         2x2                32  double
  a         1x1                 8  double
```

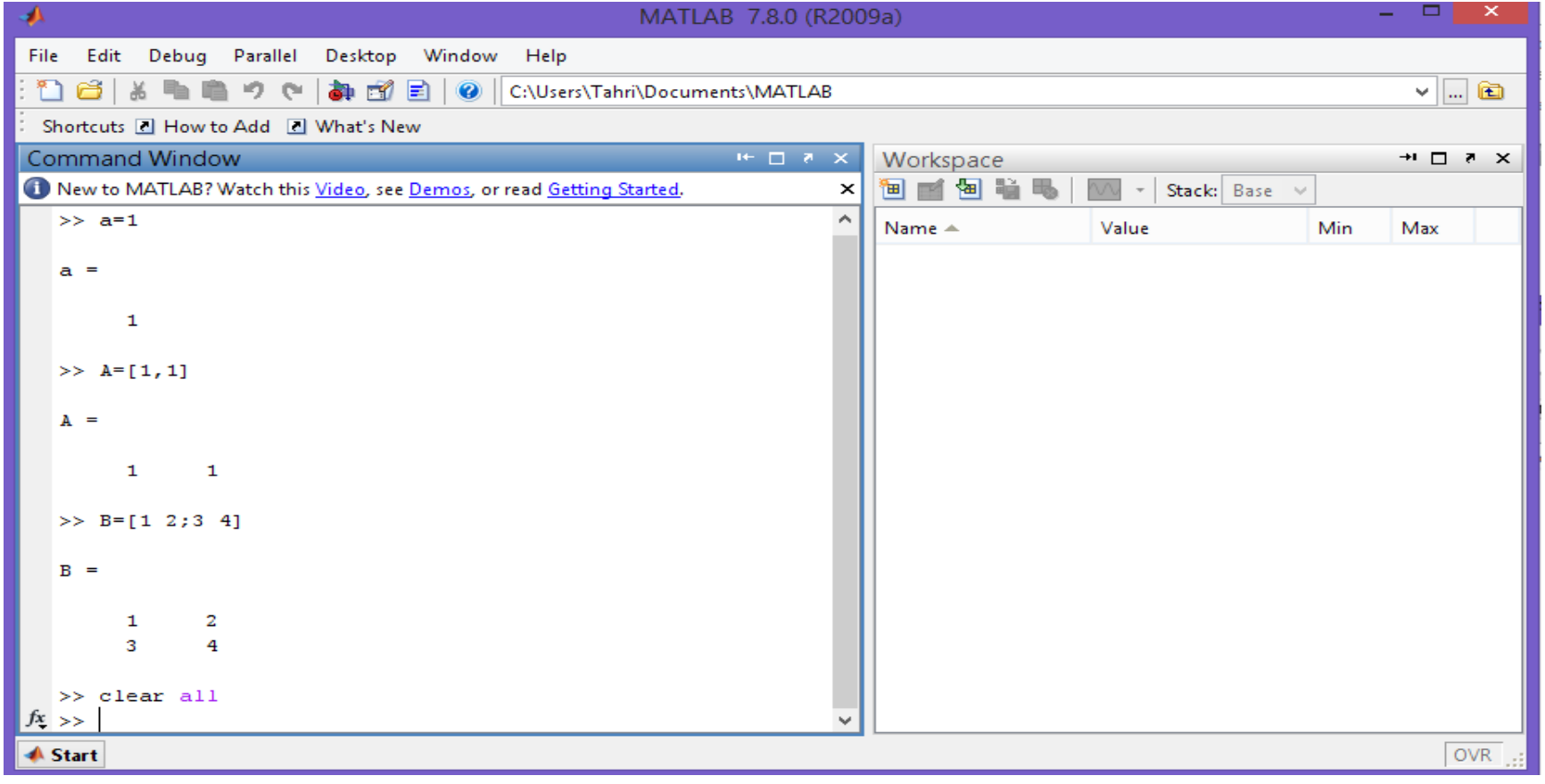
La commande clear permet d'effacer toutes les variables définies dans le workspace.



La commande clc permet d'effacer l'écran du workspace.

```
>> clc
```

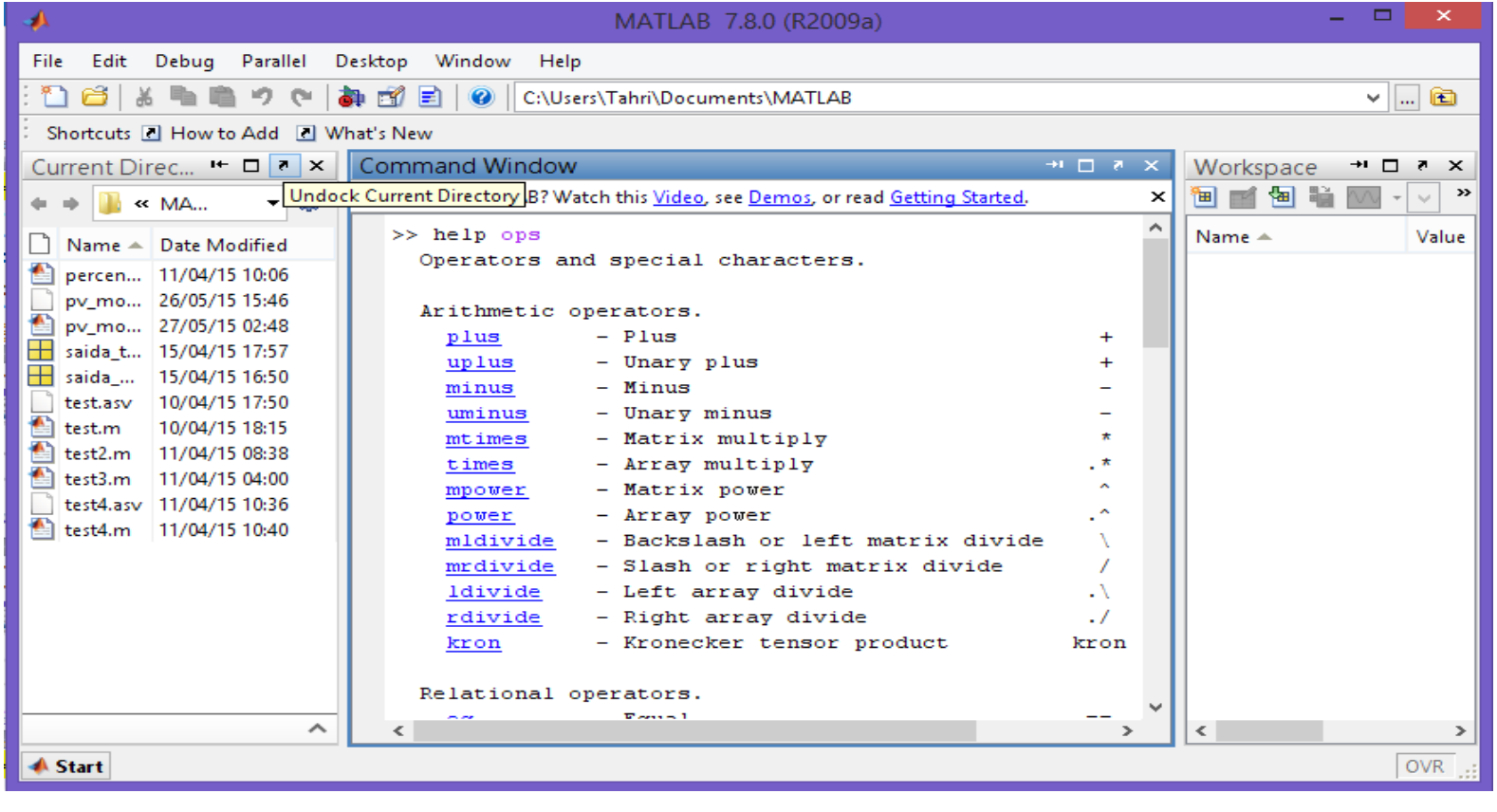
La commande clear all permet aussi d'effacer toutes les variables définies dans le workspace.



2.5 Les opérations



La commande help ops permet de visualiser toutes les opérations qui existent en Matlab.



Operateurs de base

Les opérateurs ci-dessous sont valables pour les scalaires et pour les matrices.
Matlab effectuera la bonne opération en fonction du type des termes de l'opérateur.

```
% — Définitions de matrices spéciales ———————————————————— %

'     % Transposition (matrice)
+     % Addition
-     % Soustraction
*     % Multiplication
/     % Division à droite
\     % Division à gauche
^     % Puissance
```

2.5.1 L'addition

```
>> c=1+2

c =

    3
```

2.5.2 La soustraction

```
>> d=1-2

d =

   -1
```

2.5.3 La multiplication

```
>> e=2*3

e =

    6
```

2.5.4 La division à gauche et à droite

```
>> f=9/2
```

C'est la division à droite.

```
f =

    4.5000
```

Matlab dispose de la division à gauche.

Soit le système linéaire suivant.
Trouver x1 et x2

$$\begin{bmatrix} 3 & 9 \\ 2 & 7 \end{bmatrix} \begin{bmatrix} x1 \\ x2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

```
>> A=[3 9;2 7];B=[1;2];
>> A\B
```

→ La division à gauche

$$A \quad X \quad = B$$

```
ans =

   -3.6667
    1.3333
```

$$A^{-1}AX = A^{-1}B$$

$$X = A^{-1}B$$

2.5.5 La transposition de matrice

```
>> A=[1 2;3 4]
A =
    1    2
    3    4
>> A'
ans =

    1    3
    2    4
```

2.5.6 L'inversion d'une matrice

```
>> inv(A)

ans =

   -2.0000    1.0000
    1.5000   -0.5000
```

2.5.7 Déterminant d'une matrice

```
>> A=[1 2;3 4]
A =
    1    2
    3    4
>> det(A)

ans =

   -2
```

2.6 Opérations élément par élément

Ajouter un point devant un des opérateurs classiques signifie que l'on effectue l'opération élément par élément.

```
% — Opérations élément par élément ———————————————————— %

.+    % Addition
.-    % Soustraction
.*    % Multiplication
./    % Division
.^    % Puissance
```

Plus disponible dans les nouvelles versions de Matlab.

```
>> A=[1 2;3 4]
A =
    1    2
    3    4
>> A.^2
ans =

    1    4
    9   16
```

```
>> A=[1 2;3 4],B=[5 6;7 8]

A =
    1     2
    3     4


B =
    5     6
    7     8

>> A.*B

ans =

    5    12
   21    32
```

2.7 Création de matrices particulières

Matlab permet de créer de manière simplifiée des matrices particulières.
m et n sont les dimensions de la matrice

```
% — Définitions de matrices spéciales ———————————————————— %

eye(m,n)          % Matrice unité
ones(m,n)         % Matrice dont tous les éléments sont égaux à un
zeros(m,n)        % Matrice dont tous les éléments sont égaux à zéro
rand(m,n)         % Matrice d'éléments aléatoires
diag(m,n)         % Matrice diagonale
meshgrid(m :n,k :l)  % Renvoie deux matrices de grilles définissant un quadrillage
```

```
>> A=[1 2;3 4]
A =
    1    2
    3    4
>> diag(A)
ans =
    1
    4
```

```
>> eye(2,2)
ans =
    1    0
    0    1
```

```
>> ones(2,2)
ans =
    1    1
    1    1
```

```
>> zeros(2,1)
ans =
    0
    0
```

```
>> rand(2,3)
ans =
    0.1270    0.6324    0.2785
    0.9134    0.0975    0.5469
```

2.8 Chaine de caractères



Matlab enregistre des variables sous forme de chaine de caractères

```
>> nom='Mohamed'

nom =

Mohamed
```

La fonction disp (display) permet de visualiser le contenu d'une variable

```
>> disp(nom)
Mohamed
```

num2str(x) retourne une chaine de caractères qui correspond à un nombre stocké en x

str2num(s) retourne un nombre correspondant à la chaine de caractère s

str2double(s) retourne un nombre correspondant à la chaine de caractère s

length(s) retourne le nombre de caractère dans la chaine de caractère s

lower(s) retourne la chaine de caractères en minuscule

upper(s) retourne la chaine de caractères en majuscule

sName(4) retourne le 4ème caractère de la chaine de caractère s

sName(4:6) retourne du 4ème au 6ème caractère de la chaine de caractère s

3. Constantes prédéfinies

Matlab dispose de constantes prédéfinies, voici la liste.

```
eps       % Floating-point relative accuracy
i         % Imaginary unit
Inf       % Infinity
intmax    % Largest value of specified integer type
intmin    % Smallest value of specified integer type
j         % Imaginary unit
NaN       % Not-a-Number
pi        % Ratio of circle's circumference to its diameter
realmax   % Largest positive floating-point number
realmin   % Smallest positive normalized floating-point number
```

ATTENTION : ces variables ne sont pas protégées, donc si vous les affectez, elles ne gardent pas leur valeur initiale. C'est souvent le problème pour i et j que l'on utilise souvent spontanément comme indices de boucles, de telle sorte qu'on ne peut plus ensuite définir de complexe.

```
>> 1+2i
ans =

   1.0000 + 2.0000i
>> eps

ans =

  2.2204e-016

   >> pi

   ans =

       3.1416
```

```
>> 1/0

ans =

    Inf


>> 0/0

ans =

    NaN
```

4. Le typage de données

Matlab effectue le typage de données, autrement dit il adapte le type des opérateurs de manière à permettre aux opérations de s'effectuer.

```
% — Fonctions de conversion ———————————————————————— %
cast                              % Cast variable to different data type
double                            % Convert to double precision
int8, int16, int32, int64         % Convert to signed integer
single                            % Convert to single precision
typecast                          % Convert data types without changing underlying data
uint8, uint16, uint32, uint64     % Convert to unsigned integer
```

```
% — Fonctions de conversion string vers nombre ———————————————— %
base2dec        %Convert base N number string to decimal number
bin2dec         %Convert binary number string to decimal number
cast            %Cast variable to different data type
hex2dec         %Convert hexadecimal number string to decimal number
hex2num         %Convert hexadecimal number string to double-precision number
str2double      %Convert string to double-precision value
str2num         %Convert string to number
unicode2native  %Convert Unicode characters to numeric bytes
```

Merci

Pour

votre attention

Le logiciel de simulation MATLAB
Partie 2
Graphiques

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Graphique simple

Les commandes suivantes montrent comment afficher un graphique simple. Matlab traite que des valeurs discrètes, il faut d'abord commencer par définir l'intervalle de valeurs de l'abscisse puis on calcule la valeur de la fonction (dans notre cas un cosinus) pour chaque échantillon de temps et l'on arrive au vecteur y.

```
% — Graphique simple ———————————————————————— %

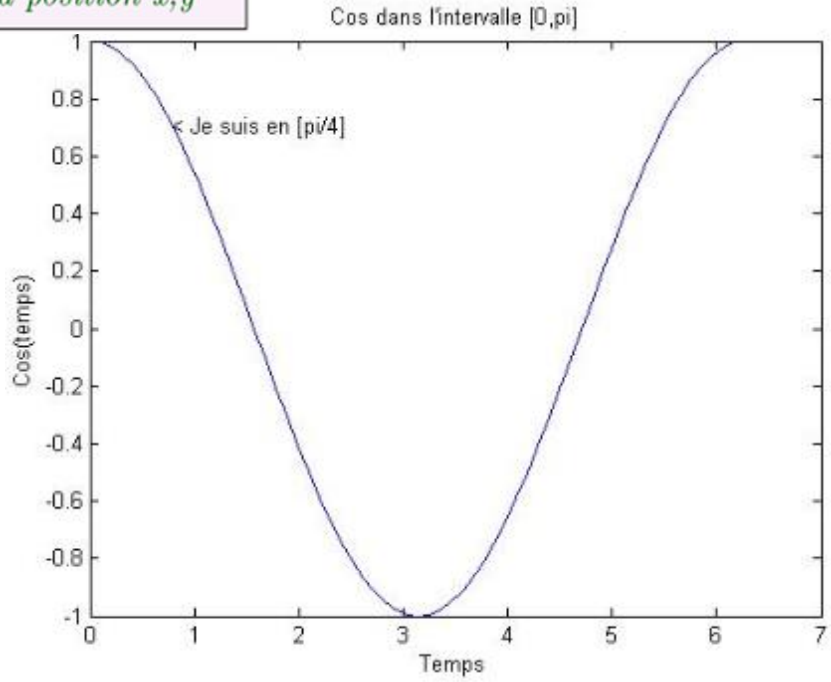
t = 0 :pi/30 :2*pi;                          % Abscisse de 0 à 2π par pas de π/30
y = cos(t);                                  % Valeurs de la fonction dans le temps

figure;                                      % Nouvelle fenêtre
plot(t,y)                                    % Tracer
title('Cos dans l"intervalle [0,pi]');       % Titre
xlabel('Temps');                             % Légende en abscisse
ylabel('Cos(temps)');                        % Légende en ordonnée
text(pi/4,cos(pi/4),'¡ Je suis en [pi/4]')   % Chaîne de caractères à la position x,y
```

```
% — Graphique simple ———————————————————————— %

t = 0 :pi/30 :2*pi;                          % Abscisse de 0 à 2π par pas de π/30
y = cos(t);                                  % Valeurs de la fonction dans le temps

figure;                                      % Nouvelle fenêtre
plot(t,y)                                    % Tracer
title('Cos dans l"intervalle [0,pi]');       % Titre
xlabel('Temps');                             % Légende en abscisse
ylabel('Cos(temps)');                        % Légende en ordonnée
text(pi/4,cos(pi/4),'¡ Je suis en [pi/4]')   % Chaîne de caractères à la position x,y
```



```
>> help plot
 PLOT   Linear plot.
    PLOT(X,Y) plots vector Y versus vector X. If X or Y is a matrix,
    then the vector is plotted versus the rows or columns of the matrix,
    whichever line up.  If X is a scalar and Y is a vector, disconnected
    line objects are created and plotted as discrete points vertically at
    X.

    PLOT(Y) plots the columns of Y versus their index.
    If Y is complex, PLOT(Y) is equivalent to PLOT(real(Y),imag(Y)).
    In all other uses of PLOT, the imaginary part is ignored.
```

Various line types, plot symbols and colors may be obtained with PLOT(X,Y,S) where S is a character string made from one element from any or all the following 3 columns:

b	blue	.	point	-	solid
g	green	o	circle	:	dotted
r	red	x	x-mark	-.	dashdot
c	cyan	+	plus	--	dashed
m	magenta	*	star	(none)	no line
y	yellow	s	square		
k	black	d	diamond		
w	white	v	triangle (down)		
		^	triangle (up)		
		<	triangle (left)		
		>	triangle (right)		
		p	pentagram		
		h	hexagram		

The X,Y pairs, or X,Y,S triples, can be followed by parameter/value pairs to specify additional properties of the lines. For example, PLOT(X,Y,'LineWidth',2,'Color',[.6 0 0]) will create a plot with a dark red line width of 2 points.

For example, PLOT(X,Y,'c+:') plots a cyan dotted line with a plus at each data point; PLOT(X,Y,'bd') plots blue diamond at each data point but does not draw any line.

PLOT(X1,Y1,S1,X2,Y2,S2,X3,Y3,S3,...) combines the plots defined by the (X,Y,S) triples, where the X's and Y's are vectors or matrices and the S's are strings.

For example, PLOT(X,Y,'y-',X,Y,'go') plots the data twice, with a solid yellow line interpolating green circles at the data points.

The PLOT command, if no color is specified, makes automatic use of the colors specified by the axes ColorOrder property. The default ColorOrder is listed in the table above for color systems where the default is blue for one line, and for multiple lines, to cycle through the first six colors in the table. For monochrome systems, PLOT cycles over the axes LineStyleOrder property.

If you do not specify a marker type, PLOT uses no marker.
If you do not specify a line style, PLOT uses a solid line.

PLOT(AX,...) plots into the axes with handle AX.

PLOT returns a column vector of handles to lineseries objects, one handle per plotted line.

2. Graphique avancé

Matlab permet de personnaliser les graphiques. Par exemple, il est possible de changer la couleur du trait, de marqué les points avec différents symboles tels que x, O, *,… etc. Voici un apperçu simple de ce qui est possible de faire

```
% — Graphique avancé ———————————————————— %

figure;                              % Nouvelle fenêtre

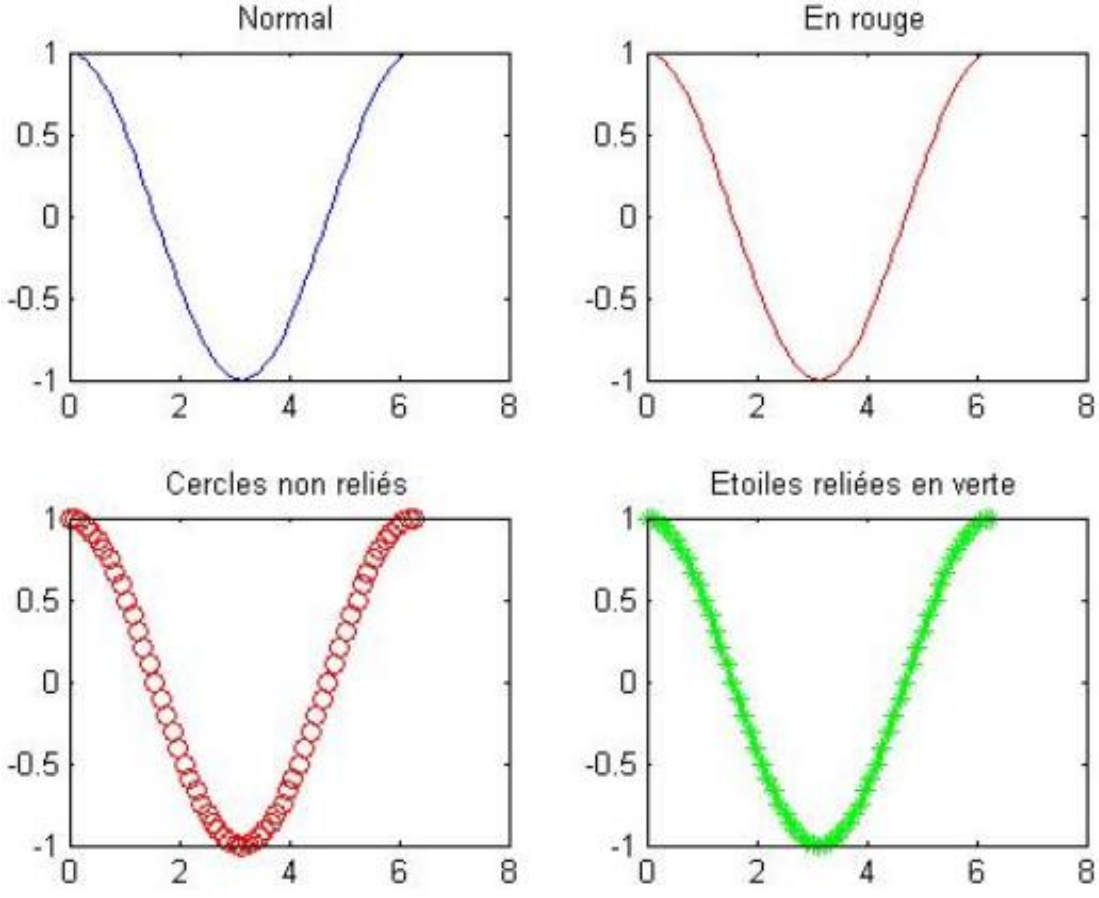
t = 0 :pi/30 :2*pi;                  % Définition de temps
y = cos(t);                          % Fonction du temps

subplot(2,2,1)                       % Zone supérieure gauche
plot(t,y);                           % Tracer
title('Normal');                     % Titre

subplot(2,2,2)                       % Zone supérieure droit
plot(t,y,'r');                       % Tracé en rouge
title('En rouge');                   % Titre

subplot(2,2,3)                       % Zone inférieur gauche
plot(t,y,'or');                      % Tracé points ronds en rouge
title('Cercles non reliés');         % Titre

subplot(2,2,4)                       % Zone inférieur droite
plot(t,y,'*-g');                     % Tracé point étoiles relié en vert
title('Etoiles reliées en verte');   % Titre
```


Normal
En rouge
Cercles non reliés
Etoiles reliées en verte
1
0.5
0
-0.5
-1
0
2
4
6
8

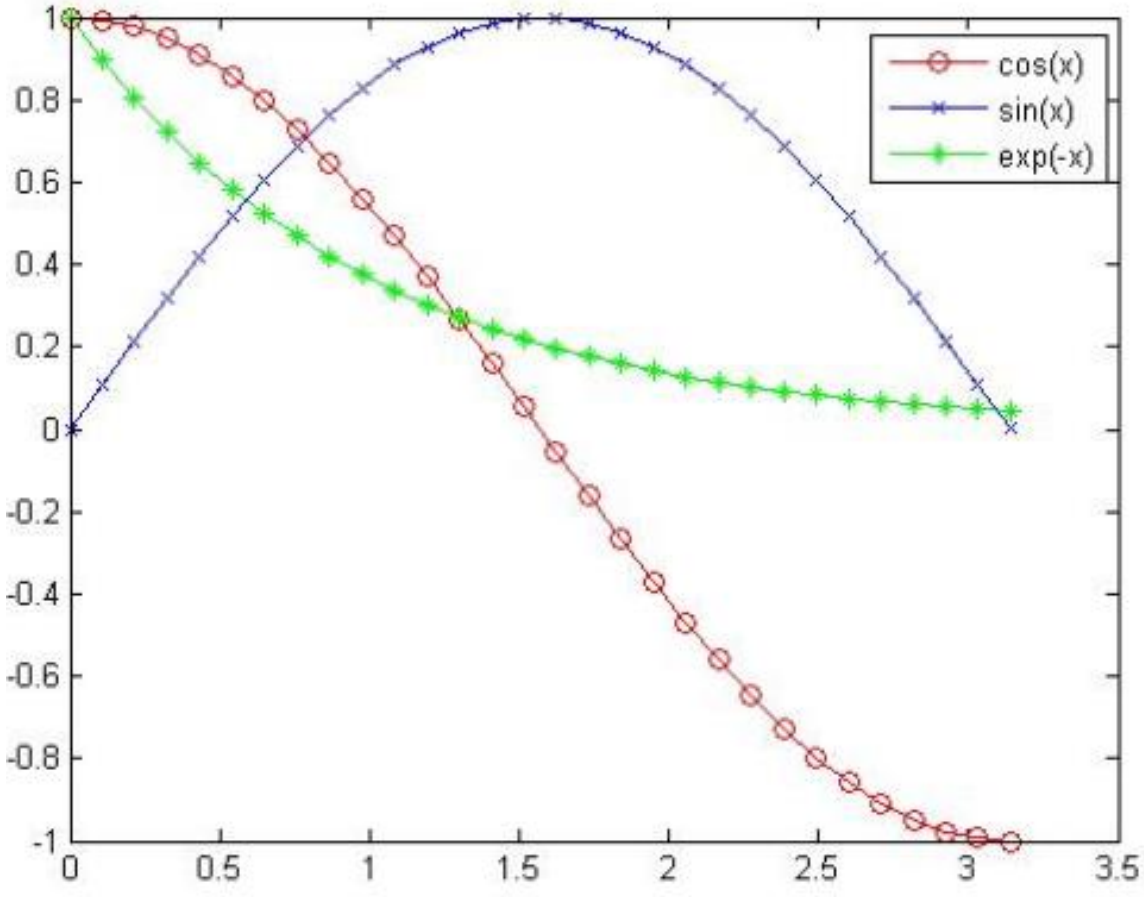
3. Graphique multiple 1

Il est possible de tracer plusieurs courbes sur le même graphique. Pour ce faire, une méthode consiste à mettre l'ensemble des fonctions à tracer dans les parenthèses qui suivent la commande plot.

```
% — Graphique multiple 1 ———————————————————————— %

figure;                                          % Nouvelle fenêtre

x = linspace(0,pi,30);                           % Fonction du temps
plot(x,cos(x),'o-r',x,sin(x),'x-b',x,exp(-x),'*-g')   % Tracer
legend('cos(x)','sin(x)','exp(-x)')              % Légende sur graphique
```


cos(x)
sin(x)
exp(-x)
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
0
0.5
1
1.5
2
2.5
3
3.5

4. Graphique multiple 2

Une autre méthode pour tracer plusieurs courbes sur un même graphique consiste à activer la commande hold on, ce qui stipule à Matlab de dessiner les graphiques les uns sur les autres.

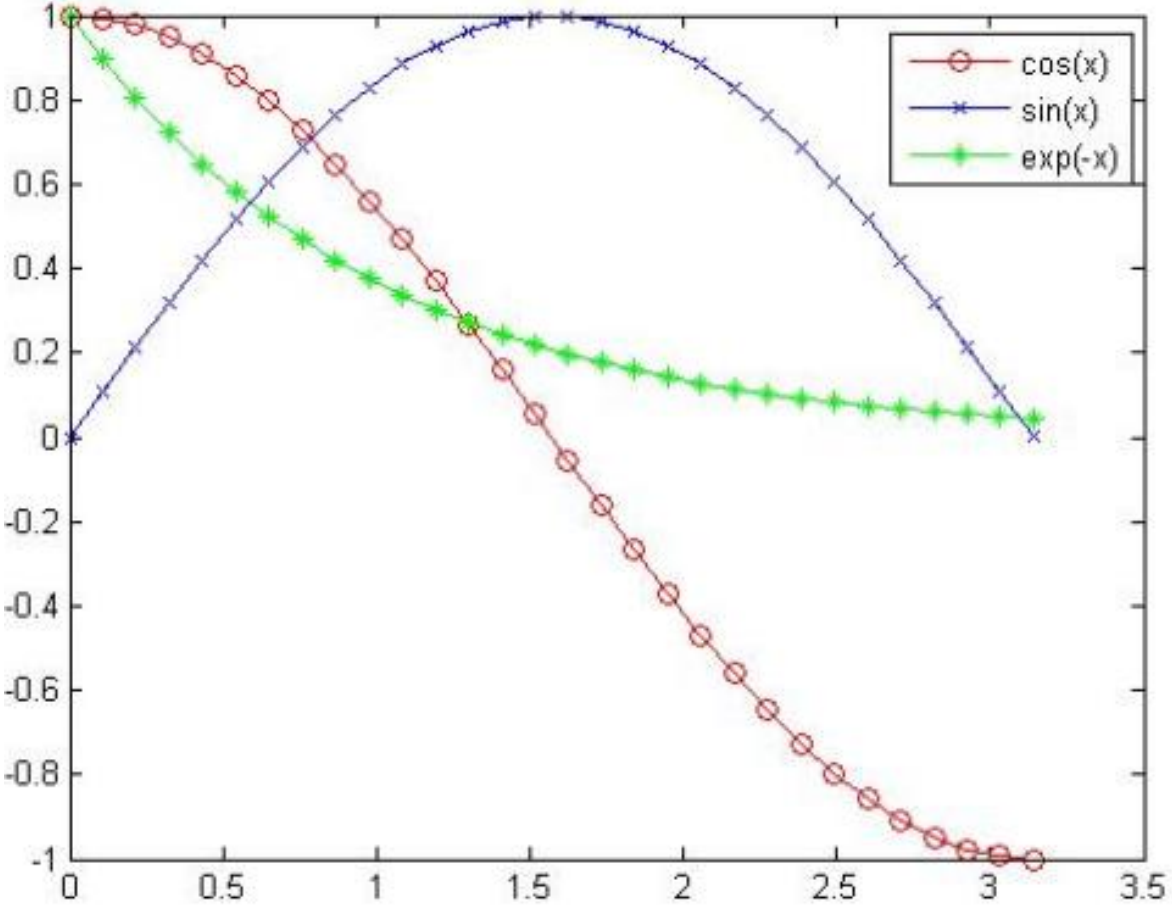
```
% — Graphique multiple 2 ———————————————————————— %

figure;                                  % Nouvelle fenêtre

x = linspace(0,pi,30);                   % Fonction du temps

hold on                                  % Maintient du graphique
plot(x,cos(x),'o-r')                     % Tracer points ronds reliés en rouge
plot(x,sin(x),'x-b')                     % Tracer points croix reliés en vert
plot(x,exp(-x),'*-g')                    % Tracer points étoiles reliés en bleu
legend('cos(x)','sin(x)','exp(-x)')      % Légende sur graphique
```

Il est à noter qu'à chaque fois hold on est activer, il faut la désactiver à la fin par hold off


cos(x)
sin(x)
exp(-x)
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
0
0.5
1
1.5
2
2.5
3
3.5

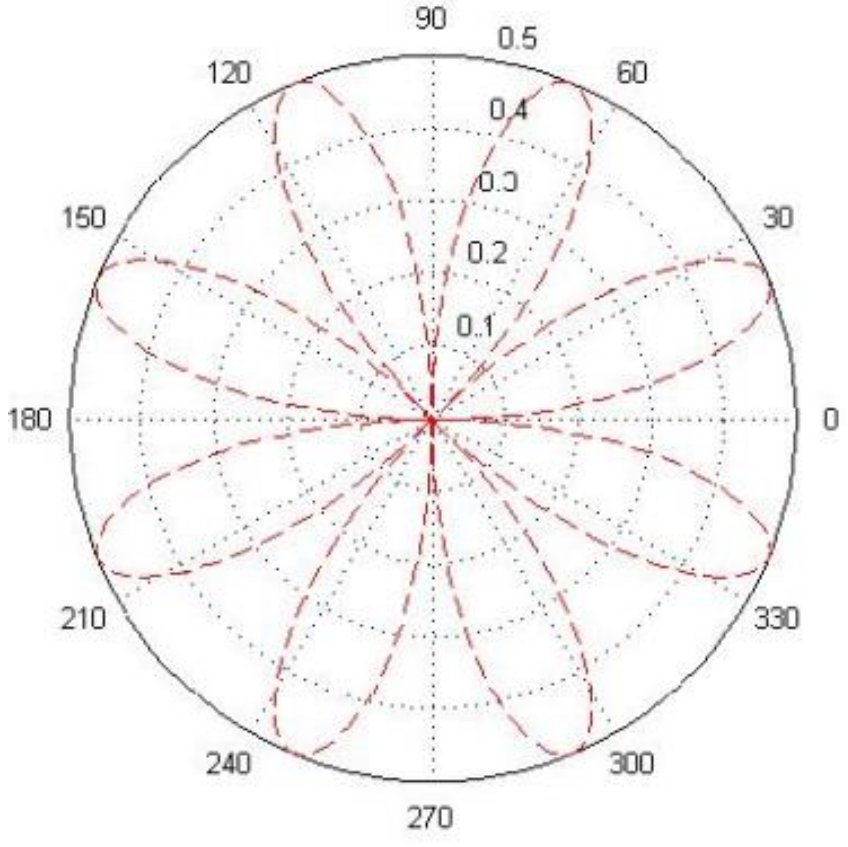
5. Graphique polaire

Il est aussi possible de tracer des graphiques polaires en Matlab.

```
% — Graphique polaire ———————————————————————————— %

figure;                              % Nouvelle fenêtre

t = 0 :.01 :2*pi;                    % Échelle du temps
polar(t,sin(2*t).*cos(2*t),'-r')     % Graphique polaire en pointillé rouge
```


90
0.5
120
60
0.4
0.3
150
30
0.2
0.1
180
0
210
330
240
300
270

6. Graphique à échelle logarithmique

Il est aussi possible de tracer des graphiques avec un ou plusieurs axes sous forme logarithmiques en Matlab.

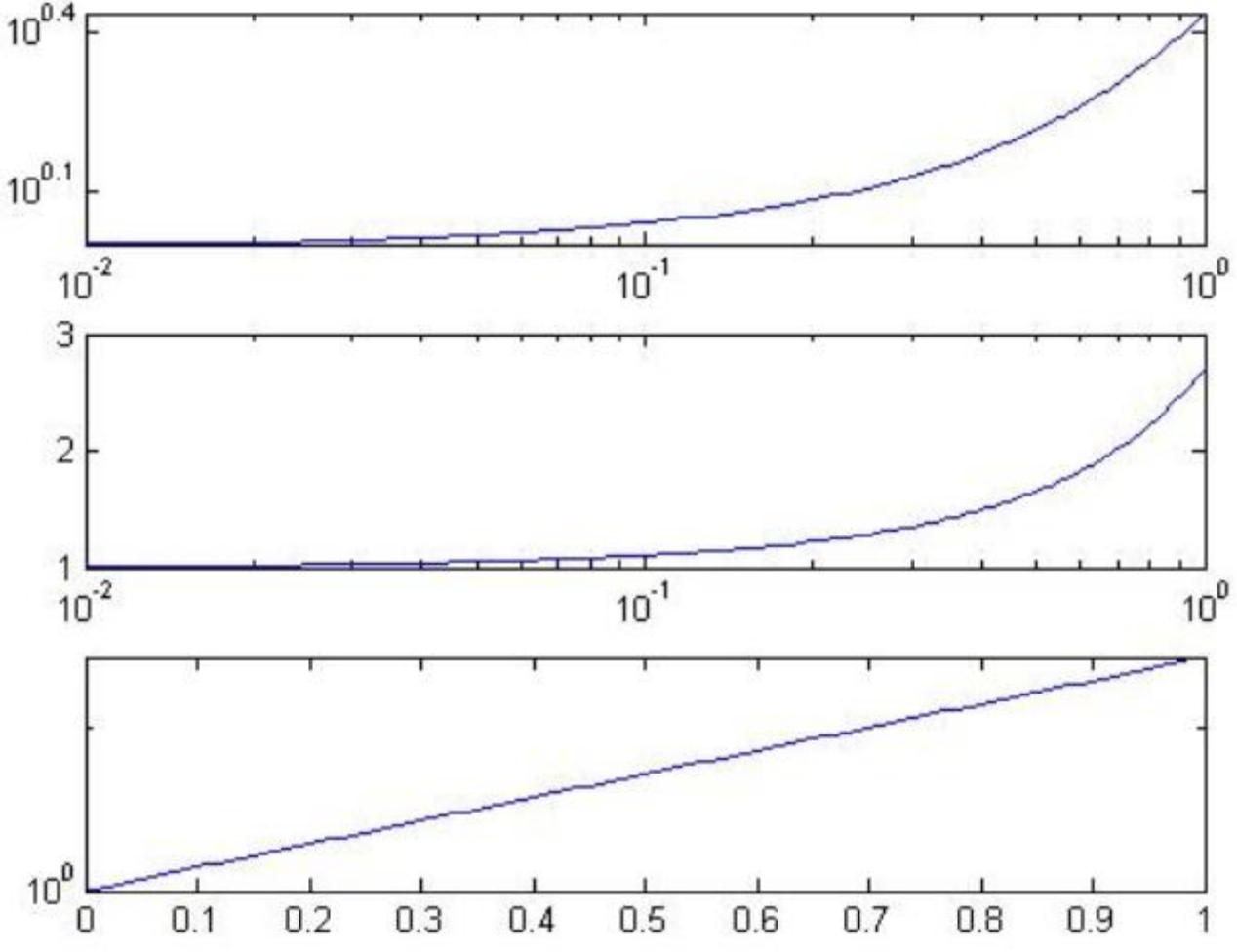
```
% — Graphique à échelles logarithmiques ———————————————————— %

figure;                 % Nouvelle fenêtre

x = 0 :0.01 :1;
subplot(3,1,1);         % Zone supérieure
loglog(x,exp(x));       % Echelle logarithmique deux axes

subplot(3,1,2)          % Zone centrale
semilogx(x,exp(x))      % Echelle logarithmique sur l'axe Ox

subplot(3,1,3)          % Zone inférieure
semilogy(x,exp(x))      % Echelle logarithmique sur l'axe Oy
```


$10^{0.4}$
$10^{0.1}$
10^{-2}
10^{-1}
10^{0}
3
2
1
10^{-2}
10^{-1}
10^{0}
10^{0}
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1

Il est aussi possible de tracer des graphiques à trois dimensions en Matlab. Voici quelques exemples de commandes que vous pouvez utiliser.

```
>> help plot3
 PLOT3  Plot lines and points in 3-D space.
    PLOT3() is a three-dimensional analogue of PLOT().

    PLOT3(x,y,z), where x, y and z are three vectors of the same
length,
    plots a line in 3-space through the points whose coordinates
are the
    elements of x, y and z.
```

```
>> help mesh
 MESH   3-D mesh surface.
    MESH(X,Y,Z,C) plots the colored parametric mesh defined by
    four matrix arguments.  The view point is specified by VIEW.
    The axis labels are determined by the range of X, Y and Z,
    or by the current setting of AXIS.  The color scaling is determined
    by the range of C, or by the current setting of CAXIS.  The scaled
    color values are used as indices into the current COLORMAP.


>> help surf
 SURF   3-D colored surface.
    SURF(X,Y,Z,C) plots the colored parametric surface defined by
    four matrix arguments.  The view point is specified by VIEW.
    The axis labels are determined by the range of X, Y and Z,
    or by the current setting of AXIS.  The color scaling is determined
    by the range of C, or by the current setting of CAXIS.  The scaled
    color values are used as indices into the current COLORMAP.
    The shading model is set by SHADING.
```

Merci

Pour

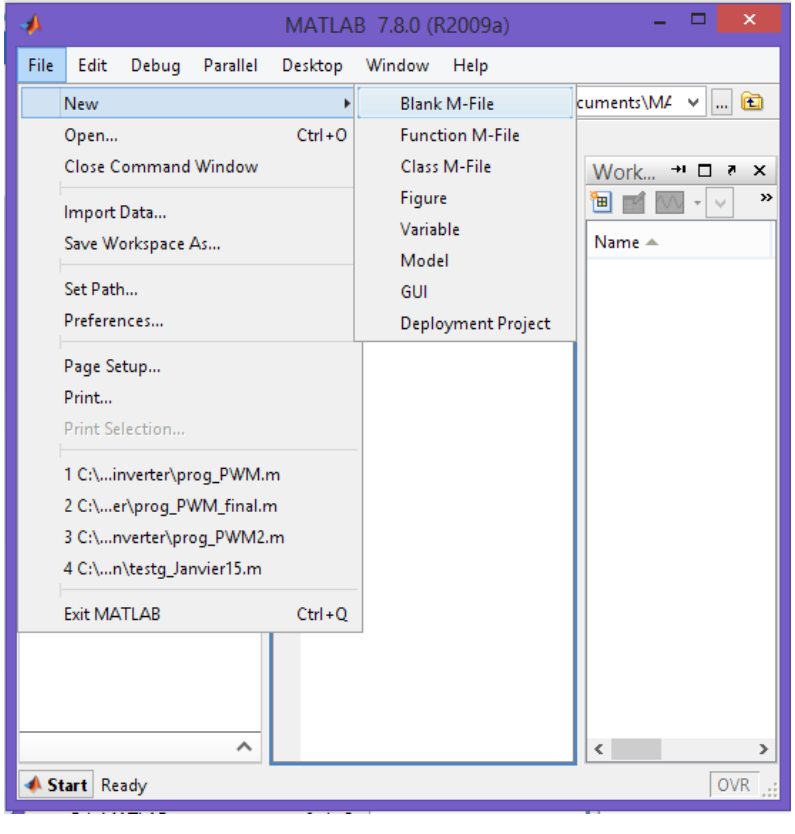
Votre Attention

Le logiciel de simulation MATLAB
Partie 3
Programmation en script

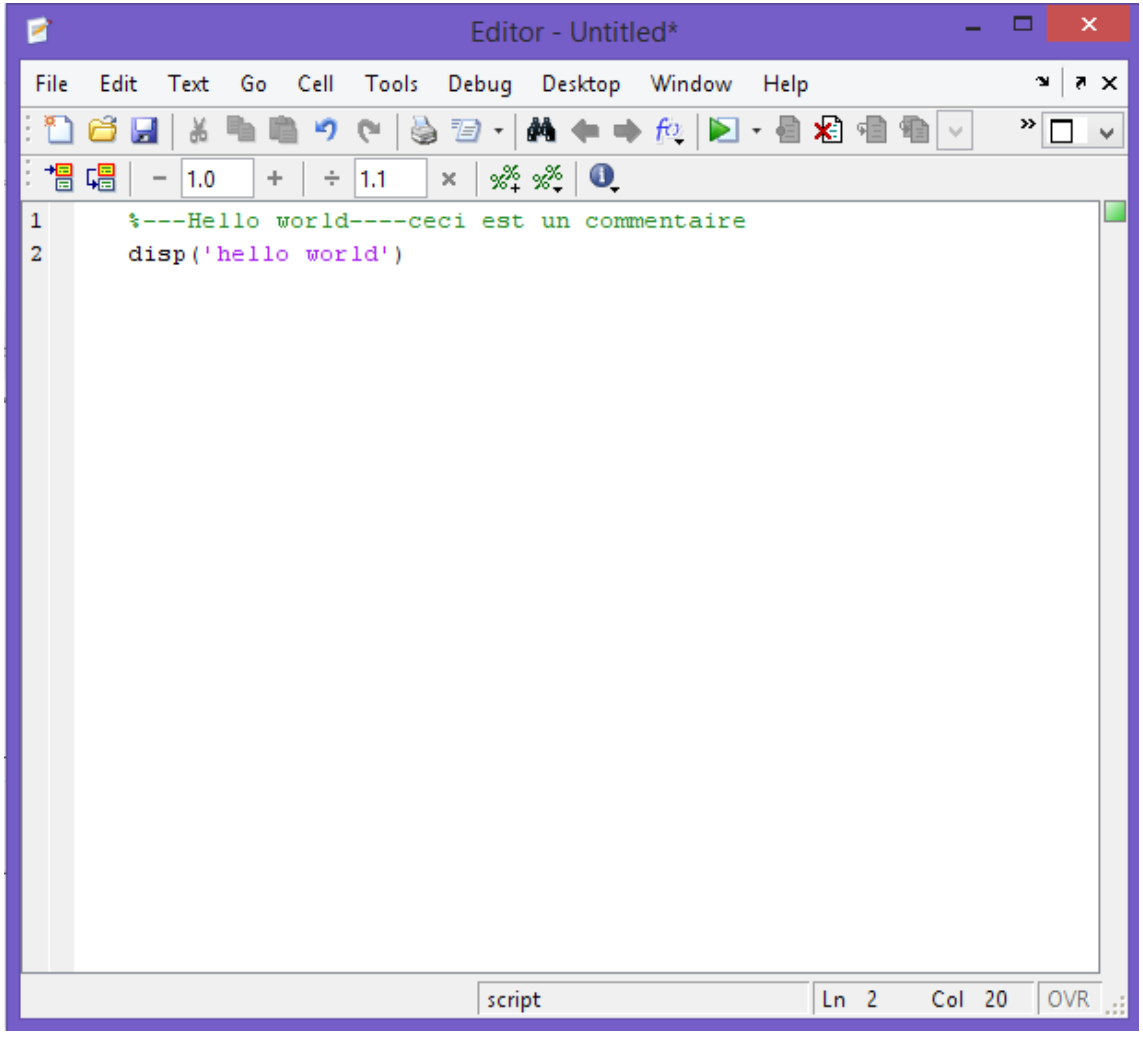
Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Les fichiers m (m-files)

Un m-file est un fichier contenant une suite d'instructions que Matlab peut exécuter.
Un m-file peut aussi être utiliser comme fichier de librairie contenant des fonctions définies par l'utilisateur.
Pour créer un m-file, utilise le menu File ⟶ new → M-file



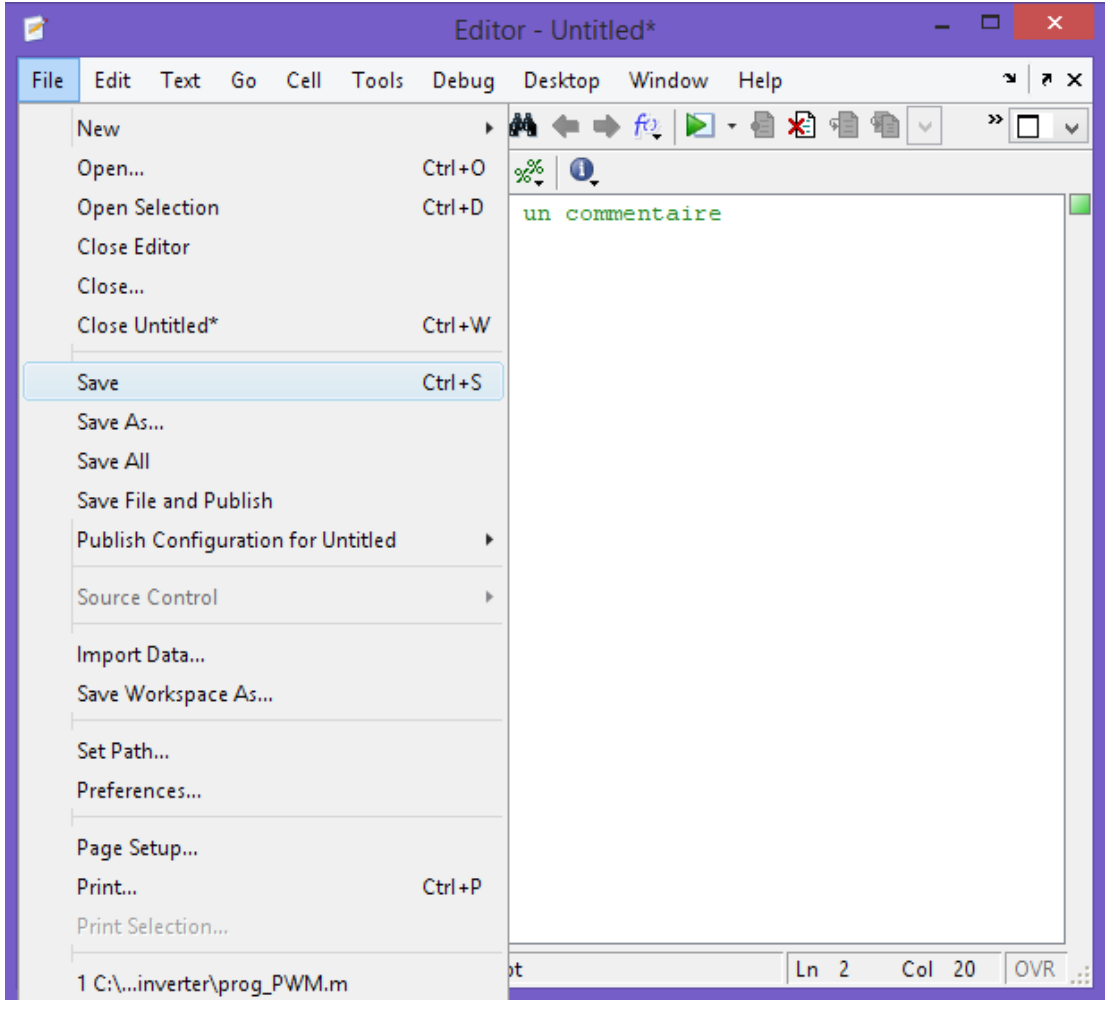
Une fois le M-file est crée, écrivez ceci



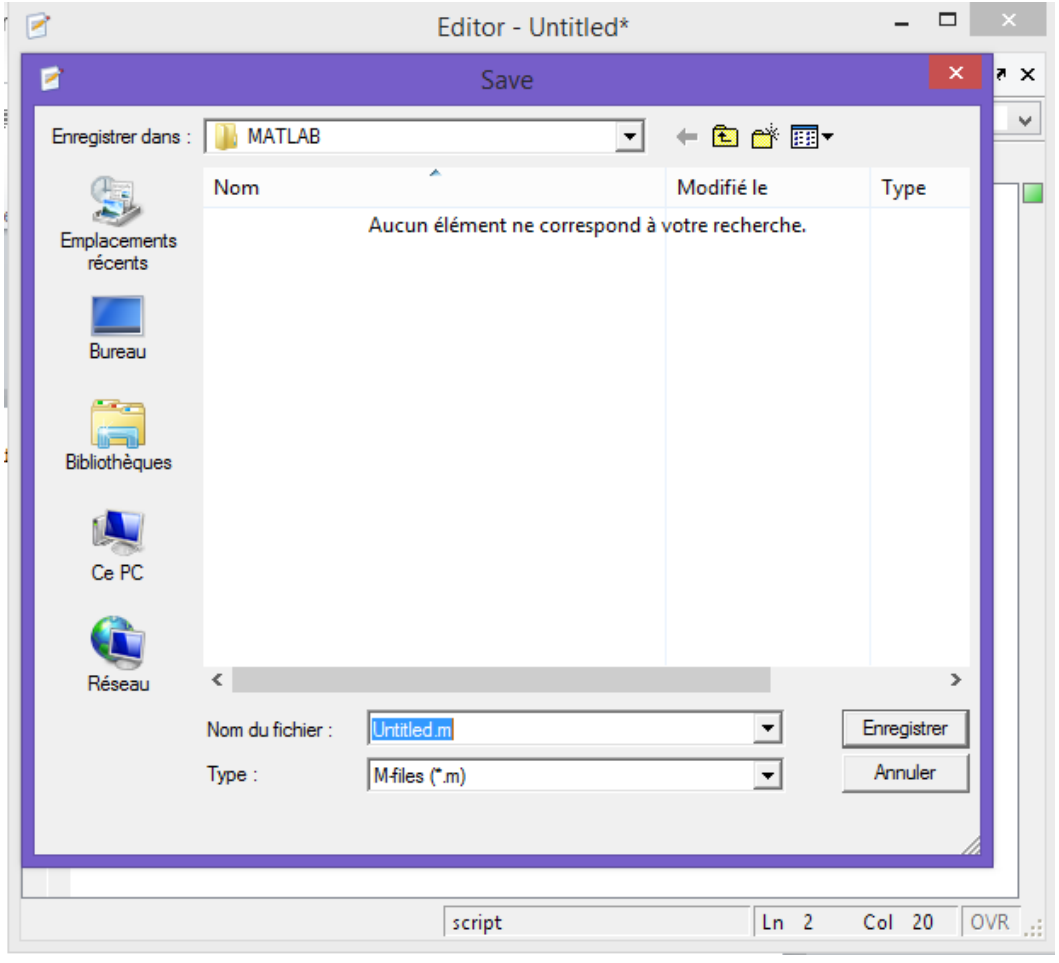
disp est une fonction qui affiche des messages ou des variables

Ce type de programme est appelé script

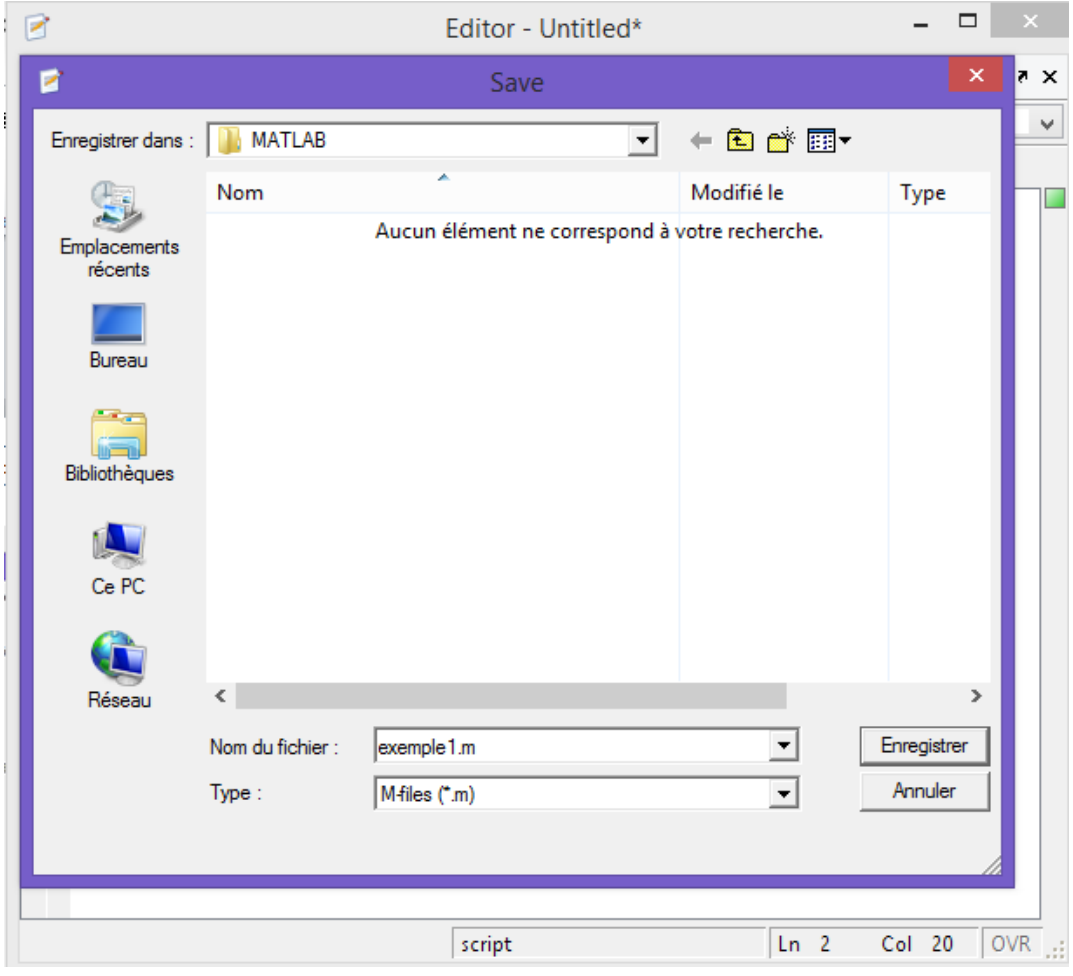
Maintenant le M-file va être sauvegarder en utilisant menu File →save



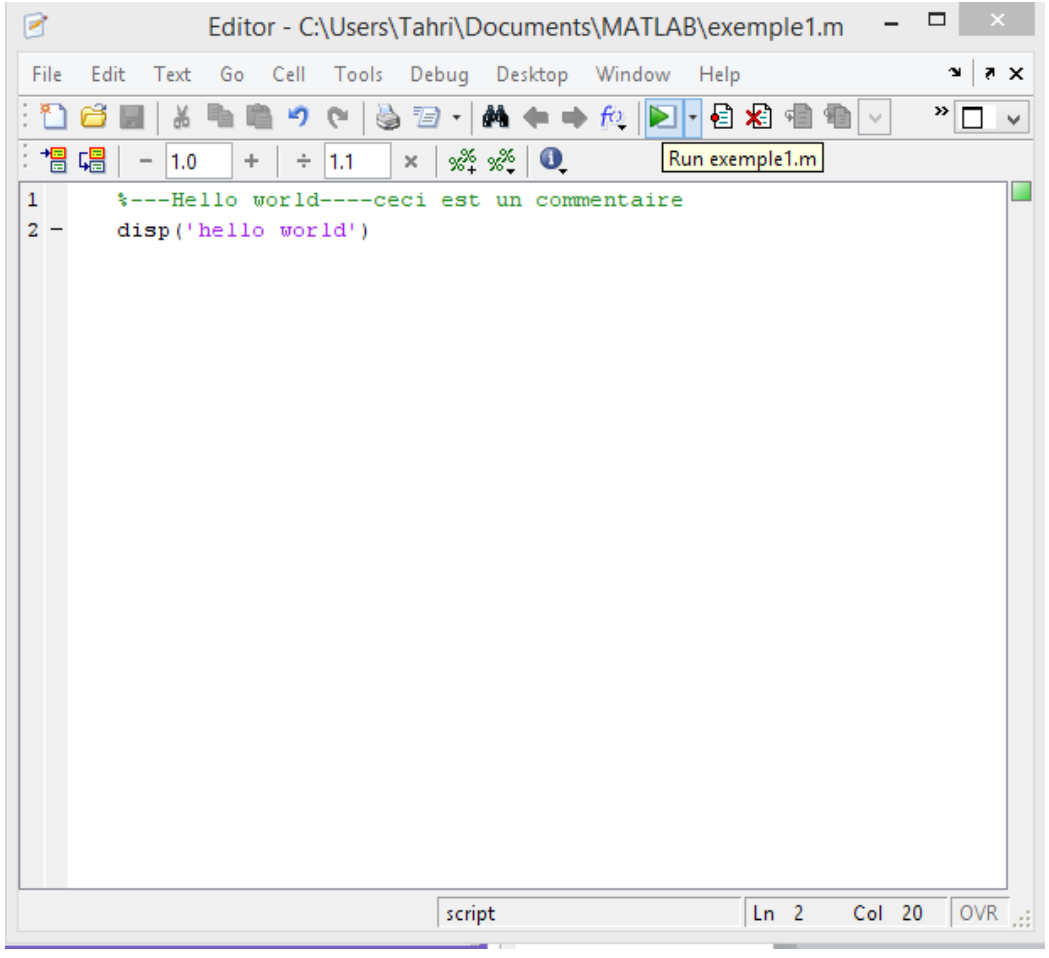
Il faut donner un nom à votre programme



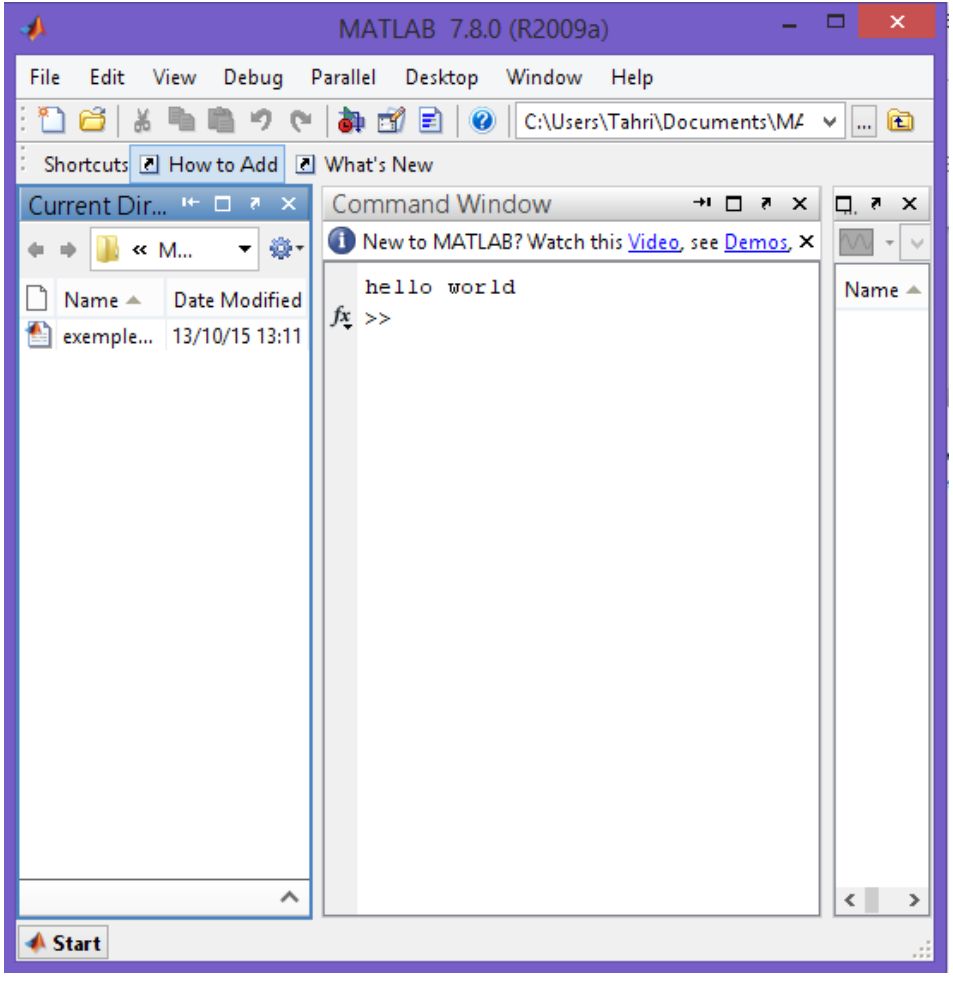
On va donner exemple1.m comme nom au programme.
Et vous cliquer sur enregistrer



Pour exécuter le programme, il suffit de cliquer sur le bouton vert (Run exemple1.m) comme sur la figure suivante.



Ainsi sur l'écran du workspace sera afficher le message



2. Les opérateurs logiques

```
% — Opérateurs logiques ———————————————————————————— %

<          ... est plus petit que ...
>          ... est plus grand que ...
<=         ... est plus petit ou égal à ...
>=         ... est plus grand ou égal à ...
==         ... est égal à ...
~=         ... n'est pas égal à ...
&          ... est vrai et ... aussi (pour tableaux)
|          ... est vrai ou ... est vrai, ou les deux (pour tableaux)
&&         ... est vrai et ... aussi
||         ... est vrai ou ... est vrai, ou les deux
~          ... n'est pas vrai
xor(x,y)   ... est vrai ou ... est vrai
any(x)     ... vrai si un des éléments de x est non nul
all(x)     ... vrai si tous les éléments de x sont nuls
```

3. Les mots gardés

Les mots ou caractères suivants ont une signification particulière dans le langage Matlab.

% — Mots/symboles gardés ——————————————————— %

:	Create vectors, subscript arrays, specify for-loop iterations
( )	Pass function arguments, prioritize operators
[ ]	Construct array, concatenate elements, specify multiple outputs from function
{ }	Construct cell array, index into cell array
.	Insert decimal point, define structure field, reference methods of object
.( )	Reference dynamic field of structure
..	Reference parent directory
...	Continue statement to next line
,	Separate rows of array, separate function input/output arguments, separate commands
;	Separate columns of array, suppress output from current command
%	Insert comment line into code
% %	Insert block of comments into code
!	Issue command to operating system
' '	Construct character array
@	Construct function handle, reference class directory

4. Entrées / Sorties

4.1 Utilisateurs

Les commandes suivantes permettent l'interaction avec l'utilisateur lors de l'exécution d'un script. L'utilisateur peut dans l'exemple ci-dessous saisir un nombre que le script pourra alors utiliser. Dans le sens inverse la commande disp permet d'afficher des variables à l'utilisateur.

```
% — Exemple 1 ————————————————————————————————————- %
n = input('Saisissez un nombre :');     Saisie de l'utilisateur
disp(n);                                Sortie vers l'affichage Matlab
```

4.2 Disques

Il est aussi possible d'exporter ou d'importer sur le disque dur via les deux commandes suivantes.

```
>> save('nomdufichier','variables')

>> save('nomdufichier','variable1','variable2')
```

Exporter vers le disque dur

Exporter du workspace vers le disque dur.

```
>> load('nomdufichier')

>> load('nomdufichier','variable1','variable2')
```

Importer du disque dur

Importer du disque dur vers le workspace.

5. Le contrôle de l'exécution

5.1 Boucle FOR

La boucle FOR permet d'effectuer des opérations pour un nombre d'itérations définis.

L'avantage de la boucle FOR sur la boucle WHILE est sa simplicité d'écriture dans le cas d'un nombre d'itérations définis et bien connu à l'avance (par exemple, le parcours d'un tableau).

Autre remarque, lors de l'exécution de la boucle FOR, la variable qui sert à boucler est accessible en lecture et en écriture . Il est donc possible de réduire ou d'augmenter le nombre d'itérations au cours de l'exécution de la boucle.

```
for i=1:n
  séquence d'instructions
  end
```

Avec pas positif égale à 1

```
for i=n:-0.2:1
  séquence d'instructions
  end
```

Avec pas négatif égale à -0.2

```
 for i=1:m
   for j=1:n
  séquence d'instructions
   end
   end
```

Deux boucle imbriquées

```
% — Exemple 1 ——————————————————————————————— %

for n = 1 :5                % Boucle pour n allant de 1 à 5 inclus par pas de 1
  disp(n)                   % Affichage
end                         % Fin de boucle

% — Exemple 2 ——————————————————————————————— %

for n = 8 :-2 :0            % Boucle pour n allant de 8 à 0 par pas de 2
  disp(n)                   % Affichage
end                         % Fin de boucle

% — Exemple 3 ——————————————————————————————— %

for n = [ 1 9 3 5 6 7 ]     % Boucle pour n égal chaque valeur du vecteur
  disp(n)                   % Affichage
end                         % Fin de boucle
```

5.2 Boucle WHILE

La boucle WHILE permet d'effectuer des opérations de manière répétée jusqu'à ce qu'une condition soit falsifiée (par exemple : tant que la solution n'est pas précise à 4 décimales, continuer à chercher une solutions plus précise.).

```
while expression logique

séquence d'instructions

end
```

```
% — Exemple 1 ———————————————— %

n = 5;
m = 8;
while (n < 10 && m > 0)     % Boucle tant que n est inférieur à 10
                            % et m est supérieur à 0
  n = n + 1;                % Incrémente n
  m = m - 1;                % Décrémente m
  disp(n);                  % Affichage
  disp(m);                  % Affichage
end                         % Fin de boucle
```

5.3 Instruction de choix IF

L'instruction IF est une instruction de choix. Autrement dit, en fonction que son gardien (expression logique) sera évalué vrai ou faux, la commande exécutera un groupe d'instruction ou l'autre.

```
if  expression logique
séquence d'instructions
end
```

```
% — Exemple 1 ———————————————— %

n = 5;
m = 8;
if (n > 0)                      % Gardien 1

  if ( n > 5 && m < 0)          % Gardien 1 et gardien 2
    disp ('ici 1')              % Affichage
  elseif (n == 5)               % Si gardien1 et non gardien2 et gardien 3
    disp ('ici 2')              % Affichage
  else                          % Si gardien1 et non gardien2 et non gardien 3
    disp('ici 3')               % Affichage
  end                           %

else                            % Si gardien 1 est faux
  disp('ici 4')                 % Affichage
end                             %
```

5.4 Instruction de choix SWITCH

L'instruction SWITCH est une instruction de choix comme le IF mais avec la particularité de pouvoir effectuer plus de branchements que le IF.

La commande SWITCH doit être utilisée dans le cas ou, par exemple, en fonction de la valeur d'une variable, on effectue différentes opérations. Attention toute fois, que le nombre de valeurs possibles de cette variable doit être restreint pour conserver une certaine lisibilité du code.

Le mot clé break signifie que l'on arrête la commande SWITCH et que l'on transfert l'exécution au mot clé end.

```
switch var

case cst1,

séquence d'instructions 1

case cst2,

séquence d'instructions 2

...

case cstN,

séquence d'instructions N

otherwise

séquence d'instructions par défaut

end
```

où

- var est une variable numérique ou une variable chaîne de caractères;
- cst_1, ..., cst_N, sont des constantes numérique ou des constantes chaîne de caractères;
- *séquence d'instructions i* est la séquence d'instructions à exécuter si le contenu de la variable var est égal à la constante cst_i ($var==cst_i$).

```
% — Exemple 1 ———————————————— %

n = input('Pour sauver jack tapez 1,        % Demande de saisie de l'utilisateur
Jessy tapez 2, Brian tapez 3 :');

switch n

  case 1                                    % Si n vaut 1
    disp('Jack est sauvé')
  break

  case 2
    disp('Jessy est sauvé')                 % Si n vaut 2
  break

  case 3
    disp('Brian est sauvé')                 % Si n vaut 3
  break

  otherwise
    disp('Vous n"avez sauvé personne')      % Si n est différent de 1,2,3

end
```

Merci

Pour

Votre Attention

Le logiciel de simulation MATLAB
Partie 4
Exemples de Programmes en script

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Lecture des éléments d'un tableau

```
% ce programme les éléments d'un tableau
T=input('saisissez le tableau :'); %Entrée par clavier
m=length(T);                       %length qui retourne la taille d'un tableau
for i= 1:m
    disp (T(i));                   % affichage des sorties sur l'ecran
end;
```

saisissez le tableau :[1 2 3 4 5]

1

2

3

4

5

2. Ordonner les éléments d'un tableau

```
% ce programme va ordonner les éléments d'un tableau
T=input('saisissez le tableau :'); %Entrée par clavier
m=length(T);                       %length qui retourne la taille d'un tableau
for i= 1:m-1
    if T(i)>T(i+1);
        A=T(i);
        T(i)=T(i+1);
        T(i+1)=A;
    end
end;
disp('Tableau ordonné')            % affichage des sorties sur l'ecran
disp(T)
```

saisissez le tableau :[1 4 3 6 5]
Tableau ordonné
1 3 4 5 6

Ce programme n'ordonne pas un tableau, si on va saisir un autre exemple de tableau, on aura un résultats faux

saisissez le tableau :[2 3 4 1 0 2]
Tableau ordonné
2 3 1 0 2 4

Pour palier ce problème, on a développé un deuxième programme pour lequel chaque élément du tableau est vérifié par rapport à tout les éléments

```
T=input('saisissez le tableau');
m=length(T);
for i=1:m-1
    for j=i+1:m
        if T(i)>T(j);
            A=T(i);
            T(i)=T(j);
            T(j)=A;
        end
    end
end;
disp('Tableau ordonné')
disp(T)
```

Deux boucles imbriquées, une pour i et une pour j

```
saisissez le tableau :[2 3 4 1 0 2]
Tableau ordonné
   0    1    2    2    3    4
```

3. Le plus grand diviseur commun: PGCD

Exemple avec la méthode des soustractions

Calculons le PGCD des nombres 96 et 36

	a=96; b=36
96-36=60	**a= a-b nouveau a=60**
60-36=24	**a= a-b nouveau a=24**
36-24=12	**a<b donc b=b-a nouveau b=12**
24-12=12	**a>b donc a=a-b nouveau a=12**
12-12=0	**maintenant a=b donc PGCD=a**

PGCD(96;36)=12

```
% calcul le PGCD de 2 nombres entiers a et b
a=input('saisissez le premier nombre');
b=input('saisissez le deuxième nombre');

while (a~=b)
    if (a>b)
        a=a-b;
    else b=b-a;
    end
end;

% affichage
disp(a);
```

saisissez le premier nombre96
saisissez le deuxième nombre36
12

4. Le factoriel d'un entier positif

```
n =input('saisissez un nombre entier positif');
% use iterations
f = 1;
if n==0
    f=1;
else
for i = 1:n
    f = f*i;
end
end;
disp(f)
```

saisissez un nombre entier positif4
  24

En Matlab, il y'a une fonction qui peut remplacer ce programme :
factorial

```
>> factorial(4)
ans =
   24
```

Par récurrence nous pouvons dire que **n! = (n-1)! * n**.
Si je manipule cette dernière expression regardez ce qui se passe :
n! = (n-1)! * n
donc n!/n = (n-1)!
d'où si n = 1 nous obtenons :
1!/1 = (1-1)!
1 = 0!
D'où le factoriel de zéro est 1 : 0! = 1.

En Matlab, il y’a une fonction qui peut aider à éviter les itérations :
prod

```
n =input('saisissez un nombre entier positif');
f = 1;
if n==0
    f=1;
else
f=prod([1 : n]);
end;
disp(f)
```

5. Programme qui vérifie si un entier est pair ou impair

```
n =input('saisissez un nombre entier ');
if rem(n,2)==0
    disp('le nombre est pair');
else
    disp('le nombre est impair');
end;
```

En Matlab, la fonction rem retourne le reste de la division

```
>> rem(5,2)

ans =

    1
```

6. Programme qui converti un nombre décimal en son équivalent binaire

```
d=input('saisir le nombre');
q=floor(d/2);
r=rem(d,2);
x=[];
x=[r,x];
while q~0;
      r=rem(q,2);
      q=floor(q/2);
      x=[r,x];
end
  disp(x);
```

Exemple :
d=5
q=partie entière (5/2)=2;
r=reste de (d/2)=1;
x=[]; vecteur vide
x=[r,x] = 1; concatenation à gauche
Est ce que q~0; cette condition est vrai
r=reste de (2/2)=0;
q=partie entière (2/2)=1;
x=[r,x]=0 1;
Est ce que q~0; cette condition est vrai
r=reste de (1/2)=1;
q=partie entière (1/2)=0;
x=[r,x]=1 0 1;
Est ce que q~0; cette condition est fausse
Arreter la boucle
Afficher x= 1 0 1

Merci

pour

votre Attention

Le logiciel de simulation MATLAB
Partie 5
Programmation en function

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Les fonctions

1.1 Les fonctions simples

Matlab permet de définir des fonctions. Une fonction est un ensemble d'instructions regroupées de manière à ne pas devoir les répéter régulièrement.

Une fonction peut prendre des arguments et renvoyer des valeurs. (Par exemple, si l'on fournit à une fonction une date de naissance, cette fonction peut renvoyer l'âge de la personne).

La définition d'une fonction se fait de la manière suivante :

function *arguments de sortie* = nomDeMaFonction (*arguments dentrée*).

Attention, pour que Matlab reconnaisse une fonction et sache l'utiliser à partir de l'espace de travail, il est obligatoire que le nom du M-file soit identique au nom donné à la fonction

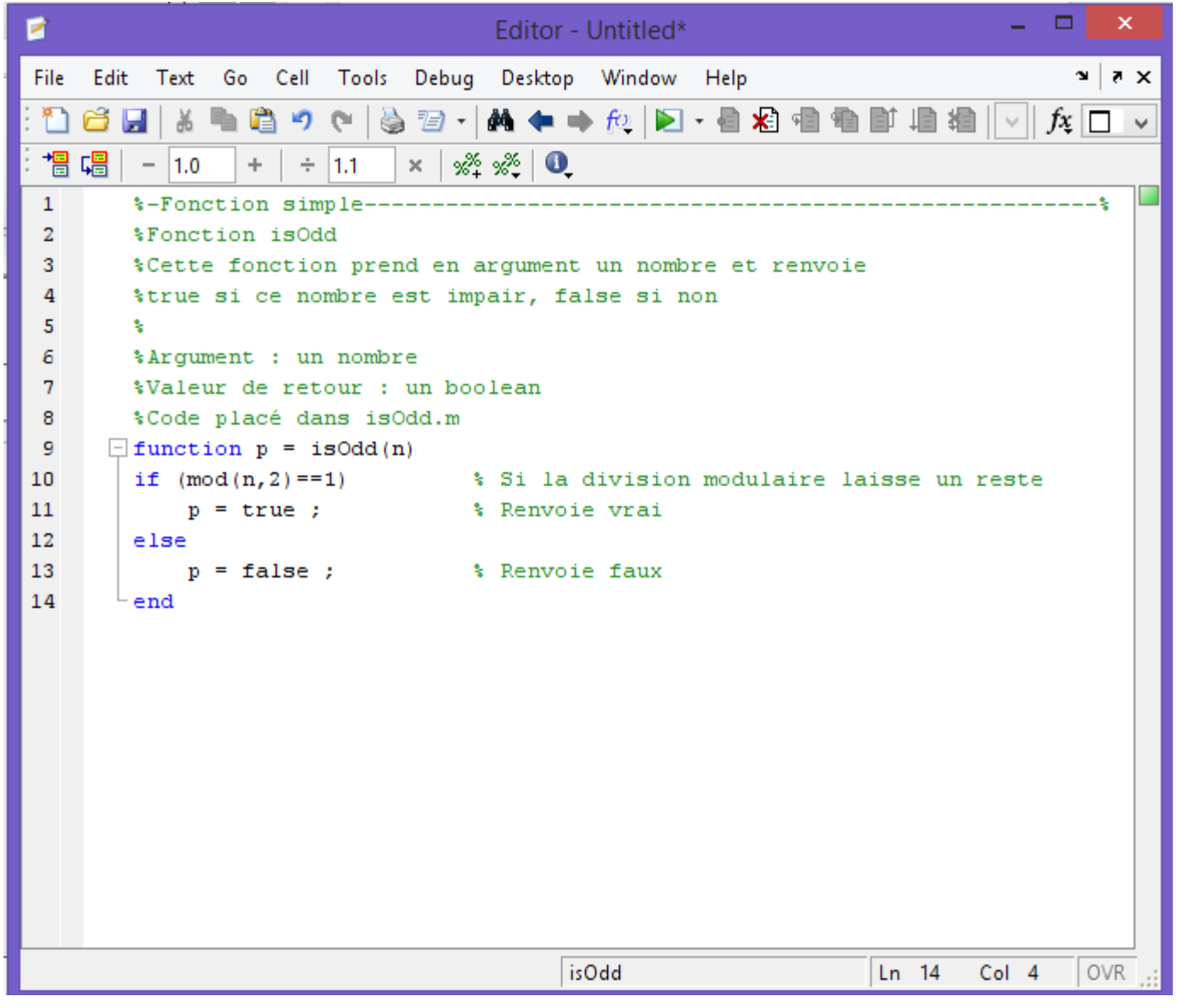
```
% — Fonction simple ——————————————————————————- %

% Fonction isOdd
% Cette fonction prend en argument un nombre et renvoie
% true si ce nombre est impair, false si non.
%
% Argument : un nombre
% Valeur de retour : un boolean
% Code placé dans isOdd.m

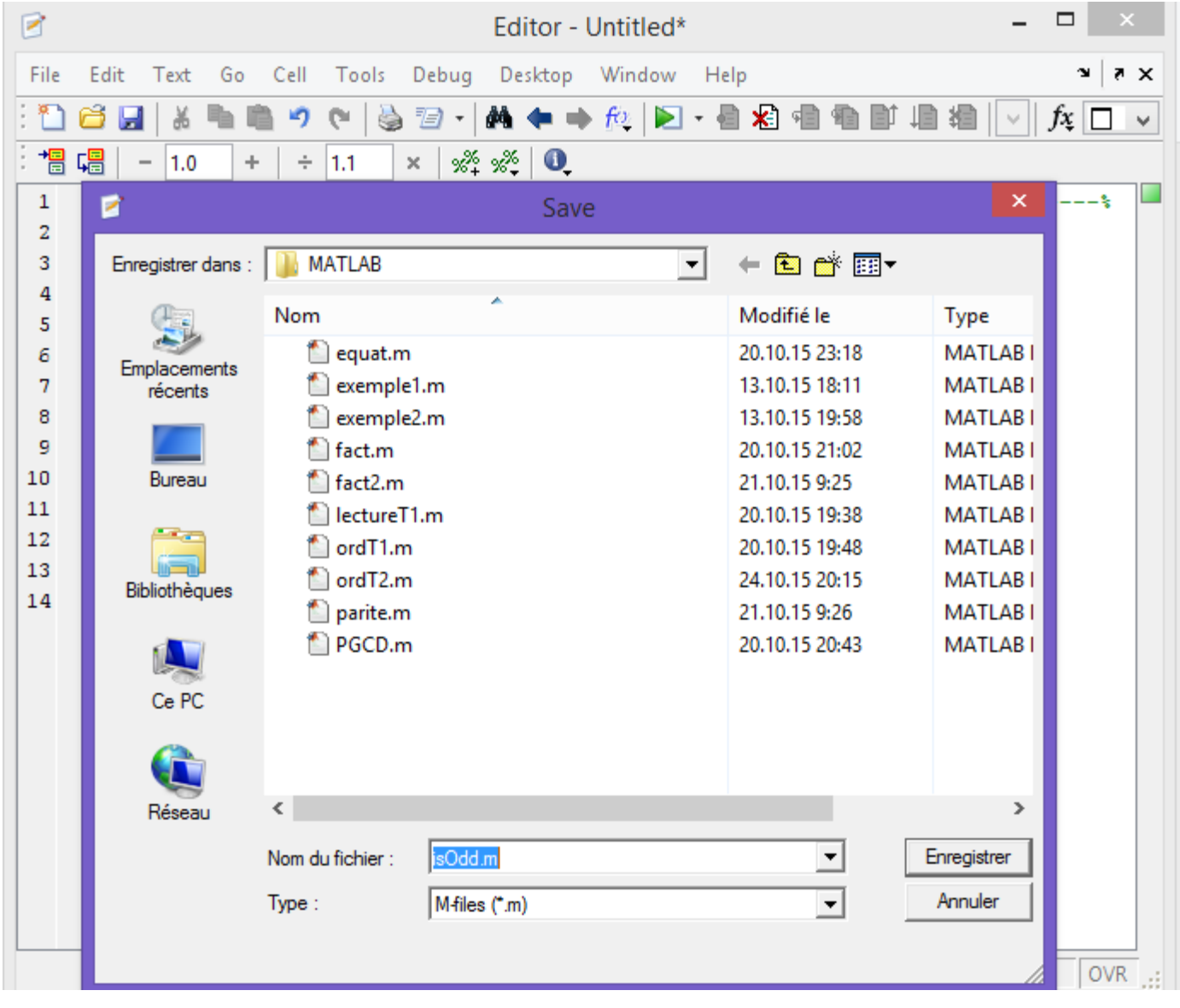
function p = isOdd(n)

if (mod(n,2) == 1)        % Si la division modulaire laisse un reste
  p = true;               % Renvoie vrai
else
  p = false;              % renvoie faux
end
```

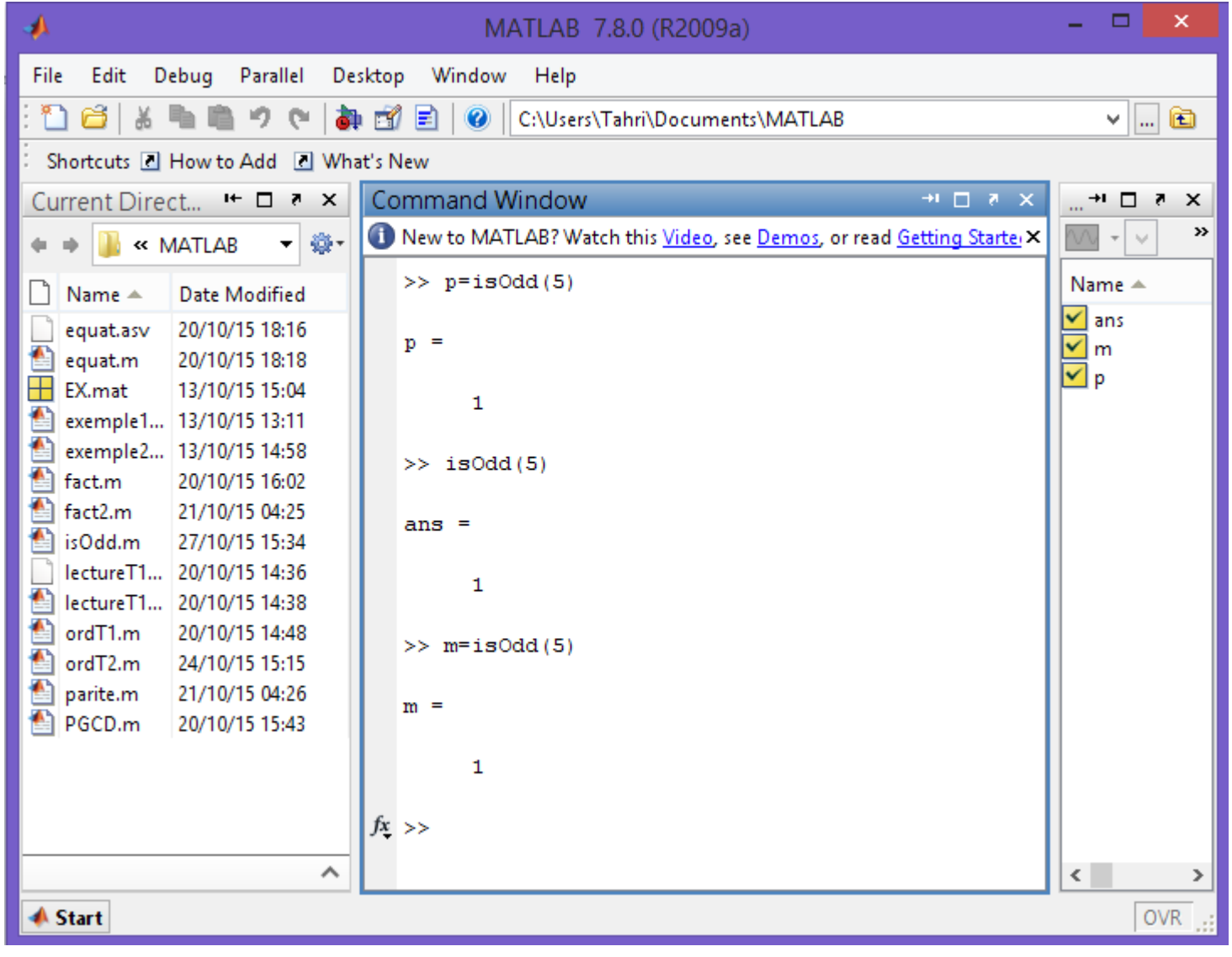
Cette fonction est écrite dans l'éditeur des M-files



Remarquez, le Matlab automatiquement donne au fichier m (M-file) le même nom que celui de la fonction.



L'appel de la fonction peut se faire dans le workspace.



1.2 Les fonctions récursives

Une fonction est dite récursive si elle s'appelle elle-même. La fonction suivante est récursive et permet de calculer la factoriel d'un nombre.

```
% — Fonction récursive ———————————————————————————— %

% Fonction fact
% Cette fonction prend en argument un entier positif
% et renvoie la factoriel correspondante.
%
% Argument : un entier positif
% Valeur de retour : un entier positif
%

function y = fact(n)

if (n <= 1)              % Si l'on est dans le cas de base
  y = 1;
else
  y = n * fact(n-1);     % Si non, appel récursif
end
```

```
>> f=fact(4)

f =

    24
```

1.3 Les fonctions primaires et sous fonctions

Un fichier de fonction peut contenir une fonction primaire qui apparait en premier et n'importe quel nombre d'autres fonctions optionnelles qu'on appellera des sous fonctions qui viennent après la fonction primaire et qui sont utilisées par cette dernière.

Les fonctions primaires peuvent être appeler de l'extérieur du fichier où elles sont définis (comme du workspace), mais les sous fonctions ne peuvent pas être appeler de l'extérieur du fichier de la fonction.
Les sous fonctions ne sont visibles qu'à l'intérieur du fichier de la fonction primaire.

```
function [x1,x2] = quadratic(a,b,c)
%this function returns the roots of
% a quadratic equation.
% It takes 3 input arguments
% which are the co-efficients of x2, x and the
%constant term
% It returns the roots
d = disc(a,b,c);
x1 = (-b + d) / (2*a);
x2 = (-b - d) / (2*a);
end % end of quadratic

function dis = disc(a,b,c)
%function calculates the discriminant
dis = sqrt(b^2 - 4*a*c);
end % end of sub-function
```

1.4 Les fonctions imbriquées

Une fonction imbriquée (nested function en anglais) est une fonction contenue à l'intérieur d'une autre (mère). Elle offre la possibilité de manipuler les variables déclarées dans la fonction mère, et d'appeler les mêmes fonctions qu'elle. On peut avoir autant de niveaux d'imbrication que l'on souhaite.

•Il faut veiller à rajouter le mot-clé **end** marquant la fin de toutes les fonctions contenues dans le fichier m.

•La définition d'une fonction imbriquée ne peut pas figurer au sein d'une instruction de contrôle. Ceci inclut les blocs **if/elseif/else**, **switch/case**, **for**,**while**, et **try/catch**.

•Une fonction imbriquée doit être appelée directement par son nom (pas d'utilisation de **feval**) ou par son handle obtenu grâce à l'**opérateur @** (pas d'utilisation de **str2func**).

•Toutes les variables d'une fonction imbriquée ainsi que celles dans laquelle elle est imbriquée doivent être définies explicitement. Aucun script ou fonction (ex. **load** appelée sans argument de sortie ou **assignin**) créant des variables qui n'existent pas déjà dans le workspace des fonctions ne peut être appelé.

Une fonction imbriquée peut être appelée depuis

• la fonction du niveau directement supérieur dans laquelle elle se trouve ;
• une autre fonction imbriquée du même niveau ;
• n'importe quelle fonction imbriquée à l'intérieur de celle-ci.

```
function [x1,x2] = quadratic2(a,b,c)
function disc  % nested function
d = sqrt(b^2 - 4*a*c);
end % end of function disc
disc;
x1 = (-b + d) / (2*a);
x2 = (-b - d) / (2*a);
end % end of function quadratic2
```

1.5 Les fonctions privées (private functions)

Une fonction privée est une fonction primaire qui est visible seulement à un groupe limité d'autres fonctions. Vous pouvez créer des fonctions privées dans un sous dossier avec un nom spécial **private,** ainsi ces fonctions ne sont visibles que par le fichier parent.

Exemple
Ecrivant une fonction *quadratic*. A ce moment la fonction *disc* qui calcule le déterminant, va être une fonction privée (private function).
Créant un sous dossier nommé private dans le dossier ou répertoire de travail. Sauvegardant la fonction suivante dans le fichier *disc.m*

```
function dis = disc(a,b,c)
%function calculates the discriminant
dis = sqrt(b^2 - 4*a*c);
end % end of sub-function
```

Creant maintenant la fonction quadratic3.m dans le répertoire de travail

```
function [x1,x2] = quadratic3(a,b,c)
%this function returns the roots of
% a quadratic equation.
% It takes 3 input arguments
% which are the co-efficient of x2, x and the
%constant term
% It returns the roots
d = disc(a,b,c);
x1 = (-b + d) / (2*a);
x2 = (-b - d) / (2*a);
end % end of quadratic3
```

Merci

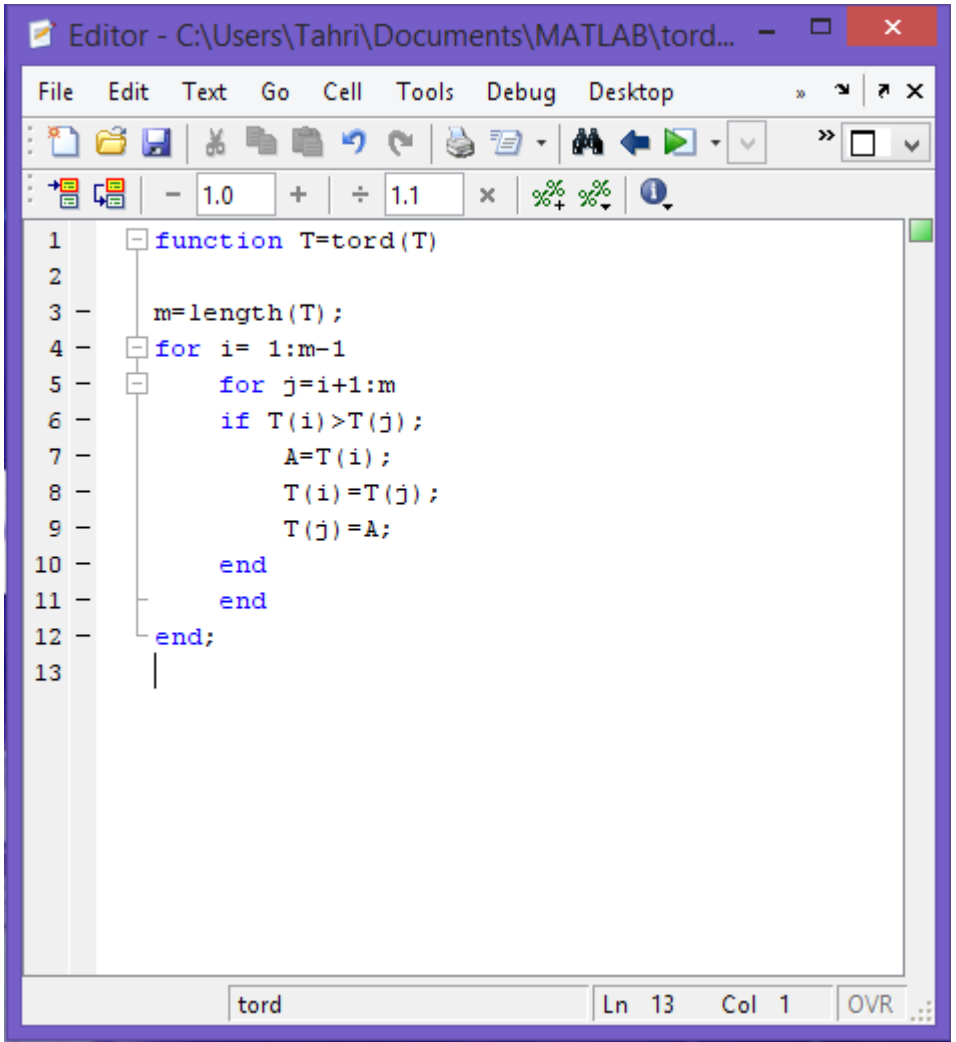
pour

votre Attention

Le logiciel de simulation MATLAB
Partie 6
Exemples de Programmes en function

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Ordonner les éléments d'un tableau



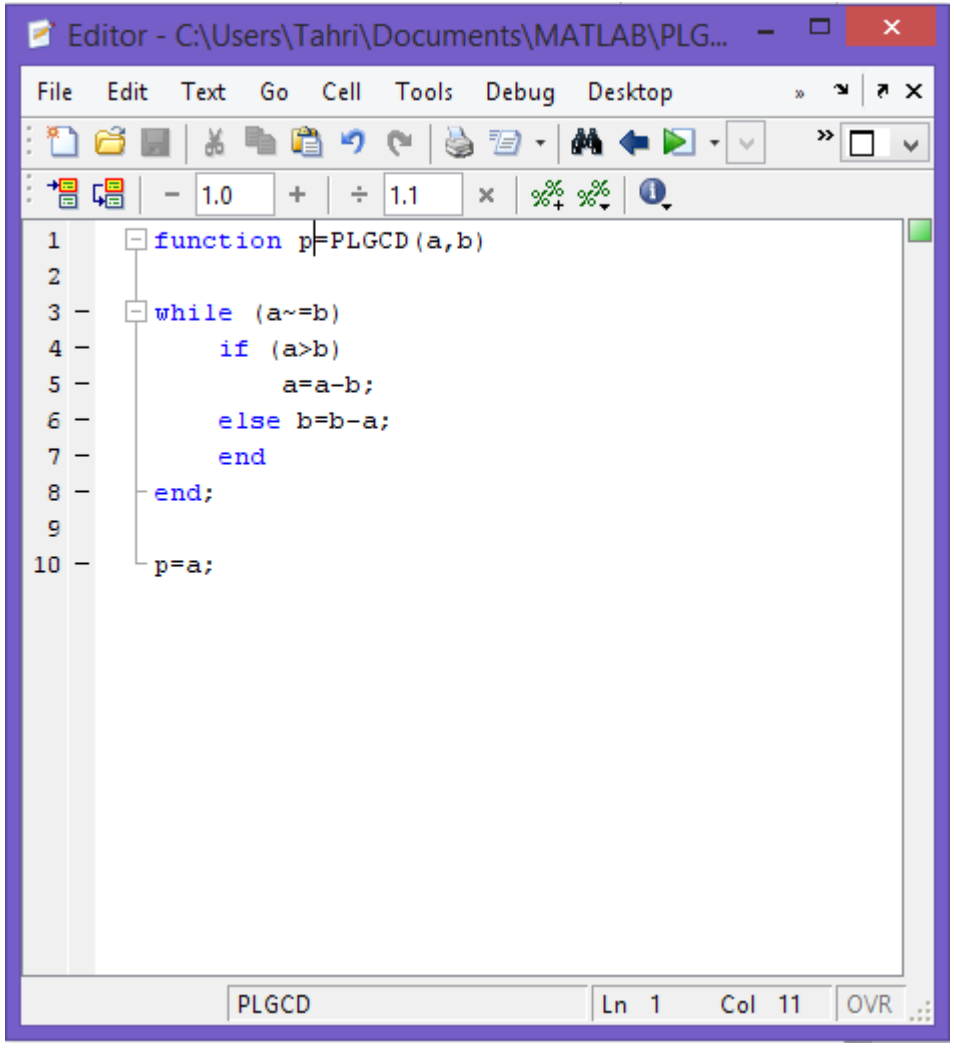
Remarquer les parenthèse de la fonction pour faire passer le tableau : [1 2 0 2 8 4]

```
>> T=tord([1 2 0 2 8 4])

T =

     0     1     2     2     4     8
```

2. Le plus grand diviseur commun: PGCD



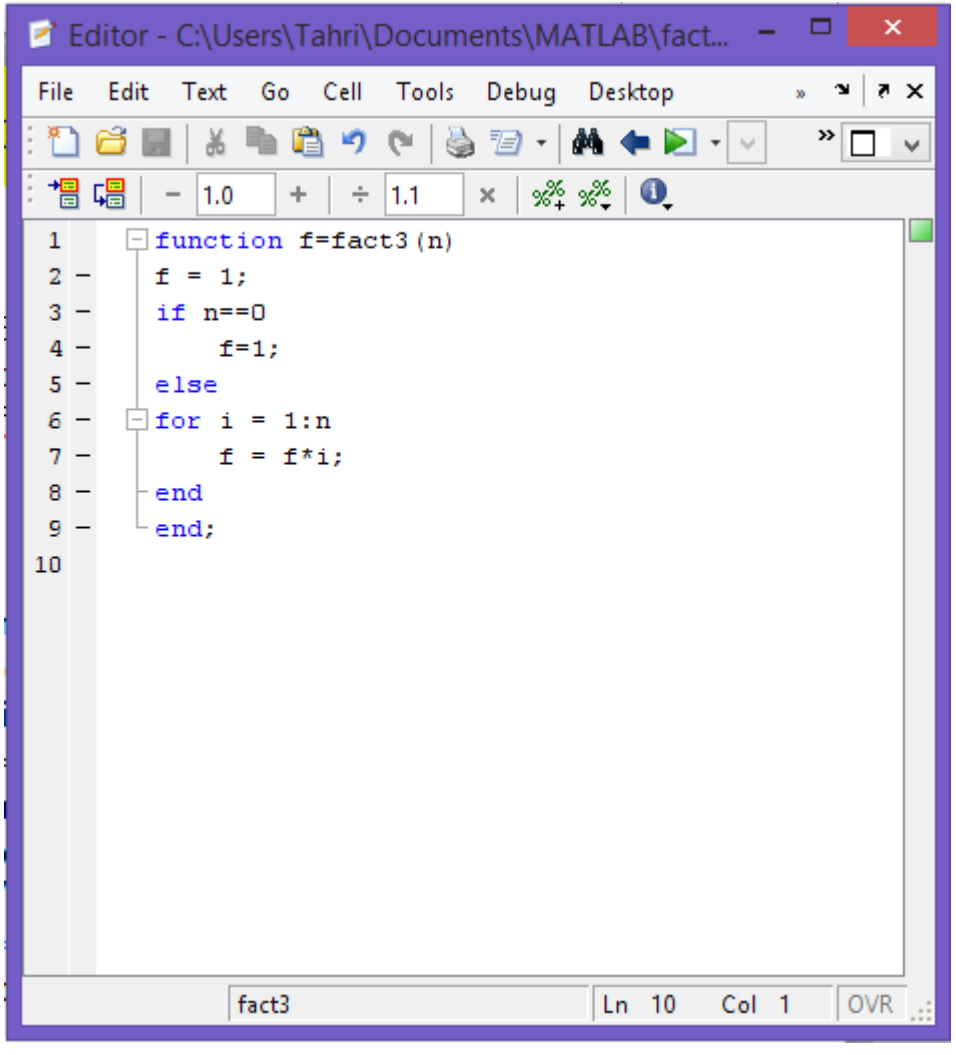
Remarquer les parenthèse de la fonction pour faire passer deux paramètres séparés par une virgule.

```
>> p=PLGCD(96,36)

p =

    12
```

3. Le factoriel d'un entier positif




```
>> f=fact3(4)

f =

    24
```

4. Conversion binaire d'un nombre décimal

```
function b=dec2bin(d);
q=floor(d/2);
r=rem(d,2);
x=[];
x=[r,x];
while q~0;
      r=rem(q,2);
      q=floor(q/2);
      x=[r,x];
end
   b=x;
```

```
>> b=dec2bin(11)
b =
   1   0   1   1
```

Merci

pour

votre Attention

Introduction au calcul formel
Symbolic Toolbox

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran
Mohamed Boudiaf

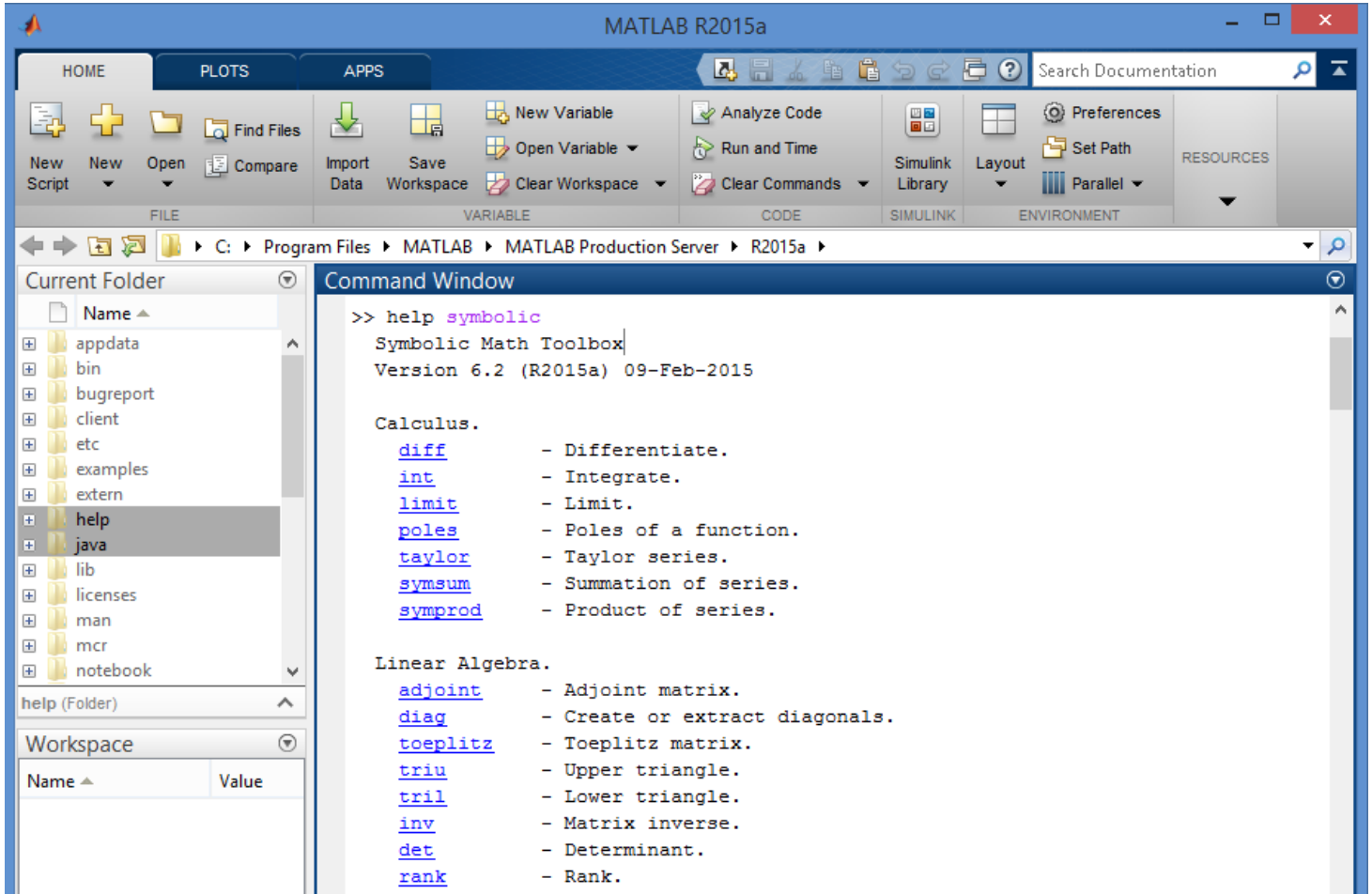
1. C’est quoi le calcul formel

Le calcul formel, ou parfois calcul symbolique, est le domaine des mathématiques et de l’informatique qui s’intéresse aux algorithmes opérant sur des objets de nature mathématique par le biais de représentations finies et exactes.

Le calcul formel est en général considéré comme un domaine distinct du calcul scientifique, cette dernière appellation faisant référence au calcul numérique approché à l'aide de nombres en virgule flottante, là où le calcul formel met l'accent sur les calculs exacts sur des expressions pouvant contenir des variables ou des nombres en précision arbitraire. Comme exemples d'opérations de calcul formel, on peut citer le calcul de dérivées ou de primitives, la simplification d'expressions, la décomposition en facteurs irréductibles de polynômes, la mise sous formes normales de matrices, ou encore la résolution des systèmes polynomiaux.

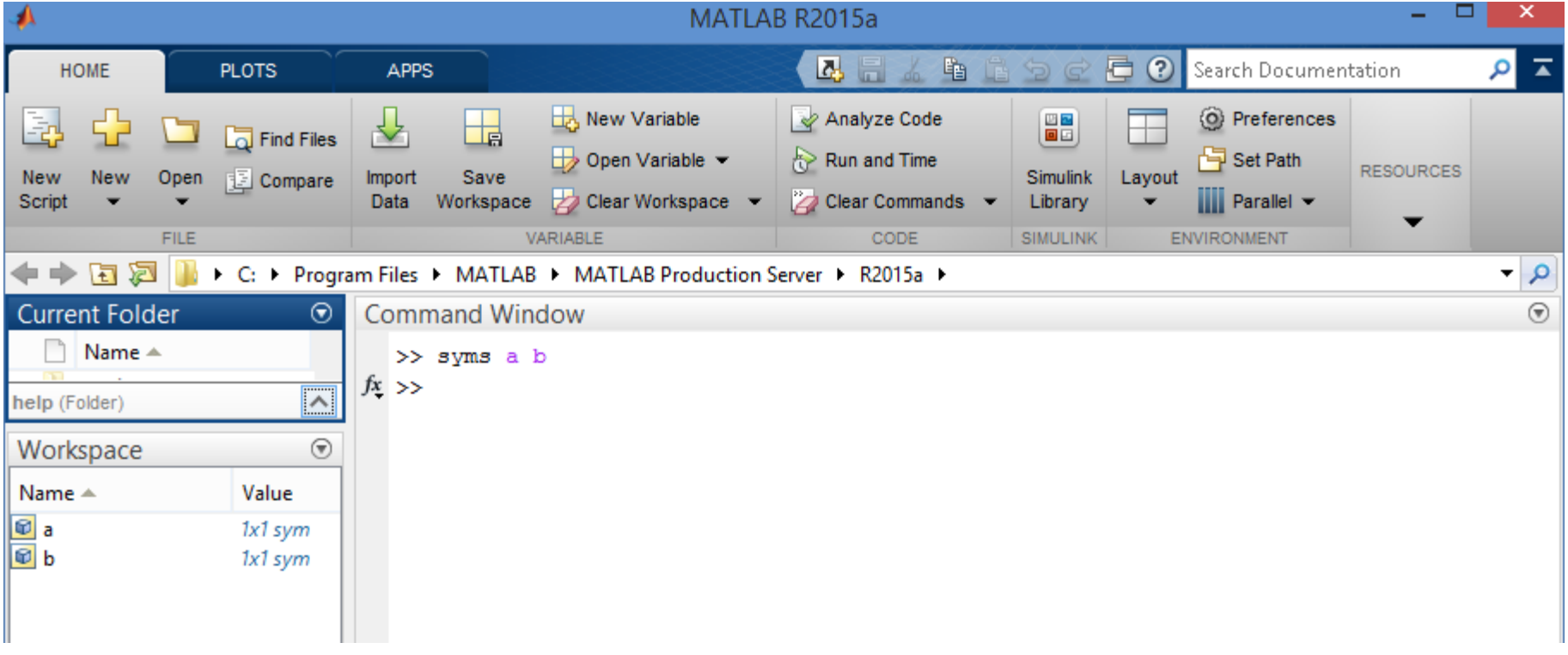
2. Symbolic Tolobox de Matlab

Symbolic toolbox de Matlab



Symbolic toolbox dispose de plusieurs fonctions prédéfinies qui permettent de faire un calcul formel aisé et facile.

Comment déclarer des variables symboliques.



```
>> sqrt(2)

ans =

    1.4142

sqrt( sym(2) )

ans =

2^(1/2)
```

```
>> 2/5

ans =

    0.4000

>> 2/5+1/3

ans =

    0.7333

>> sym(2) / sym(5)

ans =

2/5

>> sym(2) / sym(5) + sym(1) / sym(3)

ans =

11/15
```

3. Définir une expression symboliques

```
>> syms a b c x % define symbolic math variables
>> f = sym('a*x^2 + b*x + c')

f =

a*x^2 + b*x + c
```

```
>> f = sym('a*x^2 + b*x + c');
>> subs(f,x,5)

ans =

25*a + 5*b + c

>> subs(f,[x a b c],[5 1 2 3])


ans =

38
```

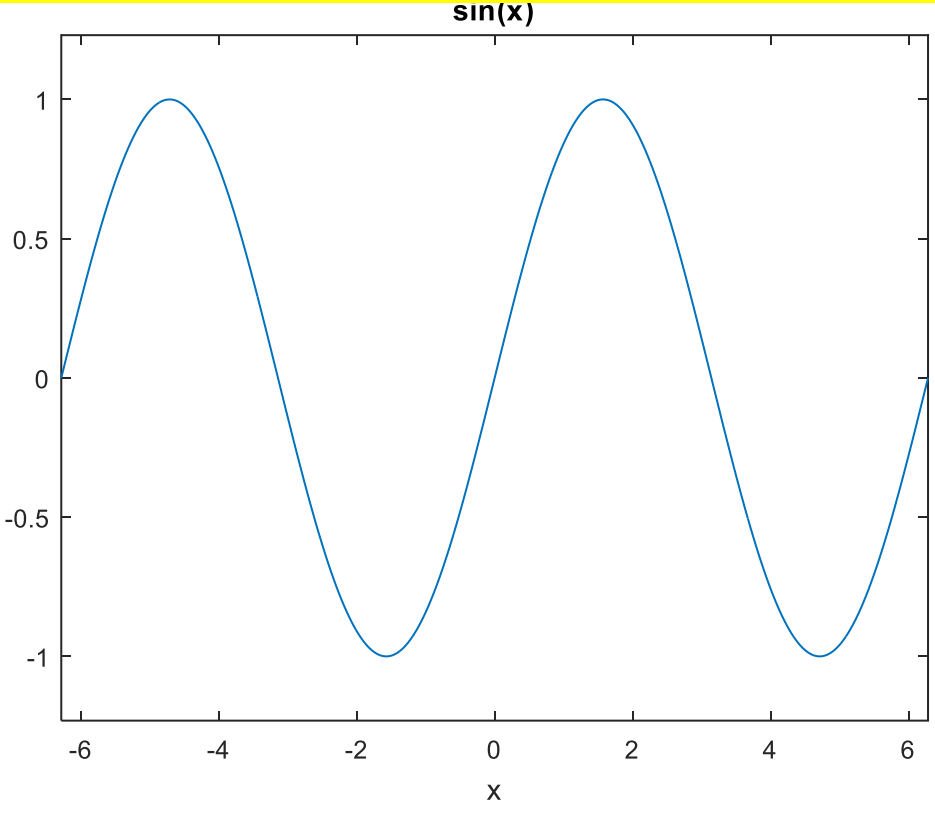
4. Tracer une fonction symbolique

```
>> syms x
>> y = sin(x)

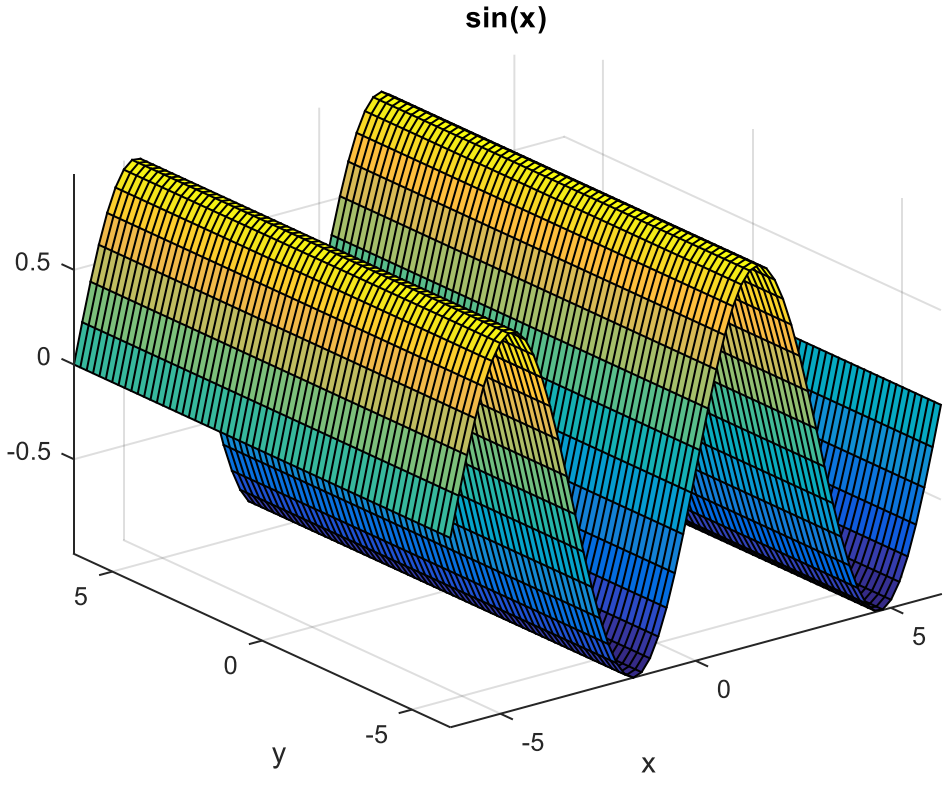
y =

sin(x)

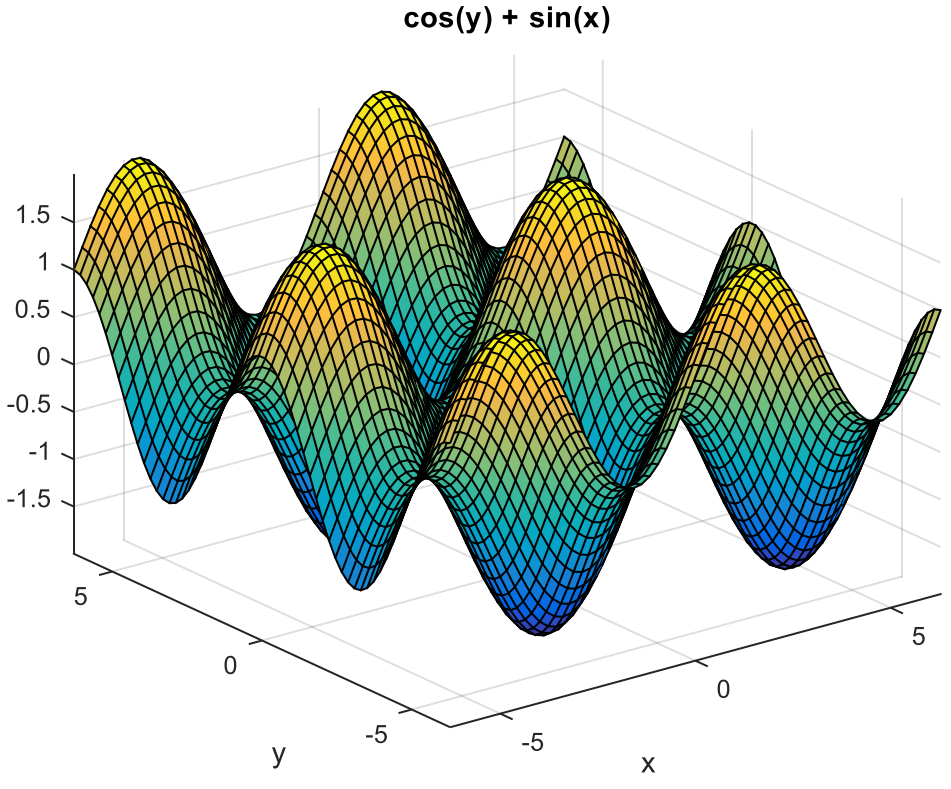
>> ezplot(y)
```



```
>> syms x
>> f = sin(x);
>> ezsurf(f)
```



```
>> syms x y
>> f = sin(x);
>> g = cos(y);
>> ezsurf(f+g);
```



5. Integration et Derivation

```
>> syms x;
      f = sin(5*x)

>> f =
      sin(5*x)

>>diff(f)

      ans =
      5*cos(5*x)
```

```
>> f = x^3

f =

x^3

>> diff(f) % 1st derivative

ans =

3*x^2
```

```
>> diff(f,2) % 2nd derivative

ans =

6*x

>> diff(f,3) % 3rd derivative

ans =

6

>> diff(f,4) % 4th derivative

ans =

0
```

```
>> syms x t;
f =  x * x ;

>>int(f)

ans =
1/3*x^3
```

```
>> int(x^3,0,10)

ans =

2500


>> int(x^3,-10,10)

ans =

0


>> int(x^3,-10,0)

ans =

-2500
```

6. Simplification d'une fonction

```
>> simplify(sin(x)^2 + cos(x)^2)
       ans =
         1
```

Merci

pour

votre Attention

Le logiciel de simulation Simulink
Partie 1
Présentation

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Introduction

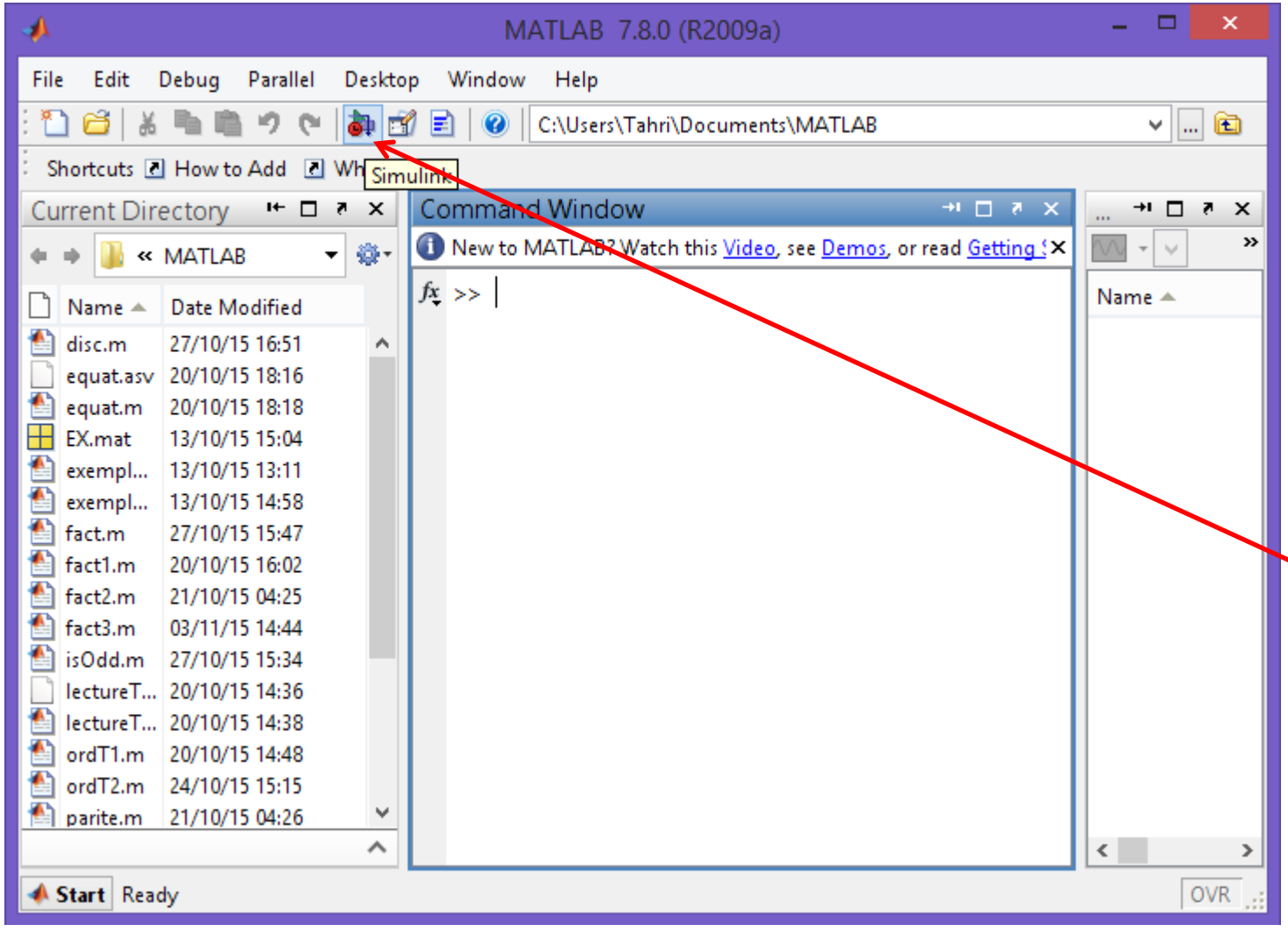
Simulink est le logiciel de simulation couplé au logiciel de calcul MATLB.
Simulink manipule des ***blocs fonctionnels*** disponibles dans des bibliothèques spécialisées et qu'on copie dans une fenêtre d'édition en les reliant selon le schéma-bloc pour représenter le système à étudier. La bibliothèque ***sources*** fournit des signaux d'excitation et la bibliothèque ***sink*** les outils d'observation comme, par exemple, le bloc oscilloscope

Particulièrement bien adapté à l'étude des schémas de l'Automatique, Simulink met en œuvre des éléments familiers, ce qui le rend facile à utiliser.

Simulink communique parfaitement avec MATLAB dont il profite de toutes les fonctionnalités. Il est possible, par exemple, de recueillir la réponse d'un modèle et de la visualiser dans MATLAB pour l'annoter et l'analyser.

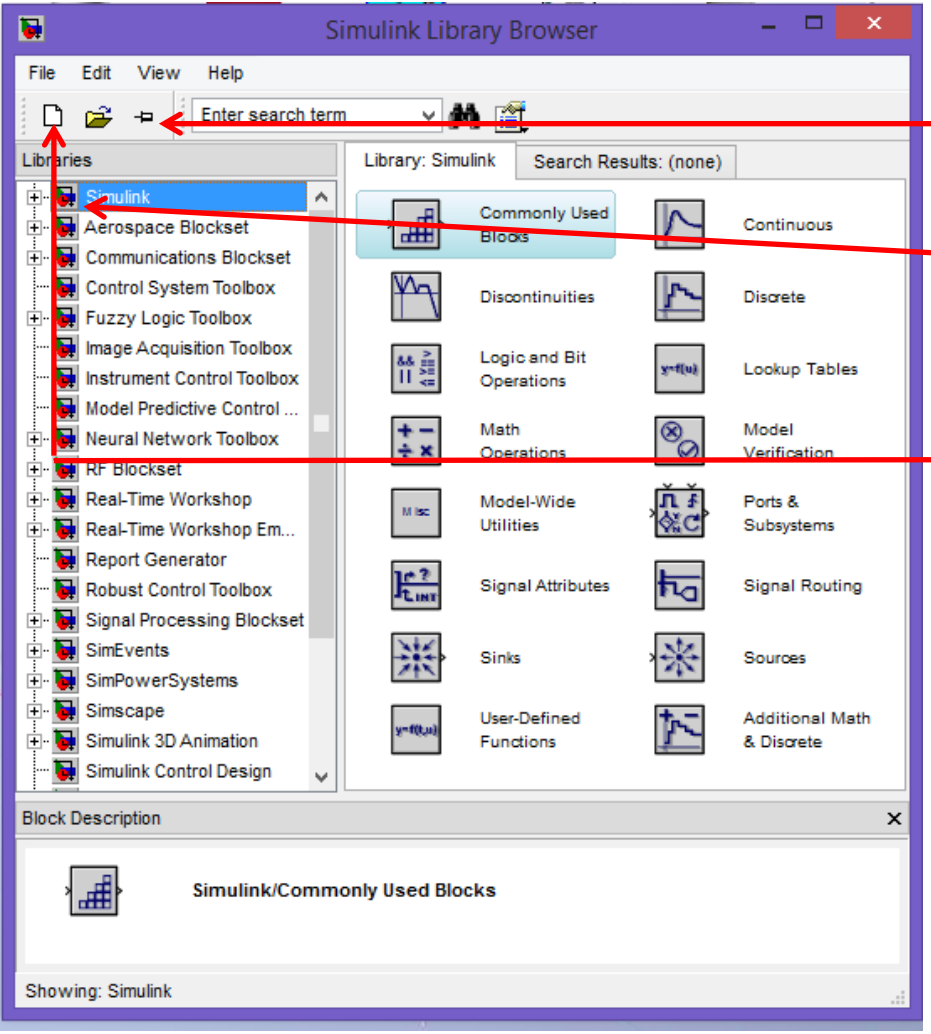
Dans Simulink un schéma se substitue aux équations pour constituer le ***modèle***. Les équations différentielles représentatives du fonctionnement du système physique modélisé sont sous-jacentes (en dessous). Le logiciel ***simule*** le comportement du modèle représentatif du système en calculant, pas après pas, les points du signal de sortie dès lors qu'il connait ceux du signal d'entrée et le point de départ (conditions initiales)

2. Lancer Simulink



Simulink peut être lancer en cliquant sur l'icone Simulink.

3. Le Simulink

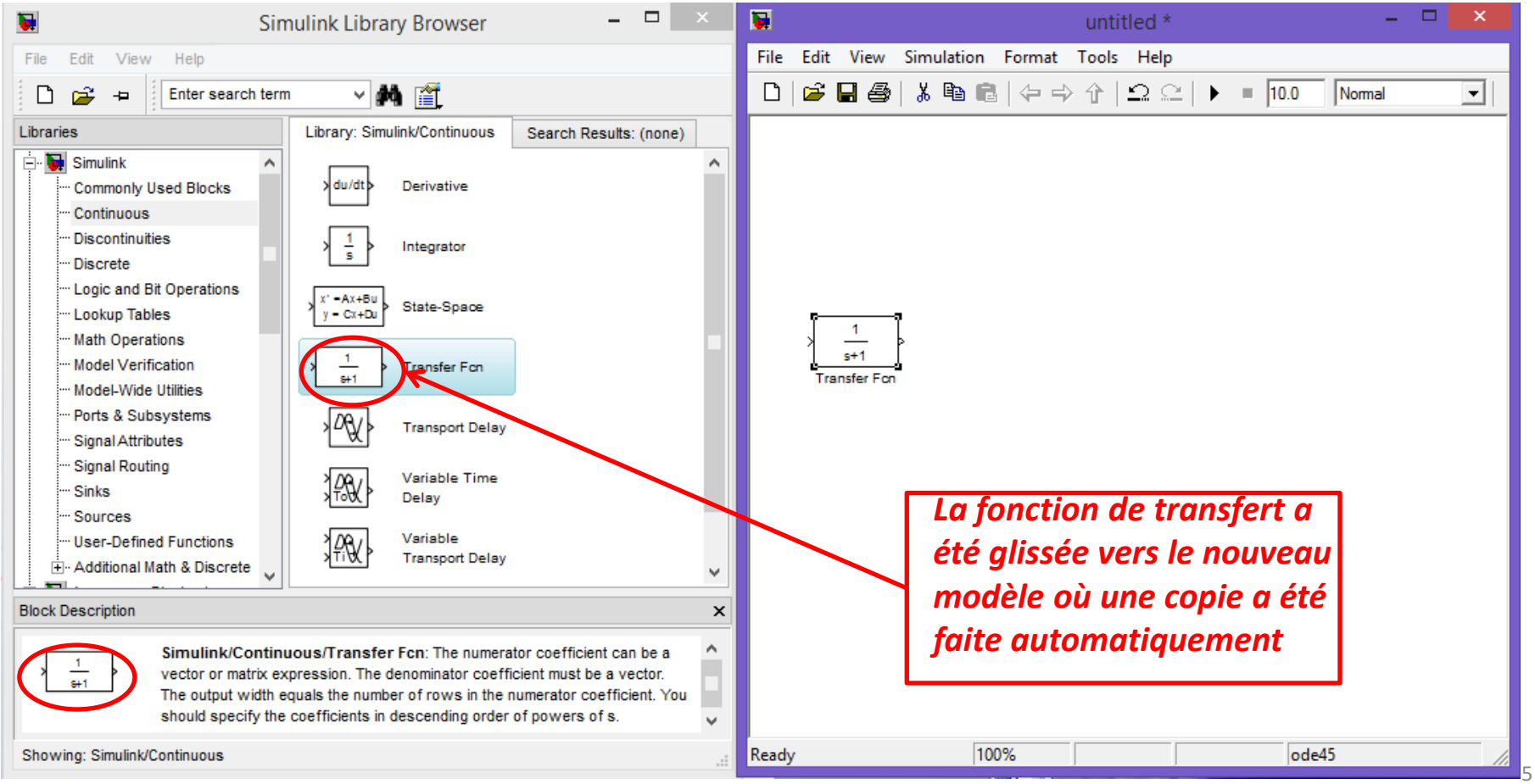


Epingler pour la visibilité permanente

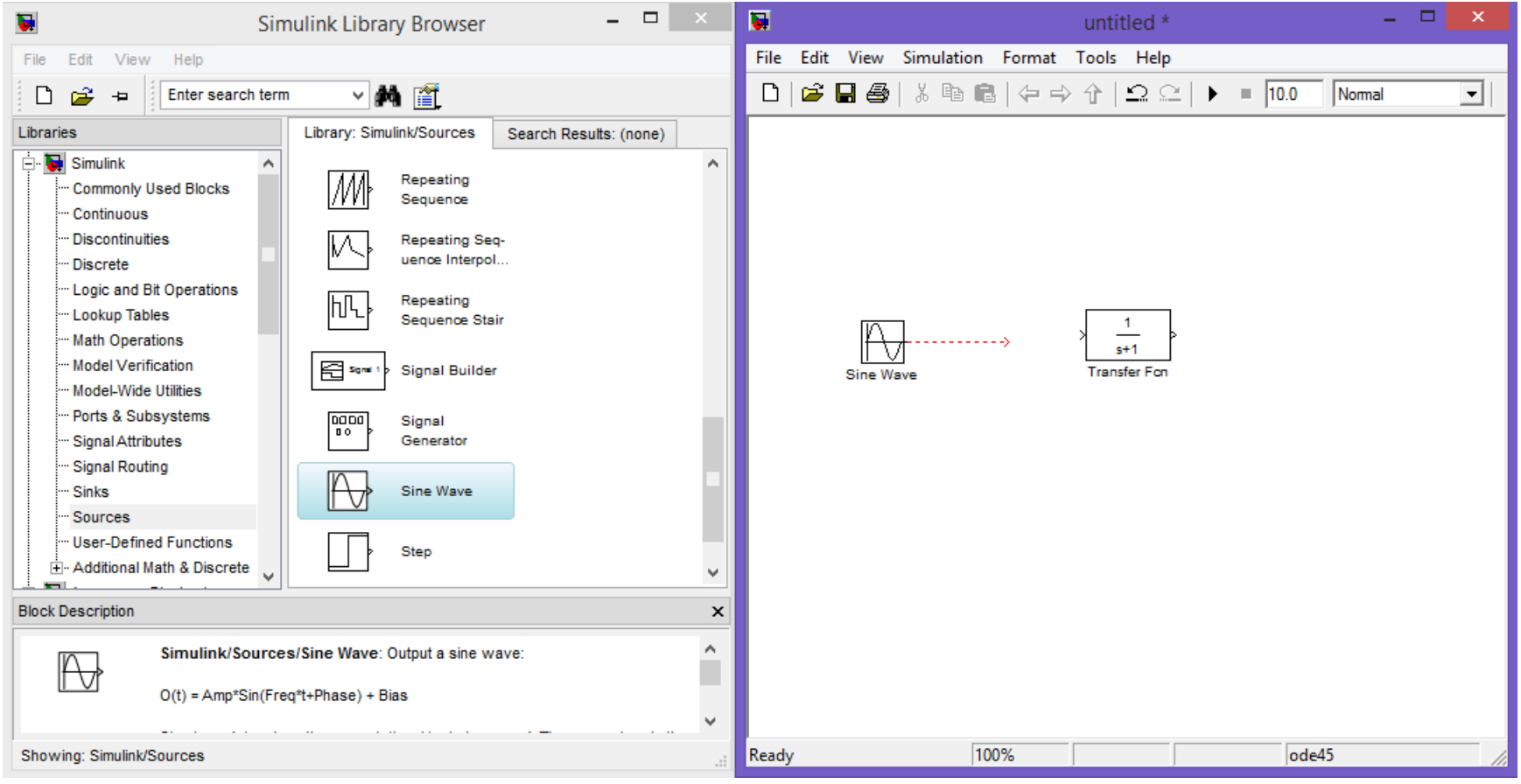
Bibliothèques de blocs

Créer un nouveau modèle

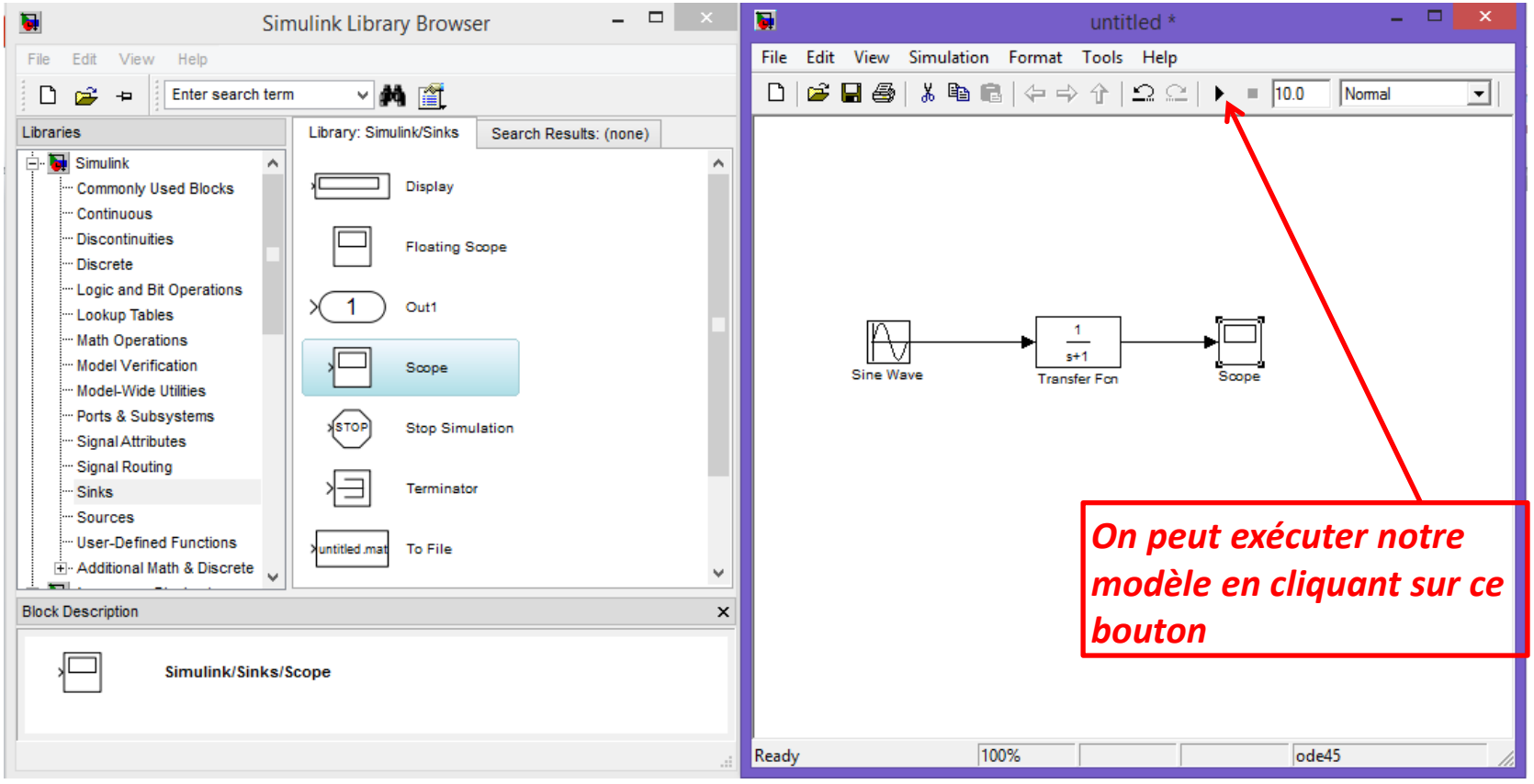
4. Créer un nouveau modèle



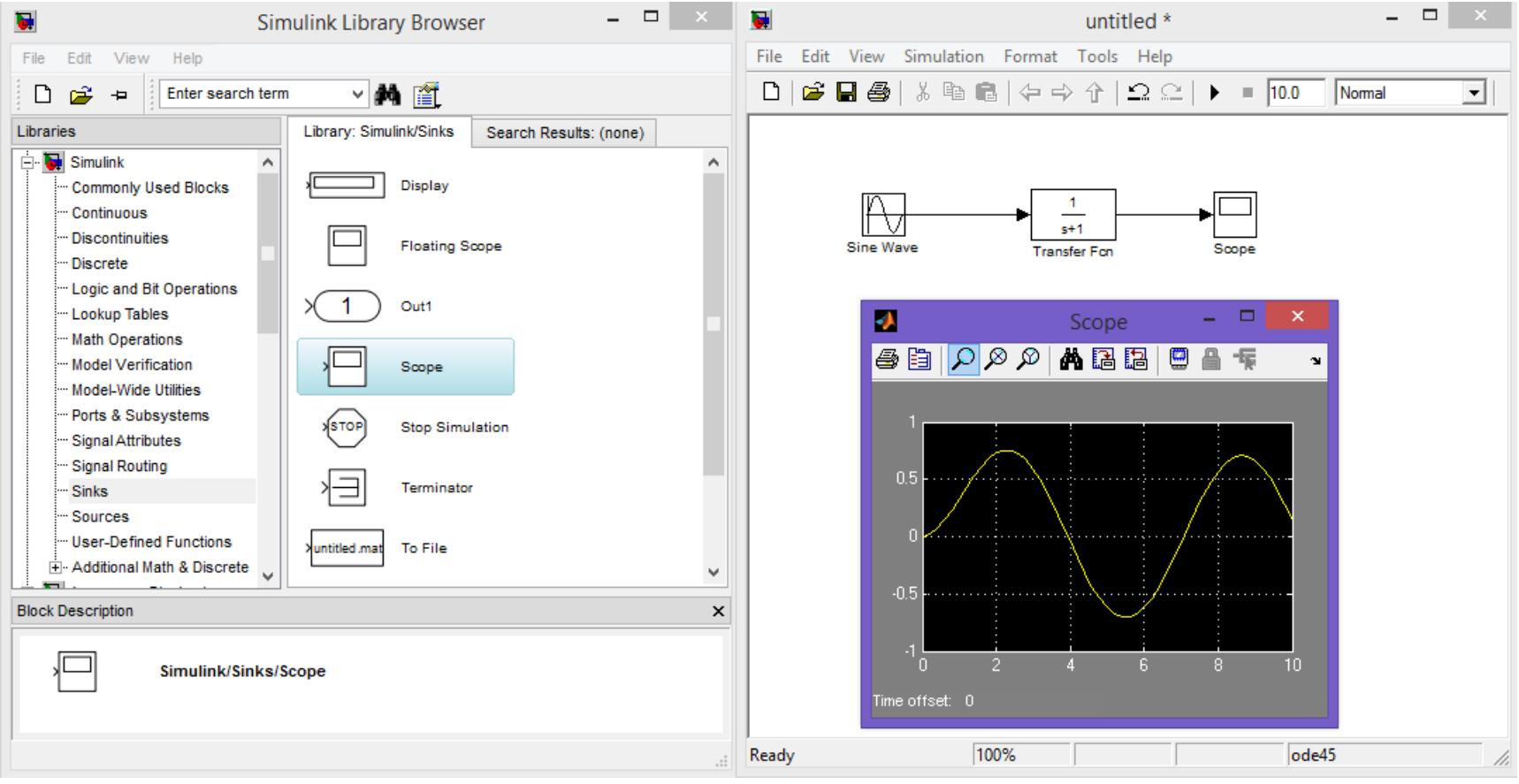
Avec la souris on peut connecter les deux blocs, le bloc signal sinus de la bibliothèque source avec la fonction de transfert.



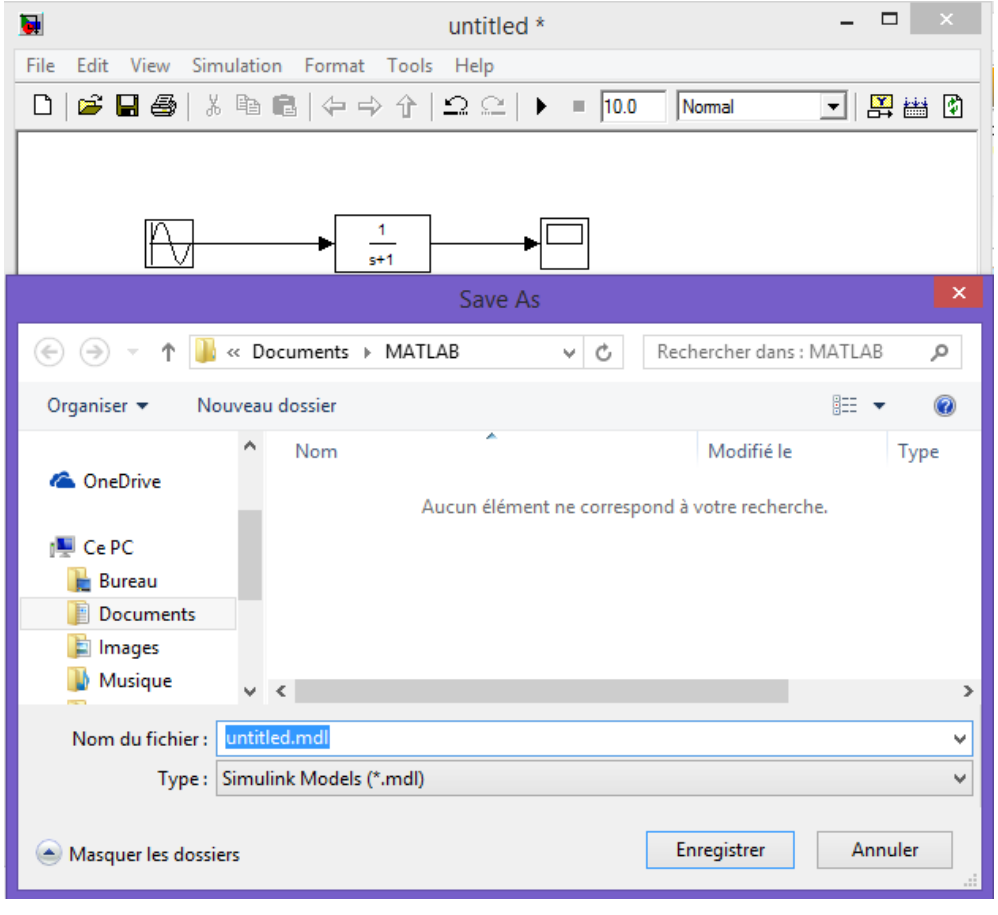
On a ajouté un oscilloscope (Scope) pour la visualisation de la sortie.



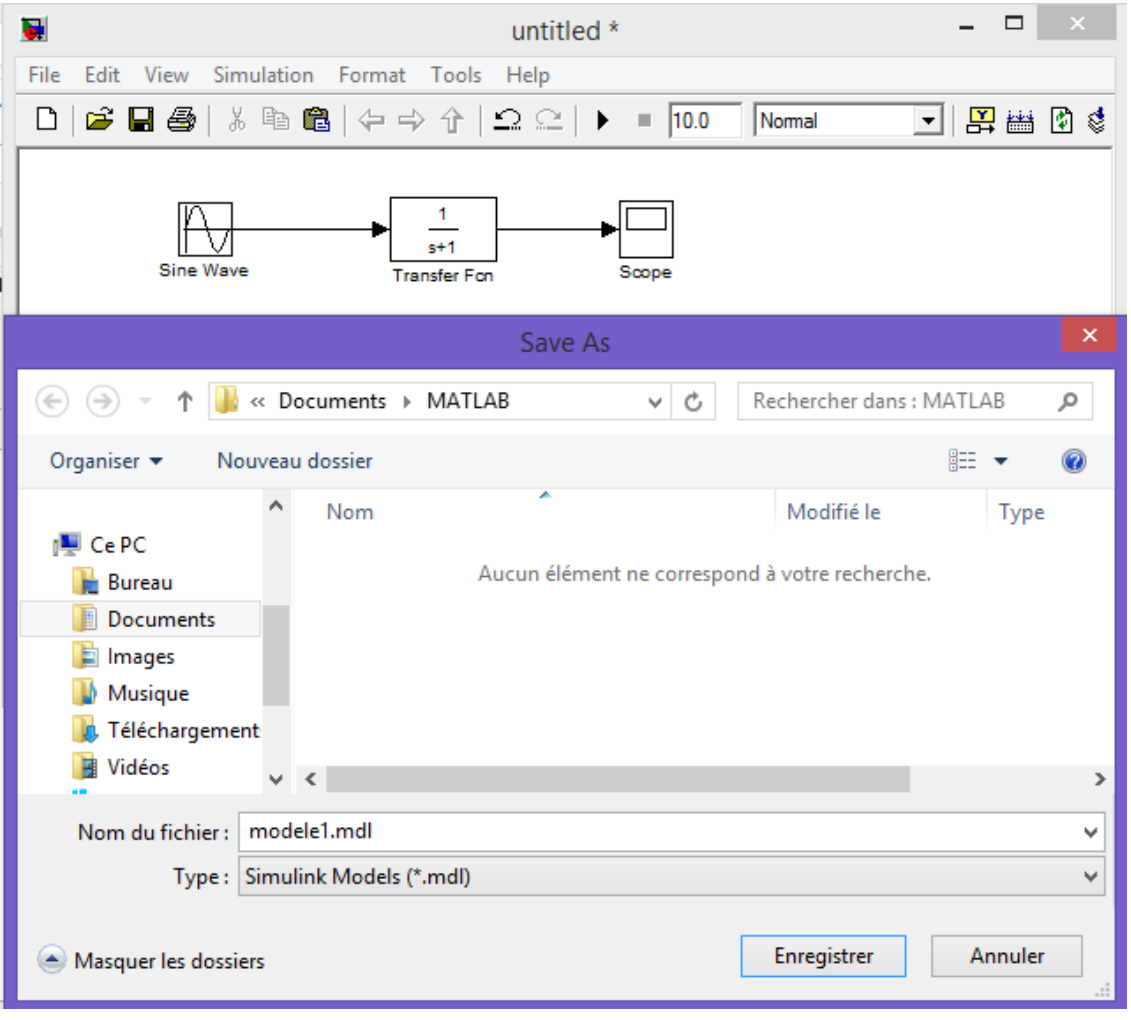
En exécutant le modèle, on clique deux fois sur l'oscilloscope (Scope) pur afficher le signal de sortie.



5. Sauvegarder un modèle

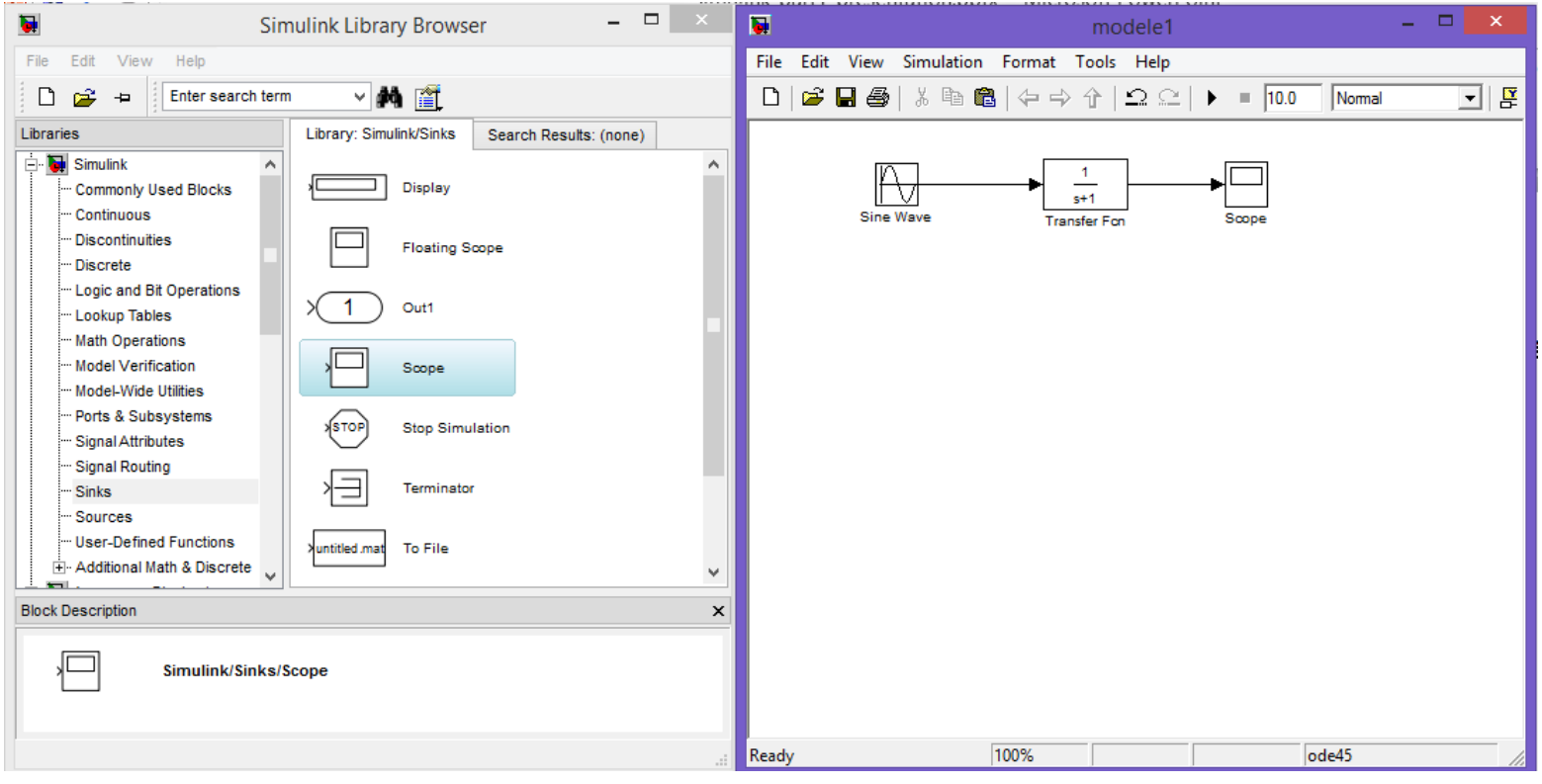


En Simulink, un modèle est sauvegardé dans un fichier avec l'extension mdl

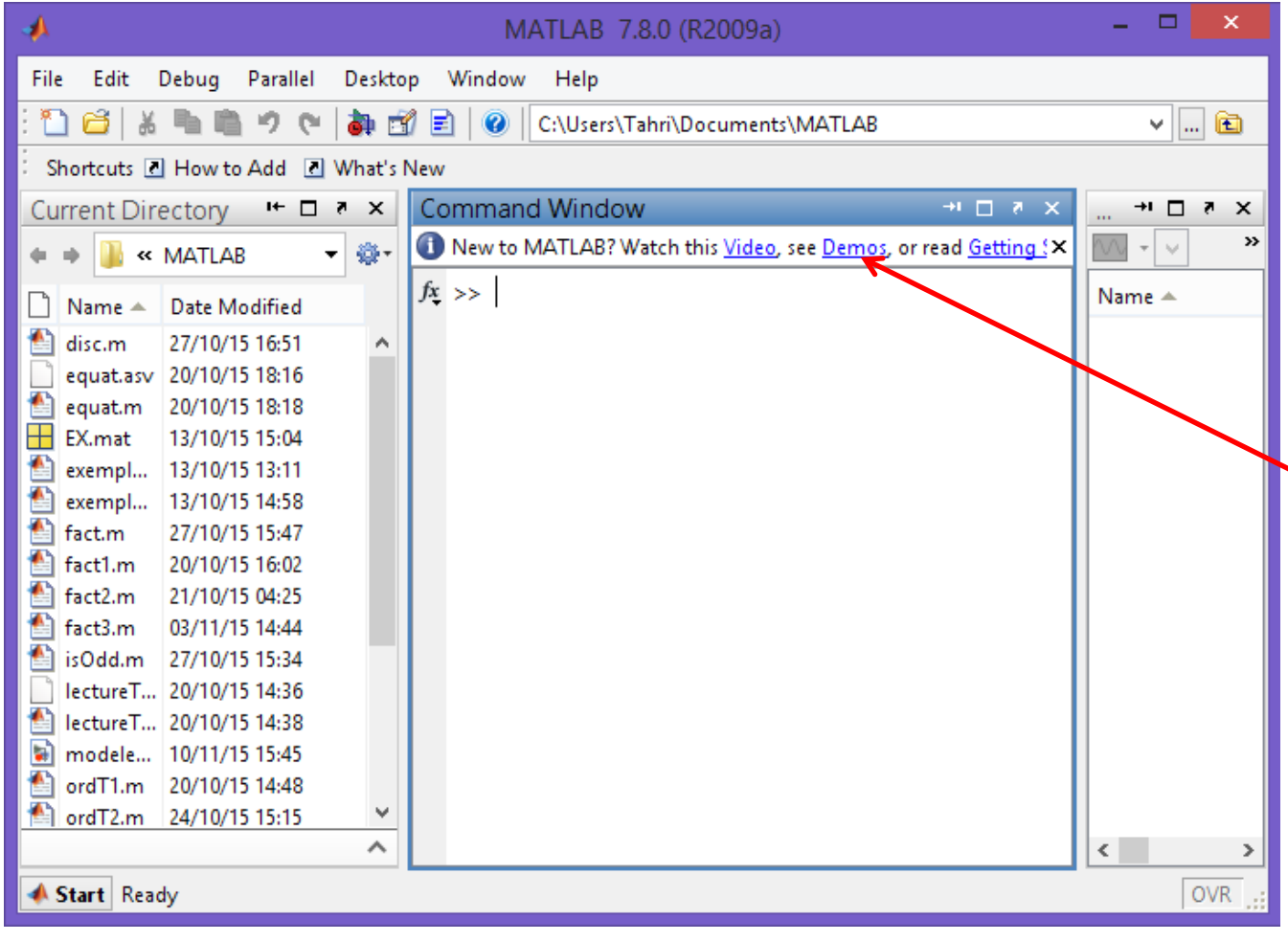


On a sauvegardé ce modèle dans un fichier modele1.mdl

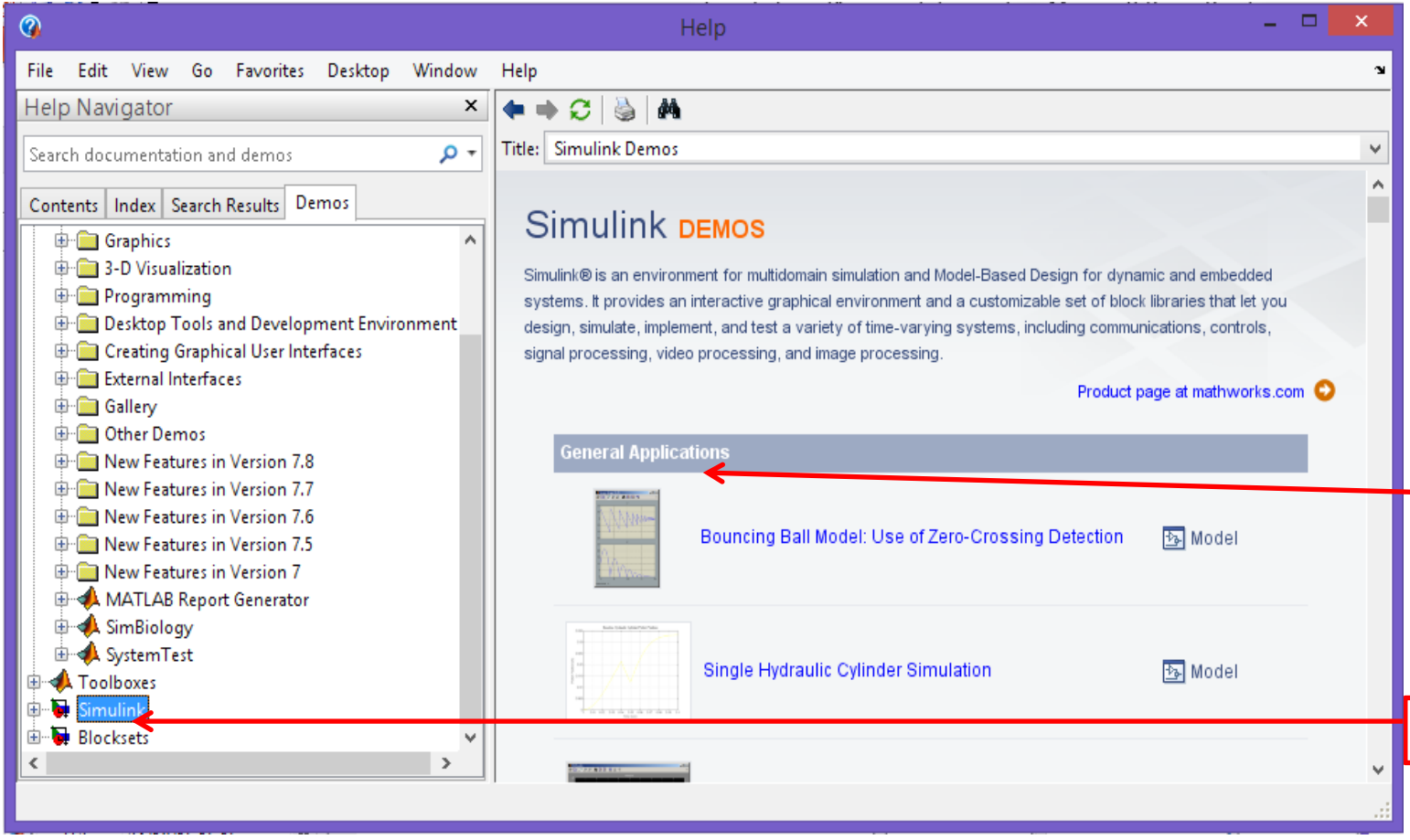
Ainsi le modèle est sauvegardé dans le fichier modele1.mdl



6. Explorer Simulink

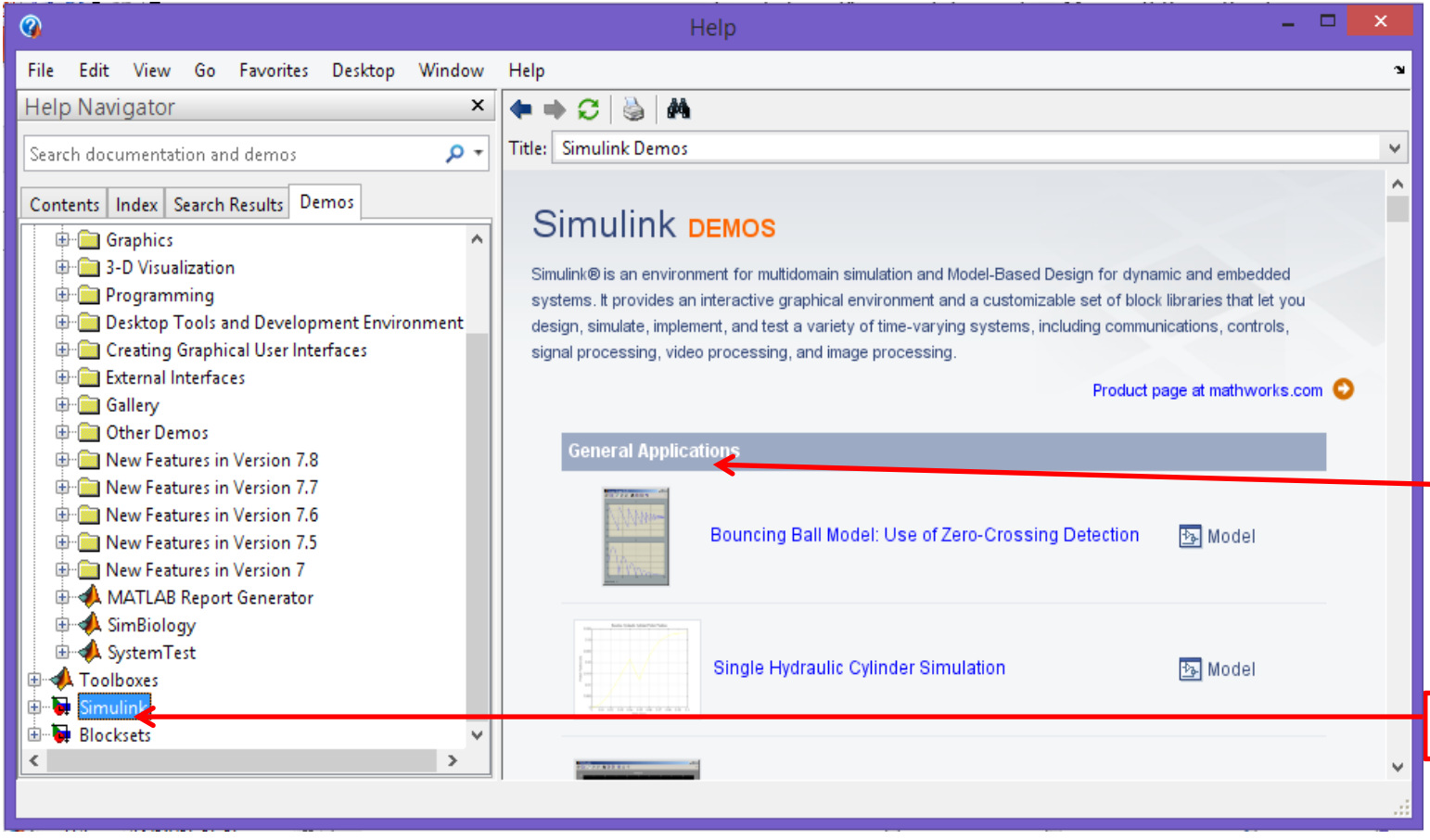


Cliquez sur Demos



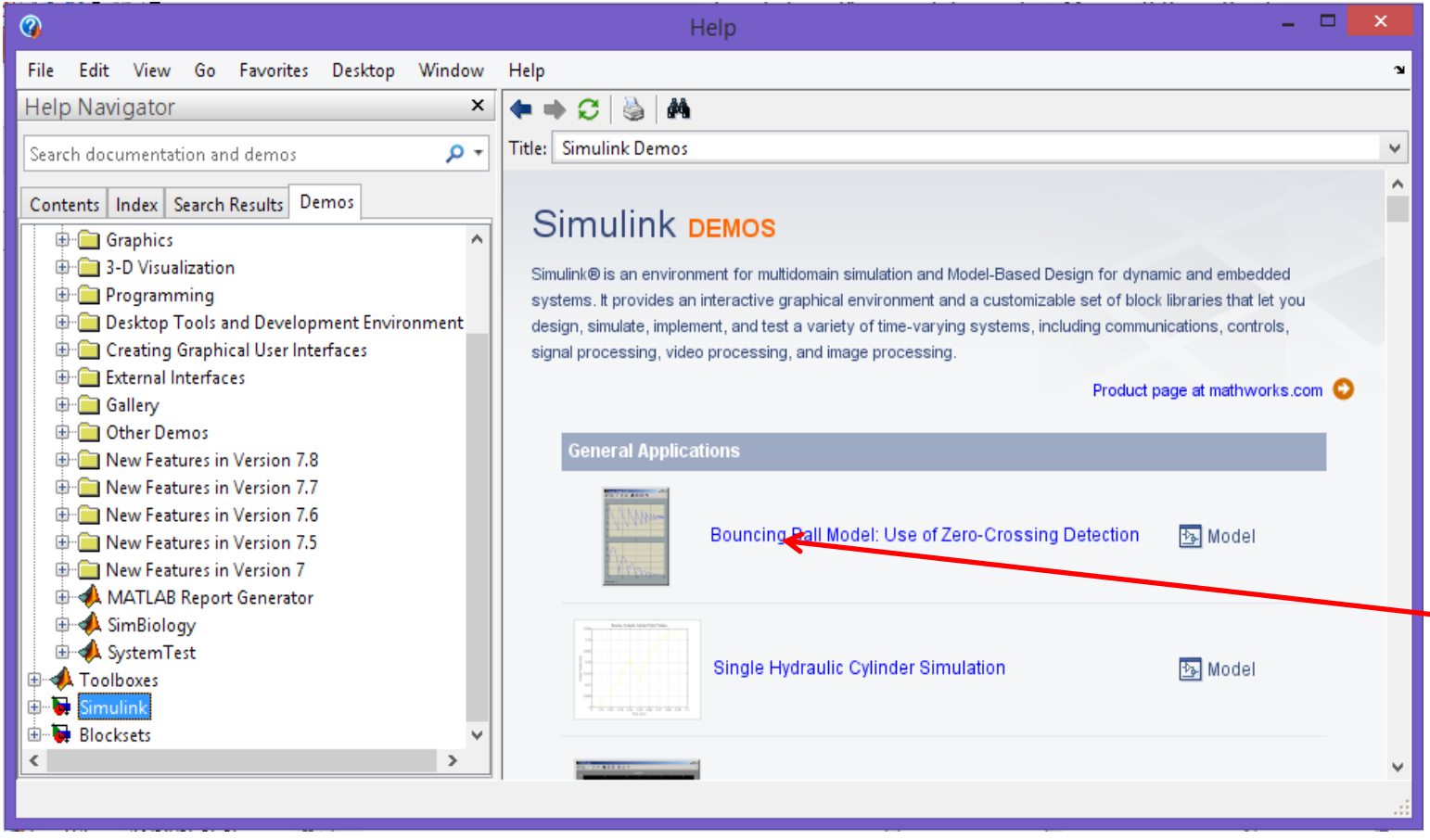
Exemples de de Simulink

Demos de Simulink



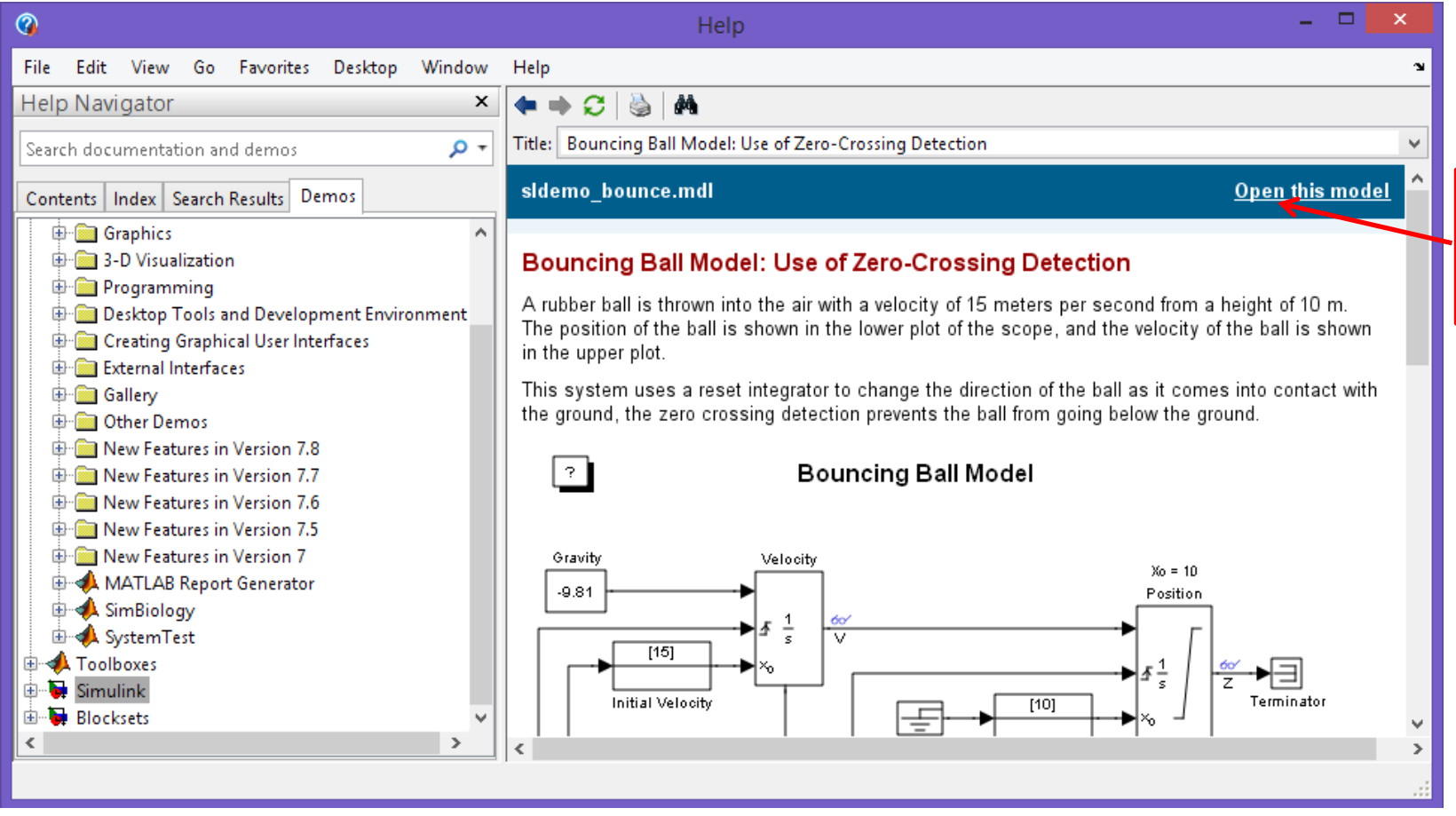
Exemples de de Simulink

Demos de Simulink



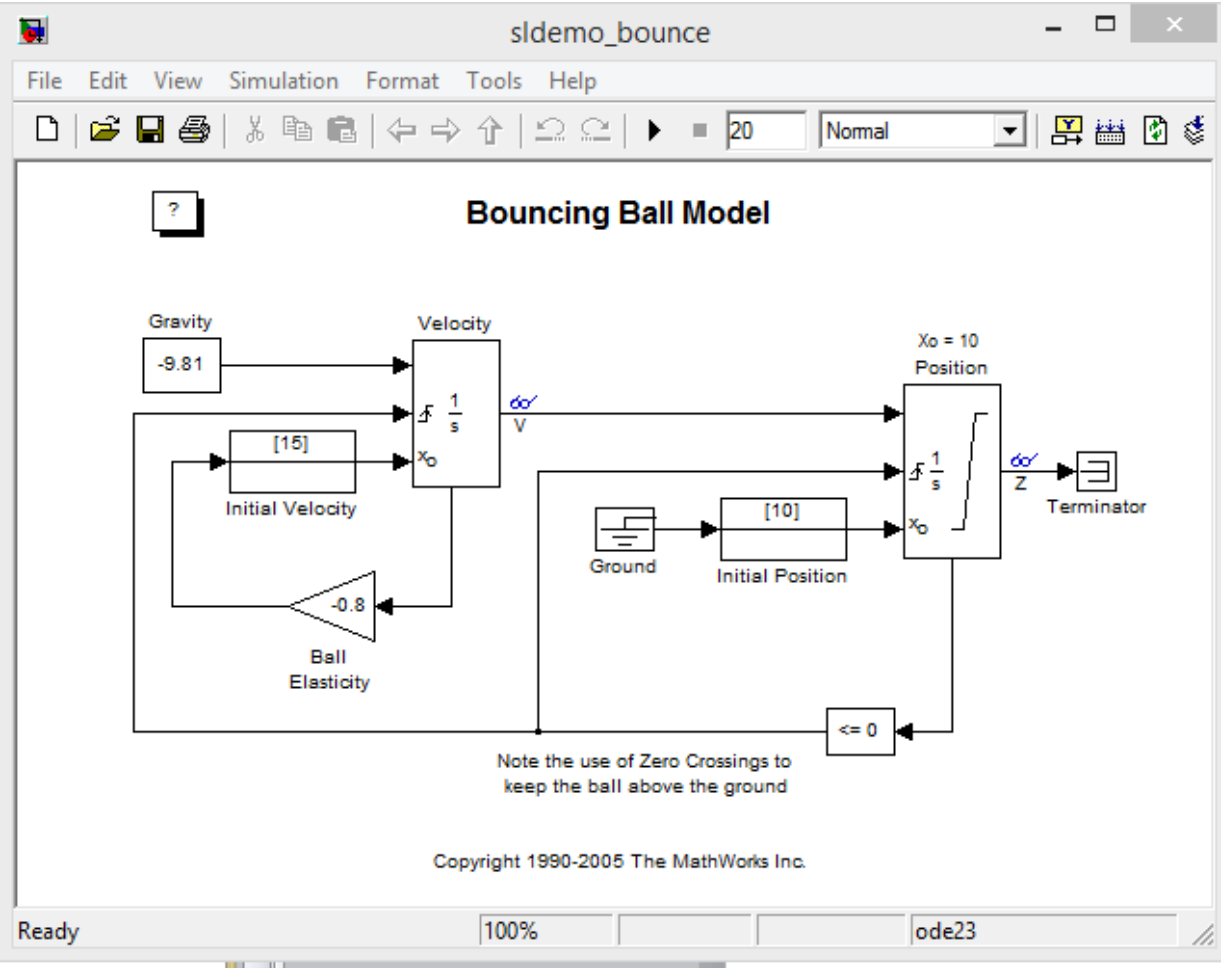
Cliquez sur cet exemple

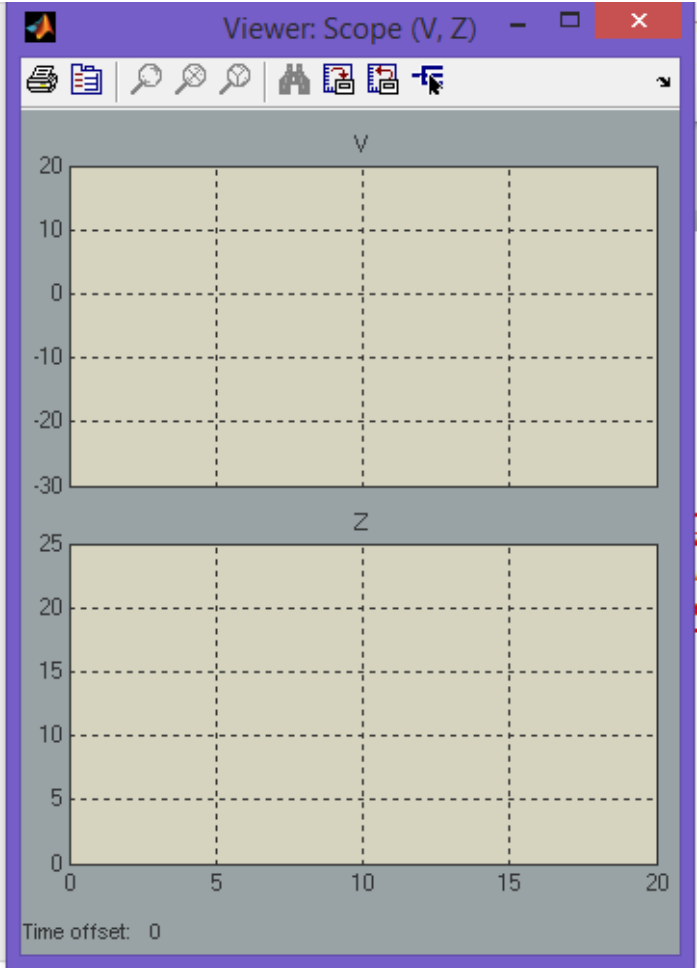
La fenêtre explique en détaille le modèle, le modèle peut être ouvert en cliquant sur ***open this model*** en haut à droite de la fenêtre.



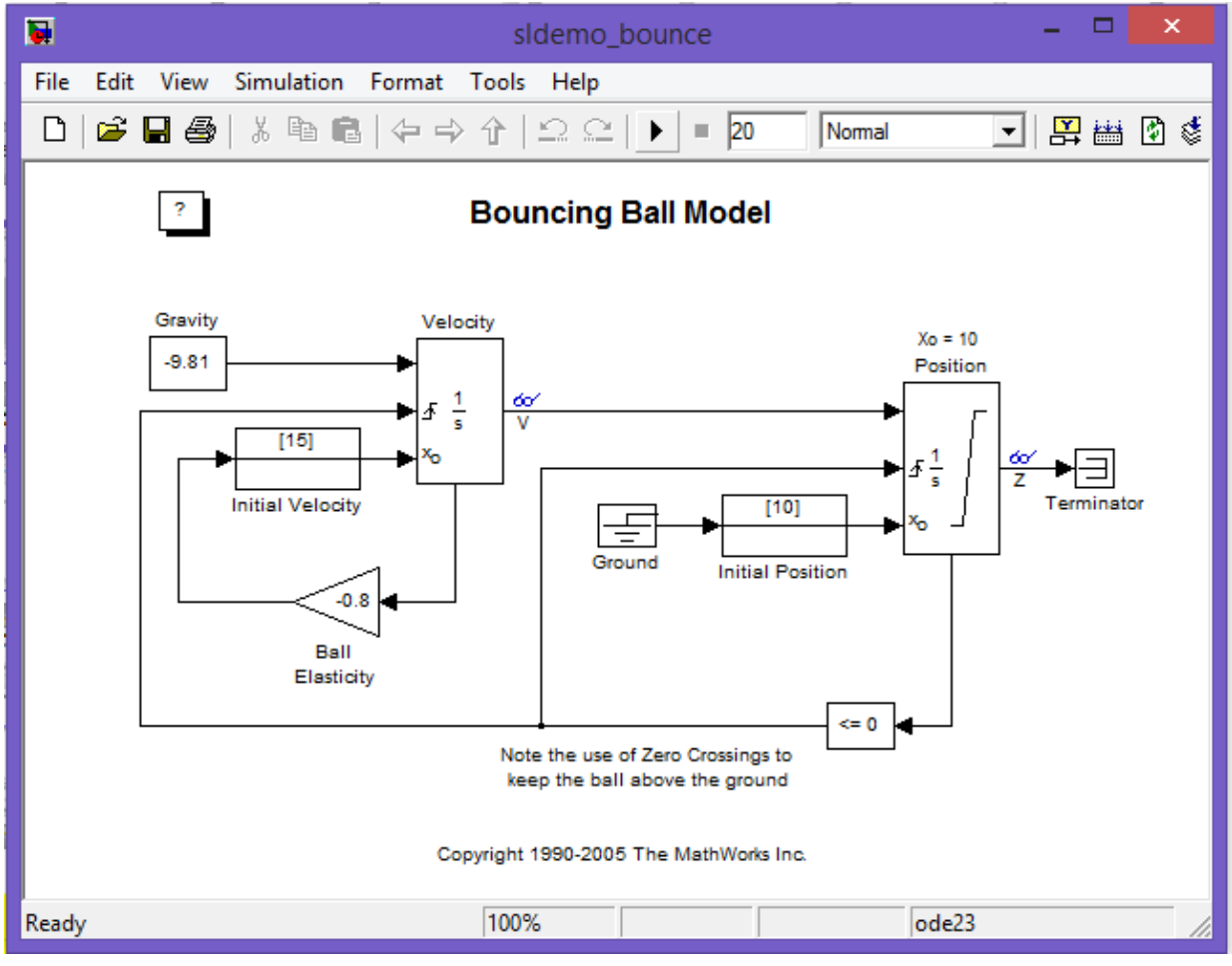
Cliquez pour ouvrir le modèle

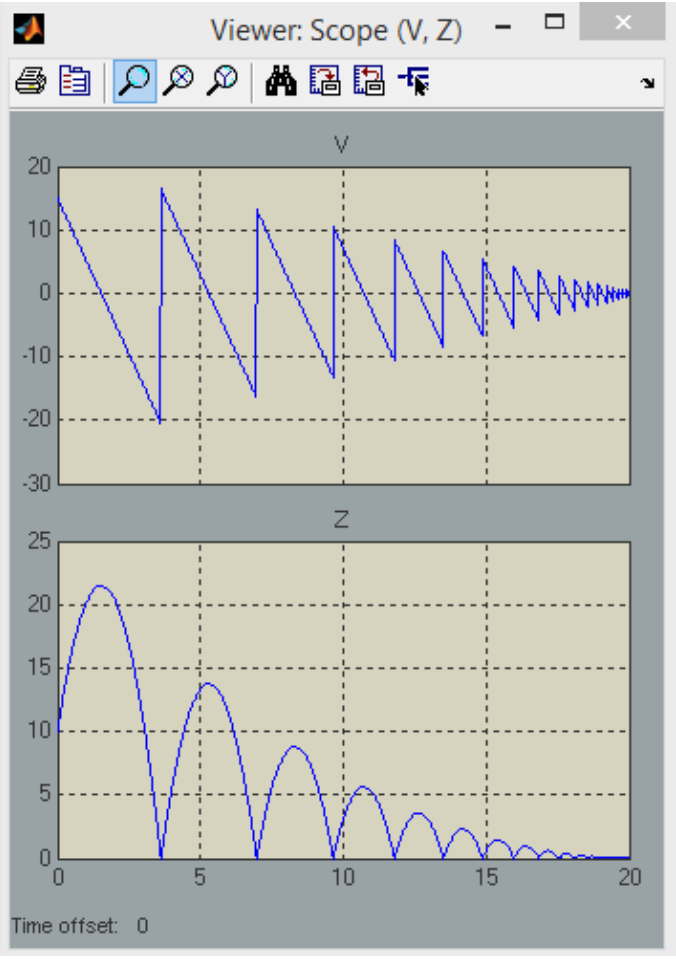
Ainsi le modèle est ouvert avec l'oscilloscope (Scope) pour la visualisation des sortie et on peut exécuter le modèle facilement.



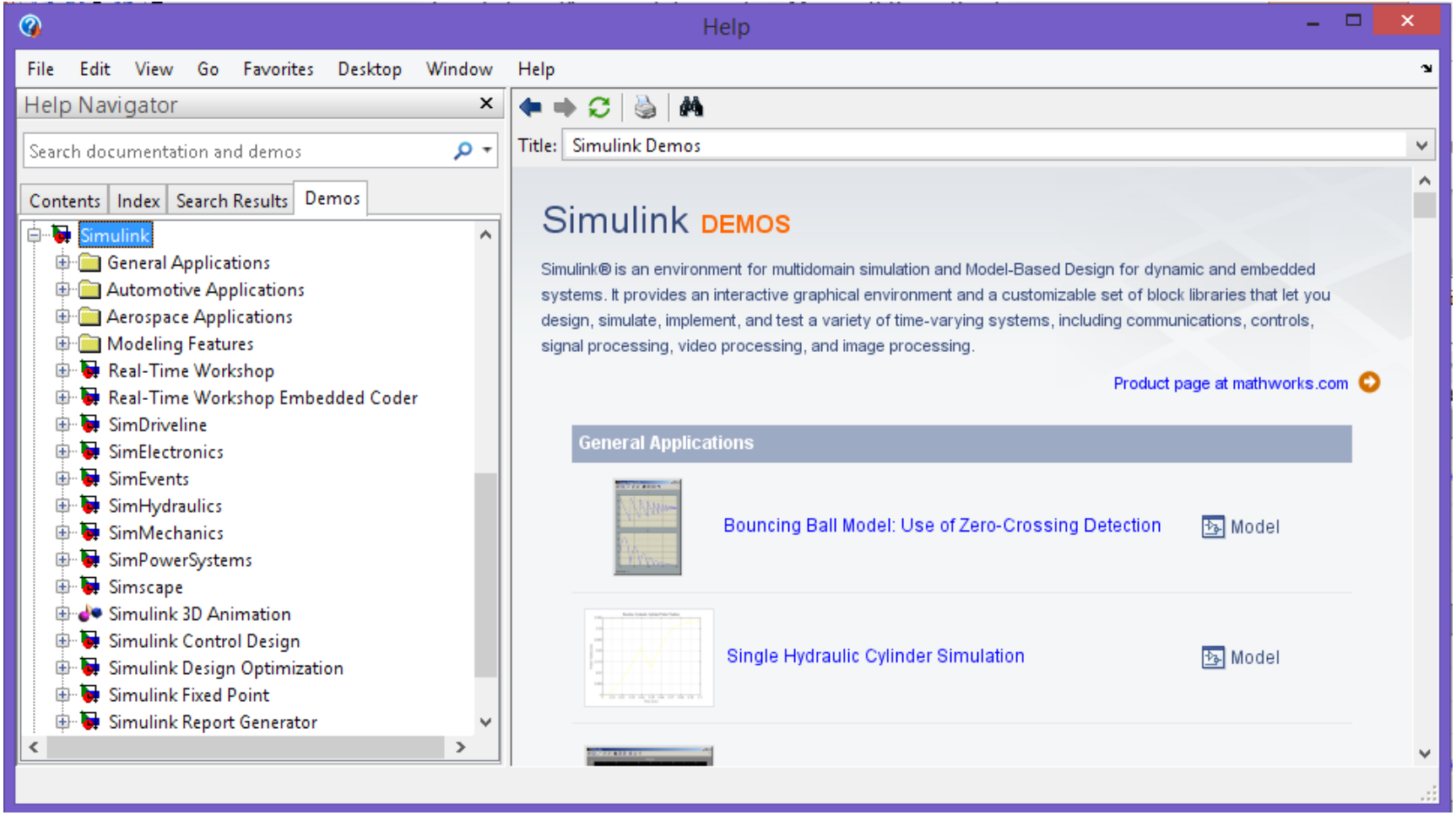


Après exécution du modèle dans l'oscilloscope (Scope), on a la visualisation de la position de la balle et sa vitesse.

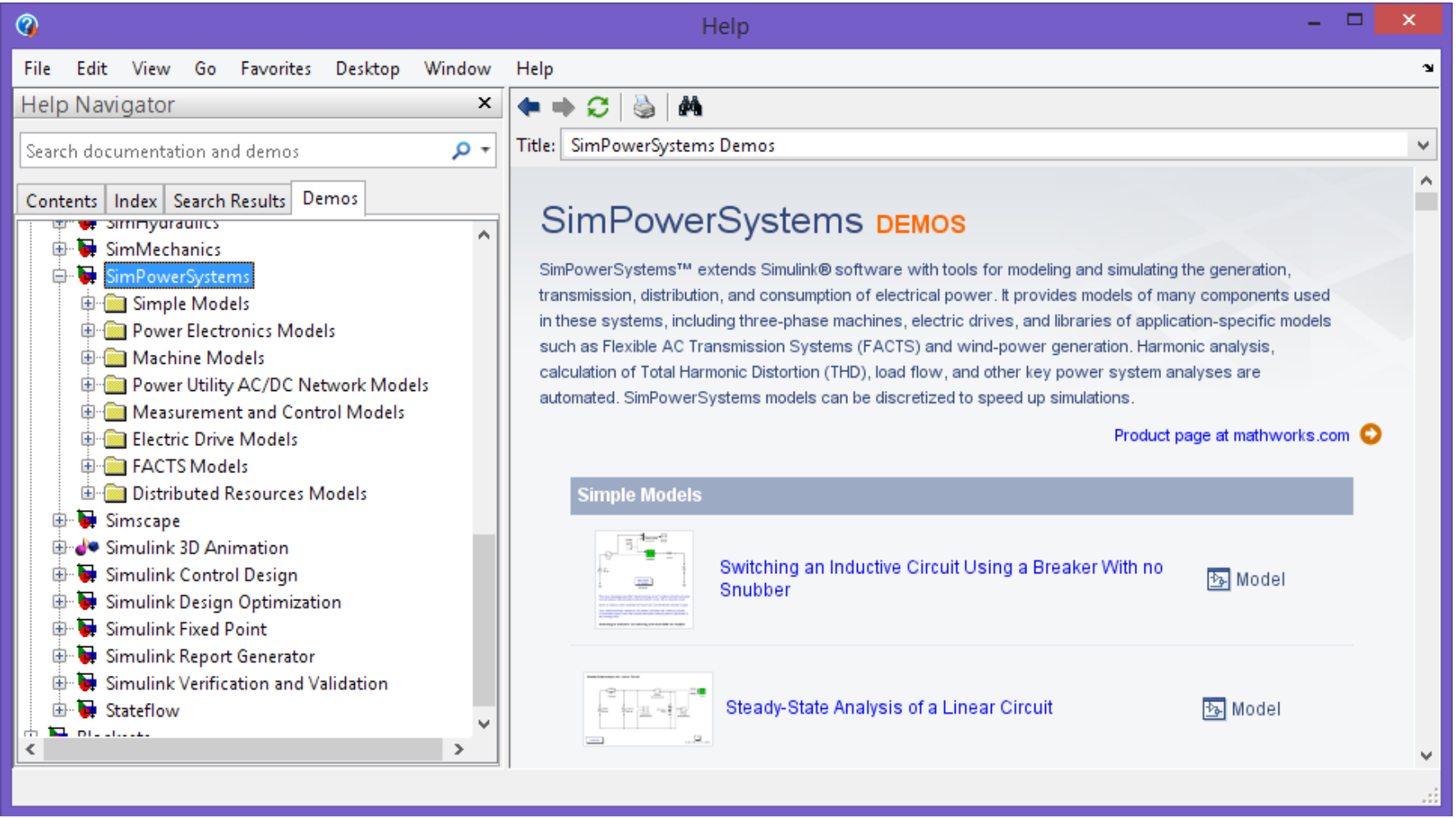




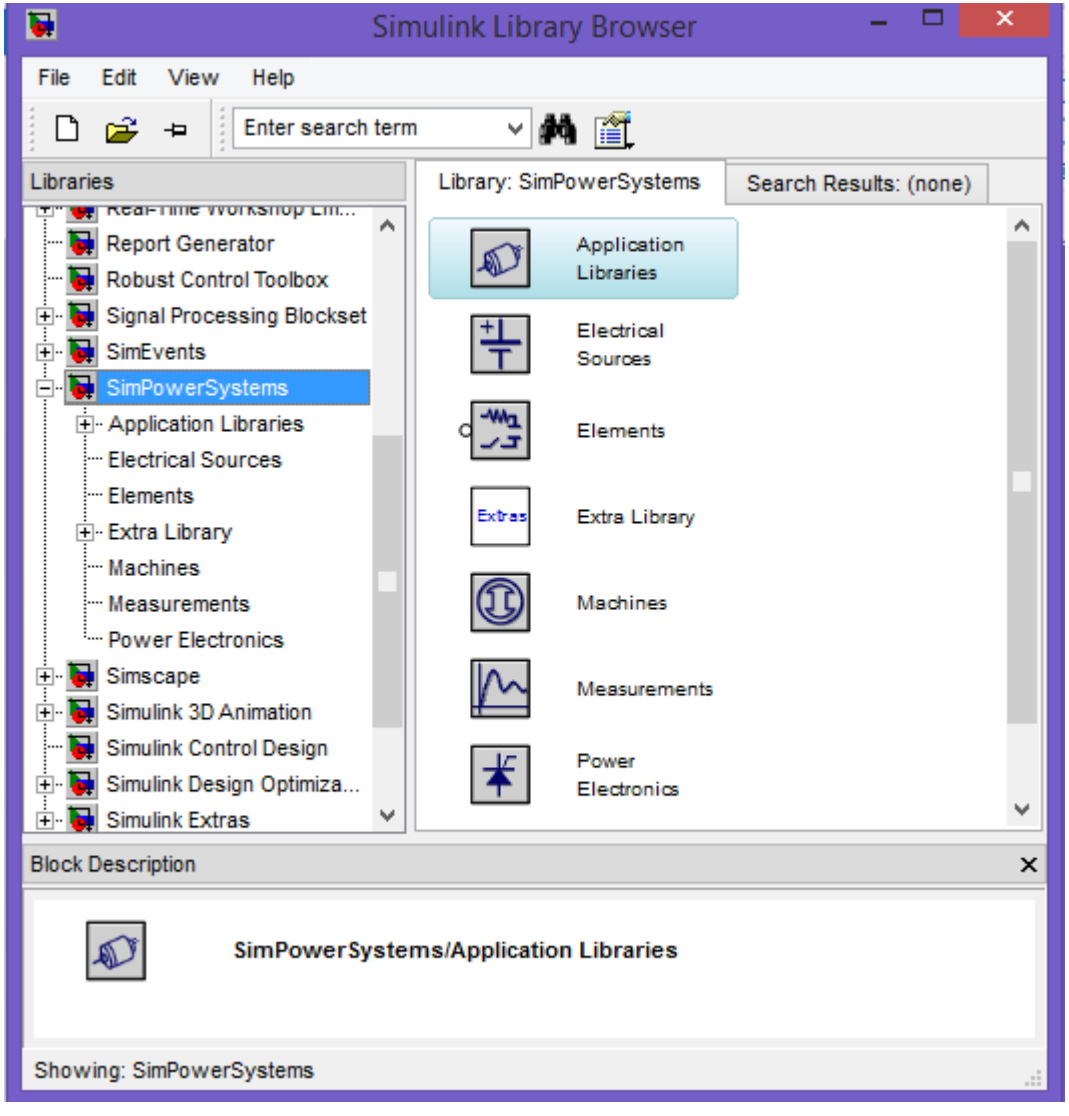
Dans la démos de Simulink, il y'a aussi beaucoup d'exemples des Toolboxes utilisées par ce dernier. Une Toolboxe nous intéresse et celle du ***SimPowerSystems***



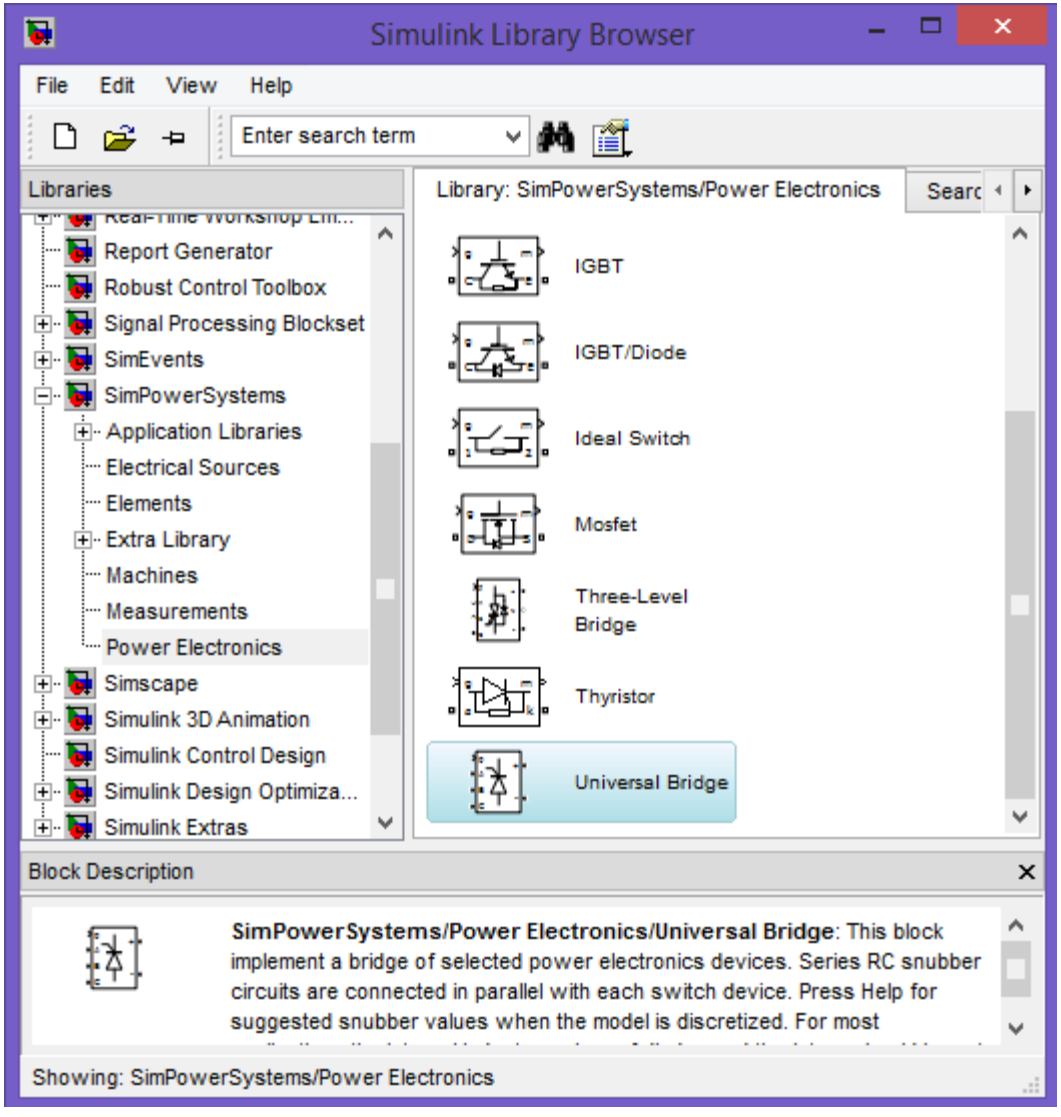
Dans la démos de ***SimPowerSystems***, il y'a aussi beaucoup d'exemples qu'on peut explorer.



Revenant au Simulink et essayant d'explorer la toolboxe ***SimPowerSystems***.

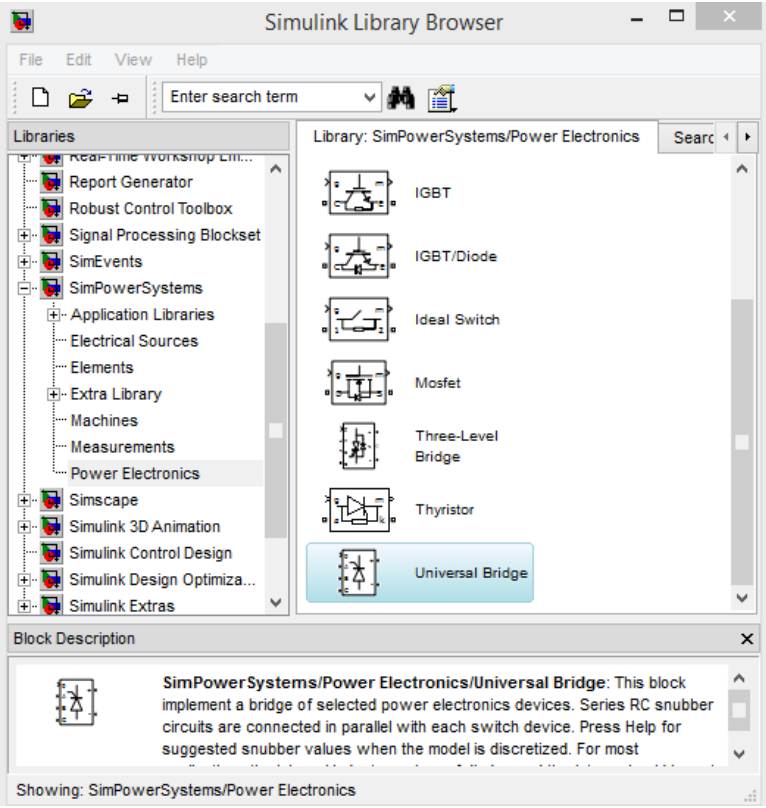


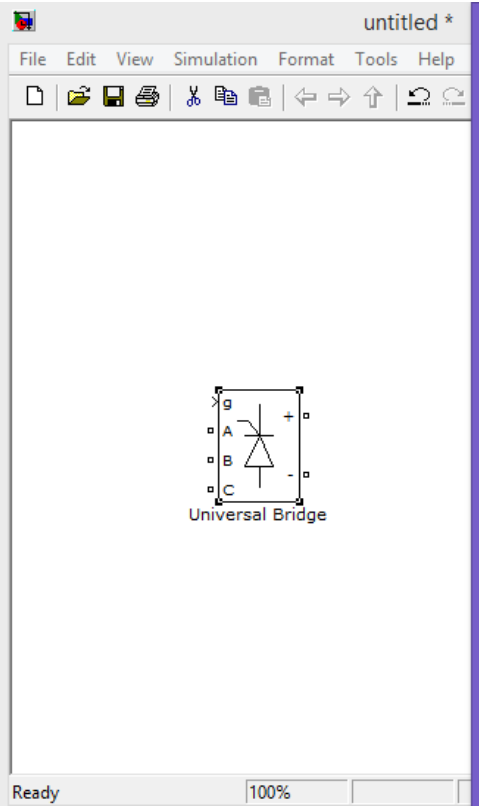
Vous remarquez que cette toolboxe est vraiment destinée aux électrotechniciens vu la richesse des blocs fonctionnels spécialisés pour tout ce qui touche le génie électrique.

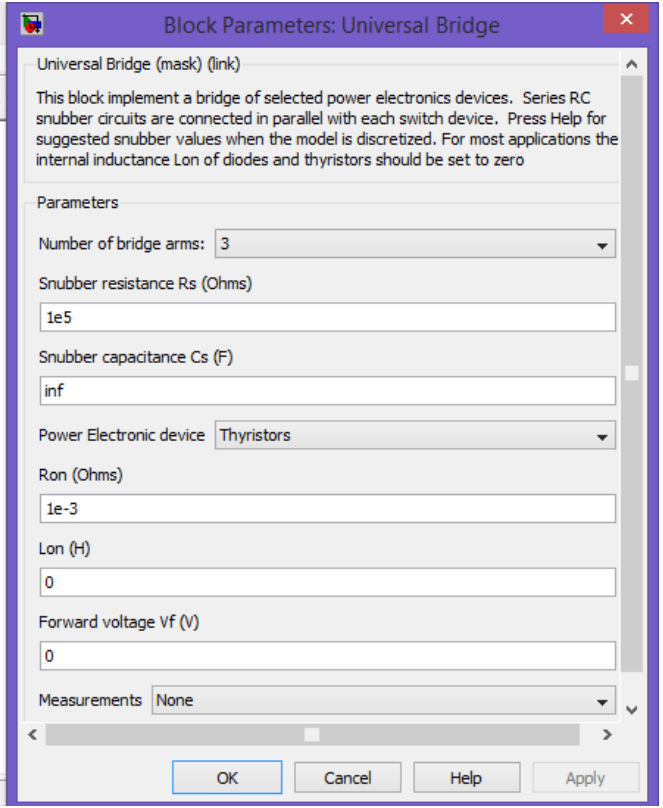


Essayant de voir par exemple le bloc ***Universal Bridge*** (pont universel).

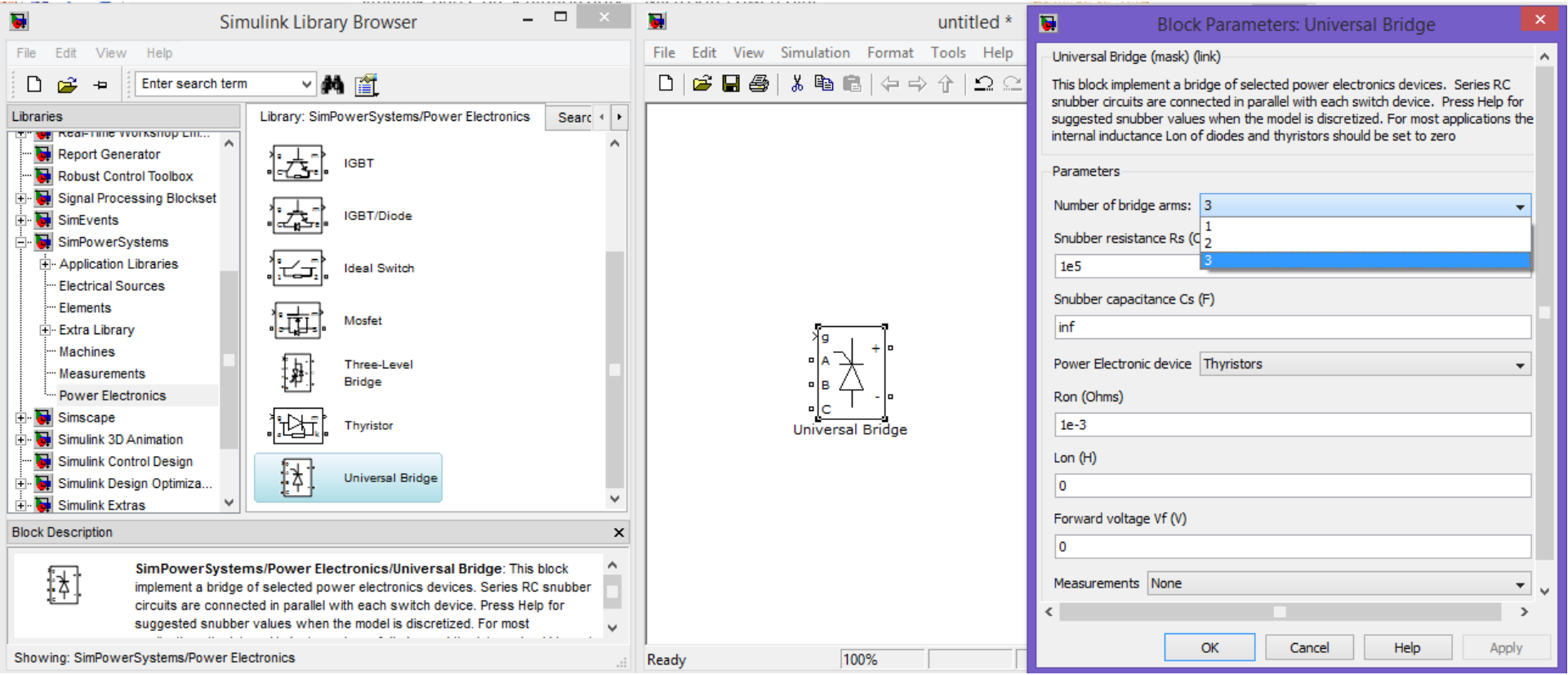
Double click sur le bloc ***Universal Bridge***, on peut remarquer quand peut changer les paramètres suivant notre application.



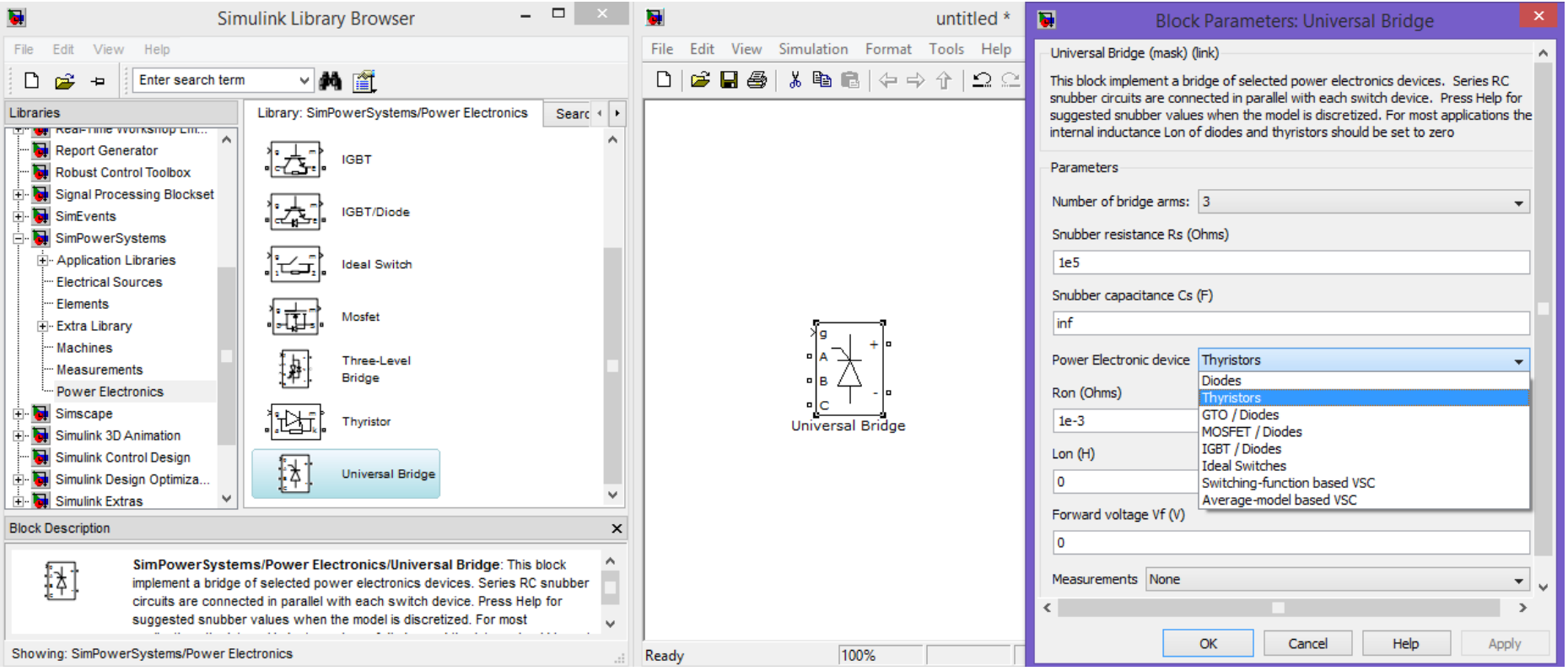




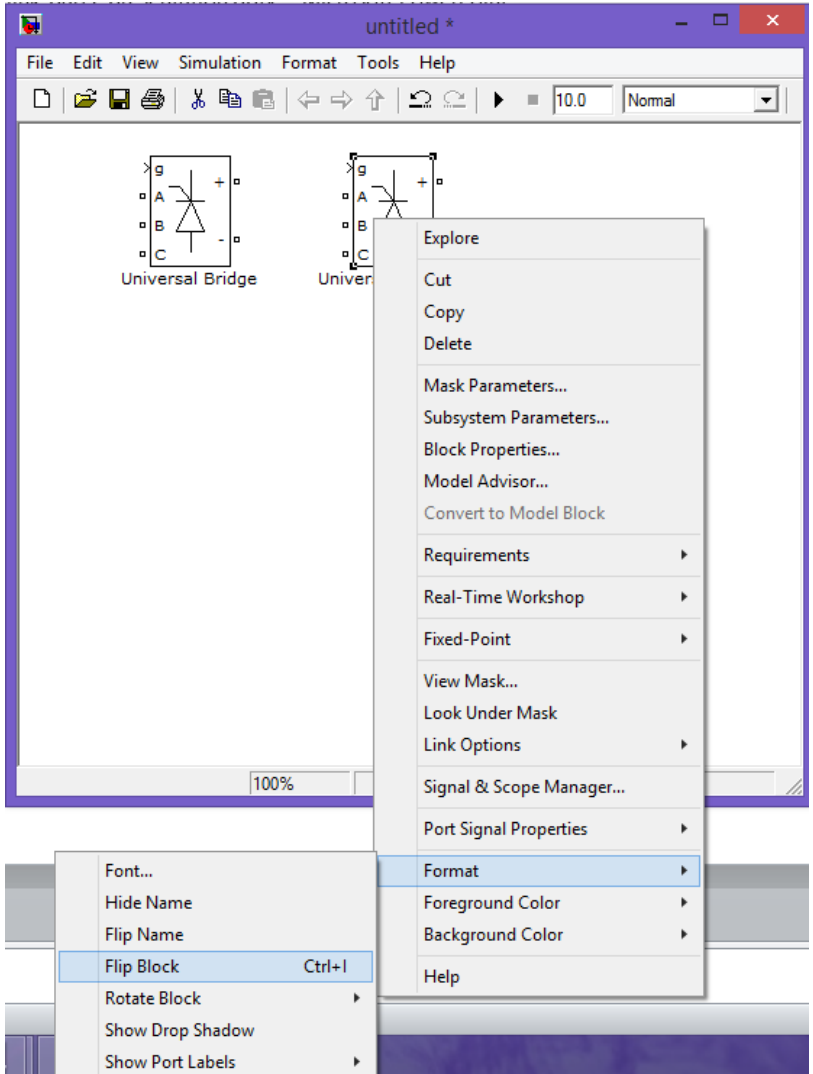
On peut choisir entre un pont à 3, 2 et 1 bras (*Number of bridge*)



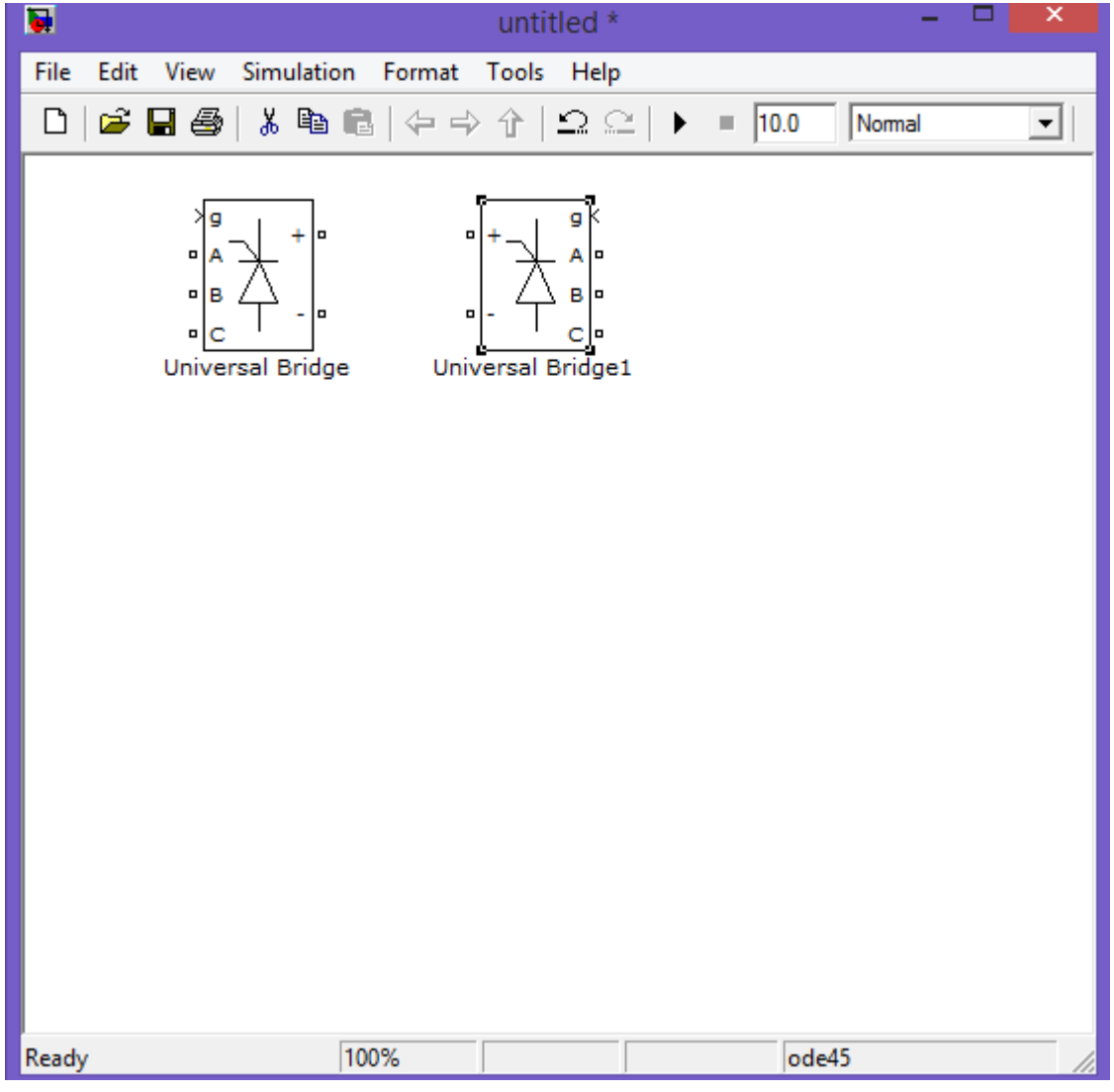
On peut aussi choisir quels sont les semi-conducteurs qu'on veut utiliser, on a le choix entre Diodes Thyristors, GTO etc…



On peut aussi inverser ou basculer le bloc en utilisant ***Flip Block***



Remarquer le deuxième bloc a été inverser



Merci

pour

votre Attention

Le logiciel de simulation Simulink
Partie 2
Exemples de modélisation

Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Introduction

Simulink est le logiciel de simulation couplé au logiciel de calcul MATLB.
Simulink manipule des ***blocs fonctionnels*** disponibles dans des bibliothèques spécialisées et qu'on copie dans une fenêtre d'édition en les reliant selon le schéma-bloc pour représenter le système à étudier.

Simulink communique parfaitement avec MATLAB dont il profite de toutes les fonctionnalités. Il est possible, par exemple, de recueillir la réponse d'un modèle et de la visualiser dans MATLAB pour l'annoter et l'analyser.

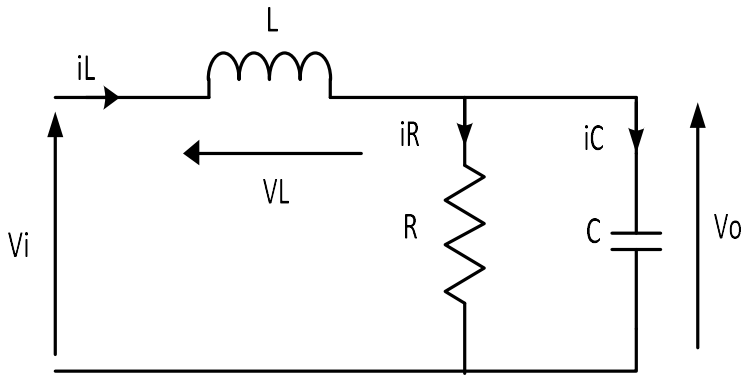
Dans Simulink un schéma se substitue aux équations pour constituer le ***modèle***. Les équations différentielles représentatives du fonctionnement du système physique modélisé sont sous-jacentes (en dessous). Le logiciel ***simule*** le comportement du modèle représentatif du système en calculant, pas après pas, les points du signal de sortie dès lors qu'il connait ceux du signal d'entrée et le point de départ (conditions initiales)

On va utiliser Simulink pour modéliser des systèmes physiques en ***blocs fonctionnels*** disponibles dans des bibliothèques spécialisées et qu'on copie dans une fenêtre d'édition en les reliant selon le schéma-bloc pour représenter le système à étudier.

2. Modélisation d'un circuit électrique

Un circuit électrique est donné par la figure ci-dessous et dont l'équation du modèle mathématique est la suivante :

Réaliser un modèle du circuit sous Simulink avec *vi* la variable d'entrée et *vo* et *iL* les variables de sortie.

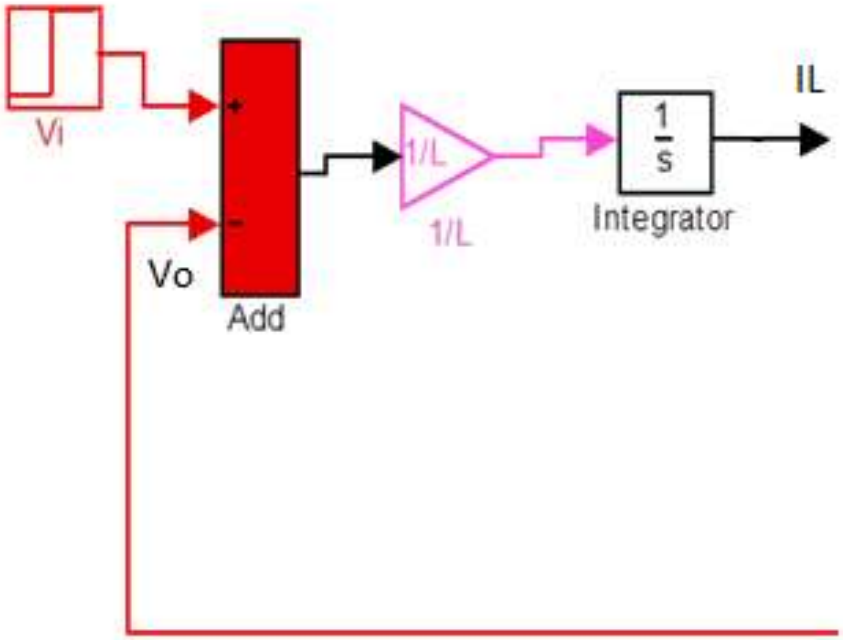


$$v_i(t) = v_o(t) + L\frac{di_L}{dt} \Rightarrow i_L(t) = \frac{1}{L}\int (v_i(t) - v_o(t))\,\mathrm{dt}$$

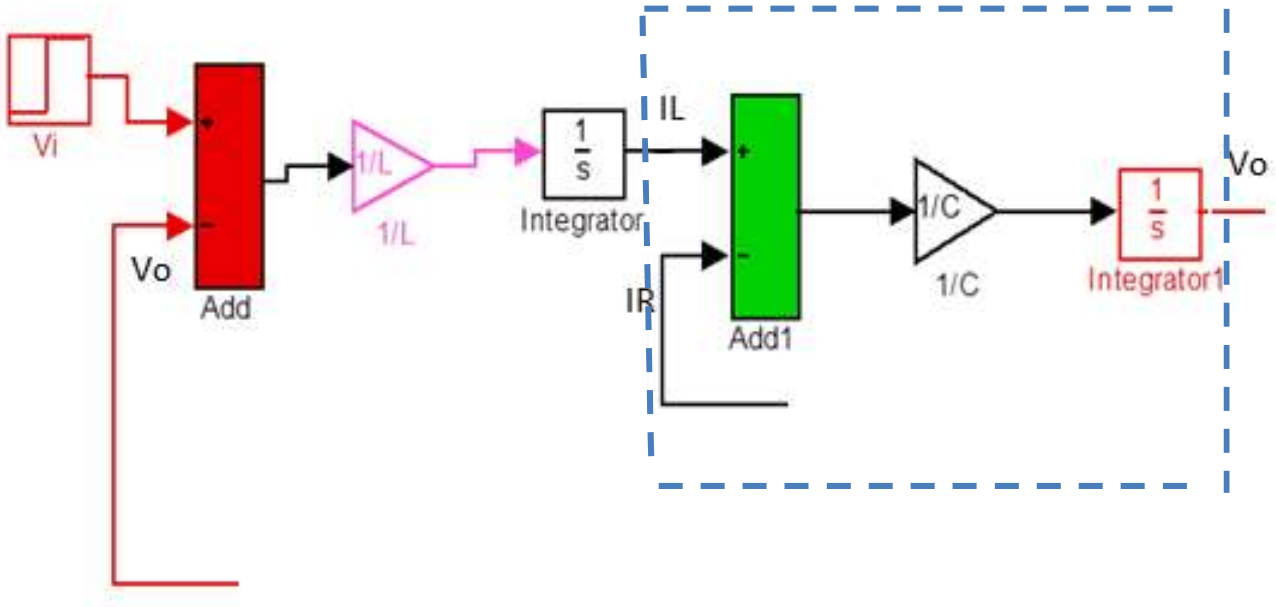
$$i_L(t) = i_R(t) + \mathrm{C}\frac{dv_o}{dt} \Rightarrow v_o(t) = \frac{1}{C}\int (i_L(t) - i_R(t))\,\mathrm{dt}$$

$$i_R(t) = \frac{v_o(t)}{R}$$

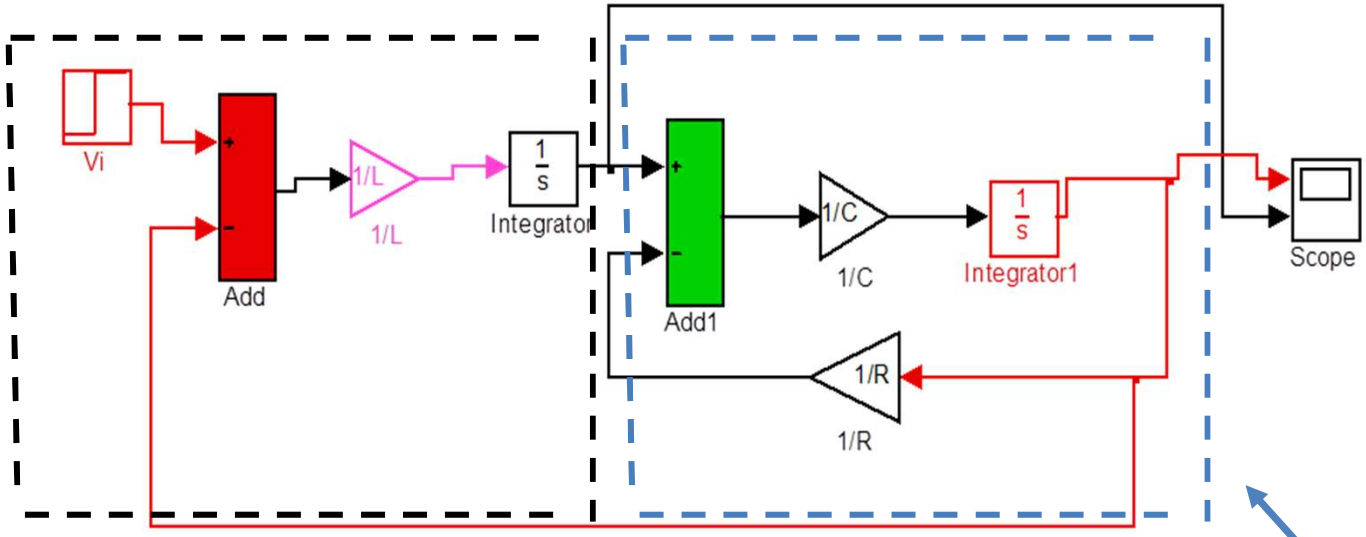
$$v_i(t) = v_o(t) + L\frac{di_L}{dt} \Rightarrow i_L(t) = \frac{1}{L}\int (v_i(t) - v_o(t))\,\mathrm{dt}$$



$$i_L(t) = i_R(t) + \mathrm{C}\frac{dv_o}{dt} \Rightarrow v_o(t) = \frac{1}{C}\int (i_L(t) - i_R(t))\,\mathrm{dt}$$



$$v_i(t) = v_o(t) + L\frac{di_L}{dt} \Rightarrow i_L(t) = \frac{1}{L}\int (v_i(t) - v_o(t))\,\mathrm{dt}$$



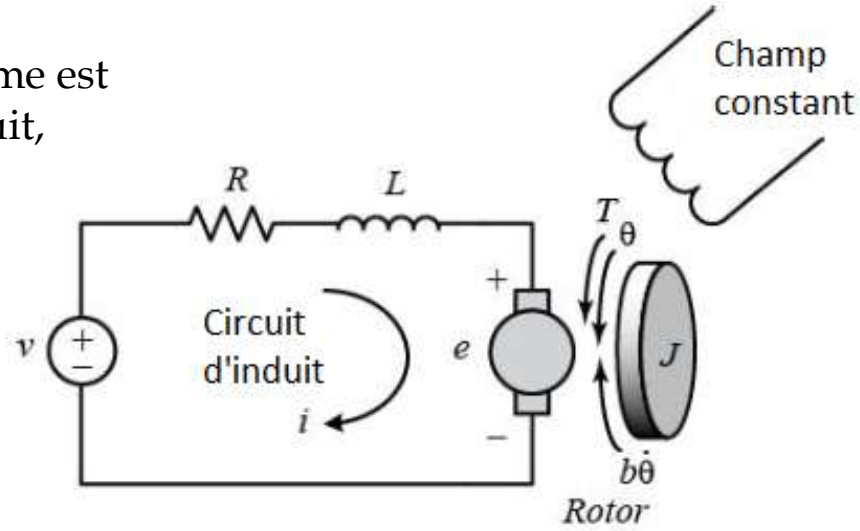
$$i_L(t) = i_R(t) + \mathrm{C}\frac{dv_o}{dt} \Rightarrow v_o(t) = \frac{1}{C}\int (i_L(t) - i_R(t))\,\mathrm{dt}$$

3. Modélisation d'un moteur à courant continu

L'actionneur le plus connu dans la commande des systèmes est le moteur à courant continu.

Pour cet exemple, on assume que l'entrée du système est la source de tension (V) appliquée au circuit d'induit, alors que la sortie est la vitesse de

rotation de l'arbre $\frac{d\theta}{dt}$.



L'arbre et le rotor sont supposés rigide et le modèle des frottements régit par le couple de frottement est proportionnel à la vitesse angulaire de l'arbre.

Les paramètres physiques pour notre exemple sont :

(J) moment d'inertie du rotor 0.01 kg.m^2
(b) constante de frottement 0.1 N.m.s
(Ke) constante de force electromotrice 0.01 V/rad/sec
(Kt) constante du couple moteur 0.01 N.m/A
(R) résistance electrique 1 Ohm
(L) inductance 0.5 H

En général le couple généré par le moteur à courant continu est proportionnel au courant d'induit et la force du champ magnétique d'excitation. Dans cet exemple, on va considérer que le champ magnétique d'excitation est constant, ainsi le couple du moteur est proportionnel seulement au courant d'induit par une constante Kt.

$$T = K_t i \qquad (1)$$

La force contre électromotrice, e, est proportionnelle à la vitesse angulaire de l'arbre par une constante de temps Ke.

$$e = K_e \dot{\theta} \qquad (2)$$

En système SI, le couple du moteur et la force contre électromotrice sont égaux, Kt = Ke; Cependant, on va utiliser dans la suite du travail K qui représentera les deux constantes celle du couple du moteur et celle de la force contre électromotrice.

En appliquant la loi de Newton et la loi de Kirchoff au système du moteur, on a les équations suivantes :

Equation mécanique

$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow \frac{d^2\theta}{dt^2} = \frac{1}{J}\left(K_t i - b\frac{d\theta}{dt}\right) \quad (3)$$

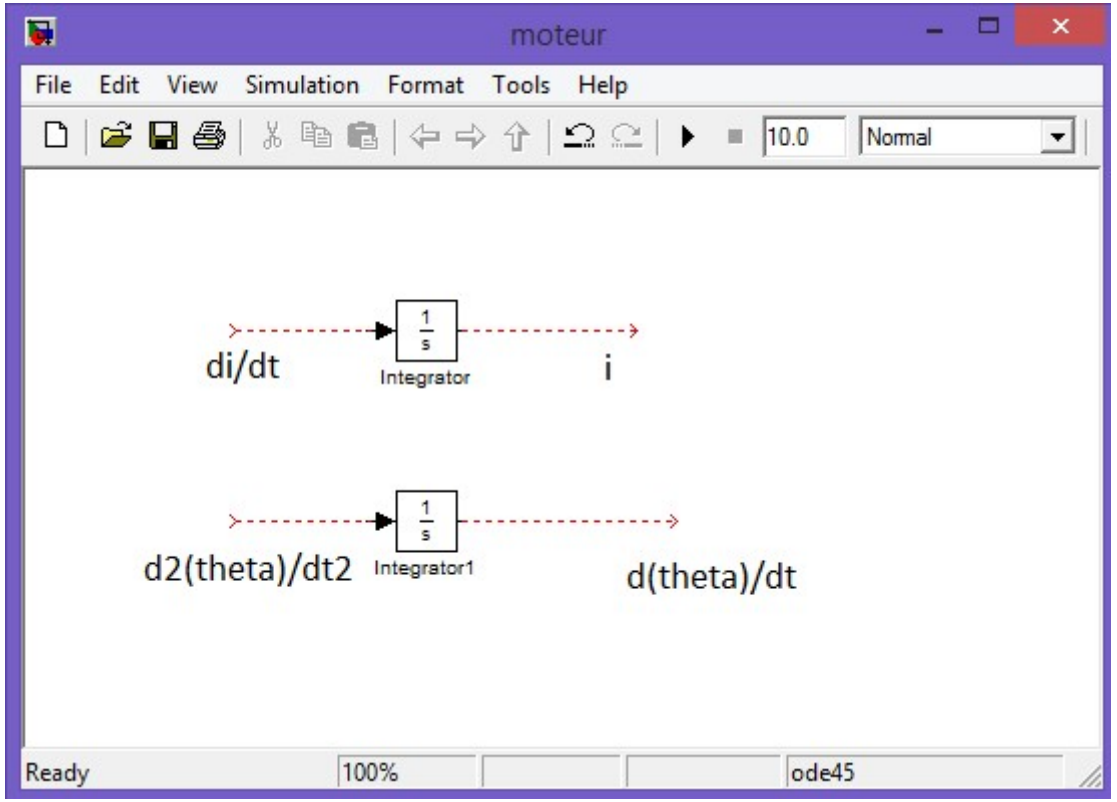
Equation électrique

$$L\frac{di}{dt} = -Ri + V - e \Rightarrow \frac{di}{dt} = \frac{1}{L}\left(-Ri + V - K_e\frac{d\theta}{dt}\right) \quad (4)$$

Ce système va être modéliser en assumant que le couple agit sur l'inertie du rotor et en intégrant l'accéleration pour obtenir la vitesse.

$$\int \frac{d^2\theta}{dt^2}\,dt = \frac{d\theta}{dt} \qquad (5)$$

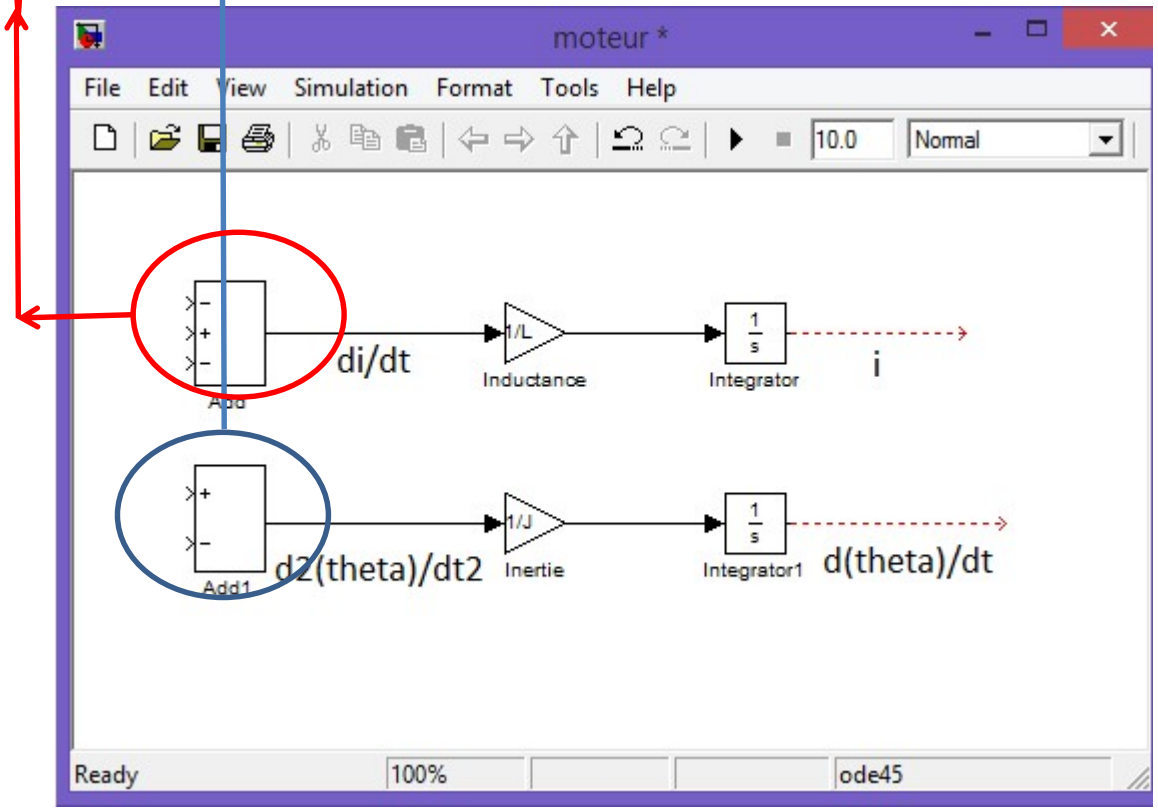
$$\int \frac{di}{dt}\,dt = i \qquad (6)$$



$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow \frac{d^2\theta}{dt^2} = \frac{1}{J}\left(K_t i - b\frac{d\theta}{dt}\right) \qquad (3)$$

$$L\frac{di}{dt} = -Ri + V - e \Rightarrow \frac{di}{dt} = \frac{1}{L}\left(-Ri + V - K_e\frac{d\theta}{dt}\right) \qquad (4)$$

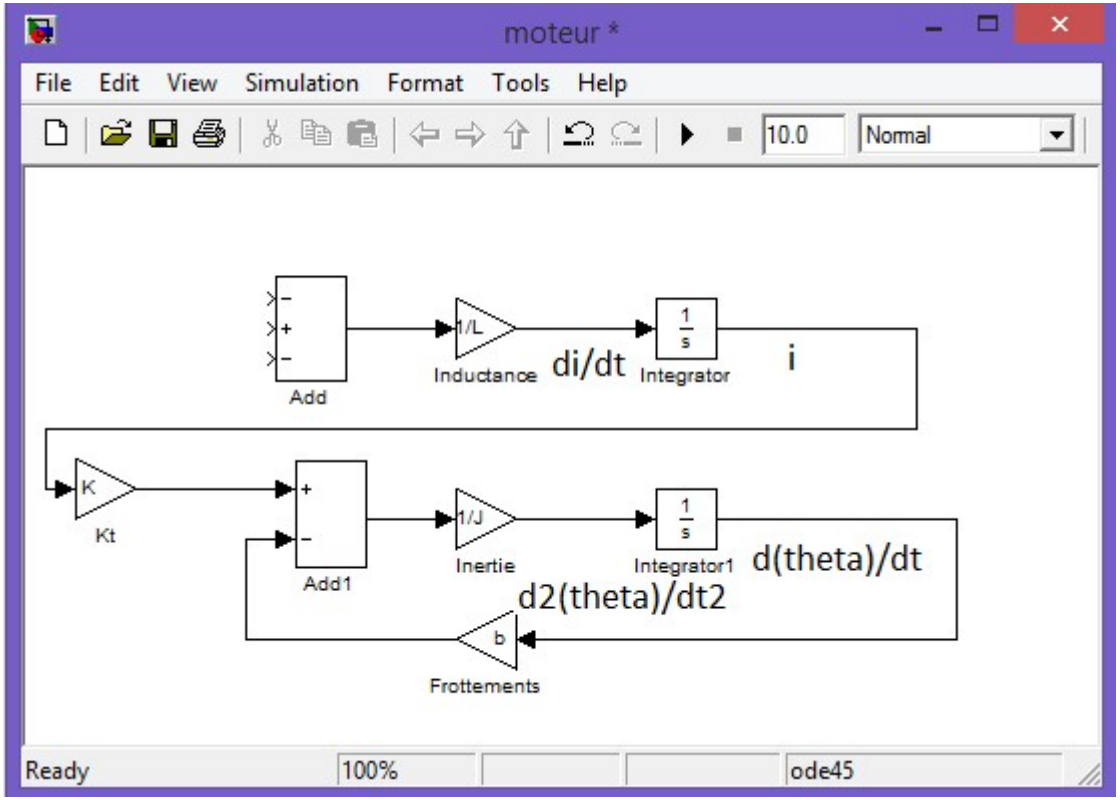
L'accélération angulaire est égale à 1 / J multiplié par la somme de deux termes (un positif et l'autre négatif).
Similaire la dérivée du courant est égale à 1 / L multiplié par la somme de trois termes (un positif et deux négatifs).



$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow \frac{d^2\theta}{dt^2} = \frac{1}{J}\left(K_t i - b\frac{d\theta}{dt}\right) \qquad (3)$$

$$L\frac{di}{dt} = -Ri + V - e \Rightarrow \frac{di}{dt} = \frac{1}{L}\left(-Ri + V - K_e\frac{d\theta}{dt}\right) \qquad (4)$$

On ajoute le bloc de l'inductance qui est le bloc Gain qu'on a renommé en Inductance et un autre bloc Gain en Inertie.
les blocs du couple Kt et des frottements b sont ajoutés au additionneur Add1

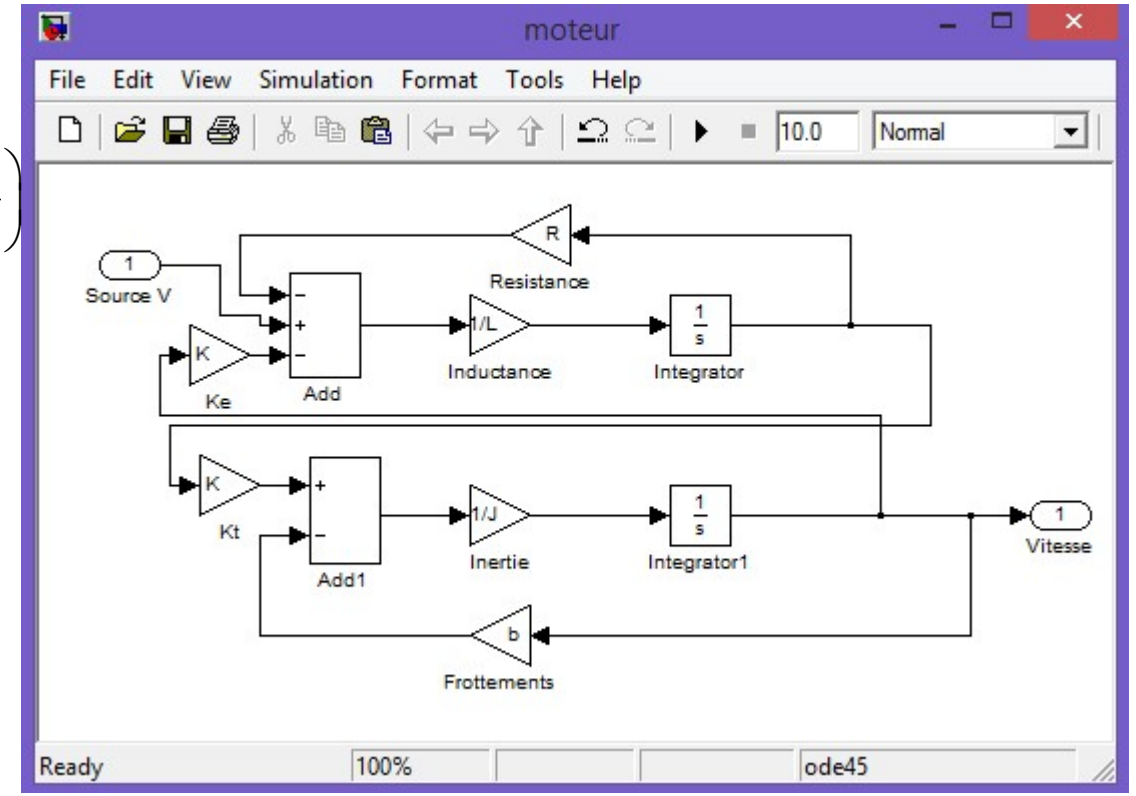


Maintenant, on va ajouter le terme de la source de tension qui est représentée dans l'équation électrique. En premier, on ajoute la chute de tension causée par la résistance.

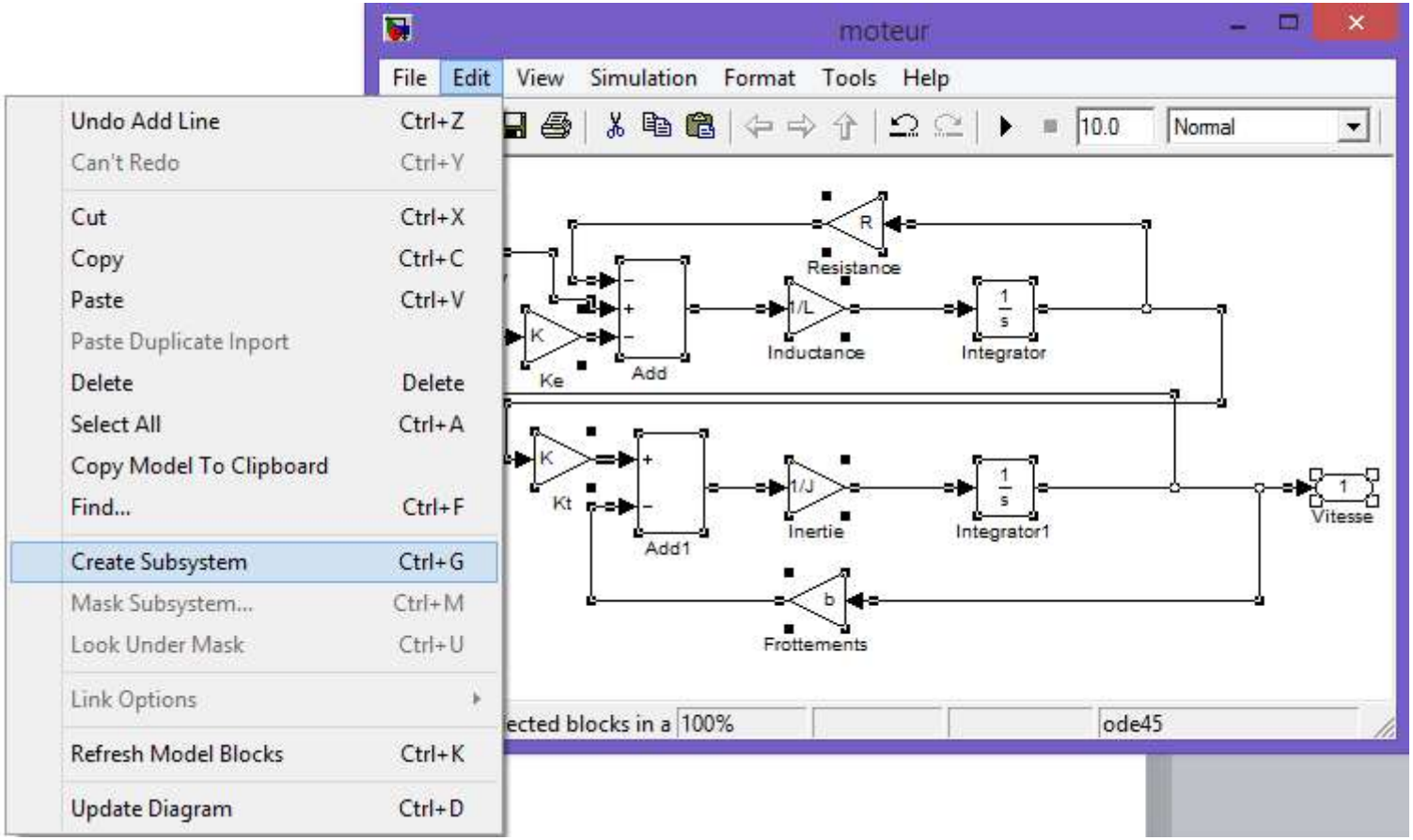
On ajoute les blocs In1 et Out1 de la library Simulink/Ports & Subsystems et respectivement on les renomme "Source V" et "Vitesse".

$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow \frac{d^2\theta}{dt^2} = \frac{1}{J}\left(K_t i - b\frac{d\theta}{dt}\right)$$

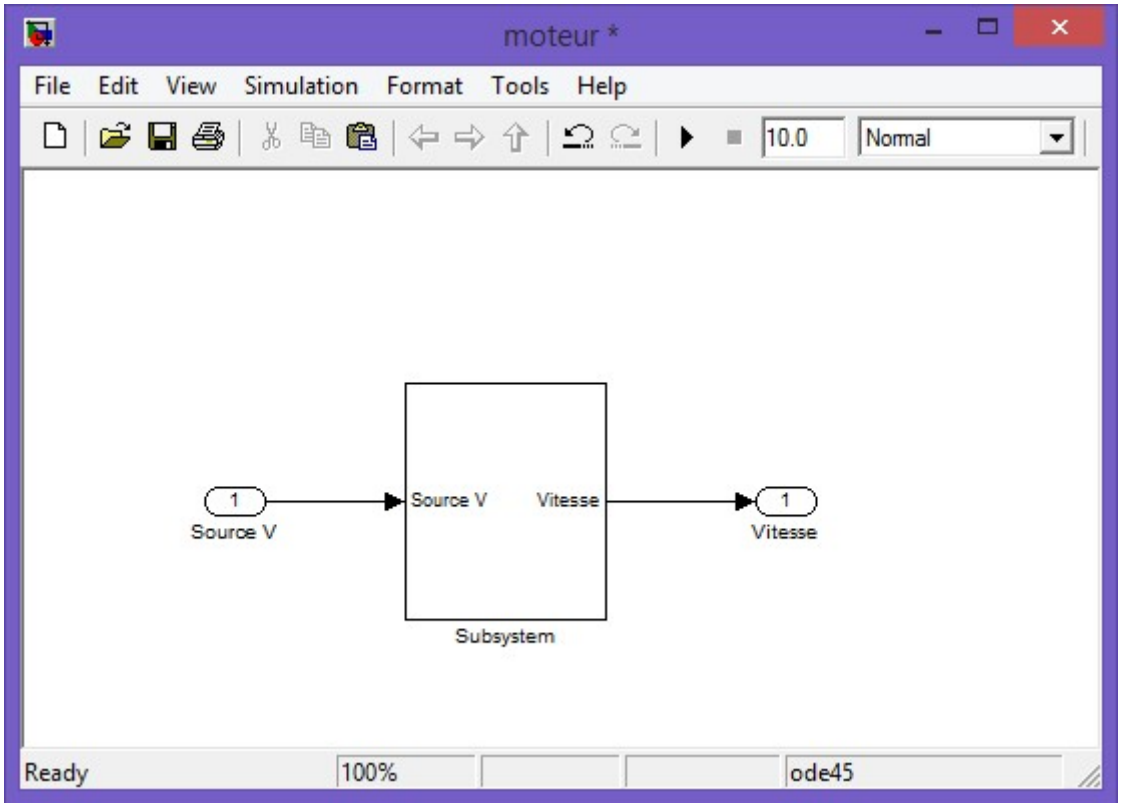
$$L\frac{di}{dt} = -Ri + V - e \Rightarrow \frac{di}{dt} = \frac{1}{L}\left(-Ri + V - K_e\frac{d\theta}{dt}\right)$$



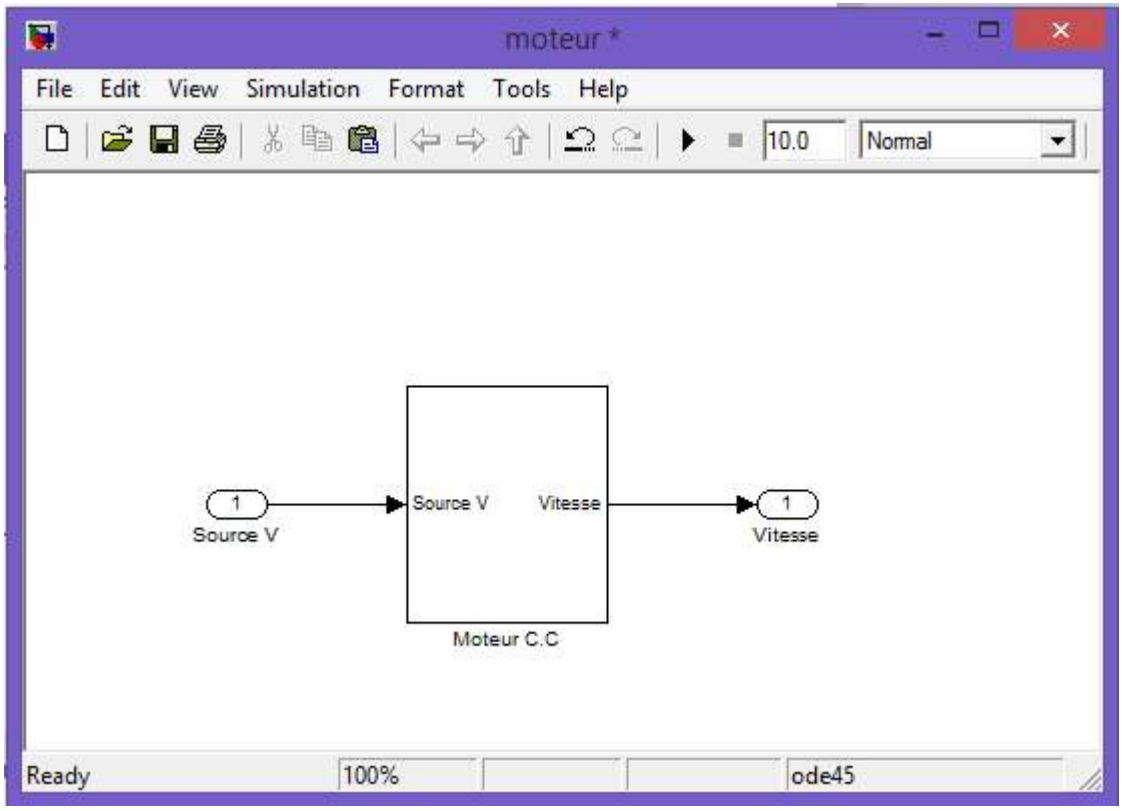
Maintenant, on va sélectionner le modèle pour créer un Subsystem.



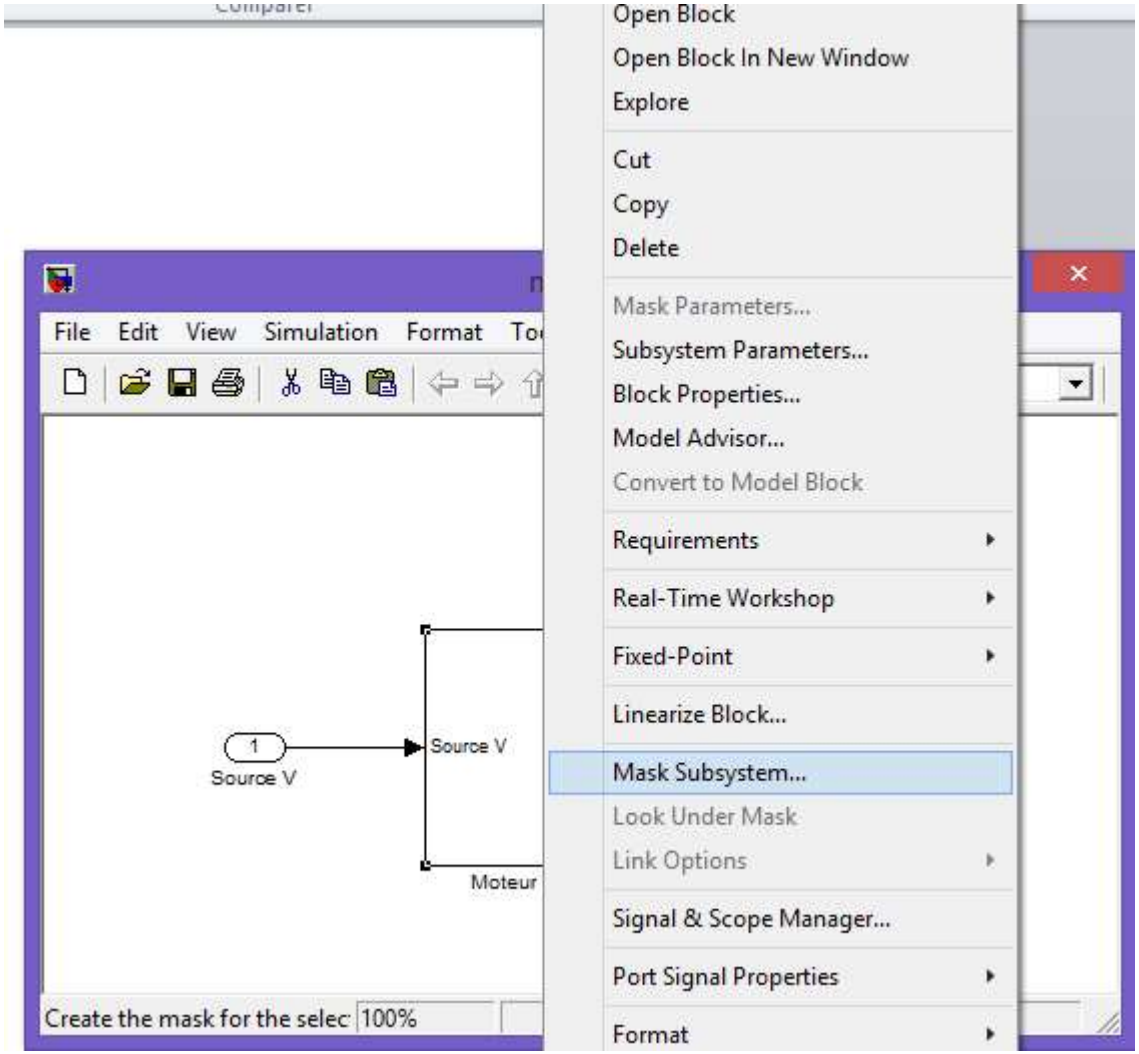
Ainsi le Subsystem a été créé.



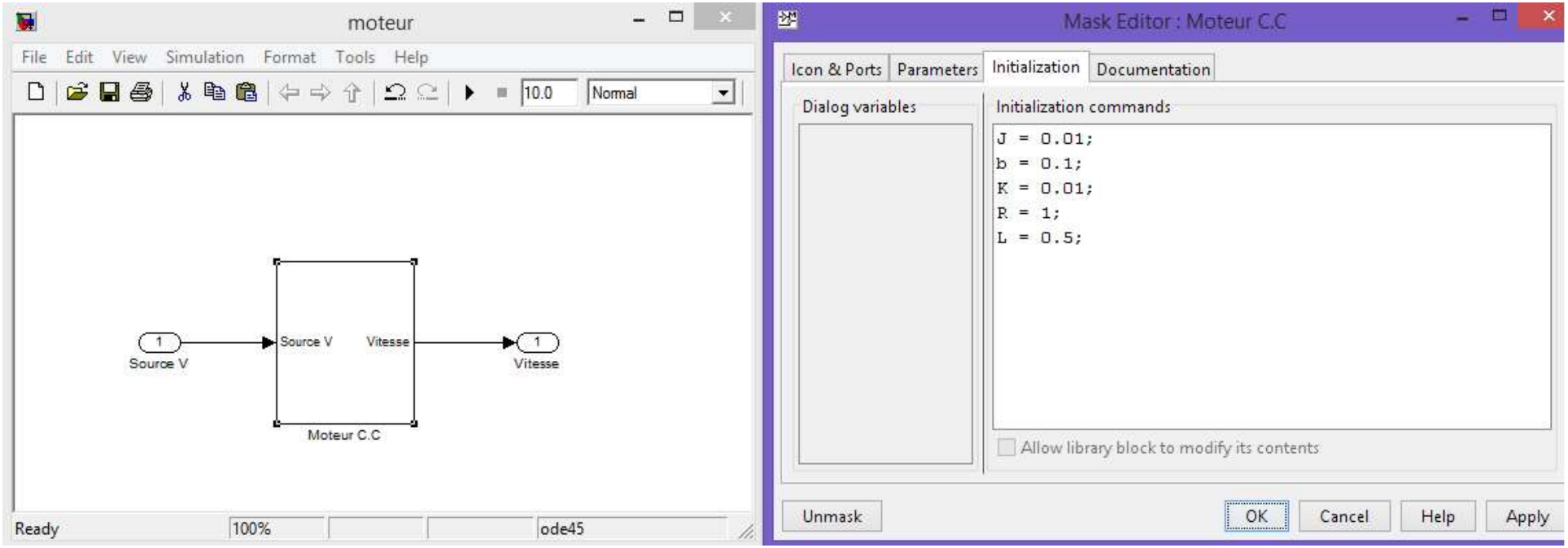
On peut renommer le Subsystem en Moteur C.C.



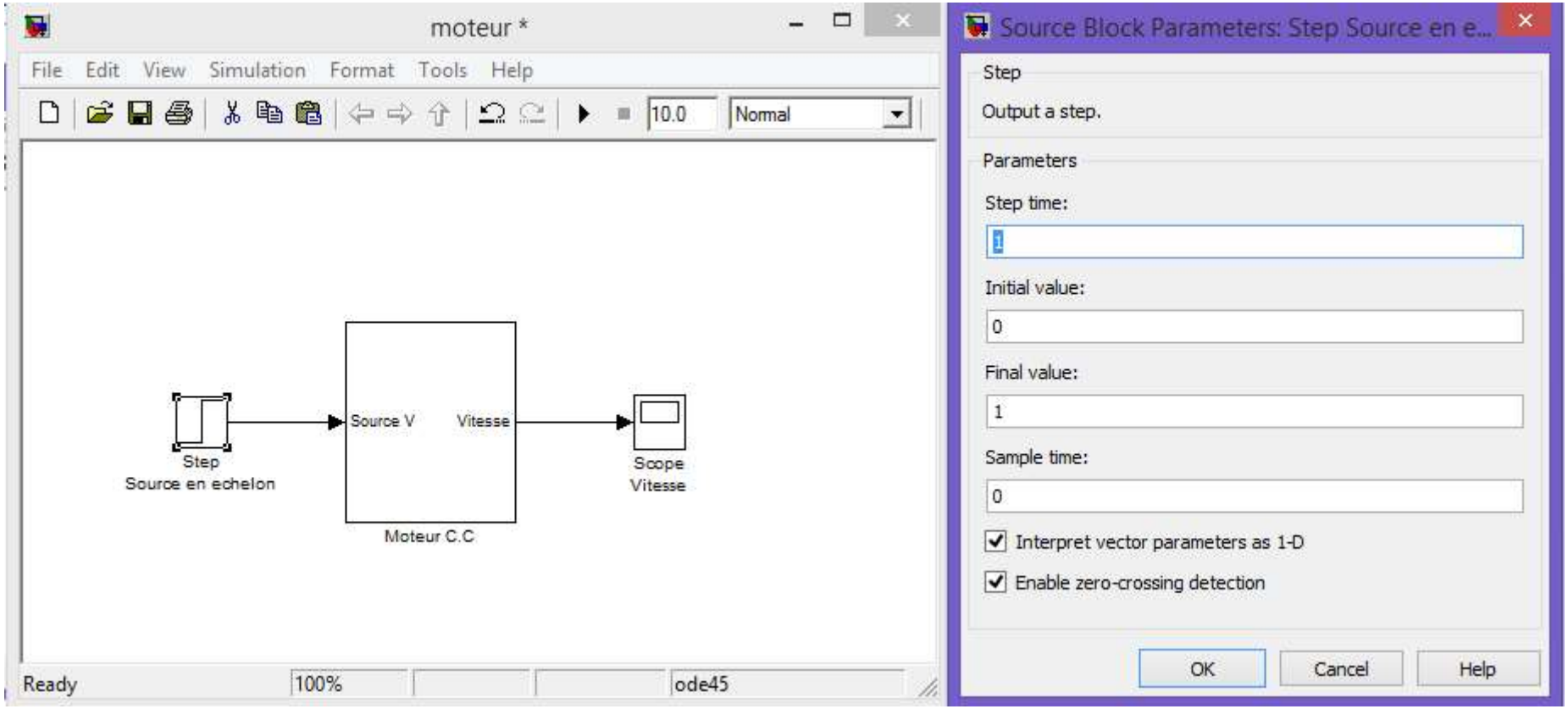
Le Subsystem en Moteur C.C peut être masquer pour introduire dans le modèle les valeurs des paramètres électriques et mécaniques.



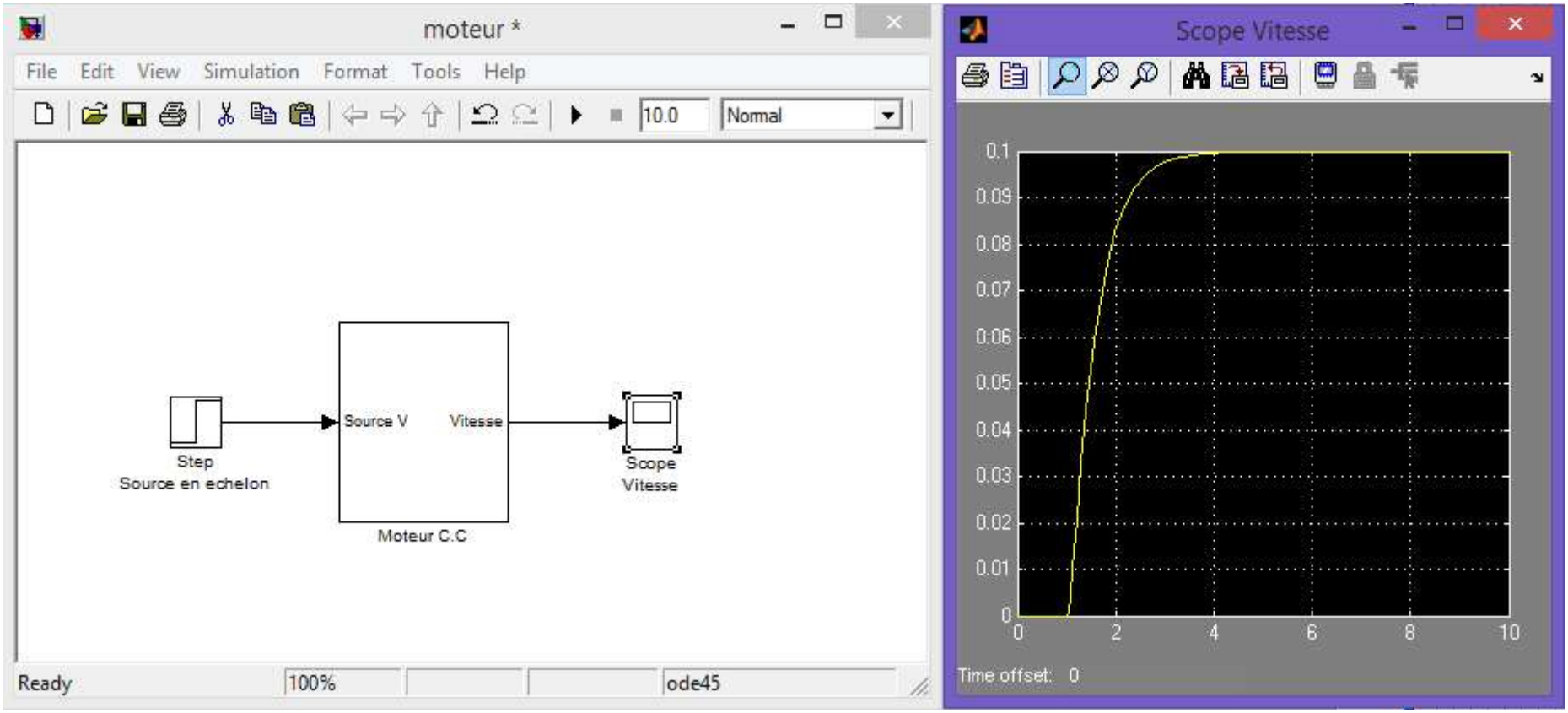
La fenêtre Mask Editor apparaît les paramètres sont copiés dans Initialisation.
Ainsi les valeurs des paramètres sont définit à l'intérieur du modèle.



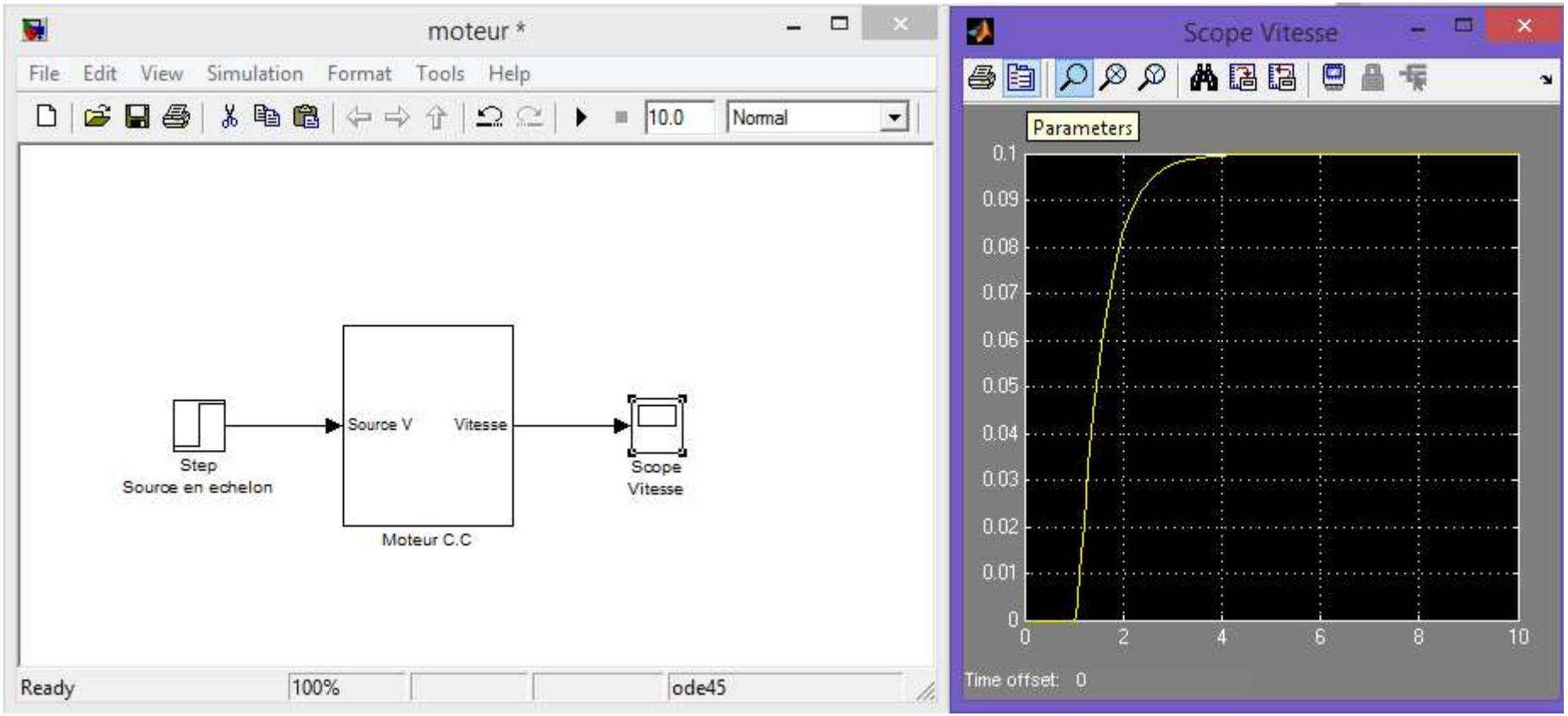
Essayant maintenant de simuler notre modèle, pour cela ajoutant un échelon à l'entrée et un oscilloscope à la sortie.



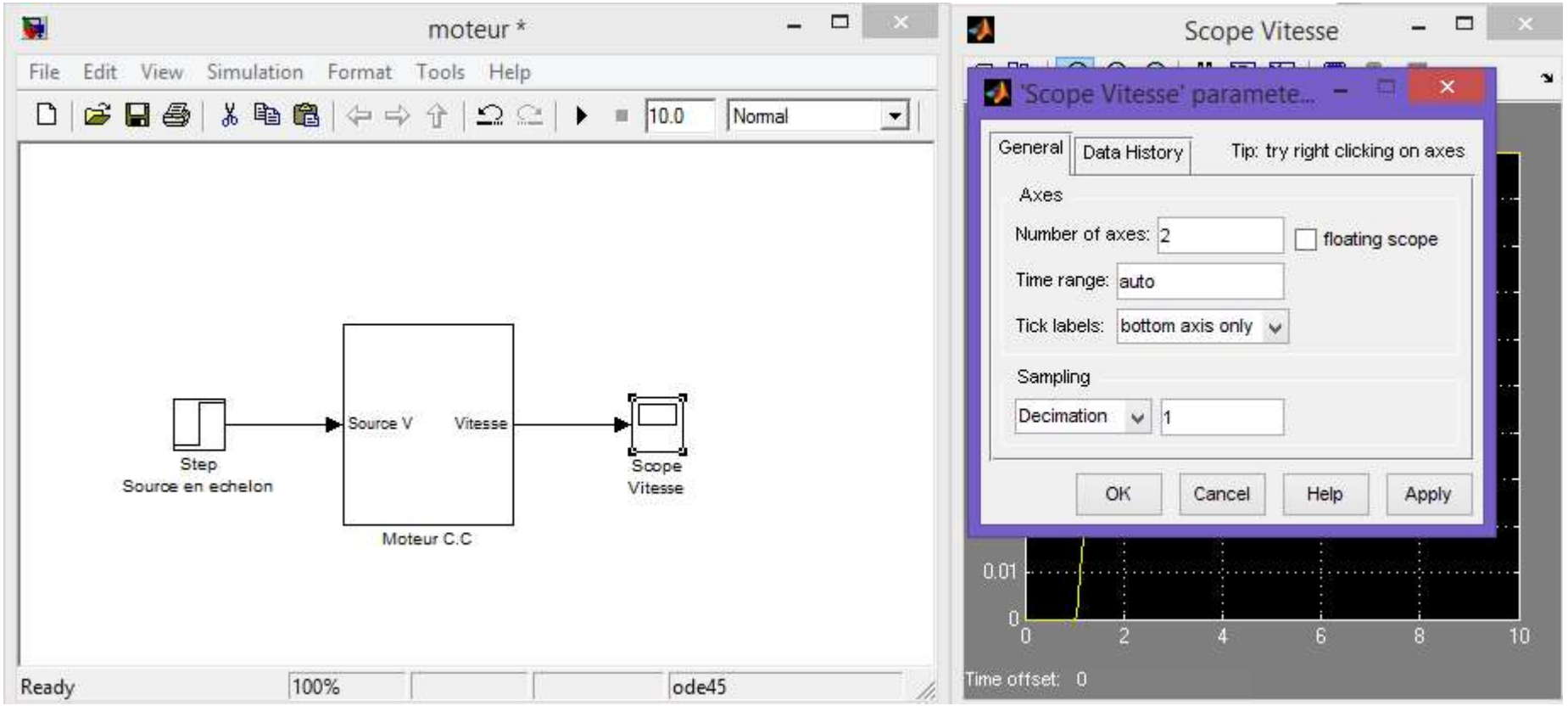
Après simulation du système et double click sur Scope, on remarque l'allure de la vitesse.



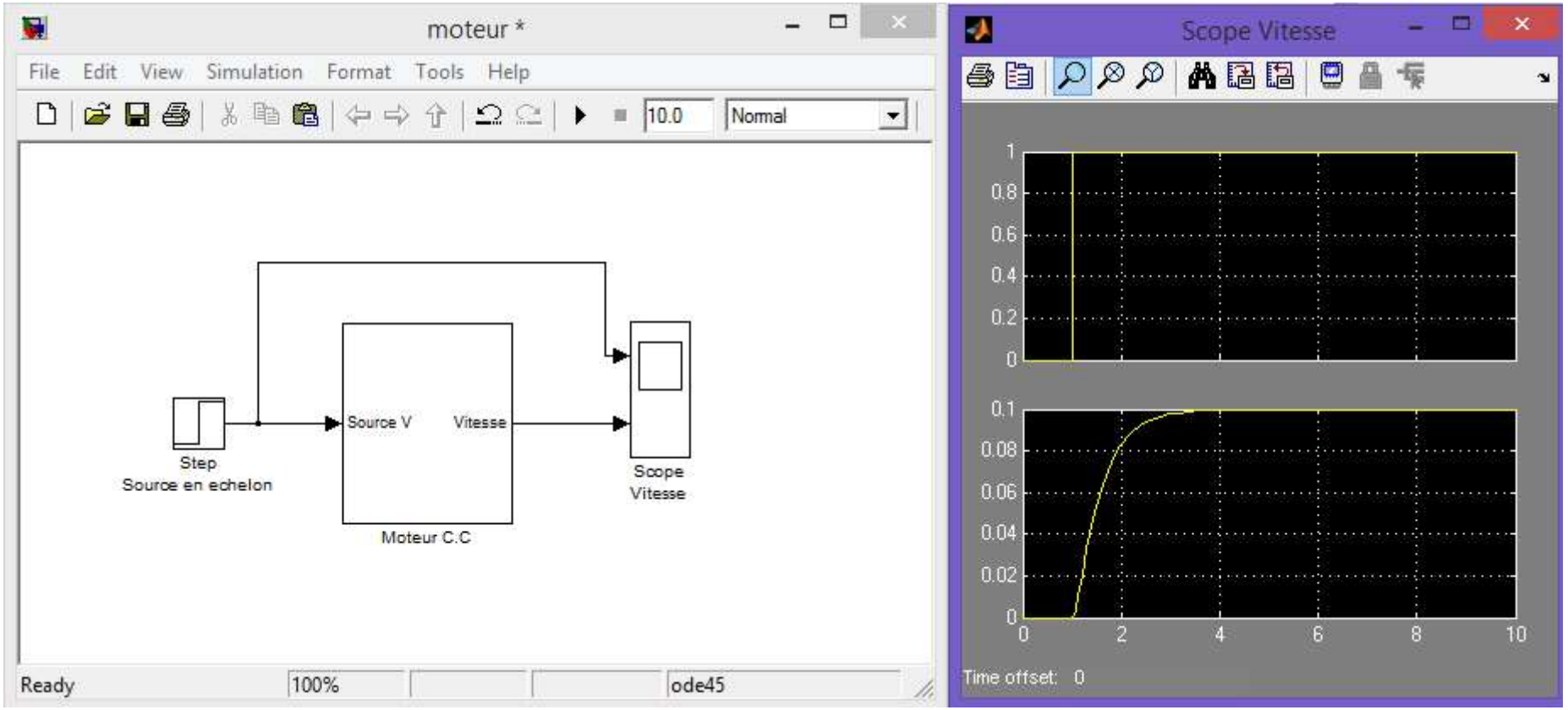
On ouvre les paramètres du Scope en cliquant sur le deuxième bouton qui est parameters.



Mettant dans la case Number of axes le nombre 2.



On peut ainsi visualiser l'entrée sur le premier axe et la sortie sur deuxième axe.



Merci

pour

votre Attention

Le logiciel de simulation Simulink
Partie 3
Exemples de modélisation

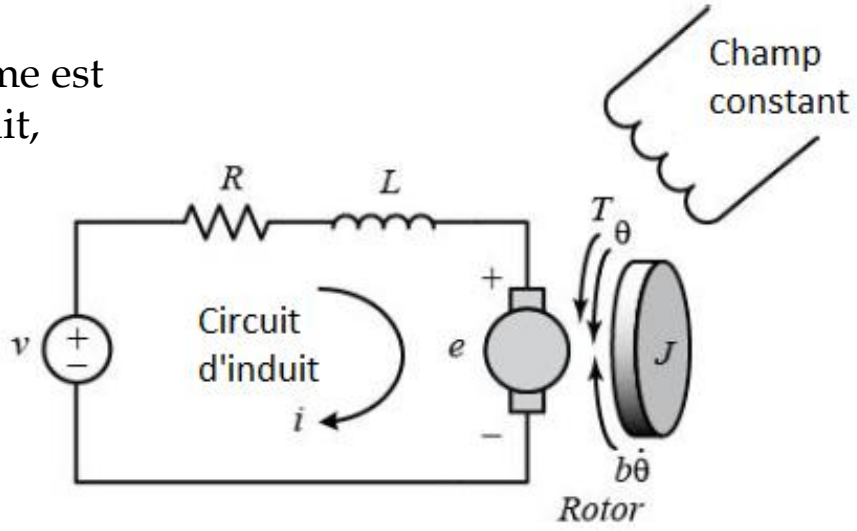
Professeur Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Modélisation d'un moteur à courant continu en espace d'état

L'actionneur le plus connu dans la commande des systèmes est le moteur à courant continu.

Pour cet exemple, on assume que l'entrée du système est la source de tension (V) appliquée au circuit d'induit, alors que la sortie est la vitesse de

rotation de l'arbre $\frac{d\theta}{dt}$.



L'arbre et le rotor sont supposés rigide et le modèle des frottements régit par le couple de frottement est proportionnel à la vitesse angulaire de l'arbre.

Les paramètres physiques pour notre exemple sont :

(J) moment d'inertie du rotor 0.01 kg.m^2
(b) constante de frottement 0.1 N.m.s
(Ke) constante de force electromotrice 0.01 V/rad/sec
(Kt) constante du couple moteur 0.01 N.m/A
(R) résistance electrique 1 Ohm
(L) inductance 0.5 H

En général le couple généré par le moteur à courant continu est proportionnel au courant d'induit et la force du champ magnétique d'excitation. Dans cet exemple, on va considérer que le champ magnétique d'excitation est constant, ainsi le couple du moteur est proportionnel seulement au courant d'induit par une constante Kt.

$$T = K_t i \qquad (1)$$

La force contre électromotrice, e, est proportionnelle à la vitesse angulaire de l'arbre par une constante de temps Ke.

$$e = K_e \dot{\theta} \qquad (2)$$

En système SI, le couple du moteur et la force contre électromotrice sont égaux, Kt = Ke; Cependant, on va utiliser dans la suite du travail K qui représentera les deux constantes celle du couple du moteur et celle de la force contre électromotrice.

En appliquant la loi de Newton et la loi de Kirchoff au système du moteur, on a les équations suivantes :

Equation mécanique

$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow \frac{d^2\theta}{dt^2} = \frac{1}{J}\left(K_t i - b\frac{d\theta}{dt}\right) \quad (3)$$

Equation électrique

$$L\frac{di}{dt} = -Ri + V - e \Rightarrow \frac{di}{dt} = \frac{1}{L}\left(-Ri + V - K_e\frac{d\theta}{dt}\right) \quad (4)$$

$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow \frac{d^2\theta}{dt^2} = \frac{1}{J}\left(K_t i - b\frac{d\theta}{dt}\right) \qquad (5)$$

$$L\frac{di}{dt} = -Ri + V - e \Rightarrow \frac{di}{dt} = \frac{1}{L}\left(-Ri + V - K_e\frac{d\theta}{dt}\right) \qquad (6)$$

$$\frac{d}{dt}\begin{bmatrix}\dot{\theta}\\ i\end{bmatrix} = \begin{bmatrix}-\frac{b}{J} & \frac{K}{J}\\ -\frac{K}{L} & -\frac{R}{L}\end{bmatrix}\begin{bmatrix}\dot{\theta}\\ i\end{bmatrix} + \begin{bmatrix}0\\ \frac{1}{L}\end{bmatrix}V \qquad (7)$$

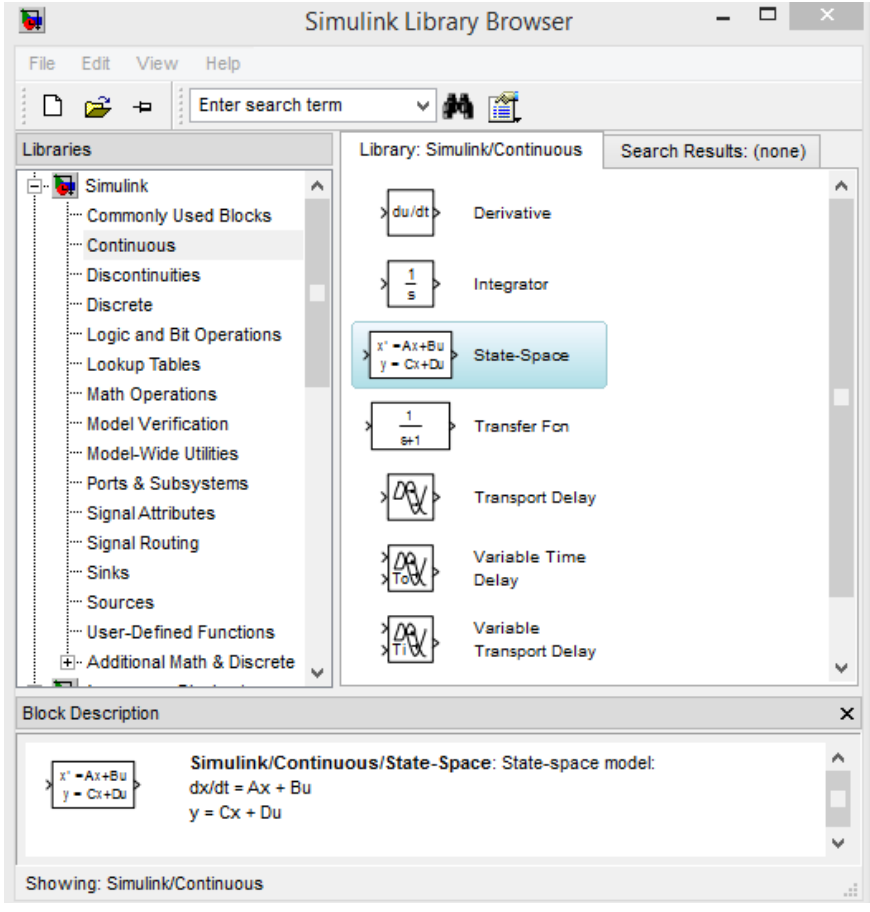
$$y = \begin{bmatrix}1 & 0\end{bmatrix}\begin{bmatrix}\dot{\theta}\\ i\end{bmatrix} \qquad (8)$$

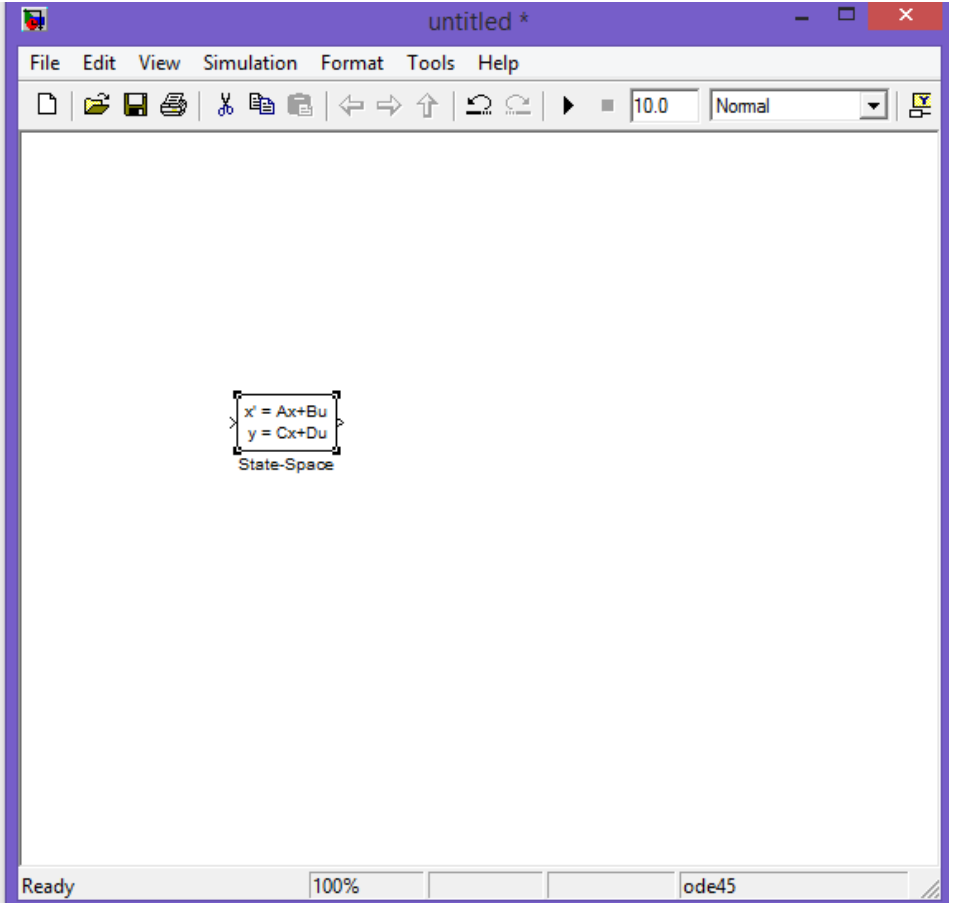
En espace d'état le modèle devient sous la forme :

$$\dot{x} = Ax + Bu$$
$$y = Cx + Du$$

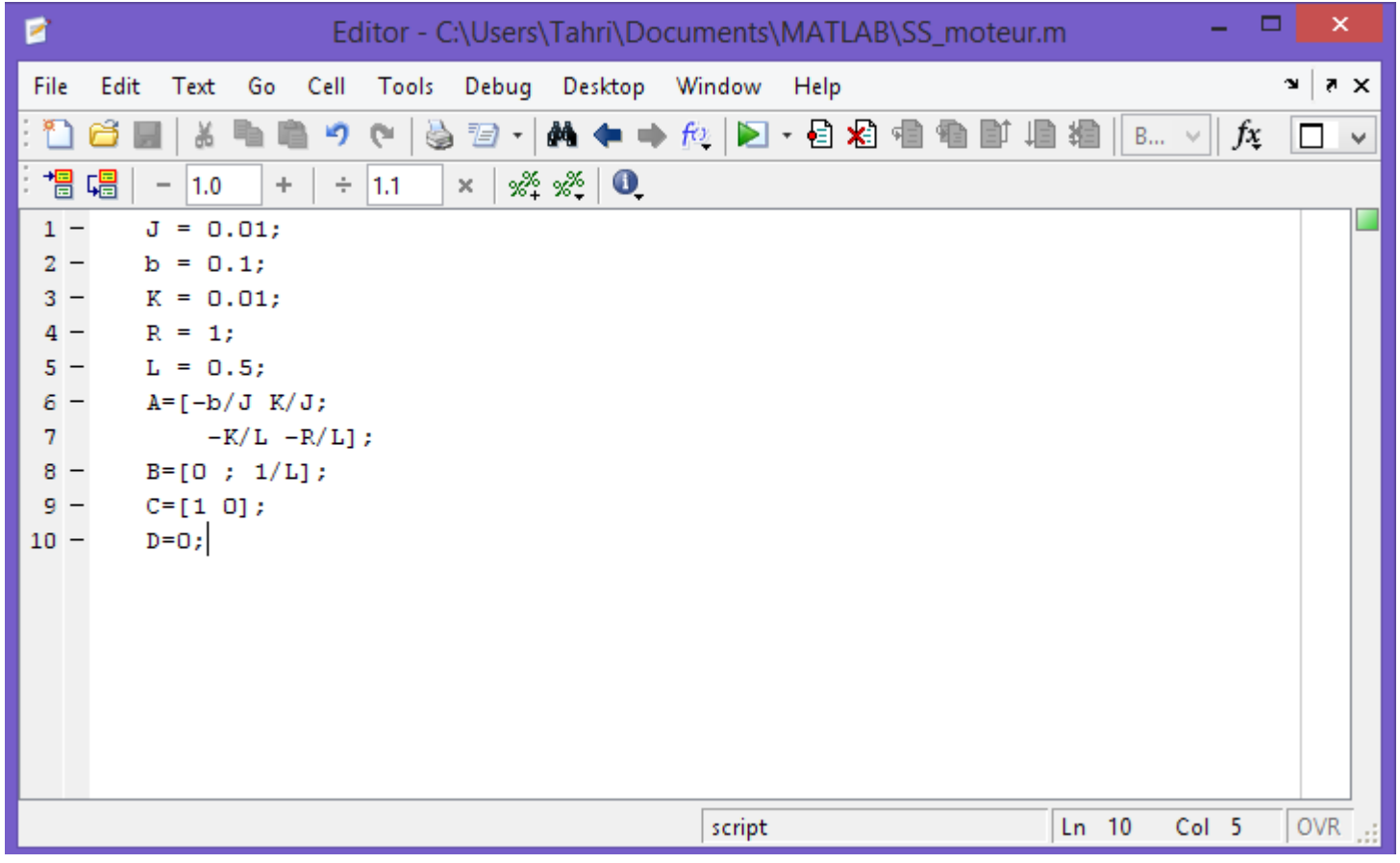
Avec :
A matrice d'état
B matrice ou vecteur de commande
C matrice ou vecteur d'observation

Dans la librairie continous, on utilise le bloc State-Space (espace d'état)

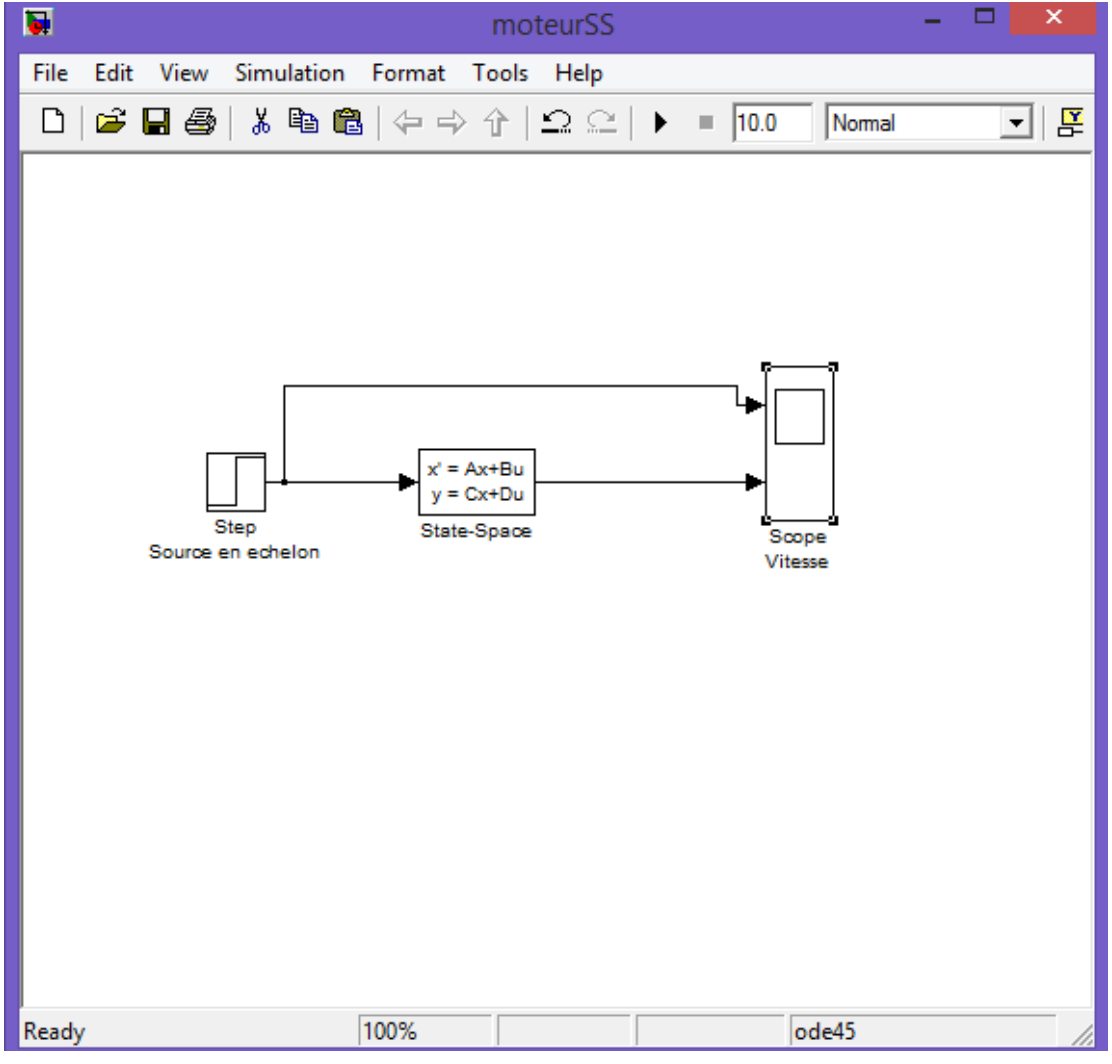




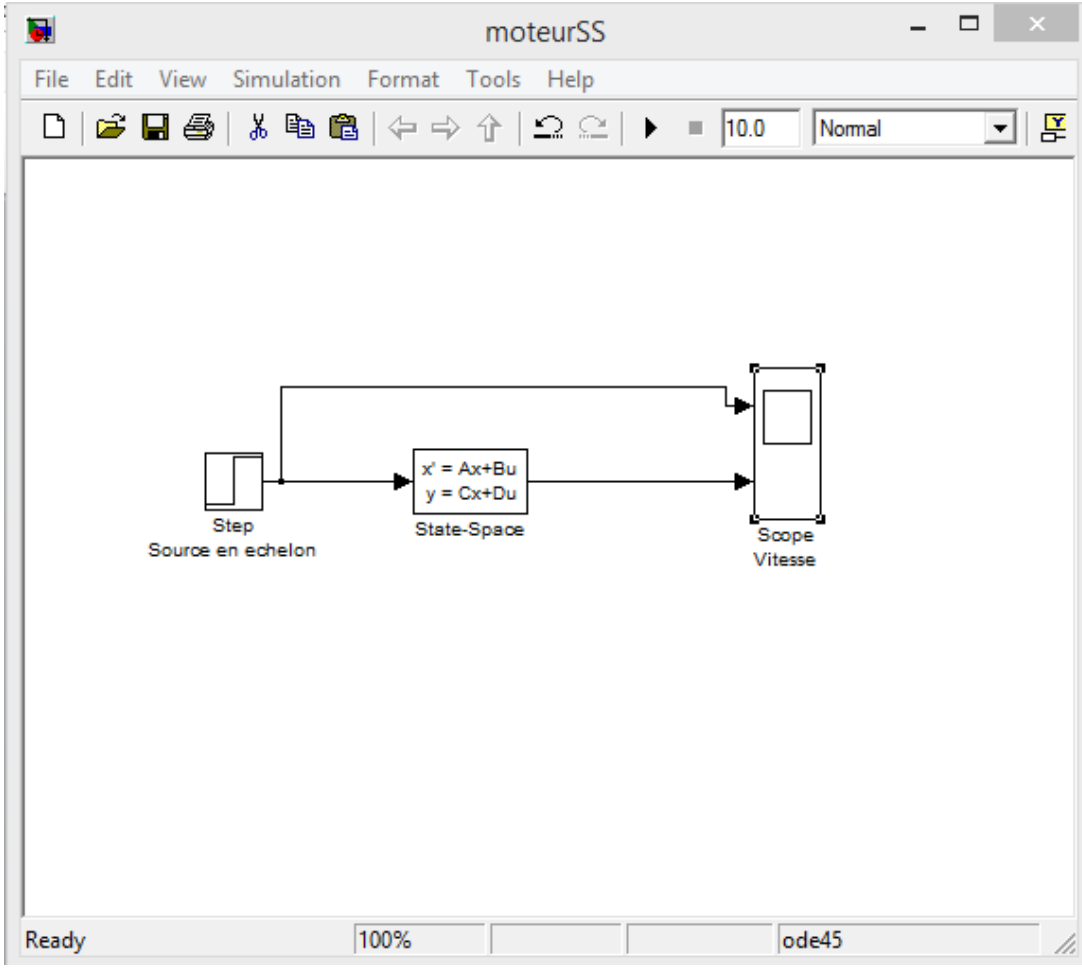
Maintenant, on va écrire les paramètres et le modèle en espace d'état du moteur dans un fichier SS_moteur.m.
Ensuite, on exécute ce programme afin que ses variables soit définis dans la mémoire du workspace, pour que Simulink ait accès à ses variables

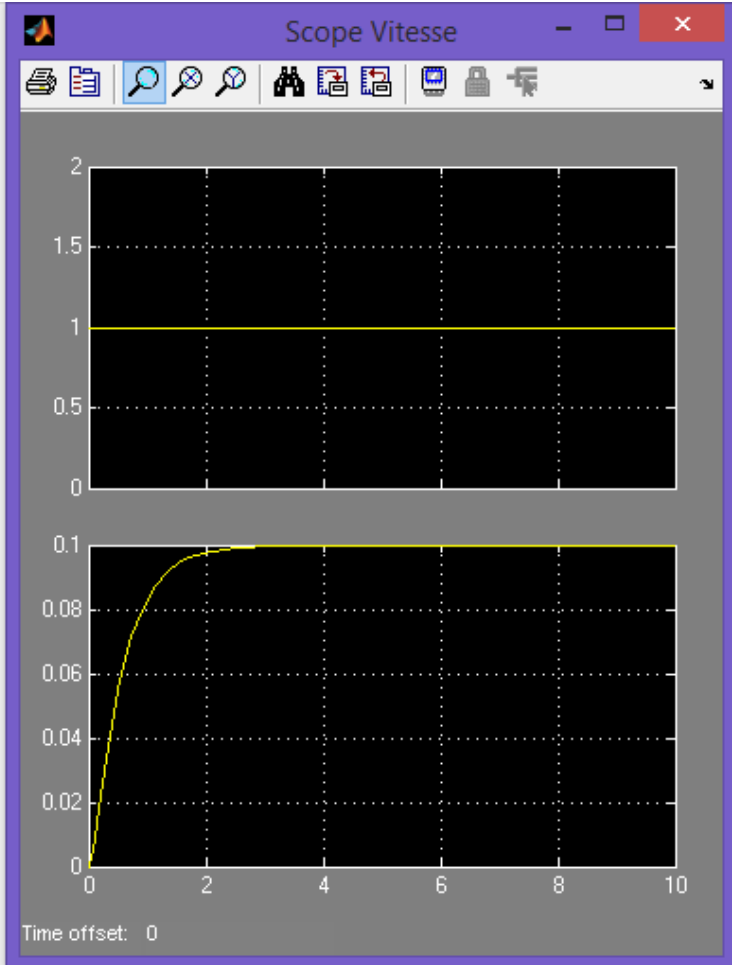


On ajoute une entrée échelon et un oscilloscope pour visualiser l'entrée et la sortie.

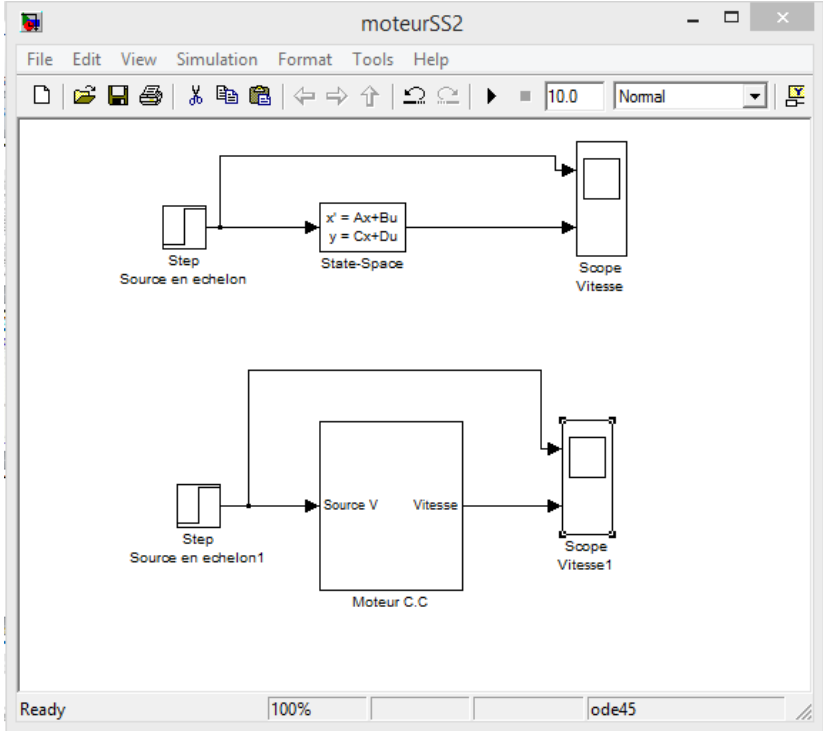


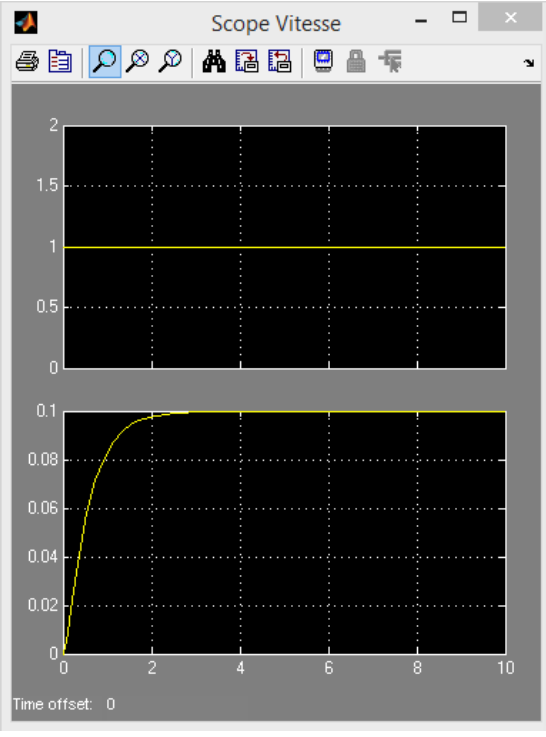
On exécute le modèle et on visualise les résultats sur l'oscilloscope.

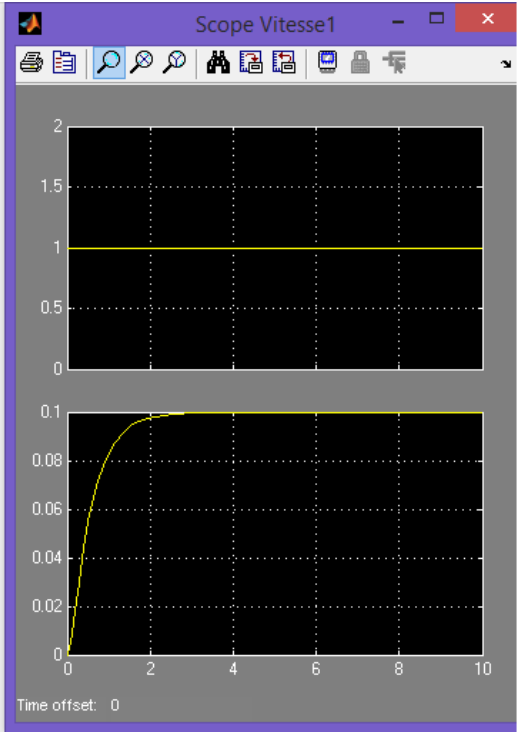




Essayant maintenant de visualiser les signaux des deux modèle celui à base de bloc fonctionnels et celui à base de bloc d'espace d'état







Mathématiquement c'est les mêmes modèles sauf la conception est différente

2. Modélisation en espace d'état avec la variable du courant comme sortie auxiliaire.

$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow \frac{d^2\theta}{dt^2} = \frac{1}{J}\left(K_t i - b\frac{d\theta}{dt}\right)$$

$$L\frac{di}{dt} = -Ri + V - e \Rightarrow \frac{di}{dt} = \frac{1}{L}\left(-Ri + V - K_e\frac{d\theta}{dt}\right)$$

$$\frac{d}{dt}\begin{bmatrix}\dot{\theta}\\ i\end{bmatrix} = \begin{bmatrix}-\frac{b}{J} & \frac{K}{J}\\ -\frac{K}{L} & -\frac{R}{L}\end{bmatrix}\begin{bmatrix}\dot{\theta}\\ i\end{bmatrix} + \begin{bmatrix}0\\ \frac{1}{L}\end{bmatrix}V$$

$$y = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}\dot{\theta}\\ i\end{bmatrix}$$

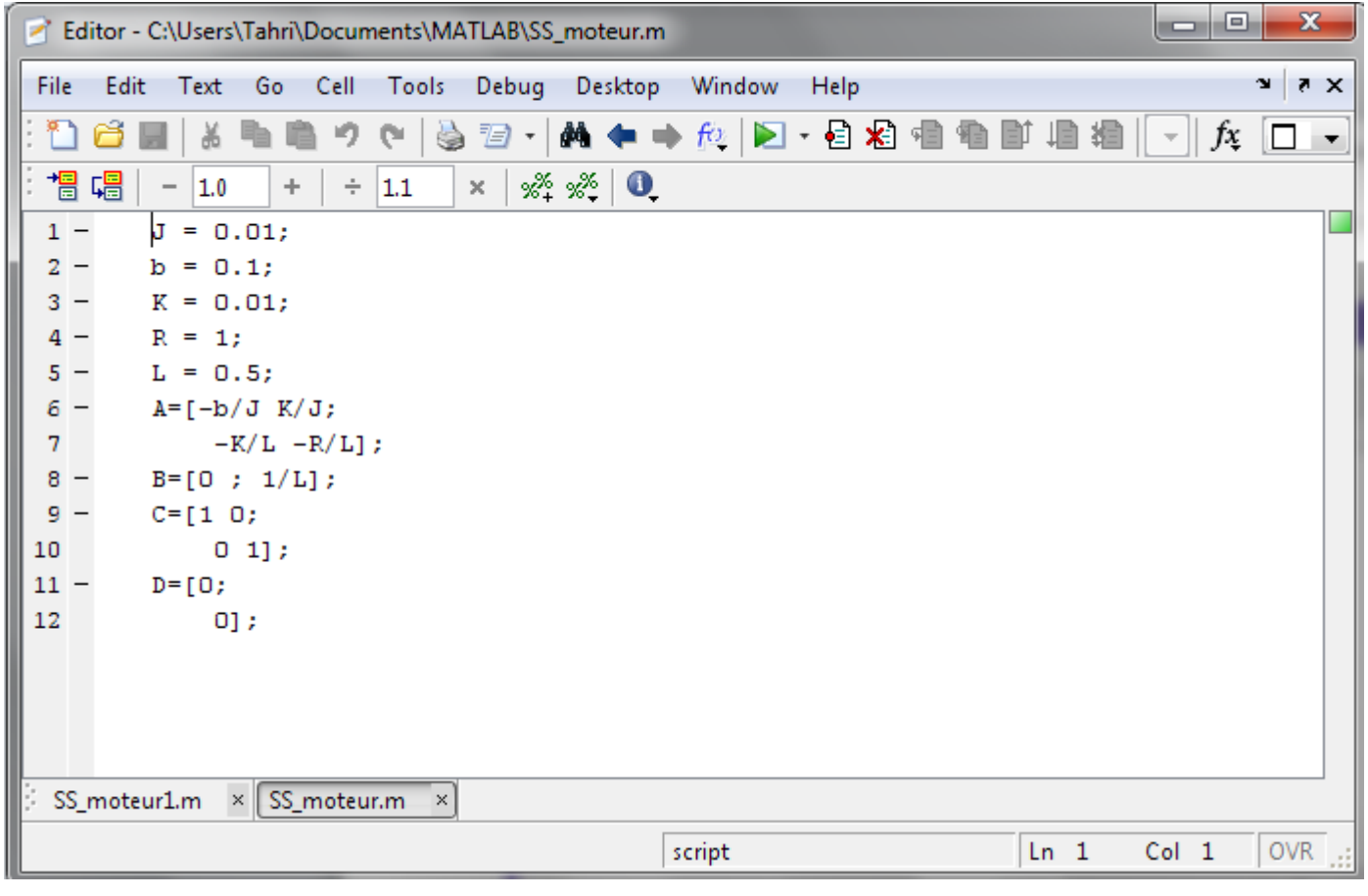
Le vecteur observation peut être réécrit de tel manière à avoir le courant d'induit comme deuxième variable de sortie

En espace d'état le modèle devient sous la forme :

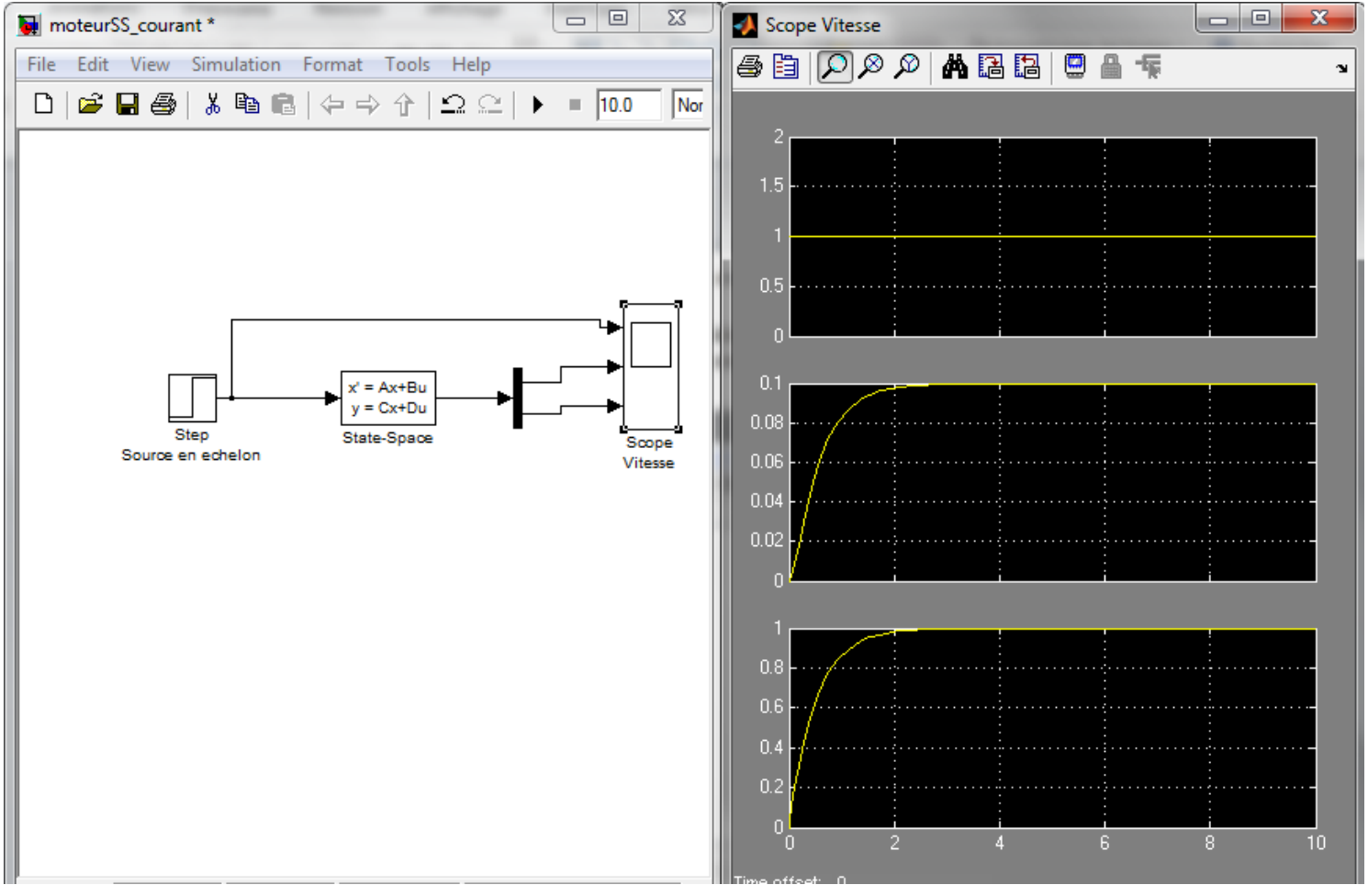
$$\dot{x} = Ax + Bu$$
$$y = Cx + Du$$

Avec :
A matrice d'état
B matrice ou vecteur de commande
C matrice ou vecteur d'observation

Maintenant, on va réécrire les paramètres et le modèle en espace d'état du moteur dans un fichier SS_moteur.m
avec la matrice C et D de tel façon à avoir la vitesse angulaire et le courant comme sorties.
Ensuite, on exécute ce programme afin que ses variables soit définis dans la mémoire du workspace, pour que Simulink ait accès à ses variables



On exécute le modèle et on visualise les résultats sur l'oscilloscope.
Un multiplexeur a été ajouté pour visualiser la vitesse angulaire et le courant d'induit séparément.



3. Modélisation en utilisant la transformé de Laplace

$$J\frac{d^2\theta}{dt^2} = T - b\frac{d\theta}{dt} \Rightarrow J\frac{d^2\theta}{dt^2} + b\frac{d\theta}{dt} = K_t i$$

$$L\frac{di}{dt} = -Ri + V - e \Rightarrow L\frac{di}{dt} + Ri = V - K_e\frac{d\theta}{dt}$$

En appliquant la transformé de Laplace :

$$J\,s^2\theta(s) + b\,s\theta(s) = K_t I(s) \qquad (9)$$

$$LsI(s) + RI(s) = V(s) - K_e s\theta(s) \qquad (10)$$

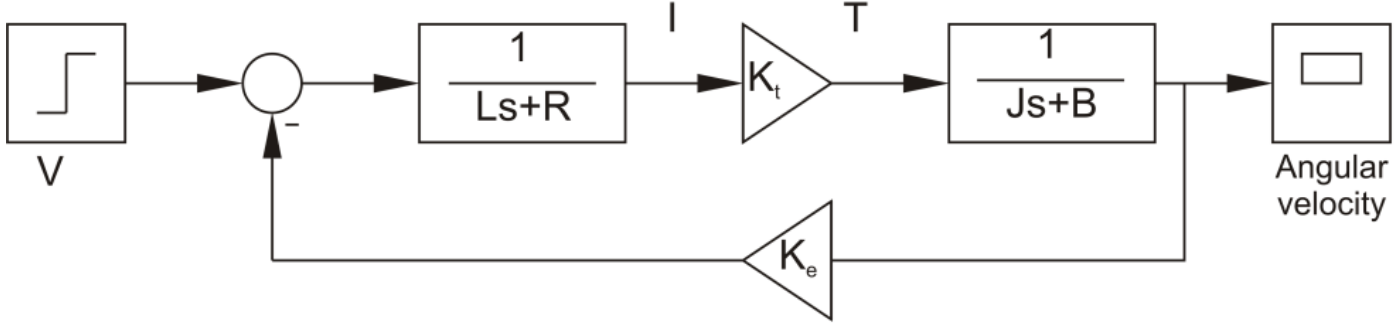
Avec s l'opérateur de Laplace

De l'équation (10), on peut écrire l'expression $I(s)$

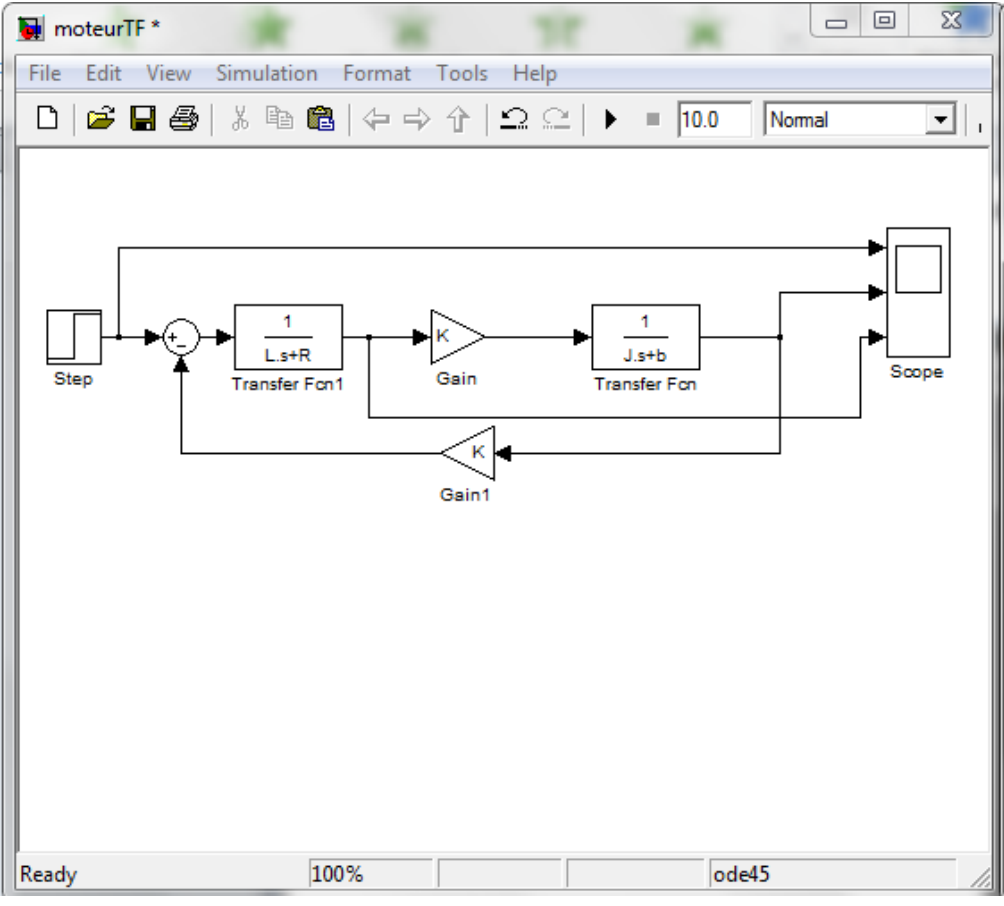
$$I(s) = \frac{V(s) - K_e s\theta(s)}{R + Ls}$$

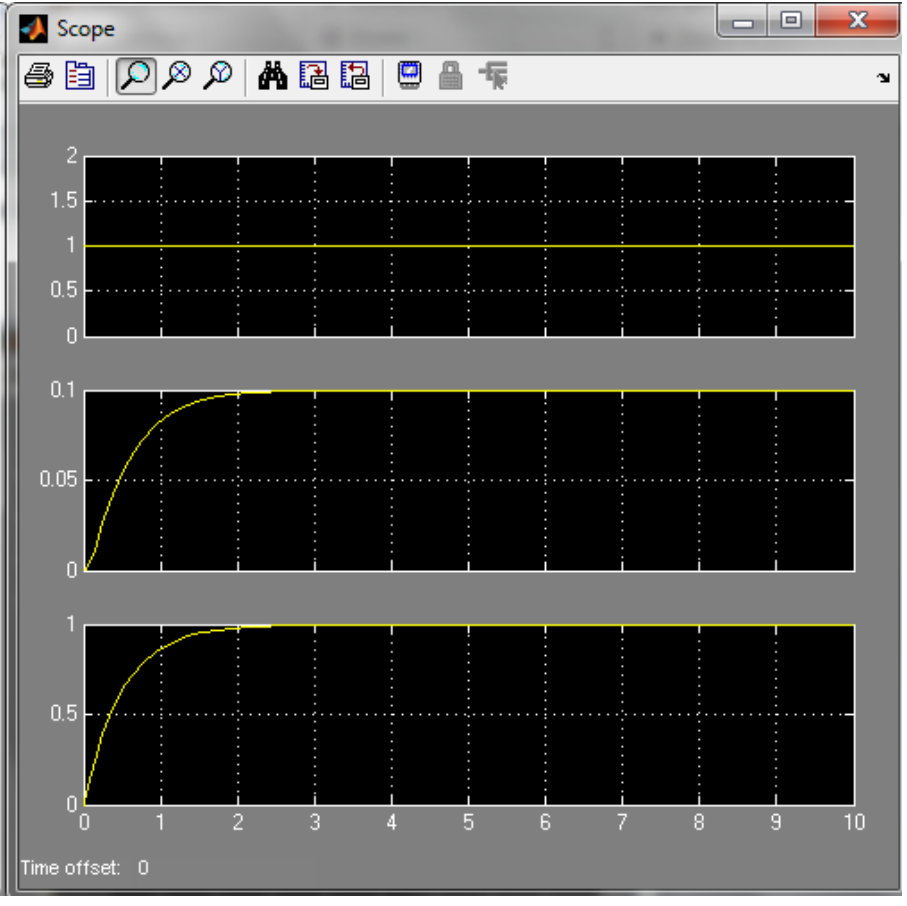
Et on substitue $I(s)$ l'équation (9)

$$J\,s^2\theta(s) + b\,s\theta(s) = K_t\frac{V(s) - K_e s\theta(s)}{R + Ls}$$



On exécute le modèle développé en fonction de transfert et on visualise les résultats sur l'oscilloscope.





Merci

pour

votre Attention

Le logiciel de simulation Simulink- Simpowersystems Toolbox
Partie 4
Exemples de modélisation

Professor Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Introduction

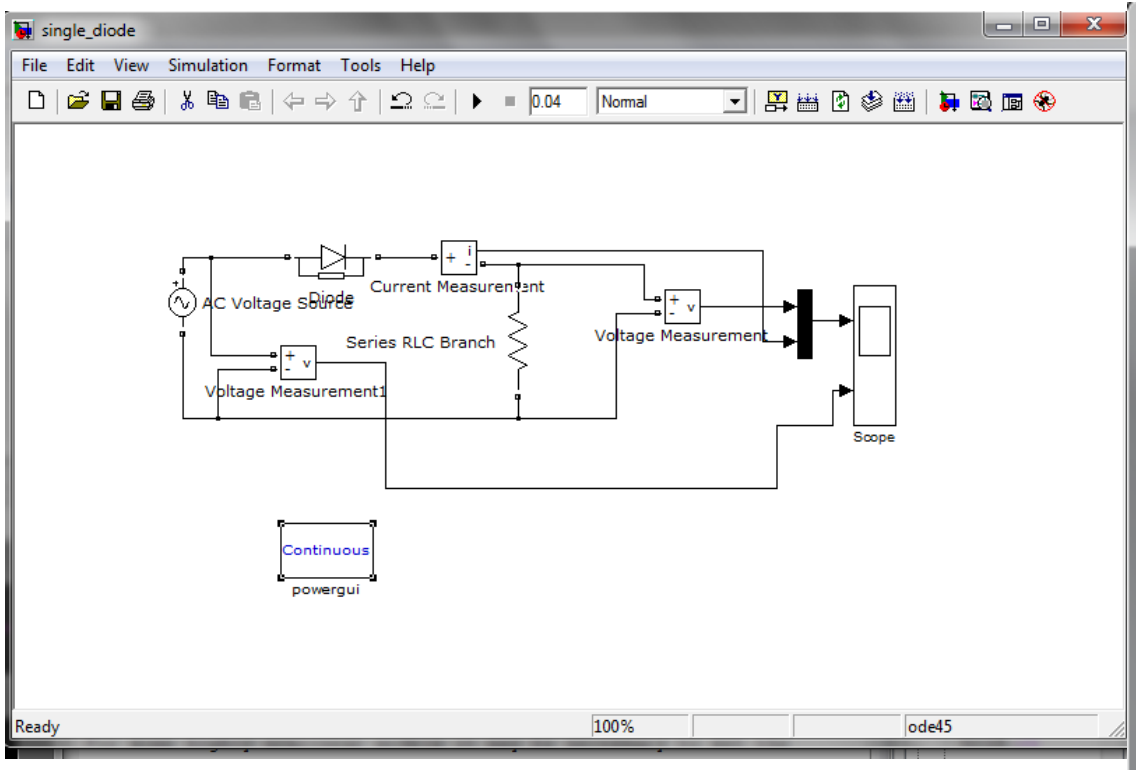


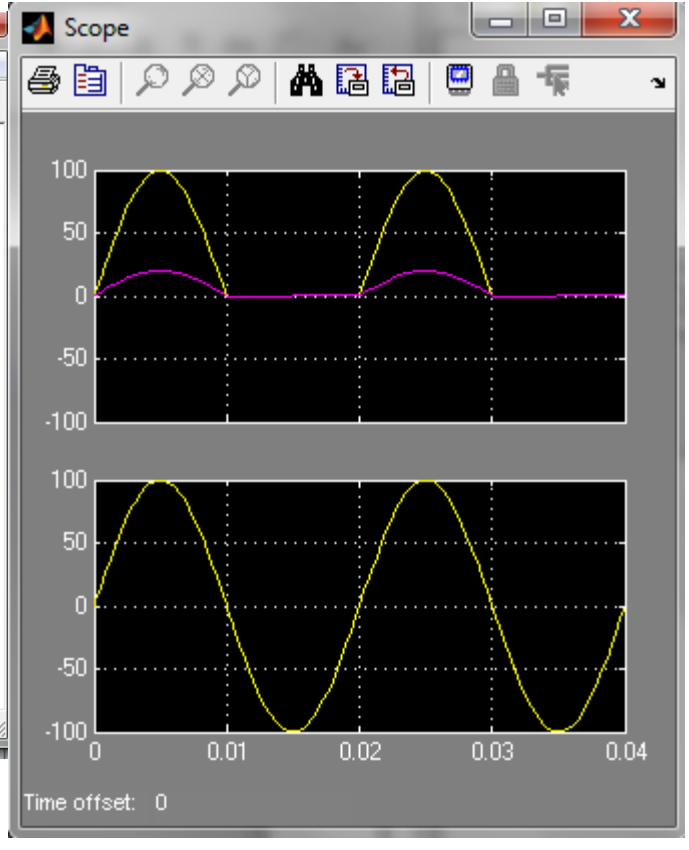
Le Simpowersystems est une toolbox dédiée à l'électronique de puissance, les machines électriques, les réseaux électriques etc…

SimPowerSystems fournit des librairies de composants et des outils d'analyse pour la modélisation et la simulation des systèmes de réseaux électriques.

Les librairies offrent des modèles de composants de réseaux électriques, comme les machines électriques triphasées, les entrainements électriques, et des composants pour des applications tels que le conditionnement des réseaux électriques (flexible AC transmission systems : FACTS) et les systèmes des énergies renouvelables. L'analyse harmoniques, le calcul du total harmonic distortion (THD), puissances et d'autres outils d'analyse sont automatisés dans la toolbox.

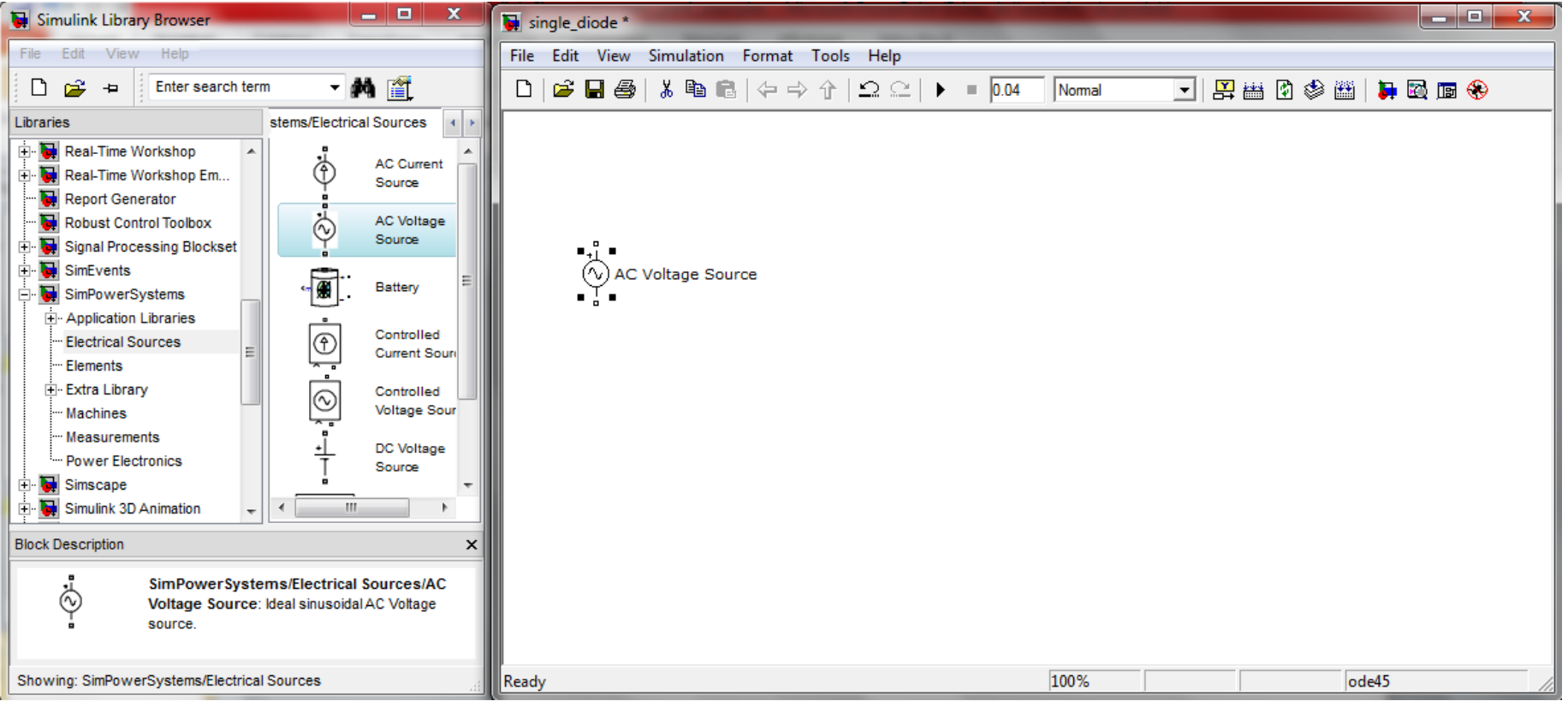
2. Redresseur monophasé mono-alternance



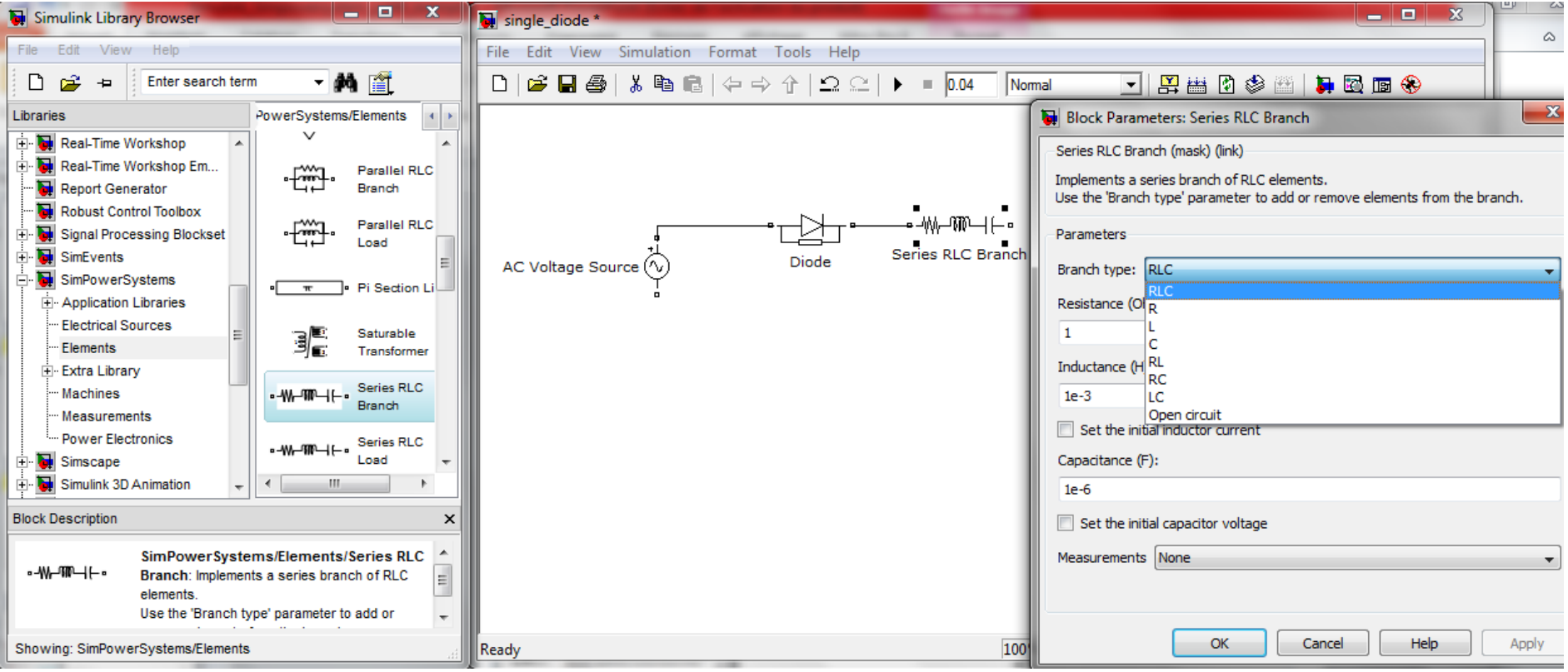


Commençant à construire ce modèle avec les composants de la toolbox Simpowersystems.

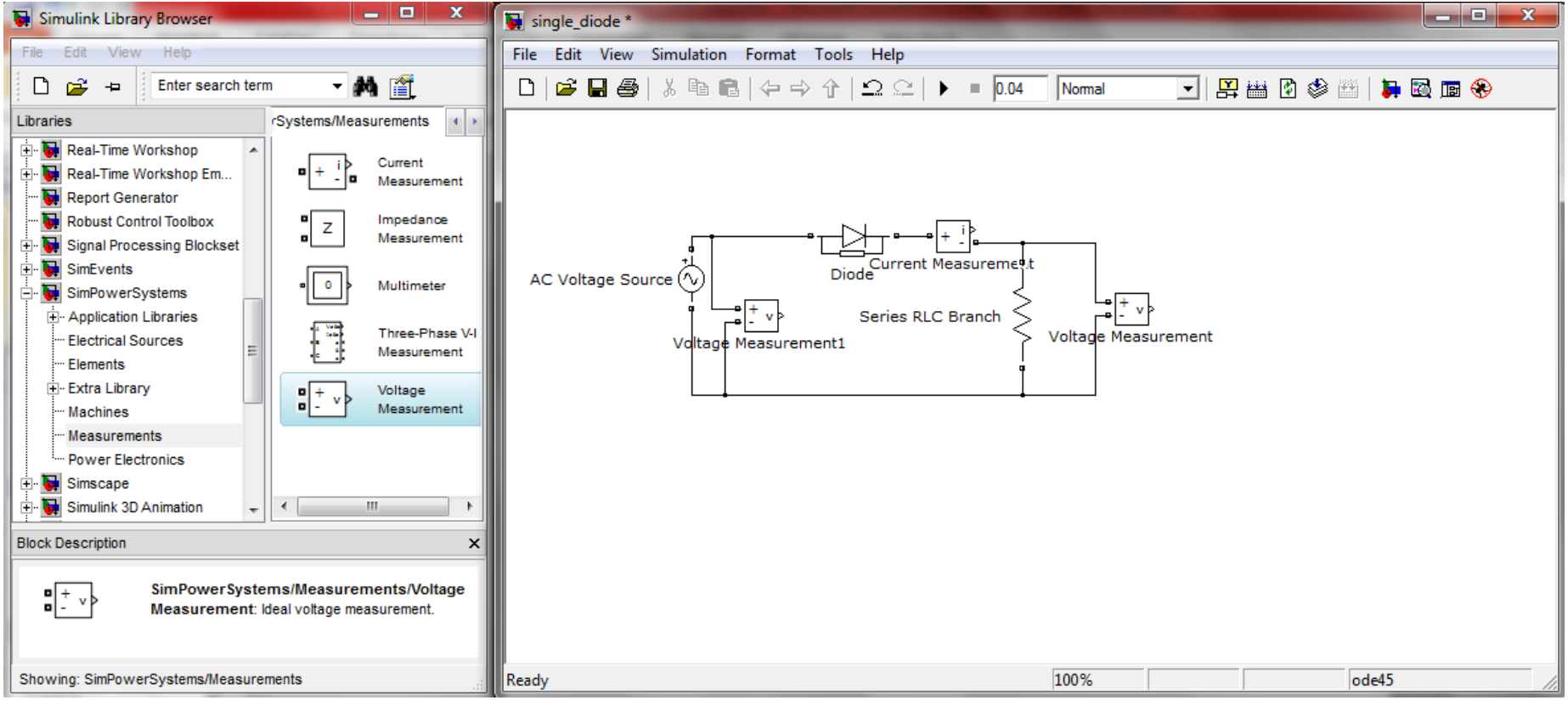
Ajoutant au modèle une source de tension sinusoïdale.



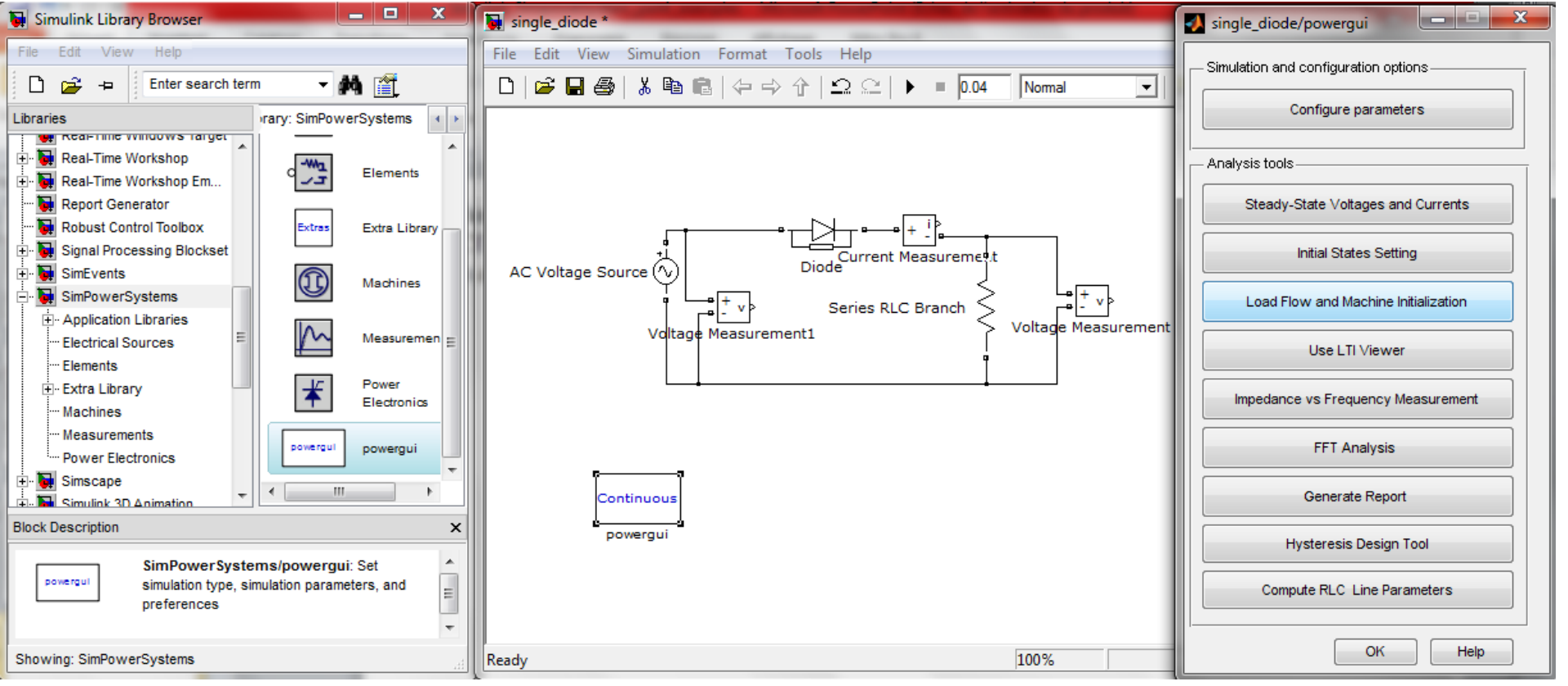
Ajoutant au modèle une diode de la librairie Power Electronics et une charge RLC de la librairie Elements, en cliquant deux fois sur cette dernière, nous pouvons changer la charge en une résistance.



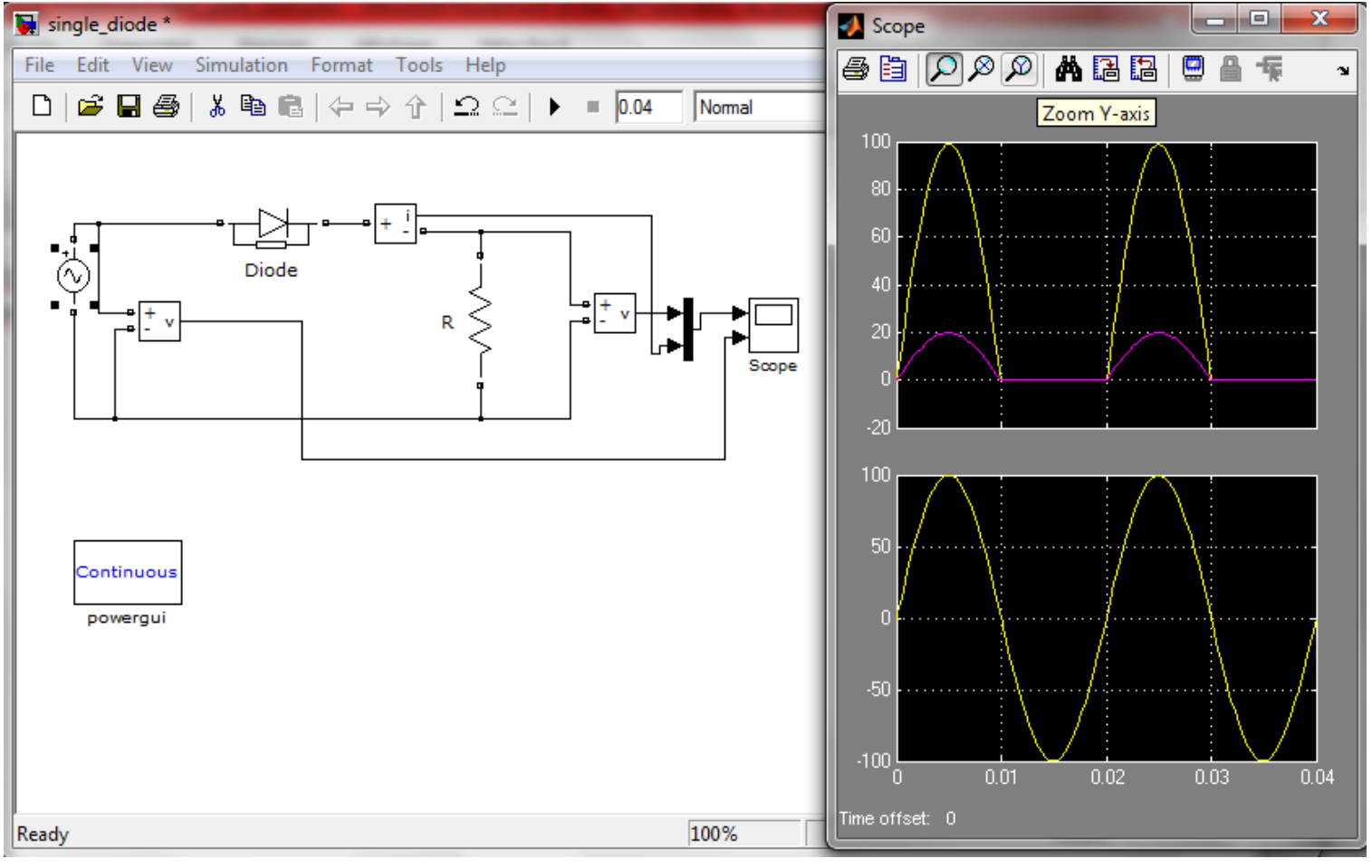
La charge résistive peut lui être appliquée une rotation en la sélectionnant et cliquer sur CTRL+R.
Nous avons ajouter deux voltmètres et un ampèremètre de la librairie Measurement.



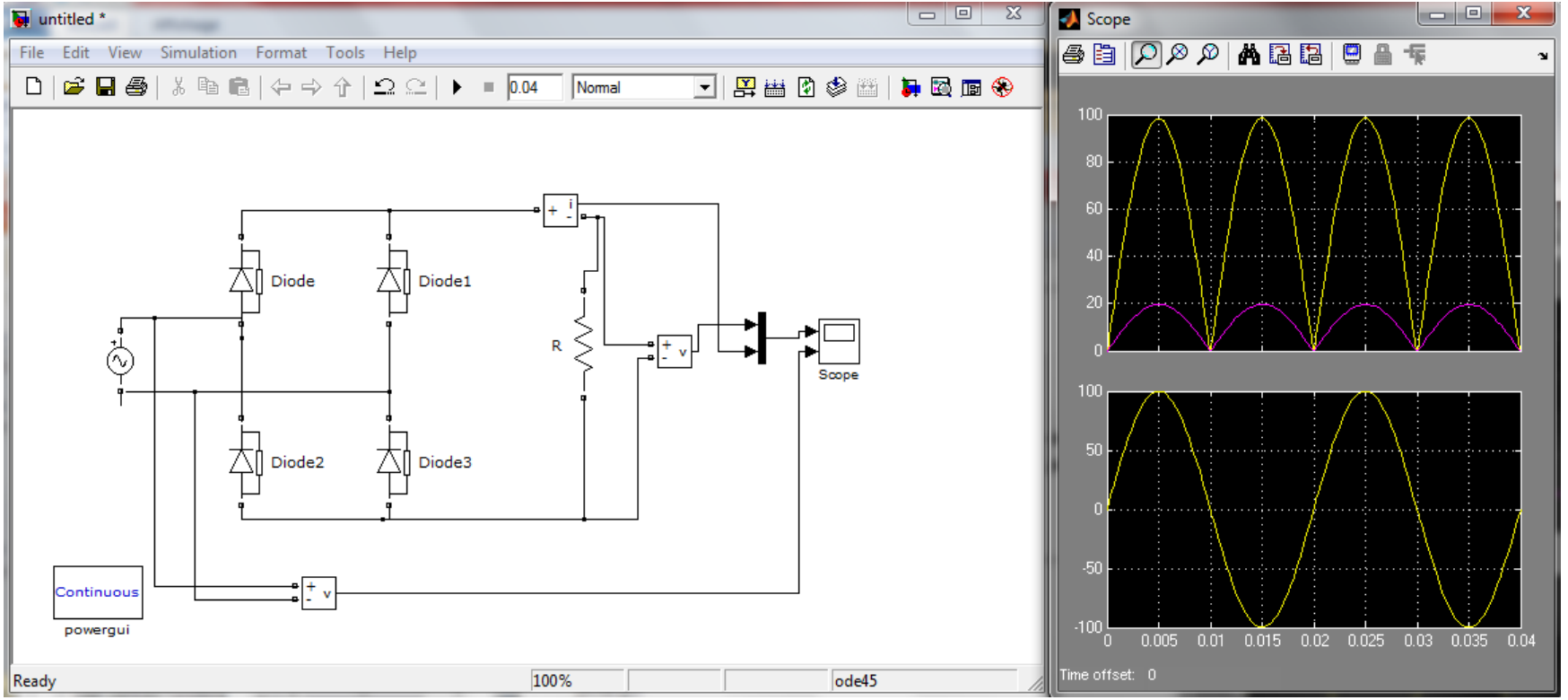
Powergui est un élément important pour la simulation de tous les modèles sous Simpowersystems, il permet de configurer plusieurs paramètres et faire l'analyse.



Maintenant on peut exécuter notre modèle pour une durée de simulation de deux périodes.

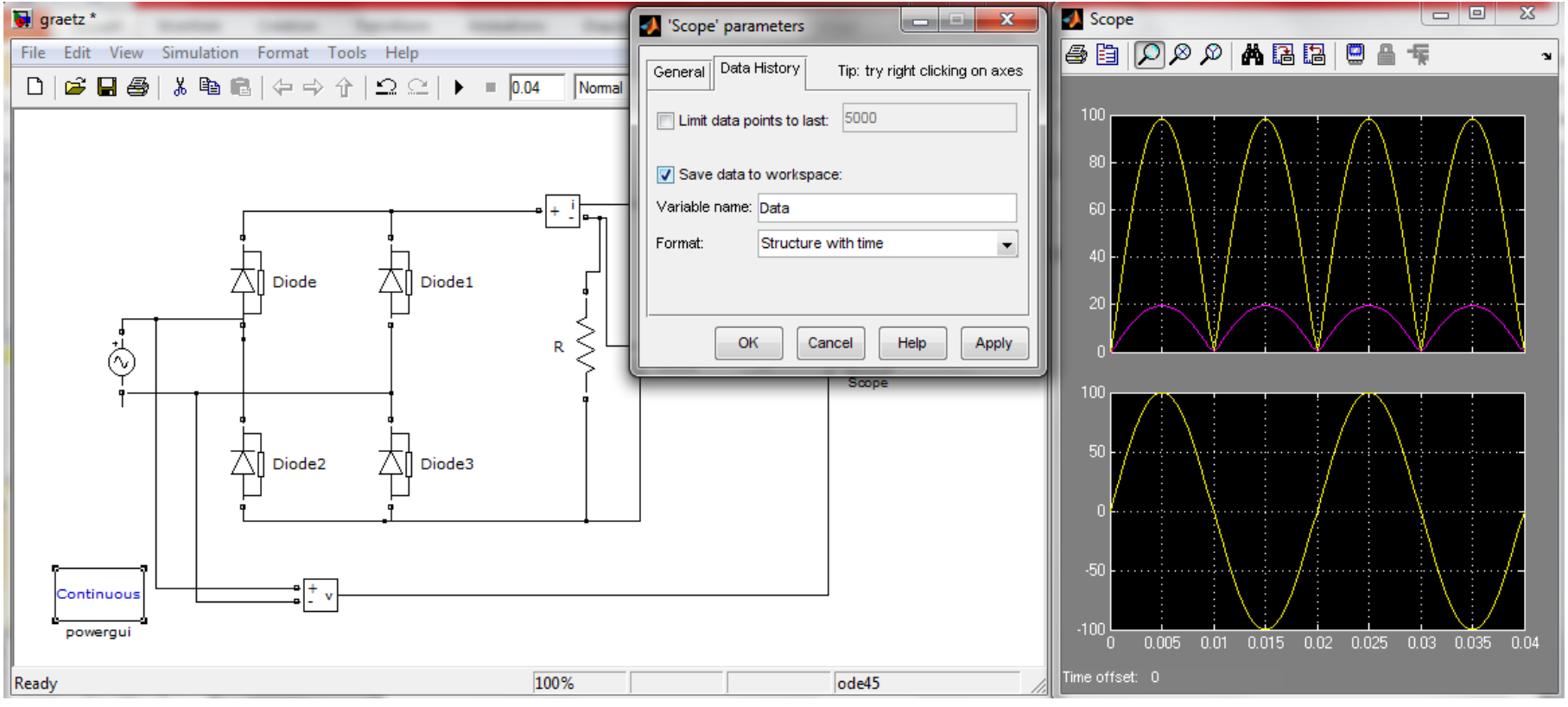


3. Redresseur monophasé en pont de Graëtz

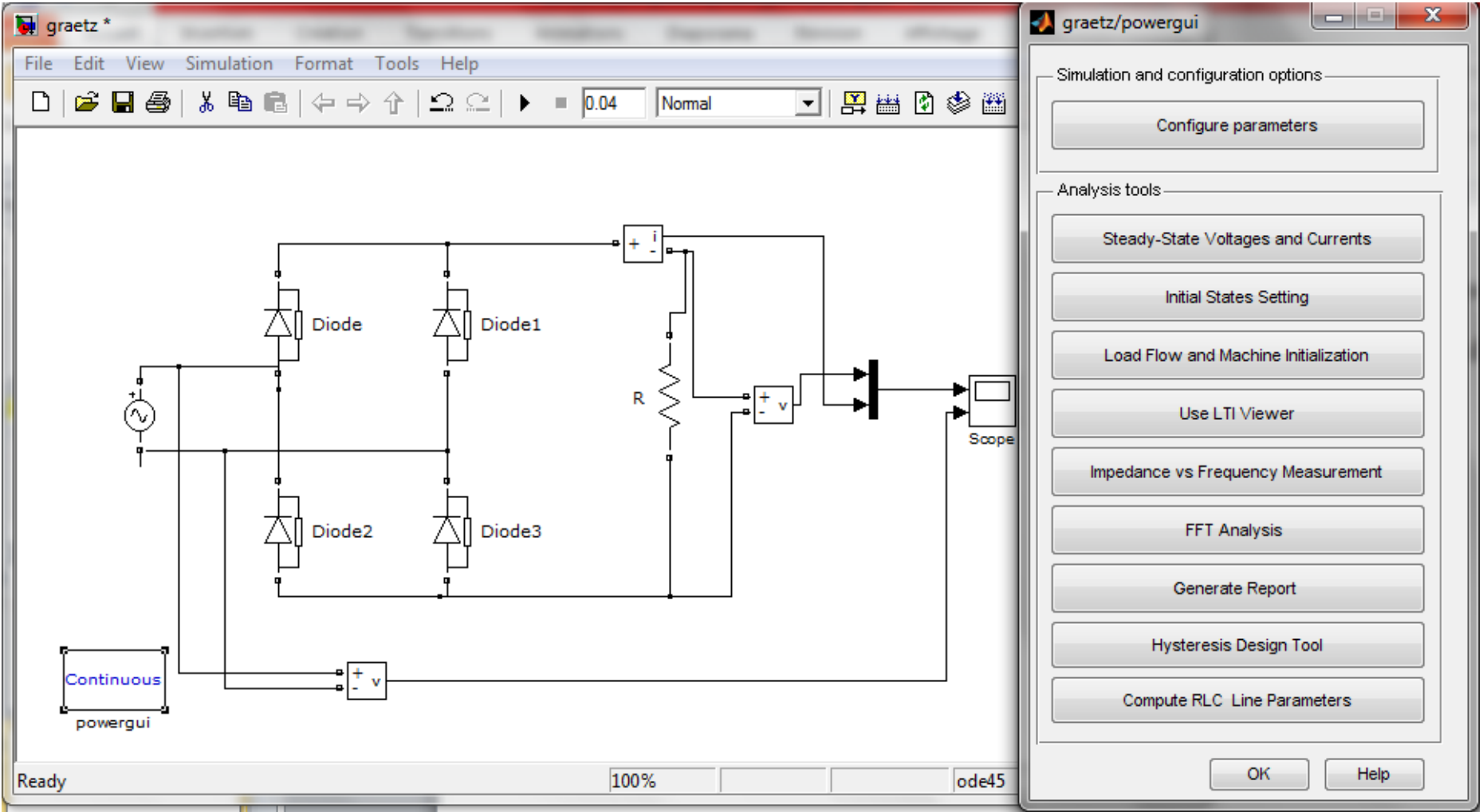


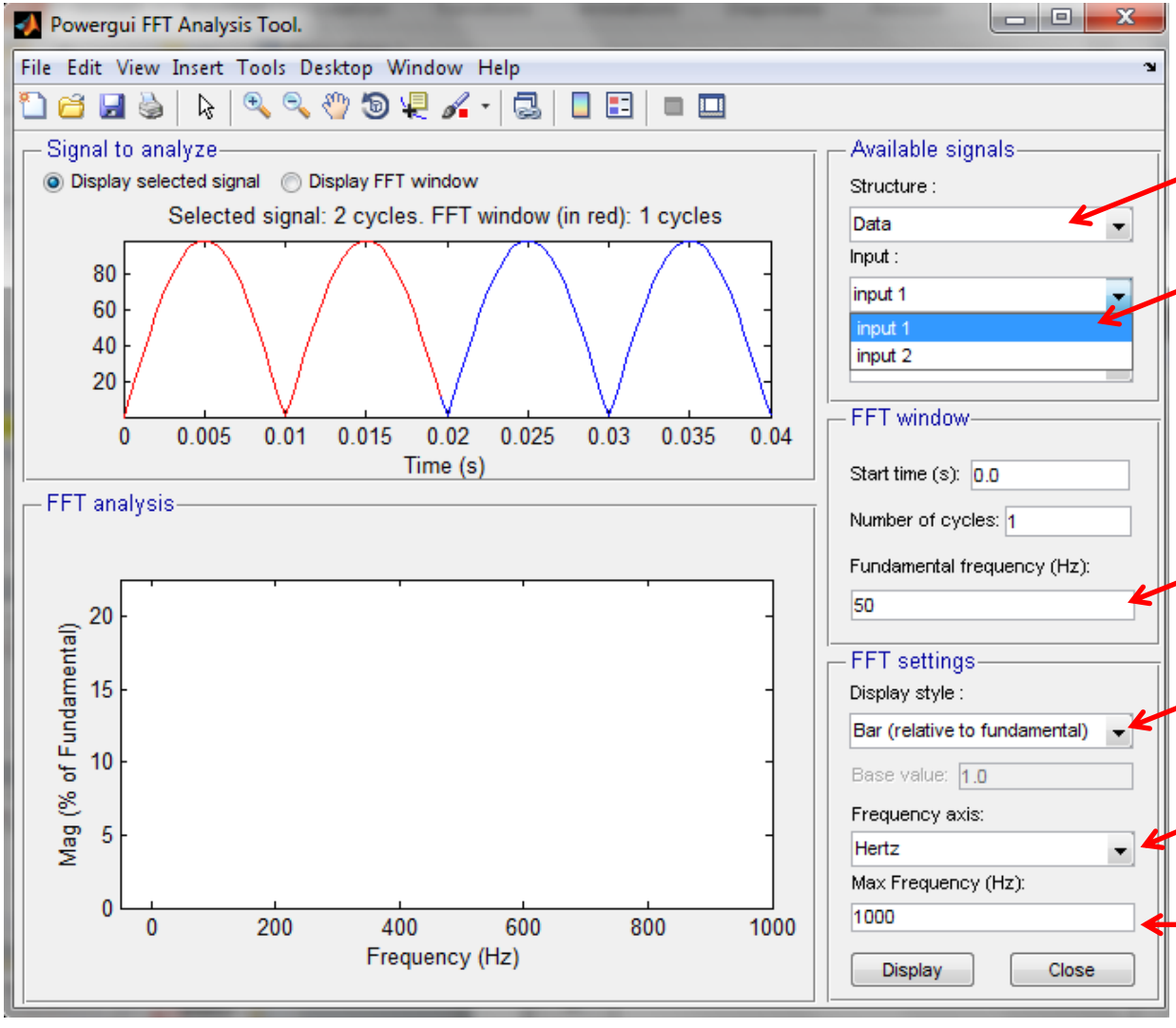
Essayant de voir ce que nous offre Simpowersystems comme outils d'analyse.

Voyant l'outil d'analyse des harmoniques, pour cela il faut sauvegarder les données dans le workspace à partir du scope en cochant save data to workspace.



Double click sur powergui la fenêtre s'ouvre cliquer sur FFT Analysis.





Data sauvegardée dans le workspace

Les données

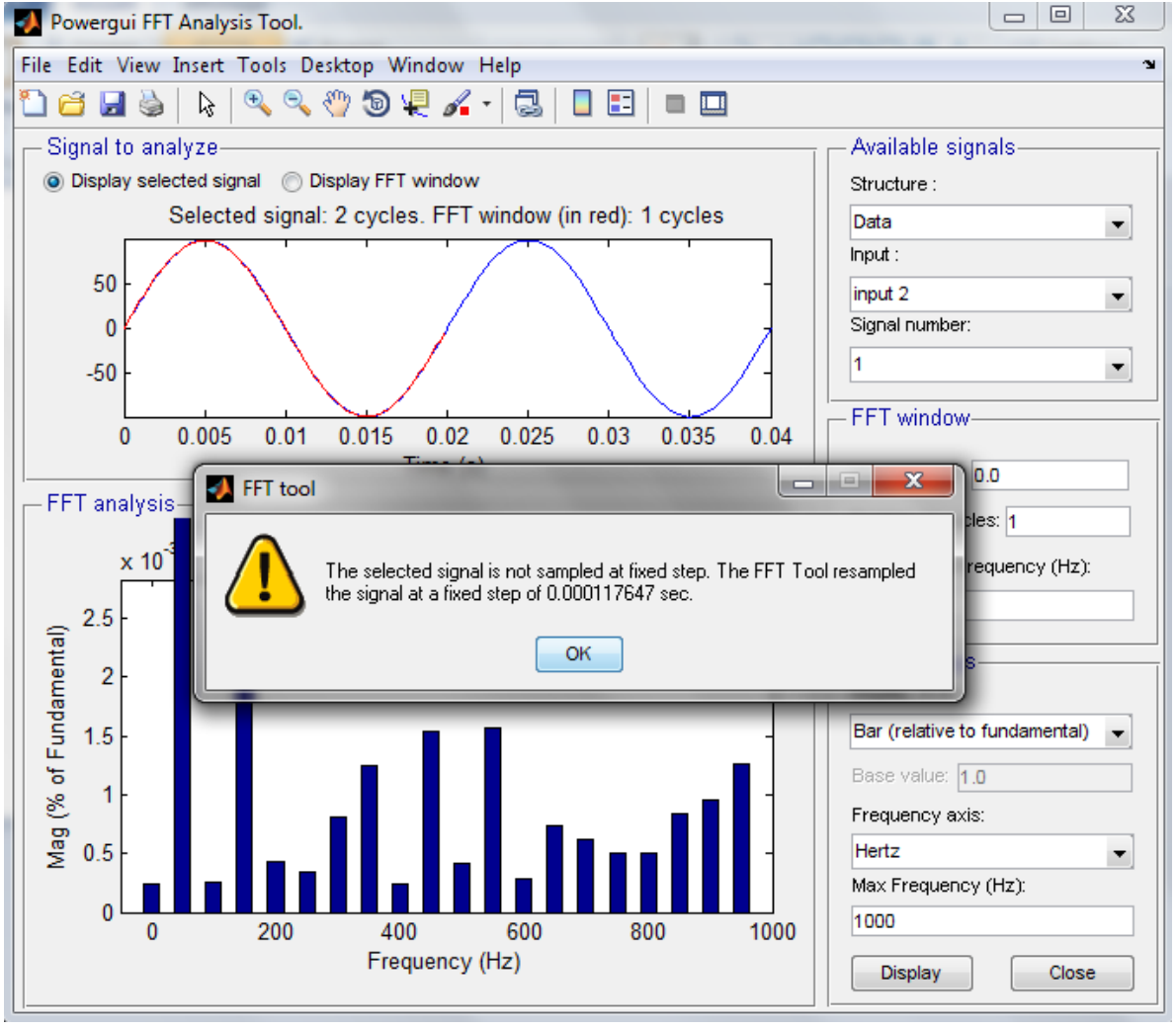
La fréquence de la fondamentale

Les styles d'affichage

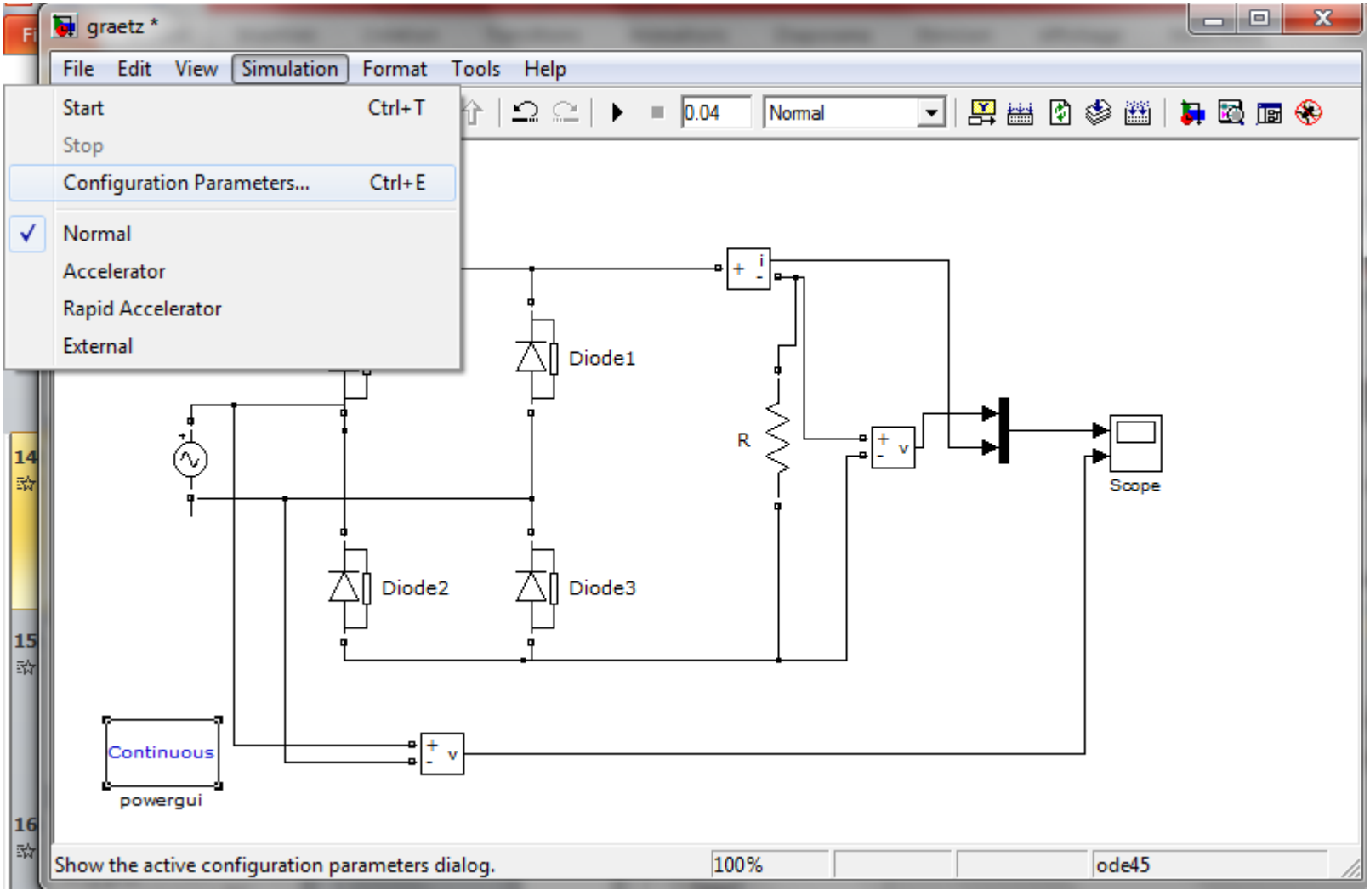
Axe des x en fréquence ou en rang

Maximum de fréquence pour l'affichage

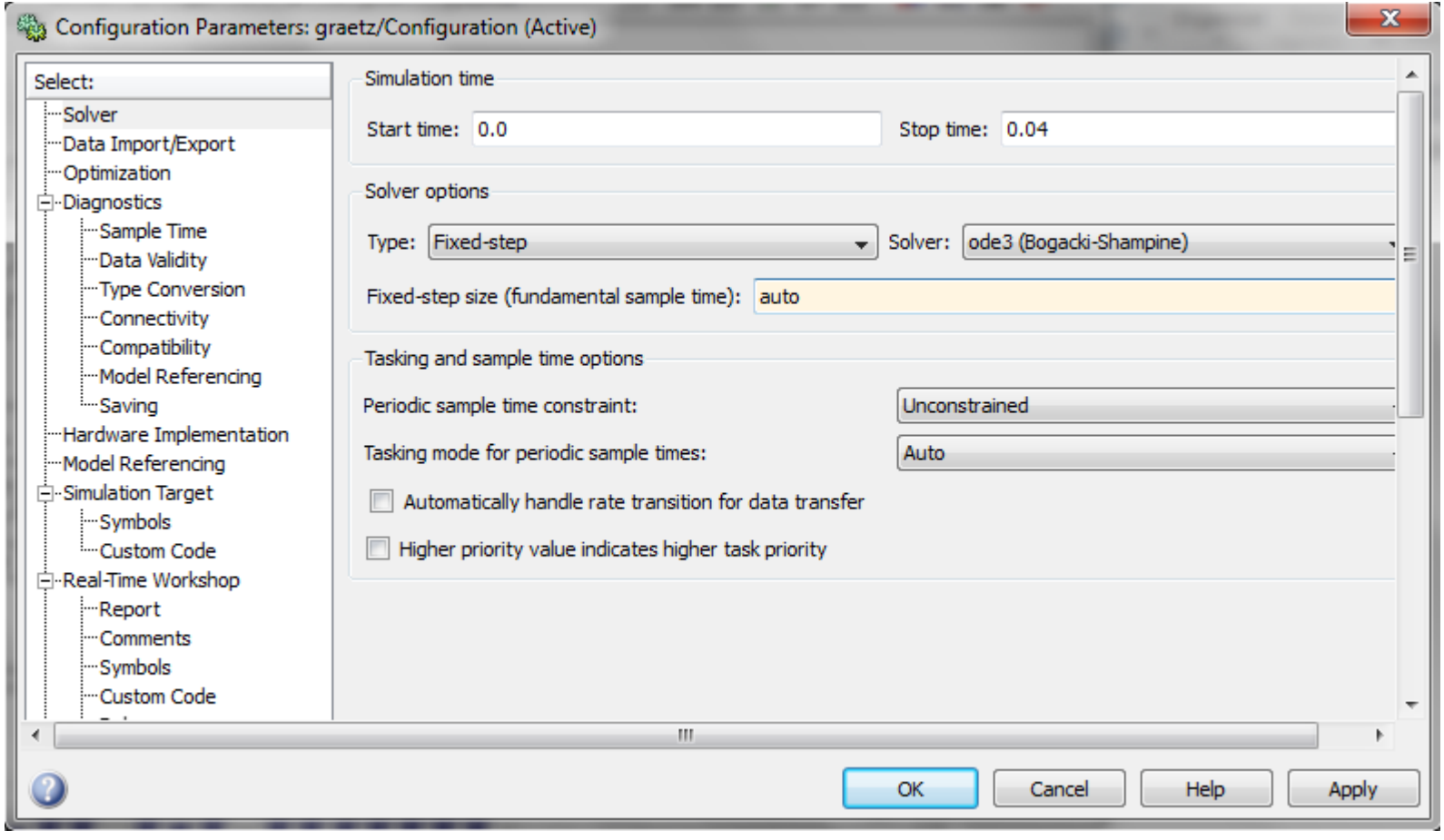
Un message d'rreur qui vous demande à ce que vos données soient simulées à un pas fixe pour pouvoir faire une analyse d'harmoniques



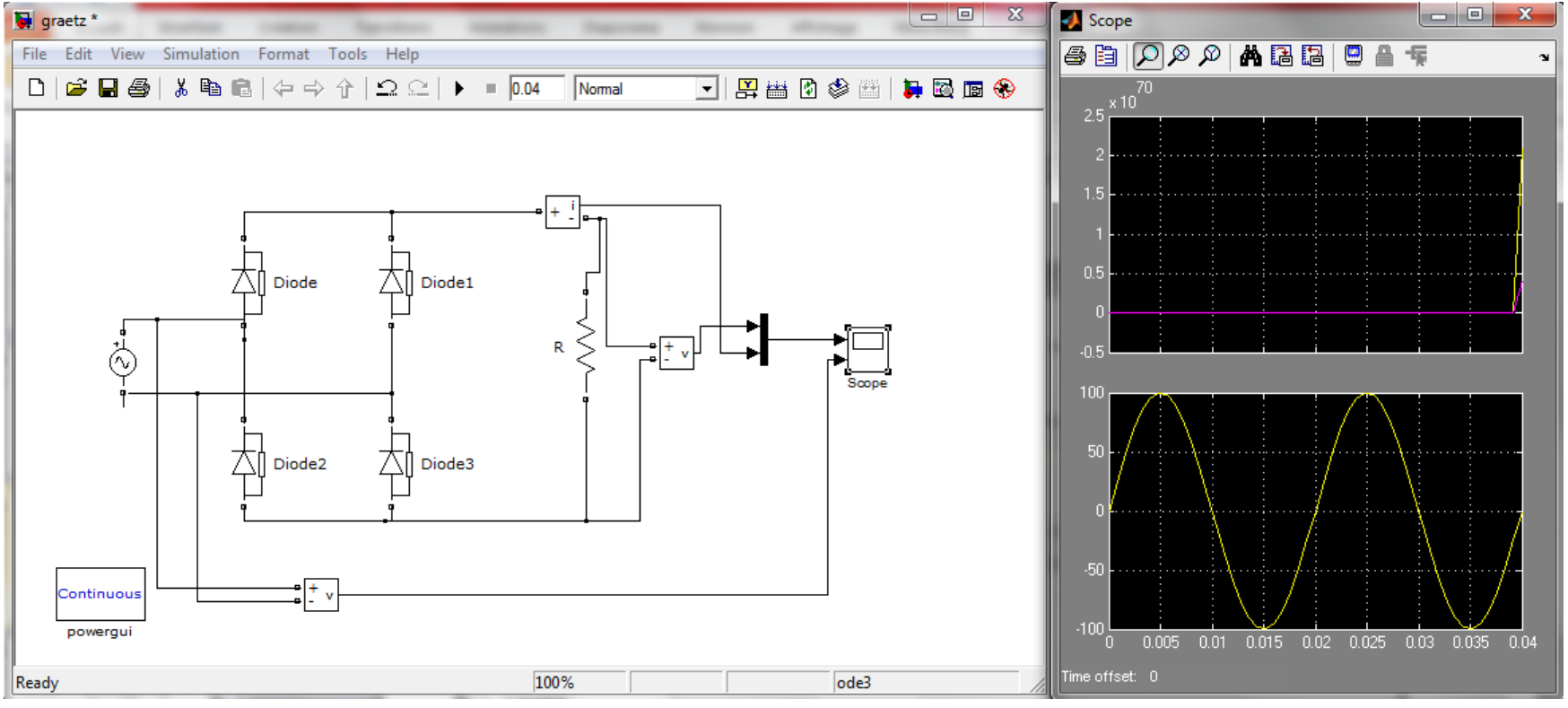
Allant à Simulation puis à Configuration Parameters dans le modèle.



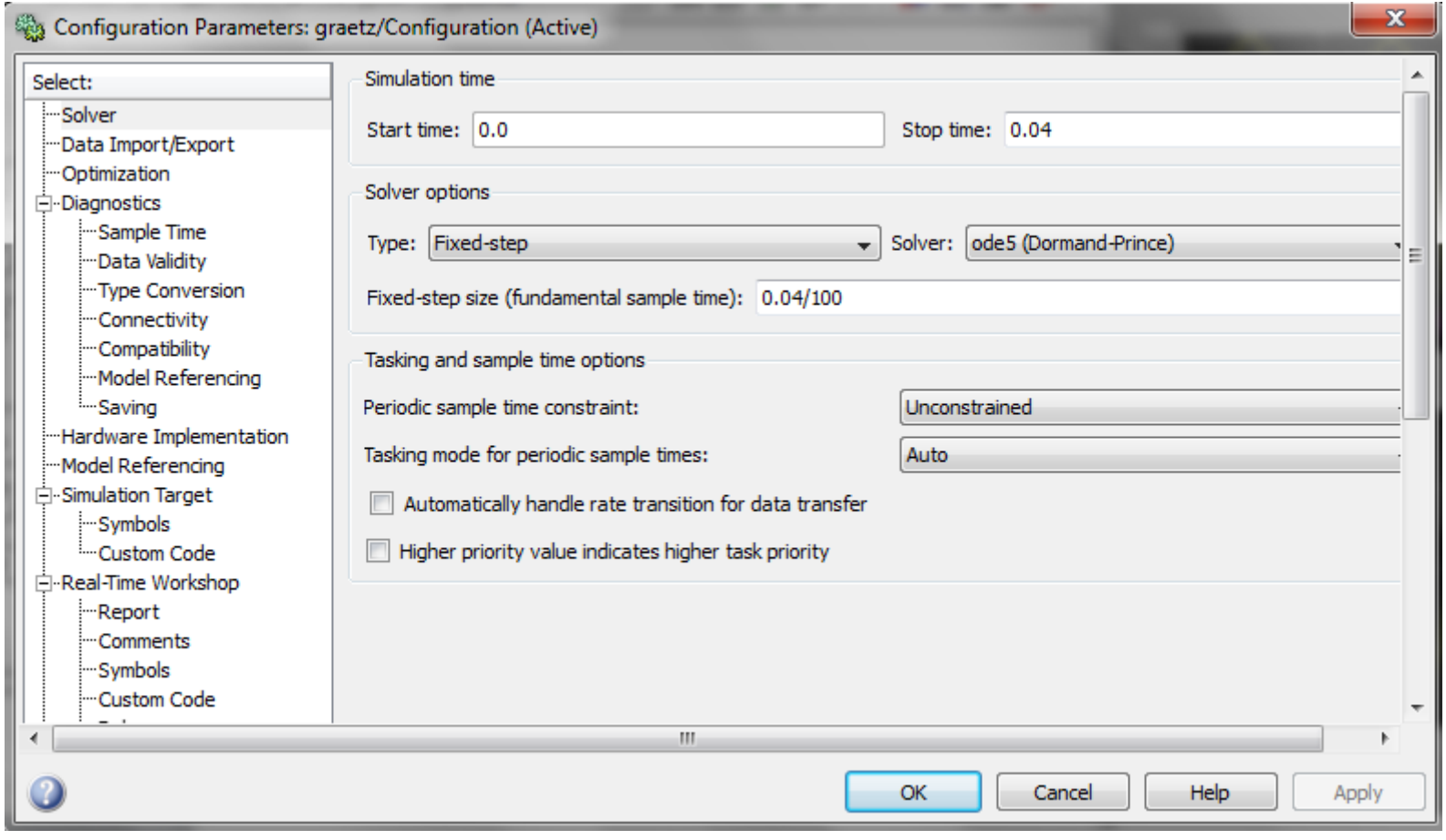
Choisissant le pas fixe et laissant le auto.



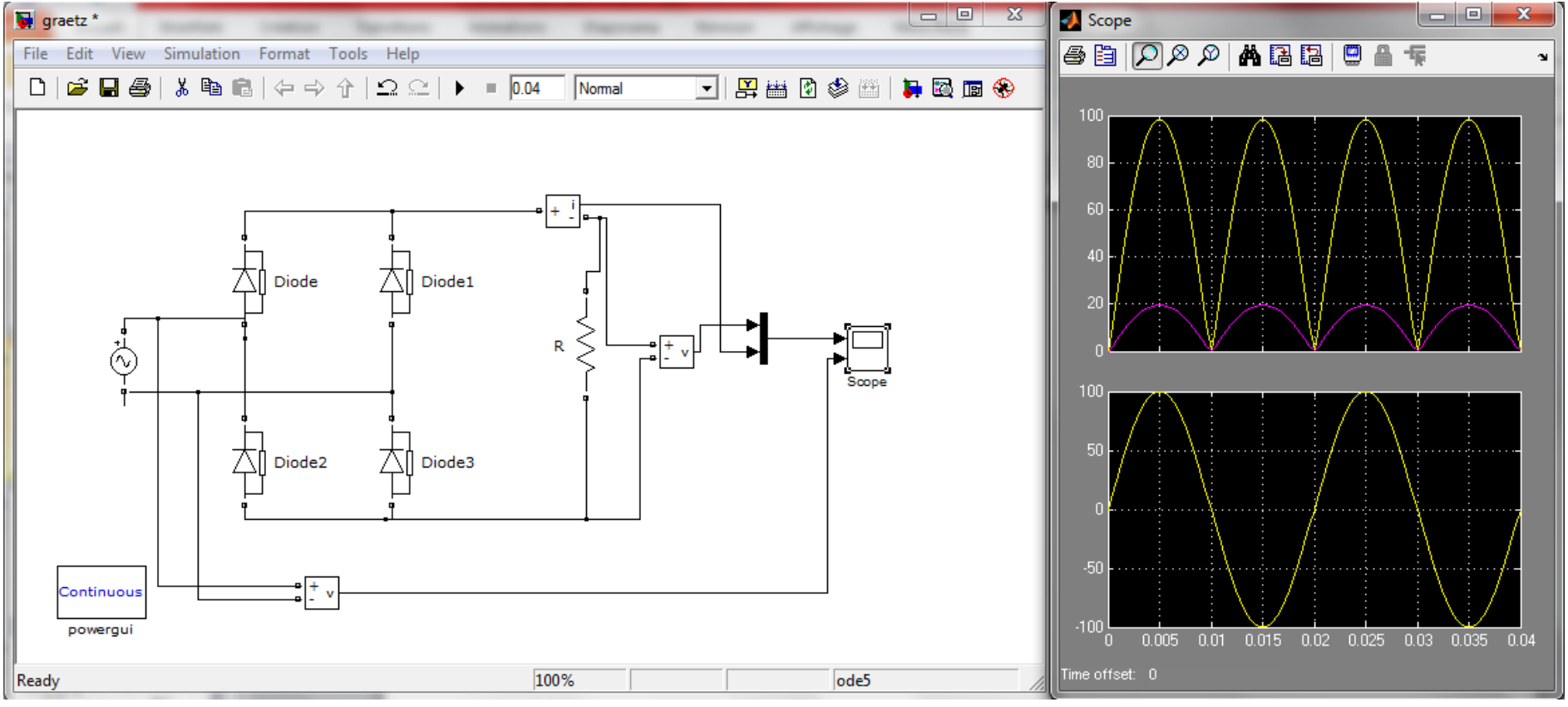
Vous voyez que les résultats sont erronés, cela est du au pas fixe qui n'est pas bien configuré.



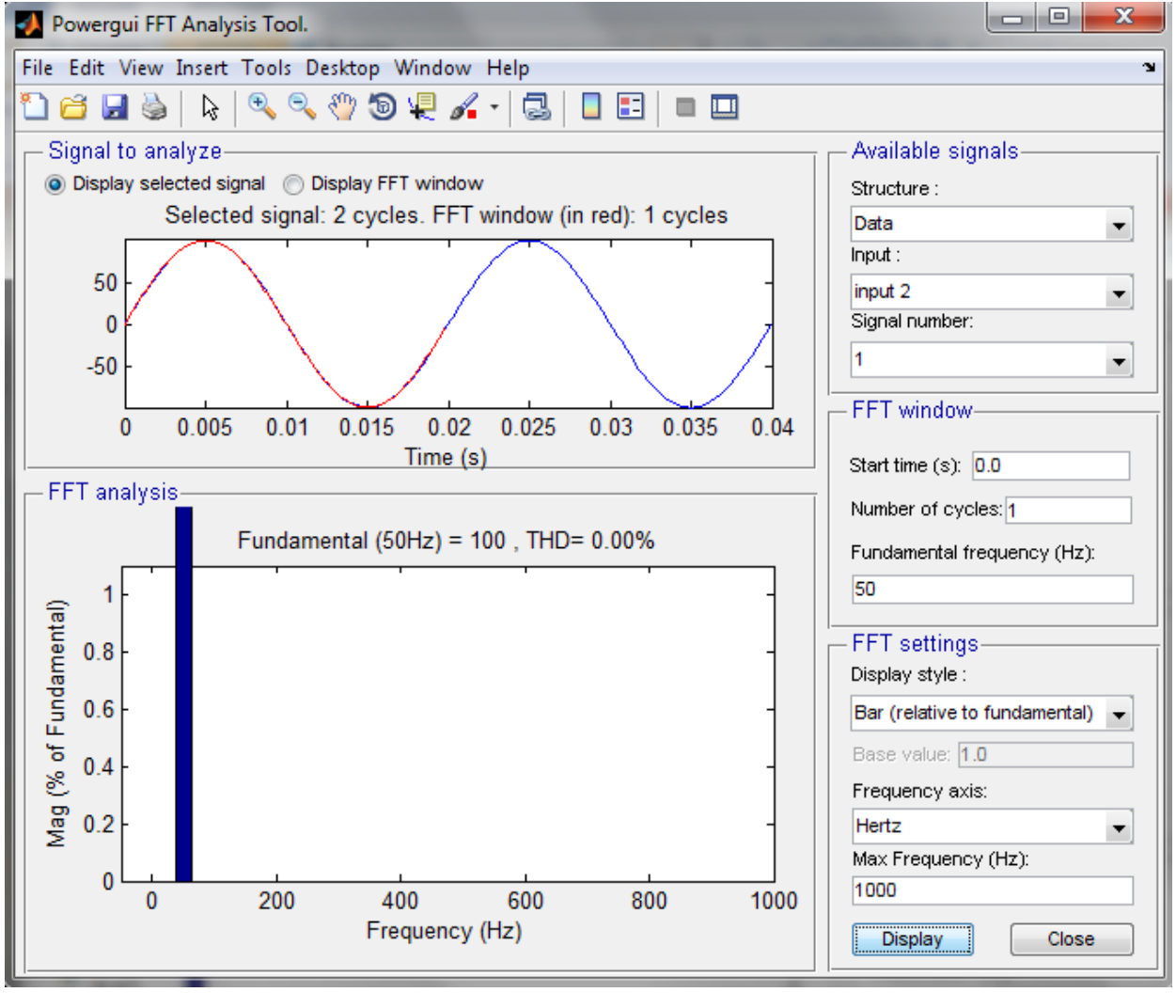
Vous pouvez prendre un pas égale au temps de simulation divisé par 100.
Et le solver la méthode ode5 qui est la plus précise.



Maintenant, on peut procéder à l'analyse d'harmoniques.

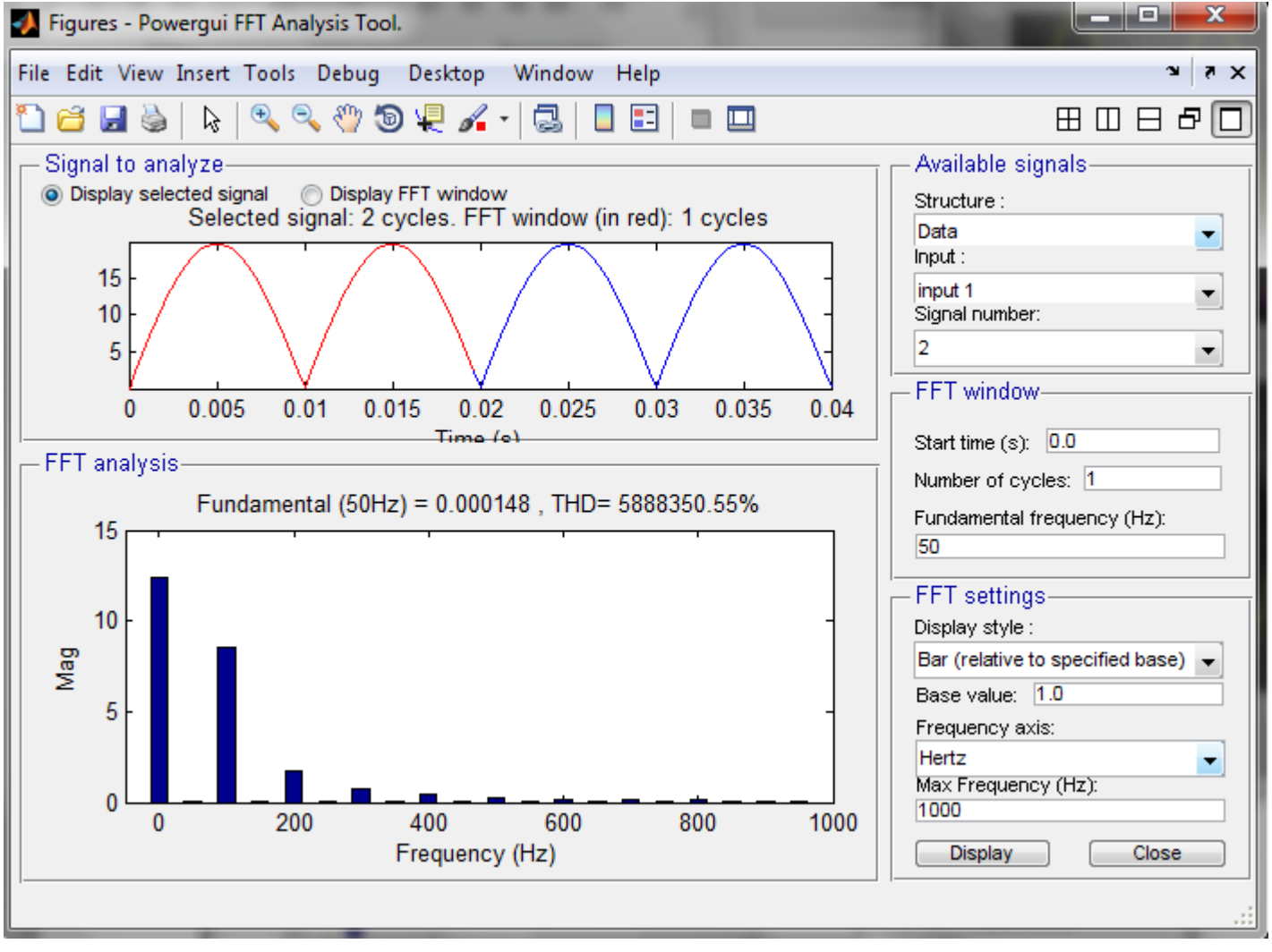


Le signal de la source alternative ne comprend que la fondamentale à la fréquence 50 Hz.
L'analyse fréquentielle est obtenue en choisissant le signal input 2 et en cliquant sur Display.



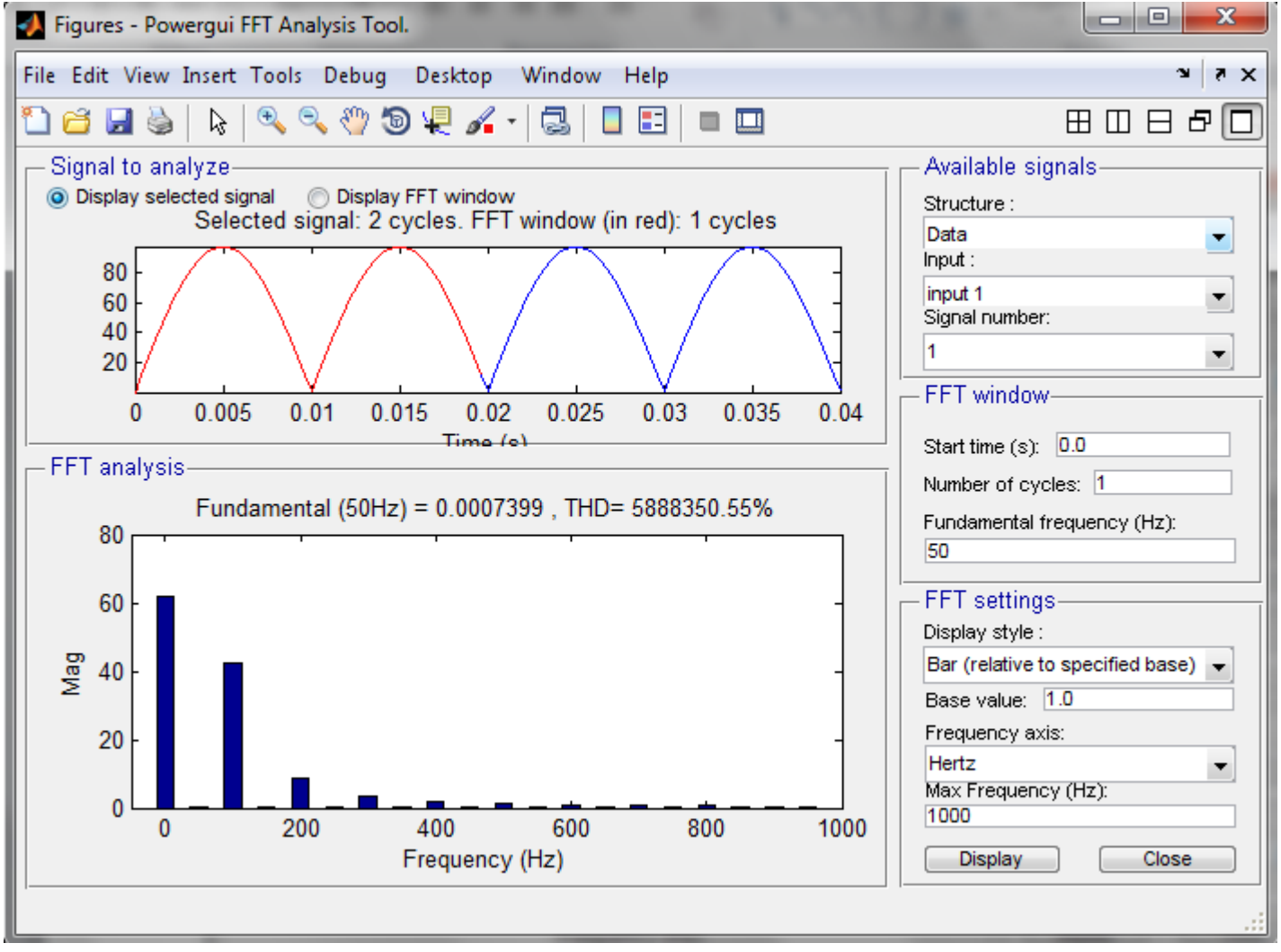
Le signal du courant (input 1, signal number 2)

La composante continue qui la valeur moyenne du courant de la charge est égale à 12.4 A



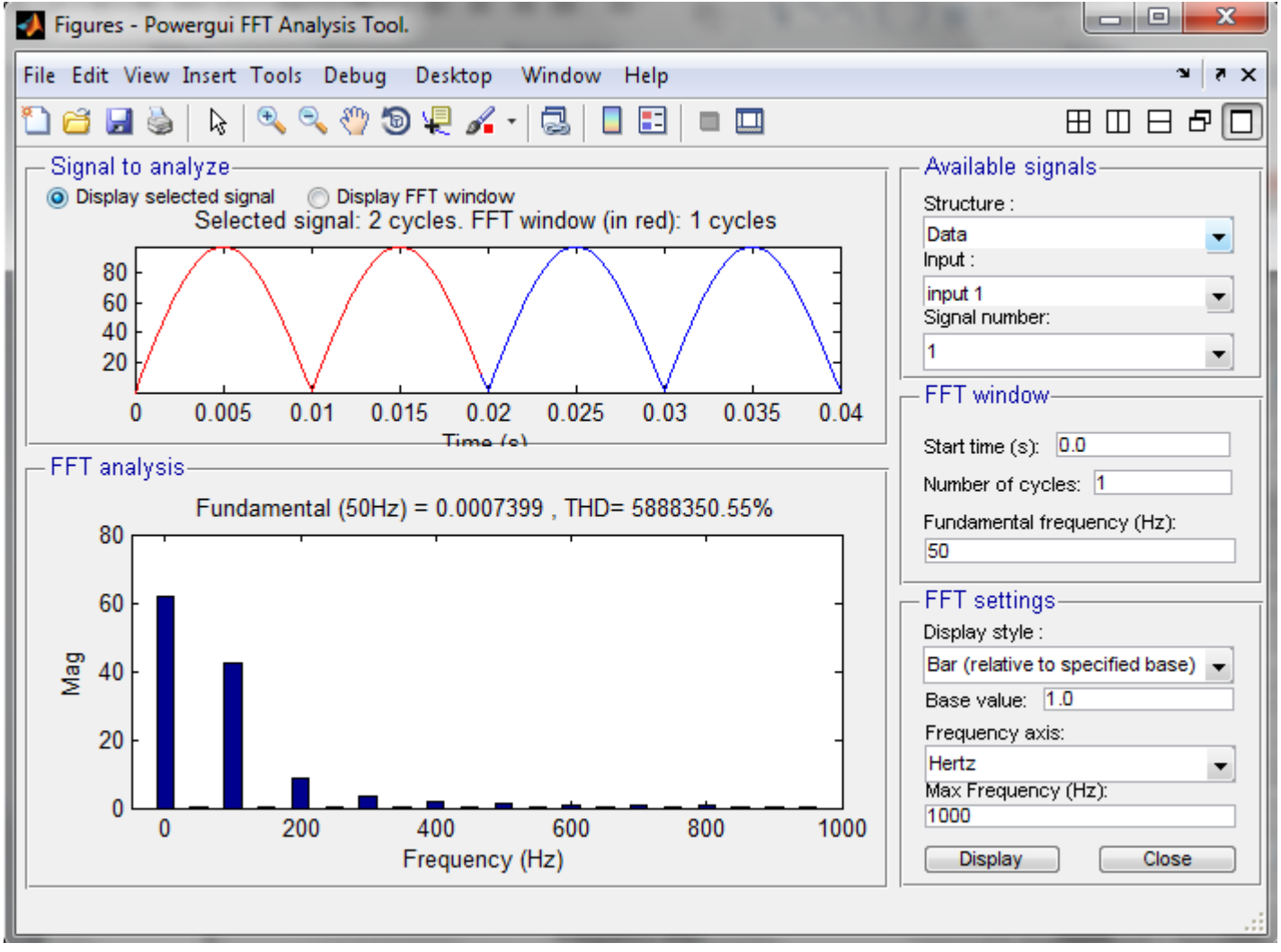
Le signal de la tension (input 1, signal number 1)

La composante continue qui la valeur moyenne de la tension redressée est égale à 62.02V

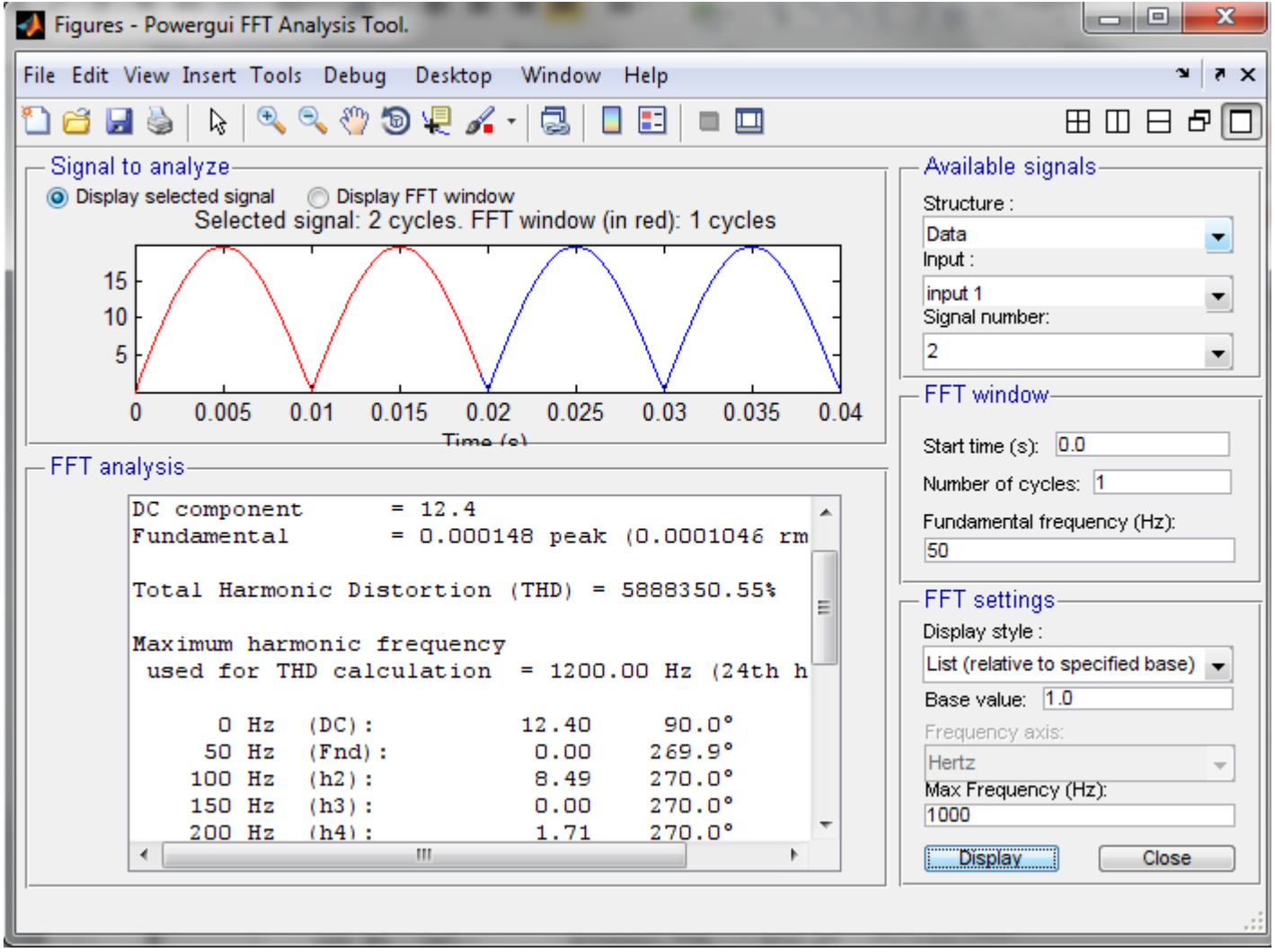


Le signal du courant (input 1, signal number 1)

La composante continue qui la valeur moyenne de la tension redressée est égale à 62.02V



L'analyse harmonique peut se faire par une list relative à la base 1 qui vous donnera les harmoniques du signal sous forme d'une liste



Merci

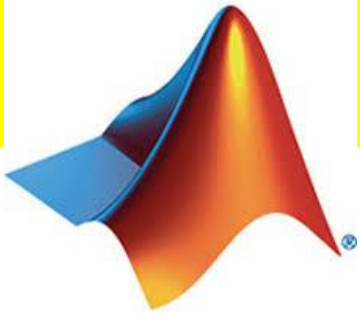
pour

votre Attention

Le logiciel de simulation Interfacer Simulink- Simpowersystems Toolbox
Partie 5

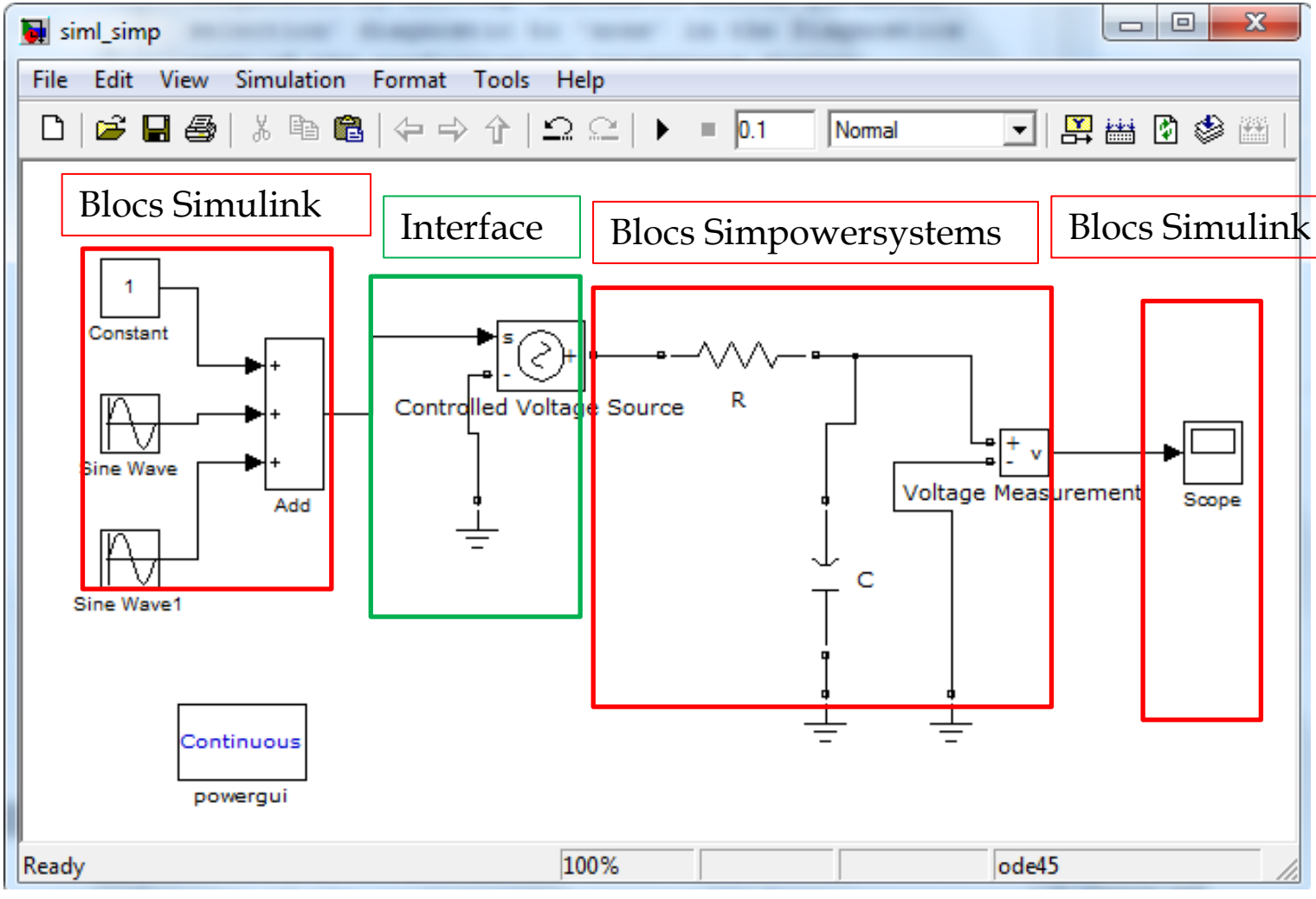
Professor Ali Tahri
Université des sciences et de la technologie d'Oran Mohamed Boudiaf

1. Introduction



Interfacer un circuit électrique de simpowersystems avec Simulink
Le bloc de mesure de tension a le rôle d'une interface entre les blocs de SimPowerSystems et les blocs de Simulink. Pour le système illustré ci-dessous,
Vous implémentez comme une interface du système électrique au système simulink.
Le bloc de mesure de tension convertit la tension mesurée en un signal de Simulink.
Notez que le bloc de mesure de courant peut être aussi utilisé pour convertir le courant en un signal de Simulink.
Vous pouvez aussi interfacer des blocs Simulink au système électrique sous Simpowersystems.
Par exemple vous pouvez utiliser le bloc source à tension contrôlée (Controlled Voltage Source block) pour injecter une tension dans un circuit électrique. Ainsi la tension est contrôlée par un signal de Simulink.

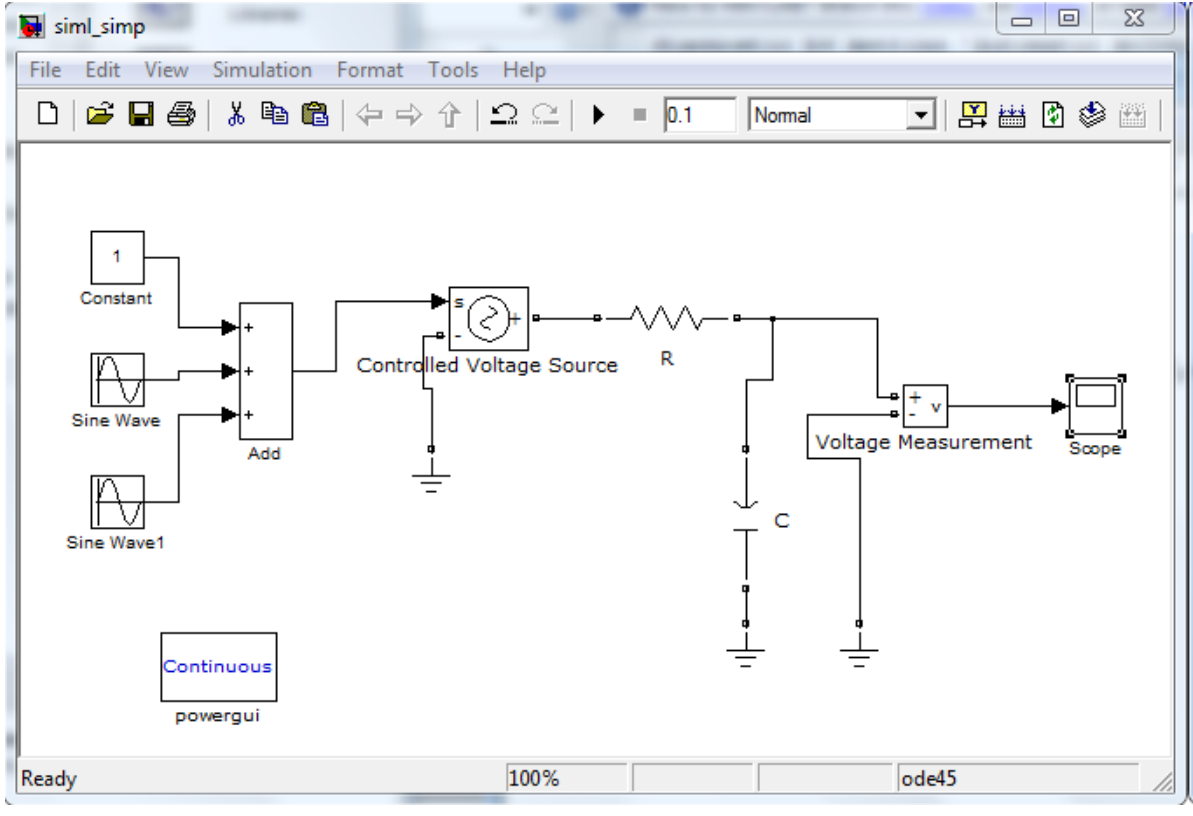
2. Exemple d'interfaçage de blocs Simulink à Simpowersystems

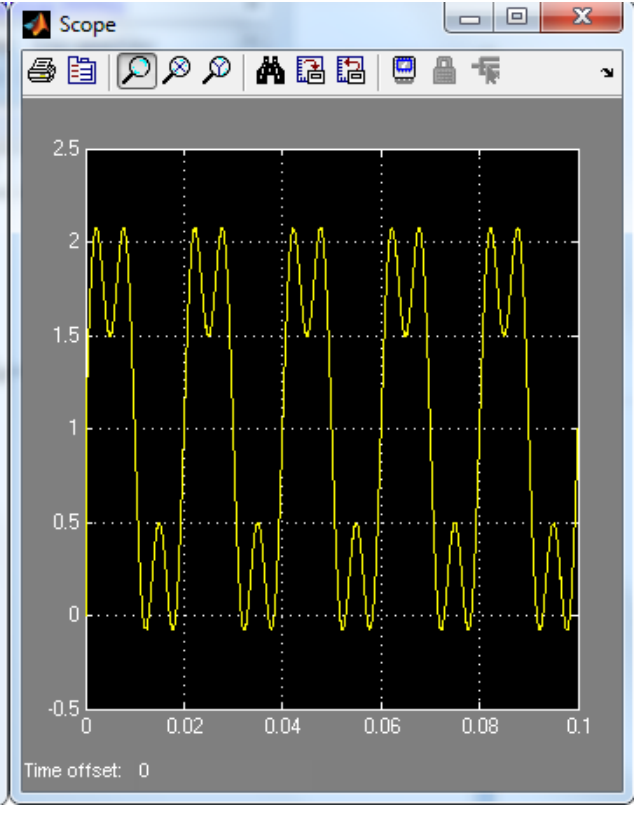


Le bloc source à tension contrôlée (Controlled Voltage Source block) pour injecter une tension dans un circuit électrique

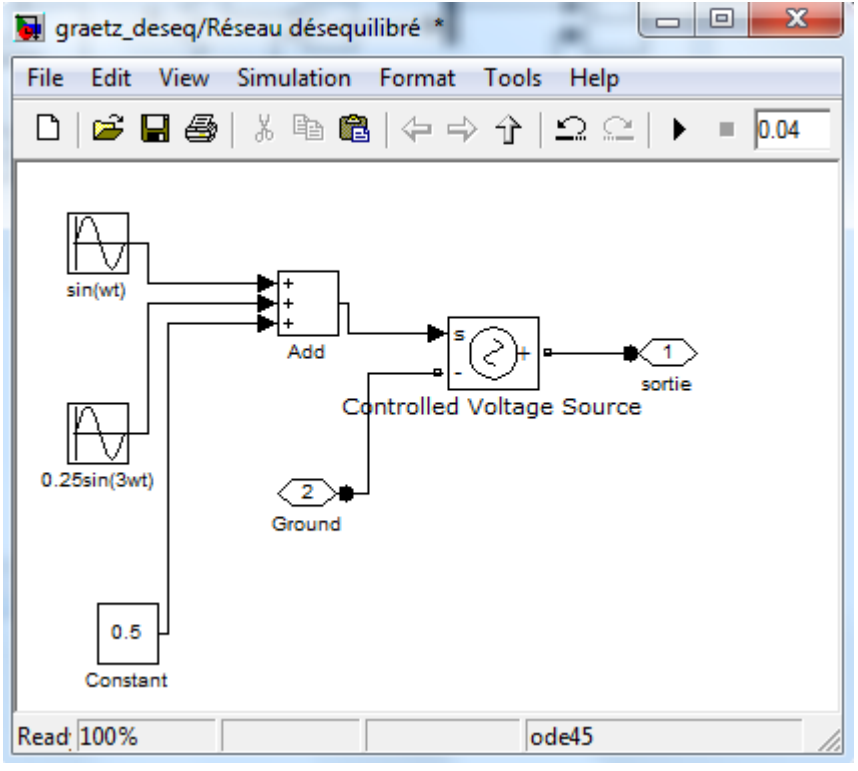
Le bloc de mesure de tension convertit la tension mesurée en un signal de Simulink.

Trois signaux, un signal continu, un signal sinusoïdal de fréquence 50 Hz et un signal sinusoïdal de fréquence 150 Hz



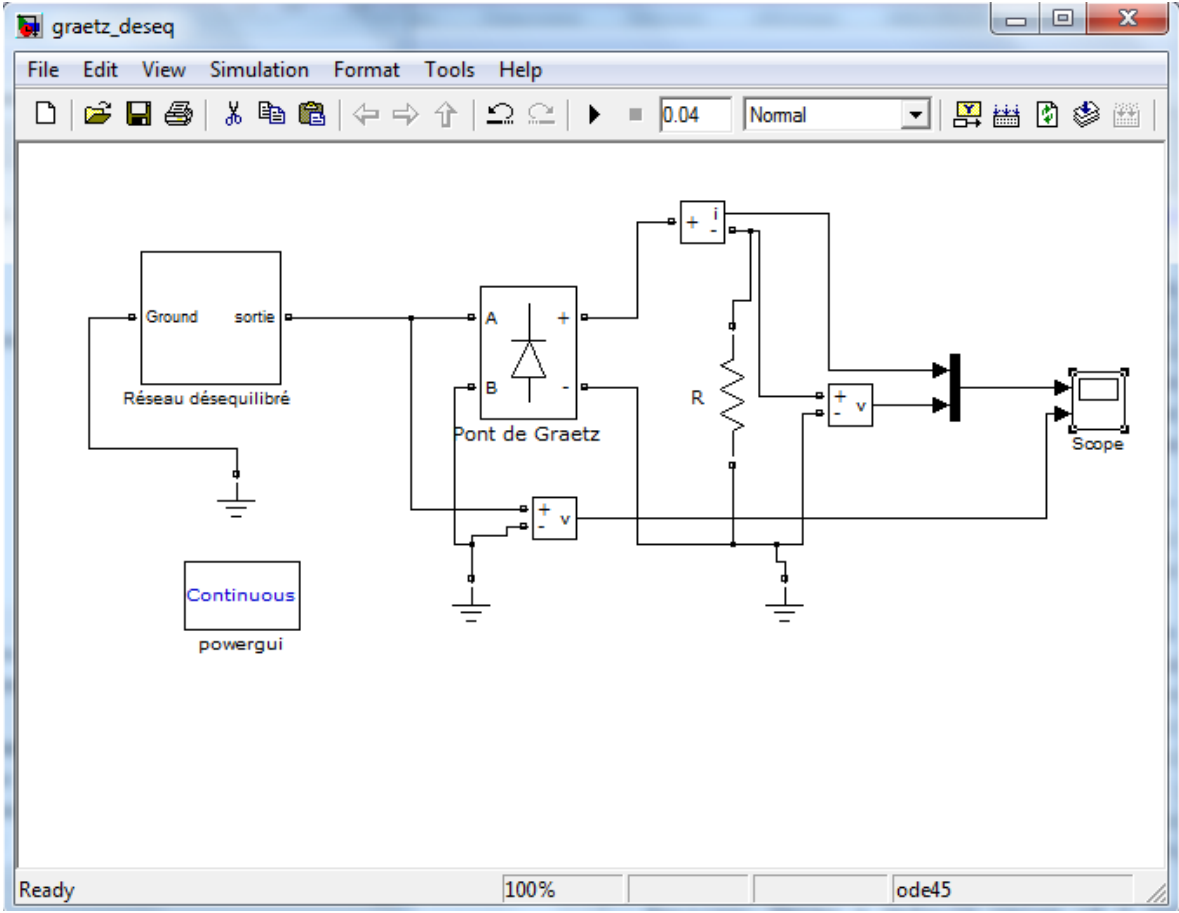


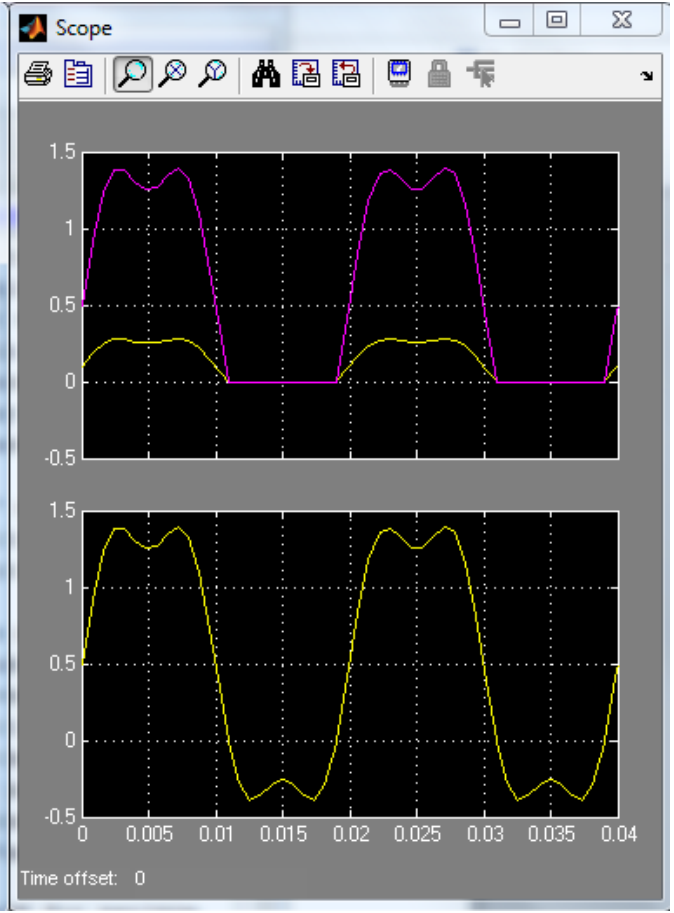
Cette méthode d'interfaçage, vous permet de modéliser un système avec des blocs Simulink puis lui ajouter un bloc de source à tension contrôlée (Controlled Voltage Source block) qui à la fin vous le rendez un sous système (subsystem), ainsi le système final obtenu pourra être incorporé dans un modèle crée en Simpowersytems.



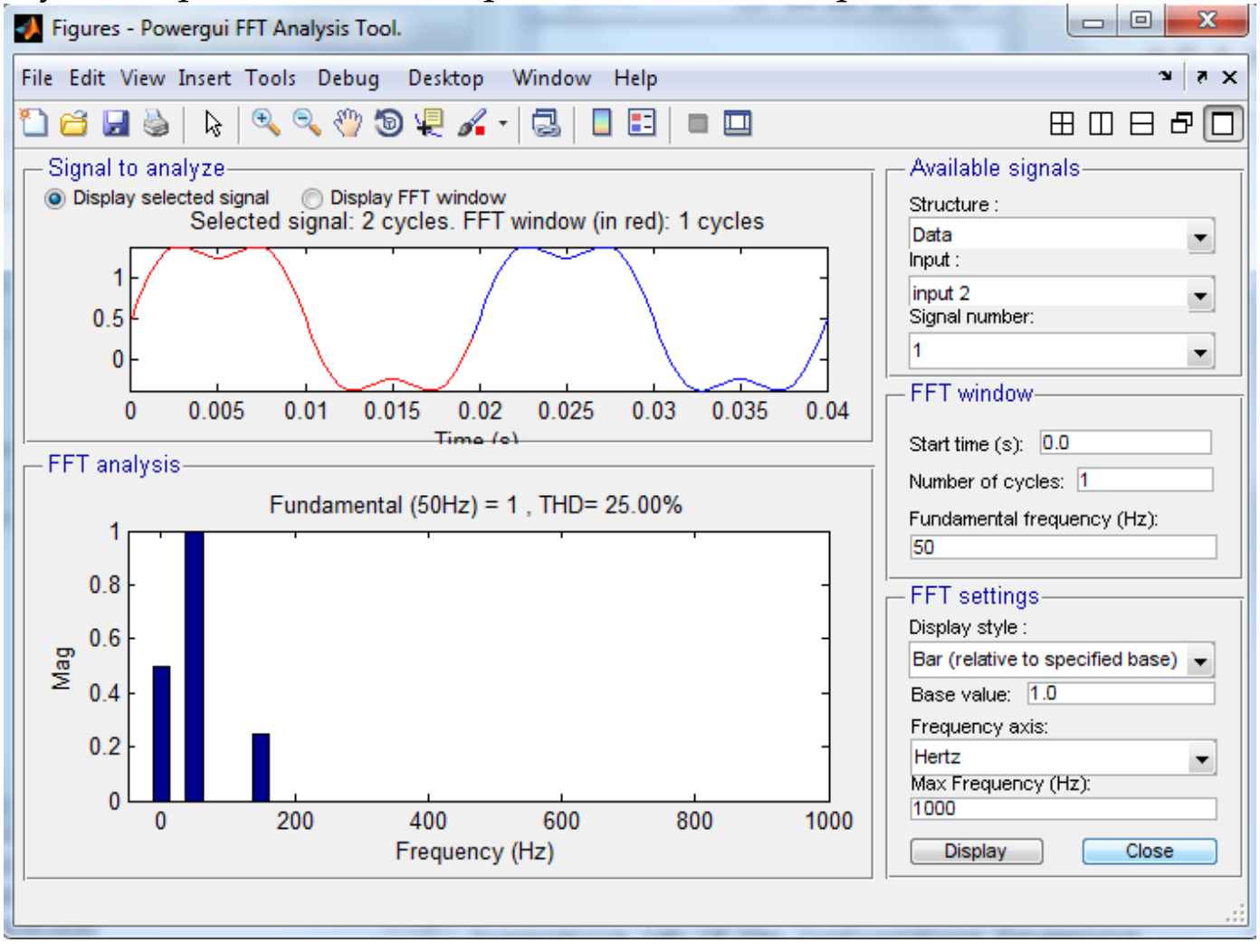
Le réseau déséquilibré avec une offset égale à 0.5 avec une harmonique d'ordre 3.
Essayant de voir le comportement du redresseur en pont de Graëtz quand il est alimenté par ce réseau.

Le sous système réseau déséquilibré réalisé avec des des blocs Simulink alimente le redresseur qui est conçu par les blocs de Simpowersystems

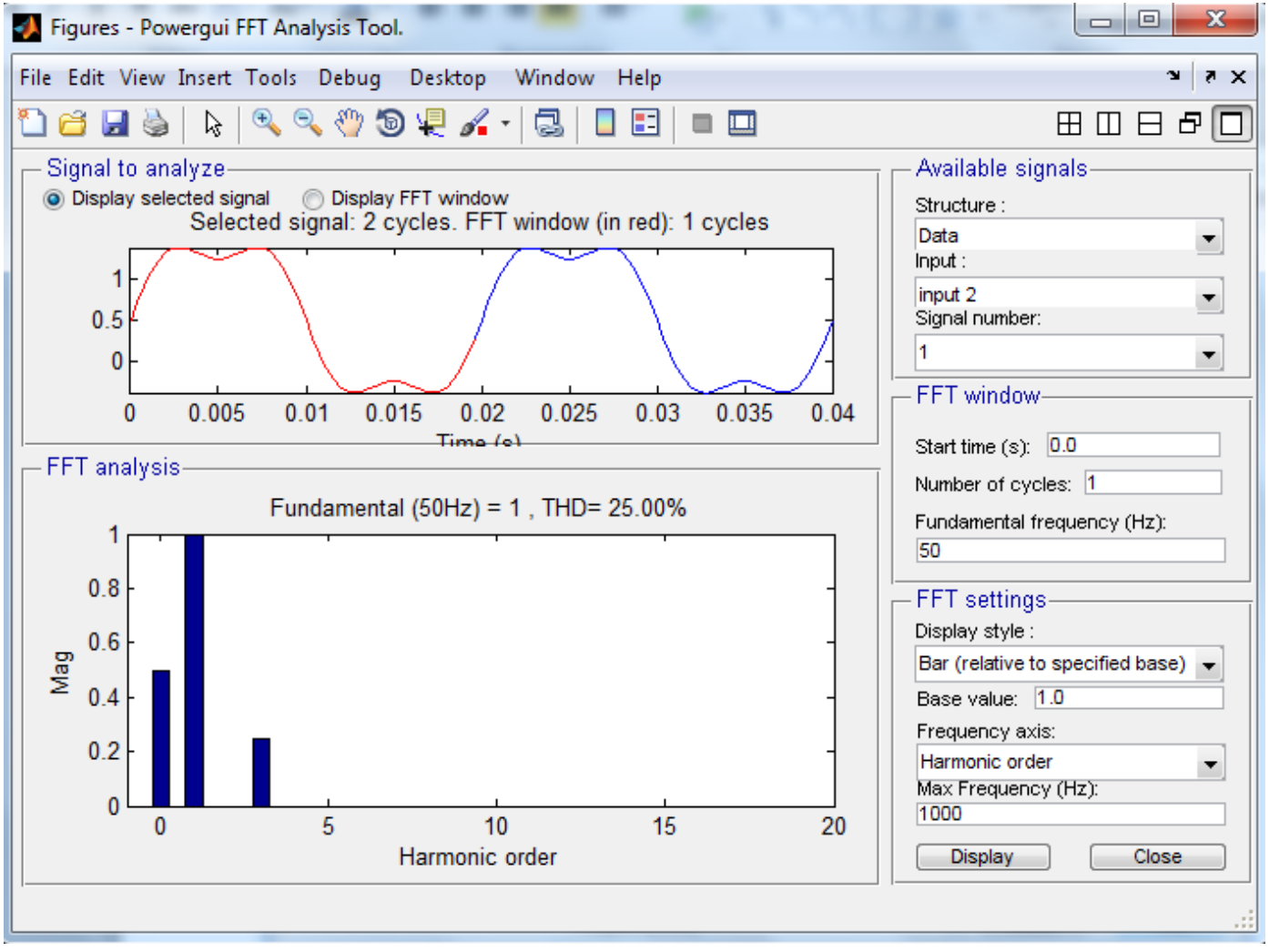




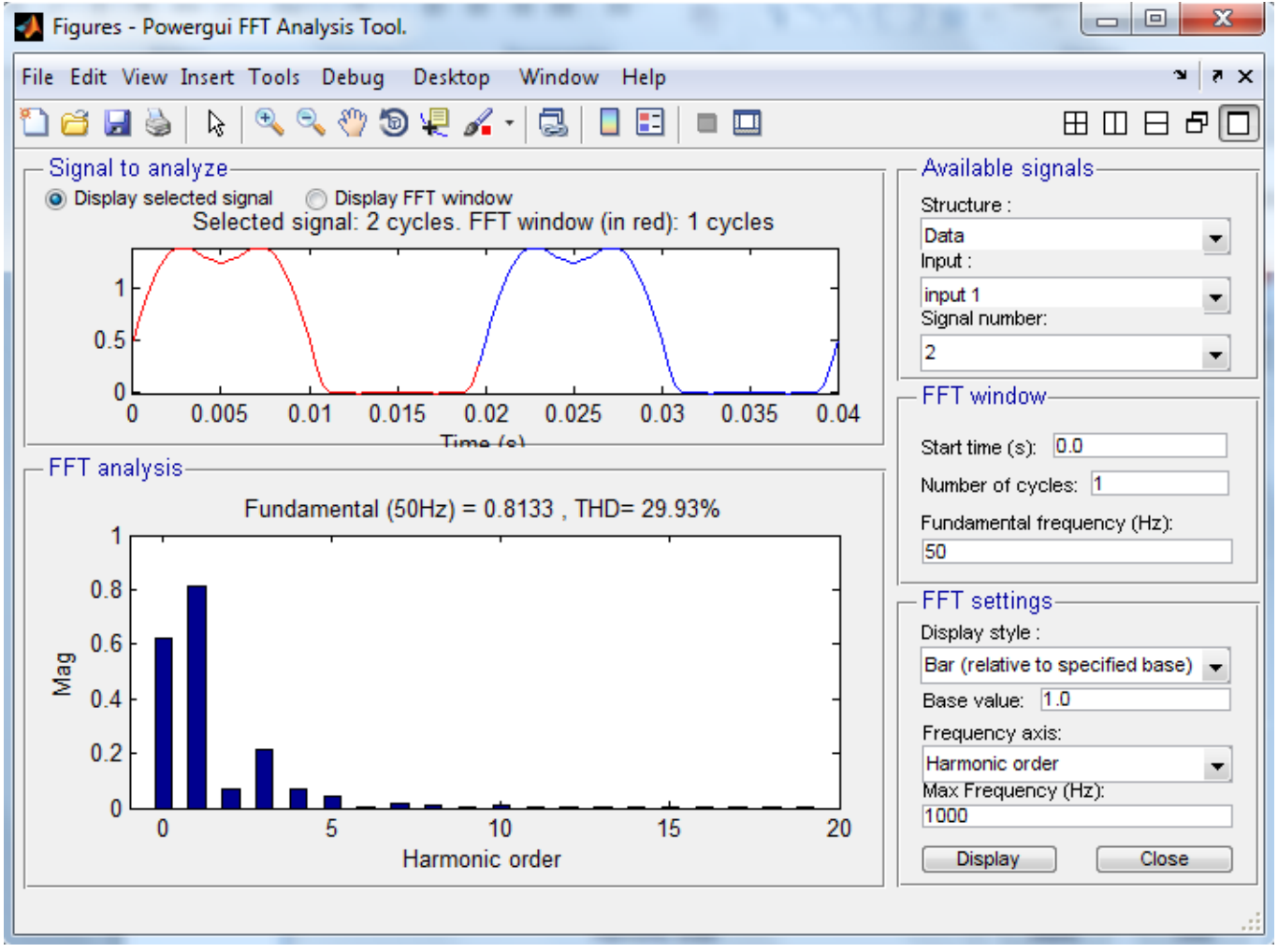
Le réseau déséquilibré avec une offset égale à 0.5 avec une harmonique d'ordre 3.
L'analyse du spectre d'harmonique confirme le déséquilibre du réseau.



L’analyse du spectre d’harmonique confirme le déséquilibre du réseau mais l’affichage donne le rang de chaque harmonique .



L’analyse du spectre d’harmonique de la tension de sortie du redresseur.



Merci

pour

votre Attention